



Automata Formal Languages & Logic

UE24CS243A

Introduction to the Course

YOUR NAME

Department of Computer Science & Engineering

Course Objectives

- ❖ Understand different classes of languages and grammars
- ❖ Analyze finite automata, PDA, and Turing Machines
- ❖ Explore language properties and computability limits
- ❖ Introduce logic-based reasoning and inference method

Why study AFLL? (APPLICATIONS)

- ❖ Foundation for compiler design and language parsing
- ❖ Applied in AI, NLP, network security, verification
- ❖ Helps in understanding computability and logic
- ❖ Boosts problem-solving and abstract reasoning skills
- ❖ Etc.....

Unit 1: Finite Automata and Regular Languages (14 hrs)

Unit 2: PDA and Context-Free Languages (14 hrs)

Unit 3: Turing Machines & CFG Properties (14 hrs)

Unit 4: Undecidability & Declarative Logic (14 hrs)

Unit 1 – Finite Automata

- Mathematical Notations and Basic Concepts
- Deterministic & Non-Deterministic Finite Accepters
- State Reduction and Regular Expressions
- Practical Regex usage and Regular Grammars

Unit 2 – PDA & Context-Free Languages

- Closure Properties of Regular Languages
- PDA: Deterministic and Non-Deterministic
- Context-Free Grammars and their Applications

Unit 3 Overview – Turing Machines & CFG Properties

- Ambiguity and Normal Forms of CFGs
- Closure Properties and Pumping Lemma for CFLs
- Turing Machines and Computability

Unit 4 Overview – Undecidability & Logic

- Recursive Languages and the Chomsky Hierarchy
- Rice's Theorem and Proof Techniques
- Propositional & First Order Logic and Inference

- ❖ JFLAP: Automata simulation and visualization
- ❖ PLY (Python Lex-Yacc): Lexical analysis & parsing
- ❖ Optional: Python for regular expressions

Course Expectations

- ☐ - Active participation and regular practice
- ☐ - Bi-Weekly assignments
- ☐ - ISA and final exam assessments
- ☐ - Application-focused learning with logic and modeling

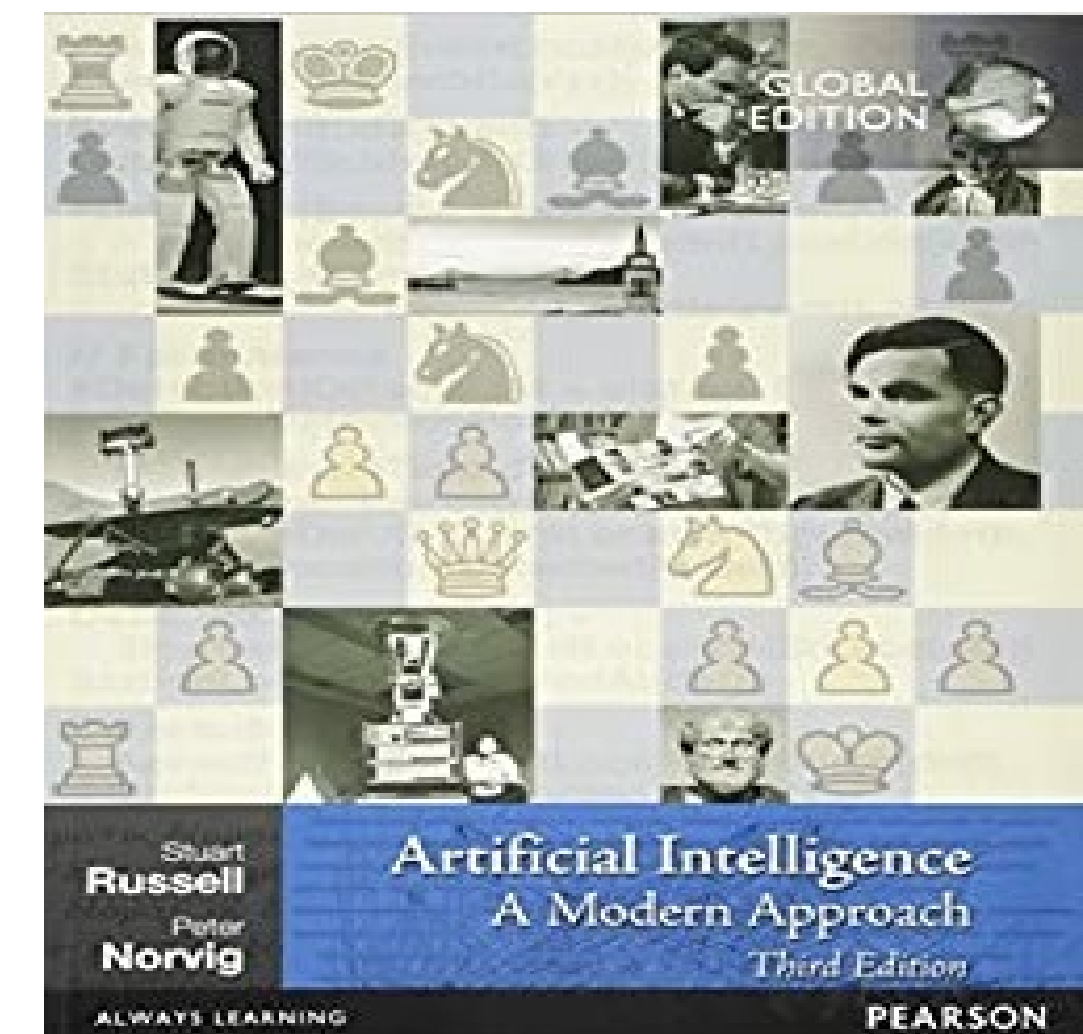
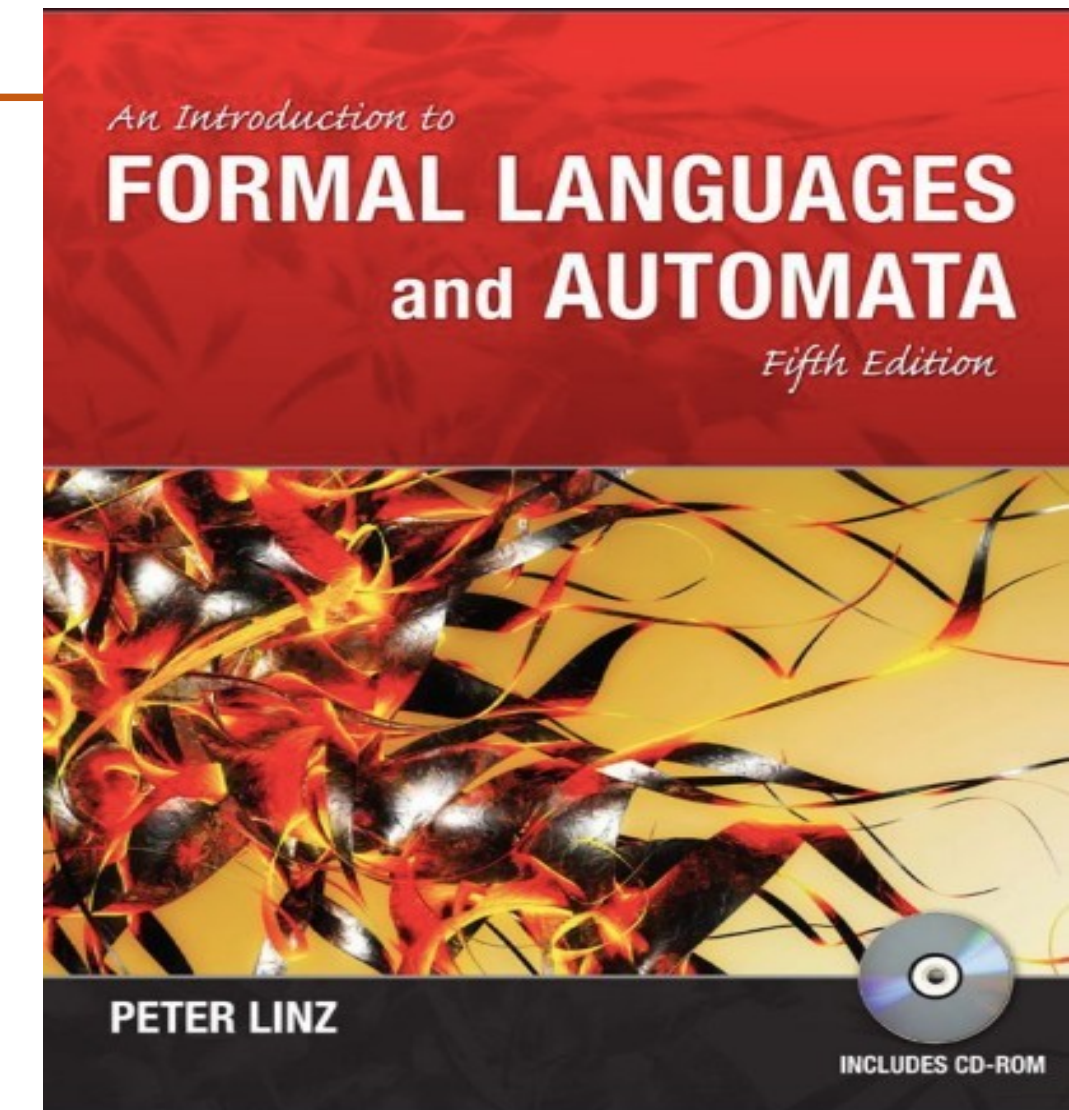
Text Books:

Text Book(s):

1. "An Introduction to Formal Languages and Automata", Peter Linz, Jones and Bartlett, New Delhi, India, 5th Edition, 2011.
2. "Artificial Intelligence – A Modern Approach", Stuart Russell and Peter Norvig, Pearson, 3rd Edition (Paperback), 2016

Reference Book(s):

3. "Theory of Computation", Michael Sipser, Cengage Learning, New Delhi, India, 2008.
4. "Introduction to Automata Theory, Languages, and Computation", John E Hopcroft, Rajeev Motwani, Jeffrey D Ullman, Pearson Education, New Delhi, India, 3rd Edition, 2009.
5. "Theory of Computation: A Problem–Solving Approach", Kavi Mahesh, Wiley India, New Delhi, 2012.



Key Definitions

What is meant by Computations?

Computation is executing an algorithm (i.e.) it involves taking some inputs and performing required operation on it to produce an output.

Input x

Output $y=f(x)$

Computation is the sequence of steps that can be performed by computer.

The computer which performs computation is not actual computers they are “abstract machines”. It can be defined mathematically.

Examples of abstract machines

- Turing Machine (Powerful as real Computers)
- Finite Automata (Simple)

What is theory of computations?

The theory of computation is the branch that deals with how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches:

1. automata theory,
2. computability theory,
3. computational complexity theory

Key Definitions

What is Automata Theory?

- Automata theory is the study of abstract machines (or abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.
- Automata which means that something is doing something by itself. Automata theory is also closely related to formal language theory, as the automata are often classified by the class of formal languages they are able to recognize.
- An automaton can be a finite representation of a formal language that may be an infinite set.

What is a formal language?

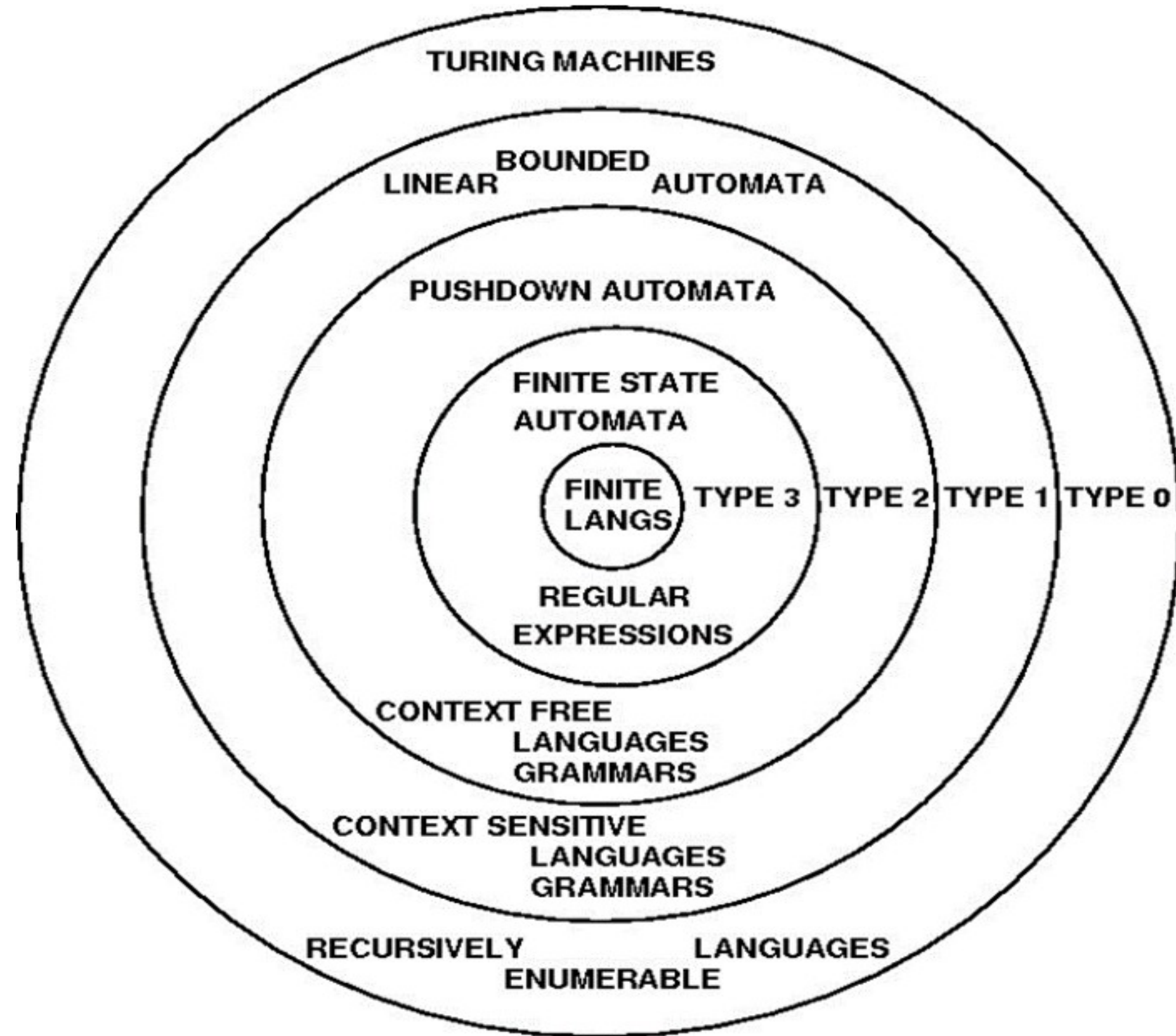
Language is a set of valid strings from some alphabet. The set may be empty, finite or infinite.

$L(M)$ is the language defined by machine M and $L(G)$ is the language defined by context free grammar.

The two notations for specifying formal languages are:

- Grammar or regular expression (Generative approach)
- Automaton (Recognition approach)

Key Definitions – Formal Languages/Grammar/



The **Chomsky hierarchy** in the fields of formal language theory, computer science, and linguistics, is a containment hierarchy of classes of formal grammars.

Course Evaluation Strategy

Evaluation Policy

Evaluation component	Test Mode	Conducted for Marks	Scaled to Marks
ISA-I	CBT	40	20
ISA-II	CBT	40	20
Experiential Learning	Syntax validation - Coding Assignment using PLY tool	6 Marks	10
	Unit-wise Problem-solving Assignment in Class: 4 questions/unit, student should solve in a sheet of paper and submit.	4 Marks (1 Mark/Unit)	
Total (ISA)			50
ESA	Pen and Paper	100	50
Total (ISA+ESA)			100



THANK
YOU
YOUR NAME

Department of Computer Science &
Engineering [@pes.edu](mailto:pes.edu)



Automata Formal Languages & Logic

Unit 1 - Finite State Machines

Preet Kanwal

Department of Computer Science &
Engineering

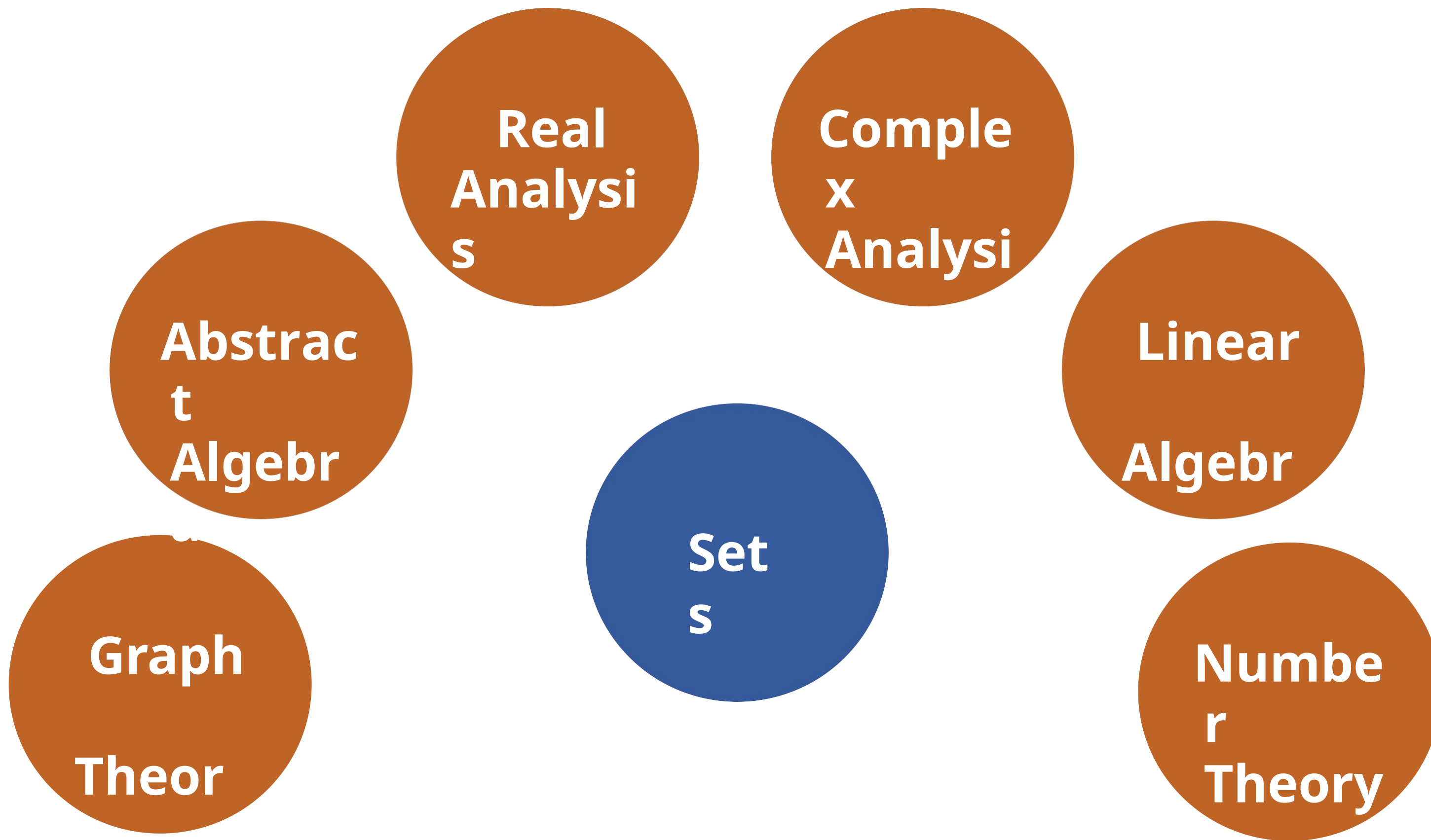
Automata Formal Languages & Logic **Mathematical** **Preliminaries**

1. Sets

2. Functions & Relations

Automata Formal Languages & Logic

Why are Sets important?



Automata Formal Languages & Logic **Sets**

A **set** is an **unordered** collection of **distinct** objects, which may be anything (including other sets).

Example :



Set Notation :

Curly braces with commas separating out

Automata Formal Languages & Logic Sets

A *set* is an **unordered** collection of distinct objects, which may be anything (including other sets).



Automata Formal Languages & Logic

Set Representation

A set can be represented in any one of the following three ways or forms.

- (i) Descriptive form
- (ii) Set-builder form or Rule form
- (iii) Roster form or Tabular form

Automata Formal Languages & Logic

Set Representation

- (i) Descriptive form -
verbal/english description of
its elements.

Example :

“The set of all even numbers.”

**“The set of all real numbers less
than 150.”**

“The set of all negative integers.”

Automata Formal Languages & Logic

Set Representation

(ii) Set-builder form or Rule form Even Natural Numbers
 $\{ n \mid n \in \mathbb{N} \text{ and } n \text{ is even} \}$

$\{ n \mid n \in \mathbb{N} \text{ and } n \text{ is even} \}$

The set of all n where n is a natural number and n is even

Automata Formal Languages & Logic

Set Representation

(iii) Roster form or Tabular form

Listing the elements of a set inside a pair of braces { } is called the roster form.

For example,

Let $A = \{x : x \text{ is an integer and } -1 \leq x < 5\}$

In roster form we write $A = \{-1, 0, 1, 2, 3, 4\}$

Automata Formal Languages & Logic

Set Membership



I
s



in the
set?

Automata Formal Languages & Logic

Set Membership



I
s



in the
set?

Automata Formal Languages & Logic

Order of a Set / Cardinality

The *cardinality* of a set S is the number of elements it contains, denoted by $|S|$.

Examples:

1) if Set $A = \{a, b, c, d, e\}$

then $|A| = 5$

2) if Set $B = \{\{a, b\}, \{c, d, e, f, g\}, \{h\}\}$

then $|B| = 3$

3) if Set $C = \{n \in \mathbb{N} \mid n < 137\}$

then $|C| = 136$

Automata Formal Languages & Logic **Order of a Set / Cardinality**

Question :

1) What is $|N|$?

2) Consider the set

$S = \{ n \mid n \in N \text{ and } n \text{ is even} \}$

What is $|S|$?

Automata Formal Languages & Logic

Order of a Set / Cardinality

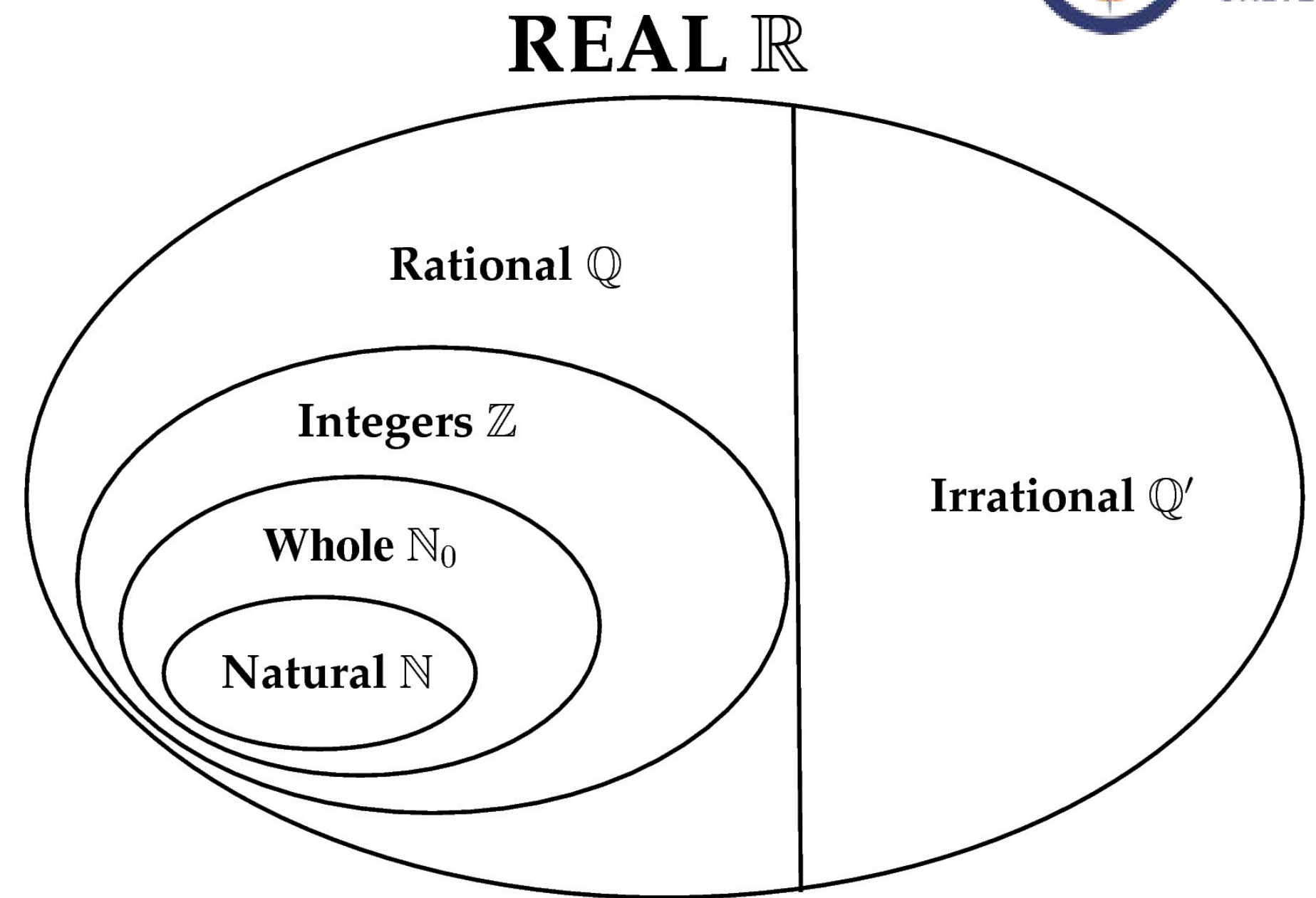
Which infinity is larger?

$\mathbb{N}$	0	1	2	3	4	5	6	7	8	...
	↕	↕	↕	↕	↕	↕	↕	↕	↕	
S	0	2	4	6	8	10	12	14	16	...

$$n \leftrightarrow 2n$$

$$S = \{ n \mid n \in \mathbb{N} \text{ and } n \text{ is even} \}$$

$$|S| = |\mathbb{N}| = \aleph_0$$



Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set

2. Singleton Set

3. Finite Set

4. Infinite Set

5. Equivalent Set

6. Equal Sets

7. Disjoint Sets

8. Subsets

9. Proper

Subset

10. Super Set

11. Universal

Set 12. Power

Set

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set

2. Singleton Set

3. Finite Set

4. Infinite Set

5. Equivalent Set

6. Equal Sets

7. Disjoint Sets

8. Subsets

9. Proper

Subset

10. Super Set

11. Universal

Set 12. Power

Set

A null set or an empty set is a valid set with no member.

 $\{ \}$ $\emptyset$

We use this symbol to denote empty set

Automata Formal Languages & Logic

Types of Sets - Empty or Null Set

Question : Are these equal ?

$$\emptyset \text{ ! } = \{ \emptyset \}$$

This set contains nothing at all.

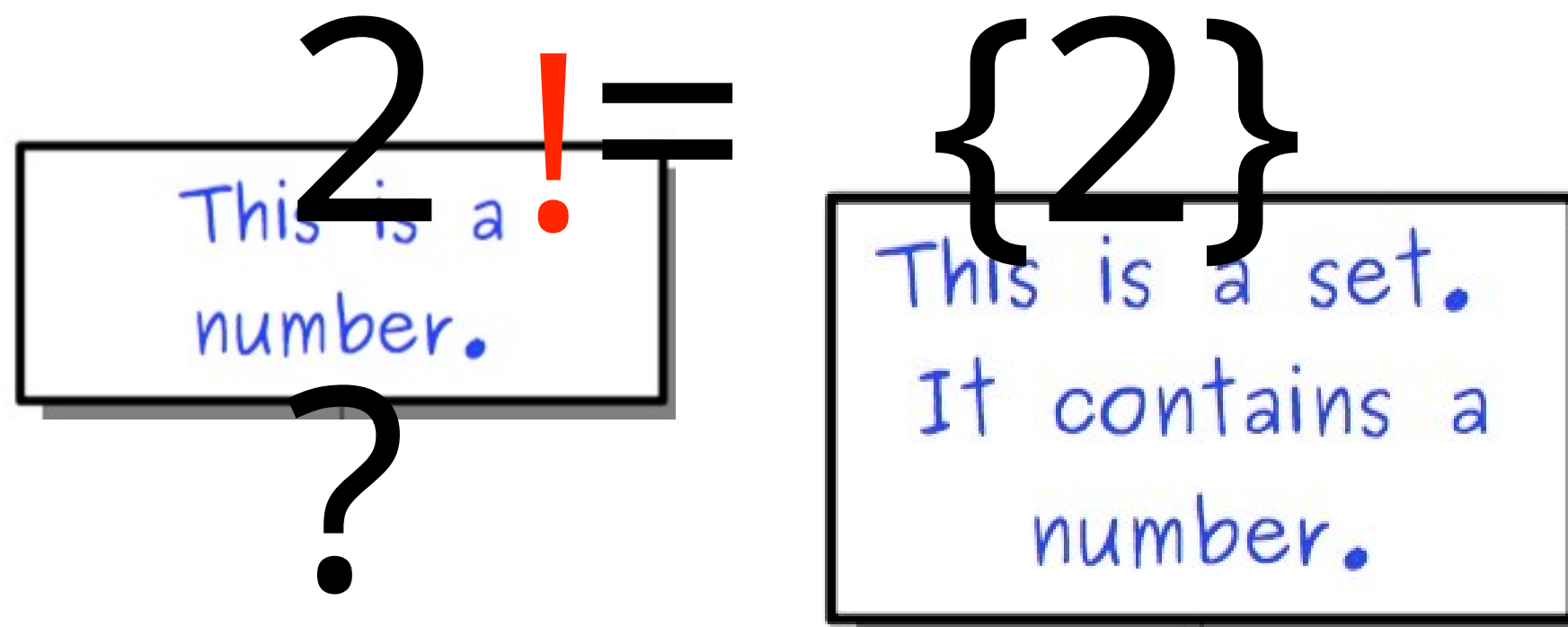
This set has one element, which happens to be the empty set.

They are not equal

Automata Formal Languages & Logic

Types of Sets - Empty or Null Set

Question : Are these equal ?



They are not equal

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set

2. Singleton Set

3. Finite Set

4. Infinite Set

5. Equivalent Set

6. Equal Sets

7. Disjoint Sets

8. Subsets

9. Proper

Subset

10. Super Set

11. Universal

Set 12. Power

Set

If a set contains only one element it is called to be a singleton set.

Example

:

Set A = {1}

Set B = {a}

Set C

= {4}

Set D

= {car}

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

A set in which number of elements are finite (can be counted) are called Finite Sets.

Example

Set A = {4, 8, 12, 16}

Set B = {90, 180}

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

A set in which number of elements are infinite are called Infinite Sets.

Example

N = Set of all natural no's

$= \{1, 2, 3, \dots\}$

Z = Set of all integers
 $= \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

If the number of elements are same for two different sets, then they are called an equivalent sets.

Example:

If $A = \{1,2,3,4\}$ and $B = \{\text{Red, Blue, Green, Black}\}$

Here, Set A and B are equivalent, since $|A| = |B| = 4$

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

The two sets A and B are said to be equal if they have exactly the same elements, order of elements do not matter.

Example:

if $A = \{1,2,3,4\}$ and
 $B = \{4,3,2,1\}$

Then, Set A and B are equal & can be denoted as :
 $A = B$

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

The two sets A and B are said to be disjoint if the set does not contain any common element.

Example

:

Set A = {1,2,3,4} and
Set B = {5,6,7,8} are
disjoint sets, because
there is no common
element between them.

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

A set S is a subset of a set T (denoted $S \subseteq T$) if all elements of S are also elements of T .

Examples:

1. $\{ 1, 2, 3 \} \subseteq \{ 1, 2, 3, 4 \}$
2. $\mathbb{N} \subseteq \mathbb{Z}$ (every natural number is an integer)
3. $\mathbb{Z} \subseteq \mathbb{R}$ (every integer is a real number)

Automata Formal Languages & Logic

Types of Sets - Subsets

Question :

Are there any sets S where $\emptyset \subseteq S$?

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set

2. Singleton Set

3. Finite Set

4. Infinite Set

5. Equivalent Set

6. Equal Sets

7. Disjoint Sets

8. Subsets

9. Proper

Subset

10. Super Set

11. Universal

Set 12. Power

Set

if B is a proper subset of A (denoted as $B \subset A$), then all elements of B are in A but A contains at least one element that is not in B.

Example :

$A = \{1, 3, 5\}$

$B = \{1, 5\}$

$C = \{1, 3, 5\}$

then

, B

$\subset A$

But,

$C \subseteq$

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set

2. Singleton Set

3. Finite Set

4. Infinite Set

5. Equivalent Set

6. Equal Sets

7. Disjoint Sets

8. Subsets

9. Proper

Subset

10. Super Set

11. Universal

Set 12. Power

Set

if $B \subseteq A$ (B is a subset of A)

then, $A \supseteq B$ (A is a superset of B)

Here,

if $B \subset A$ (B is a proper subset of A)
then,

$A \supset B$ (A is a proper superset of B)

Consider Example

: $A = \{1, 3, 5\}$

$B = \{1, 5\}$

$C = \{1, 3, 5\}$

Here, $A \supseteq C$ or $C \subseteq A$

and $A \supset B$

Automata Formal Languages & Logic

Types of Sets - Subsets

Question :

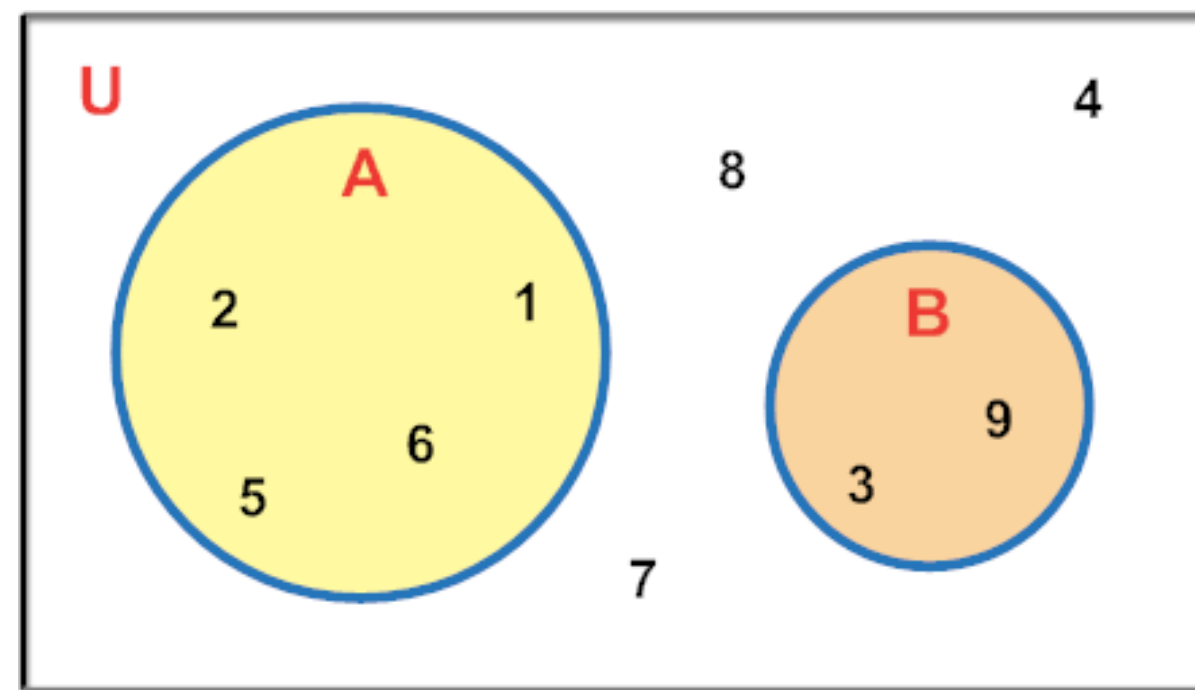
Is every set a subset and superset of itself??

Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

A universal set, denoted by capital U is all the elements, or members, of any group under consideration, including its own elements.



Automata Formal Languages & Logic

Types of Sets

1. Empty Set or Null Set
2. Singleton Set
3. Finite Set
4. Infinite Set
5. Equivalent Set
6. Equal Sets
7. Disjoint Sets
8. Subsets
9. Proper Subset
10. Super Set
11. Universal Set
12. Power Set

Power set of any set S is the set of all subsets of S , including the empty set and S itself

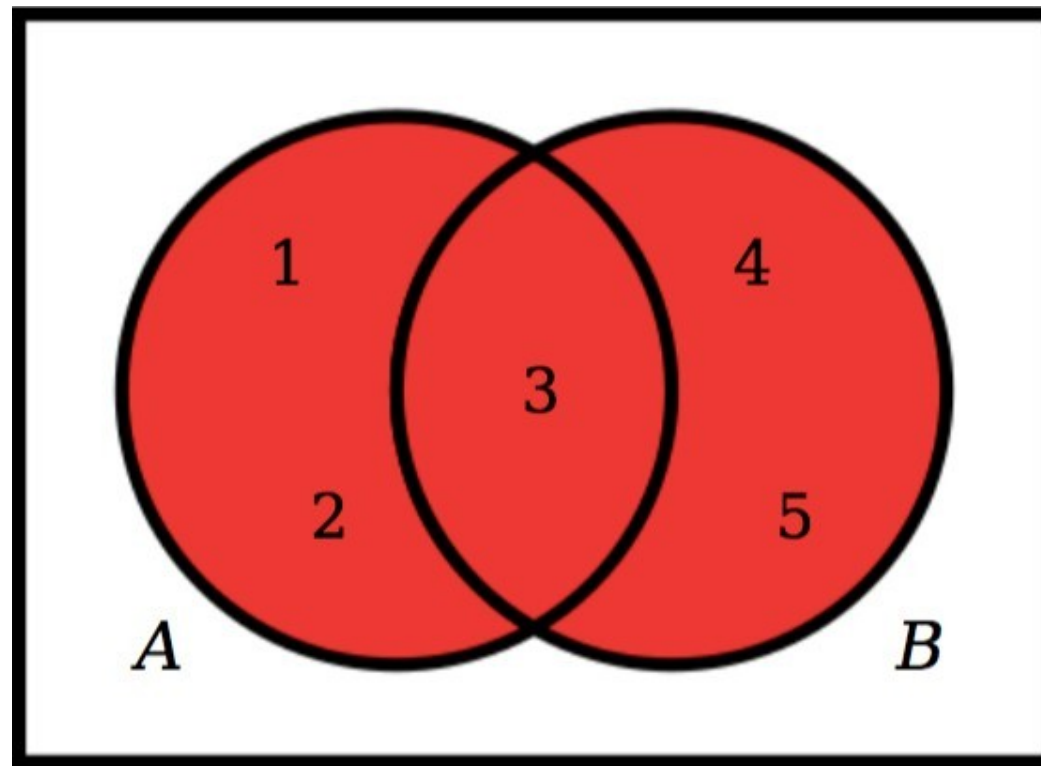
$$S = \{a, b, c, d\}$$

$$P(S) = \{$$

 $\emptyset,$ $\{a\},$ $\{b\},$ $\{c\},$ $\{d\},$ $\{a, b\},$ $\{a, c\},$ $\{a, d\},$ $\{b, c\},$ $\{b, d\},$ $\{c, d\}$

Automata Formal Languages & Logic

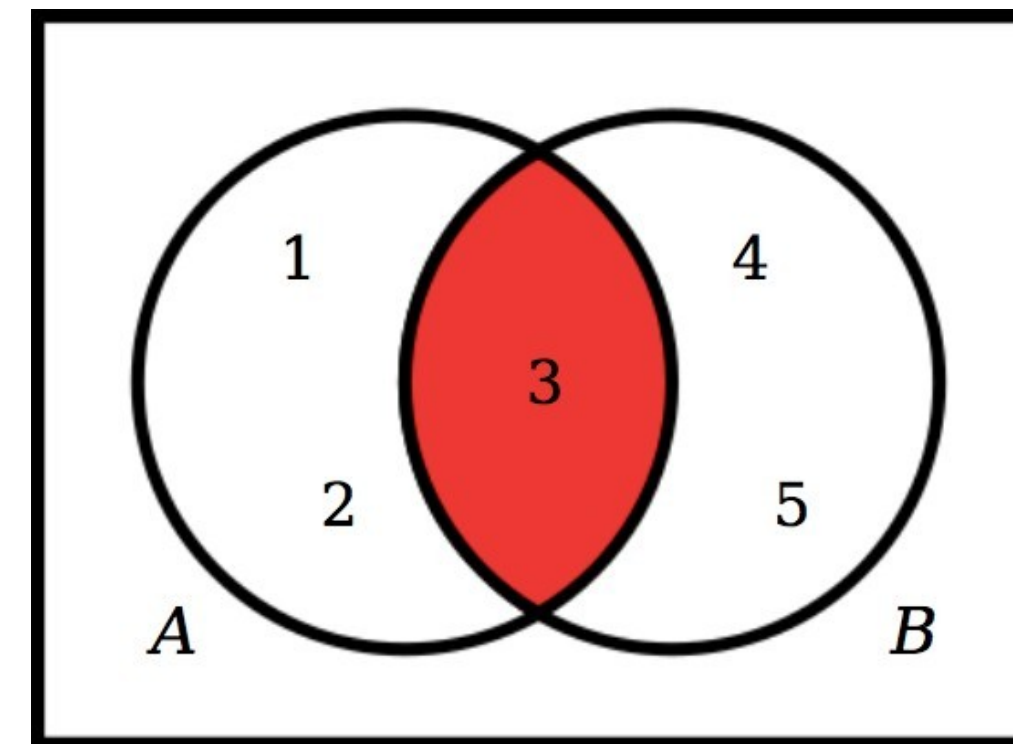
Combining Sets - Operations on Set



$$A = \{ 1, 2, 3 \}$$

$$B = \{ 3, 4, 5 \}$$

Union
 $A \cup B$
 $\{ 1, 2, 3, 4, 5 \}$



$$A = \{ 1, 2, 3 \}$$

$$B = \{ 3, 4, 5 \}$$

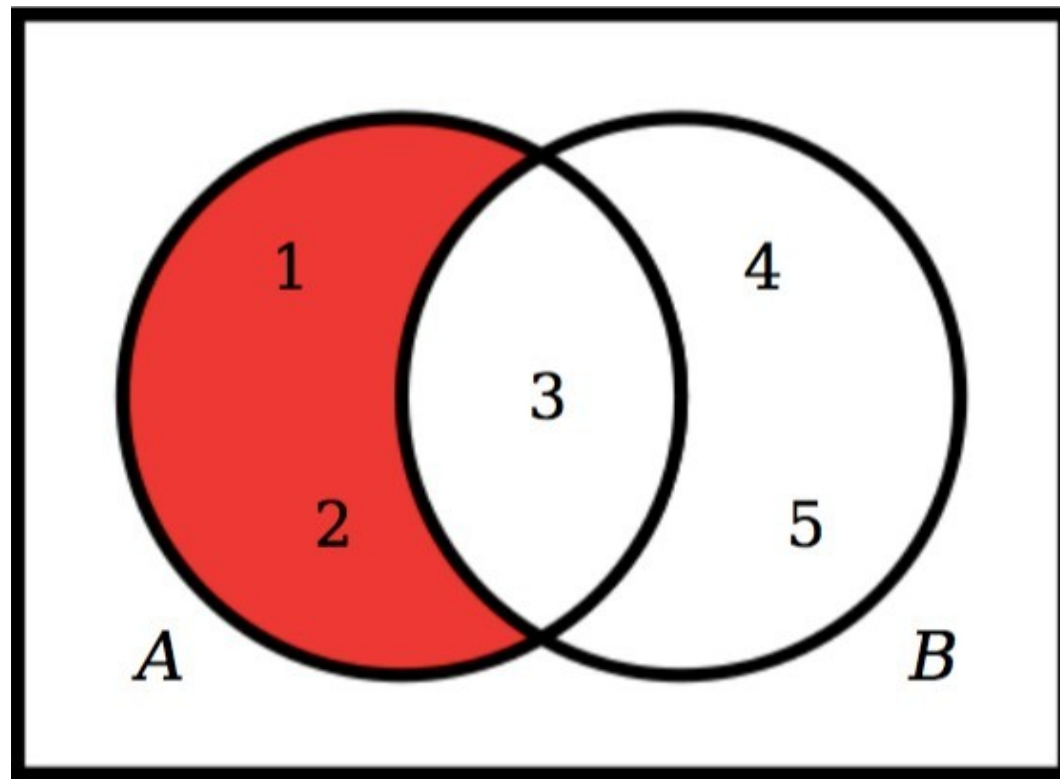
Intersection
 $A \cap B$
 $\{ 3 \}$

$$A \cup B = \{ x : x \in A \text{ or } x \in B \}$$

$$A \cap B = \{ x : x \in A \text{ and } x \in B \}$$

Automata Formal Languages & Logic

Combining Sets

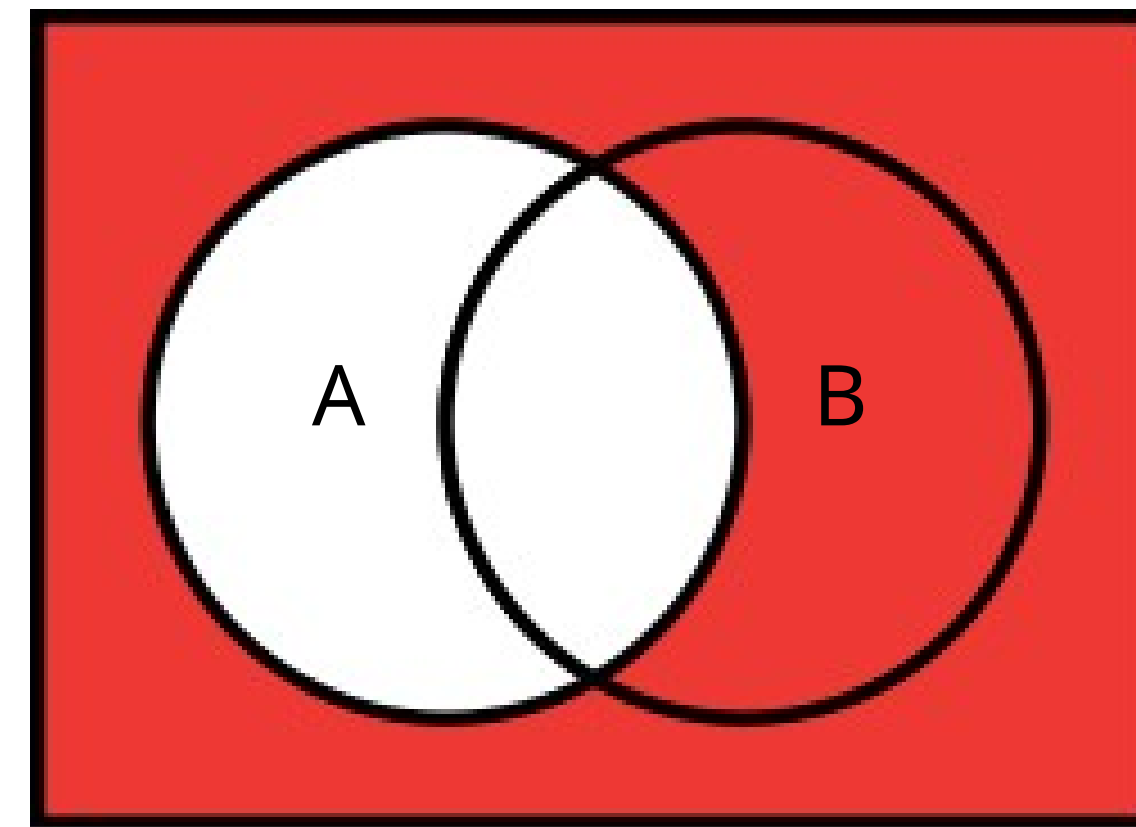


Difference
 $A - B$
 $\{ 1, 2 \}$

$$A = \{ 1, 2, 3 \}$$

$$B = \{ 3, 4, 5 \}$$

$$A - B = \{ x : x \in A \text{ and } x \notin B \}$$



Complime
 $\bar{A}' = \bar{A} = A^C$

$$A' = \{ x : x \in U \text{ and } x \notin A \}$$

Automata Formal Languages & Logic

Set Identities

$$\emptyset' = U$$

<i>Identity</i>	<i>Name</i>
$A \cap U = A$ $A \cup \emptyset = A$	Identity laws
$A \cup U = U$ $A \cap \emptyset = \emptyset$	Domination laws
$A \cup A = A$ $A \cap A = A$	Idempotent laws
$\overline{\overline{A}} = A$	Complementation law
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Commutative laws
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive laws
$\overline{A \cap B} = \overline{A} \cup \overline{B}$ $\overline{A \cup B} = \overline{A} \cap \overline{B}$	De Morgan's laws
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Absorption laws
$A \cup \overline{A} = U$ $A \cap \overline{A} = \emptyset$	Complement laws

Automata Formal Languages & Logic

Cartesian Product of Sets

The Cartesian product of two sets A and B , denoted $A \times B$, is the set of all possible ordered pairs where ,

the elements of A are first and the elements of B are second.

$$A \times B = \{ (a, b) : a \in A \text{ and } b \in B \}$$

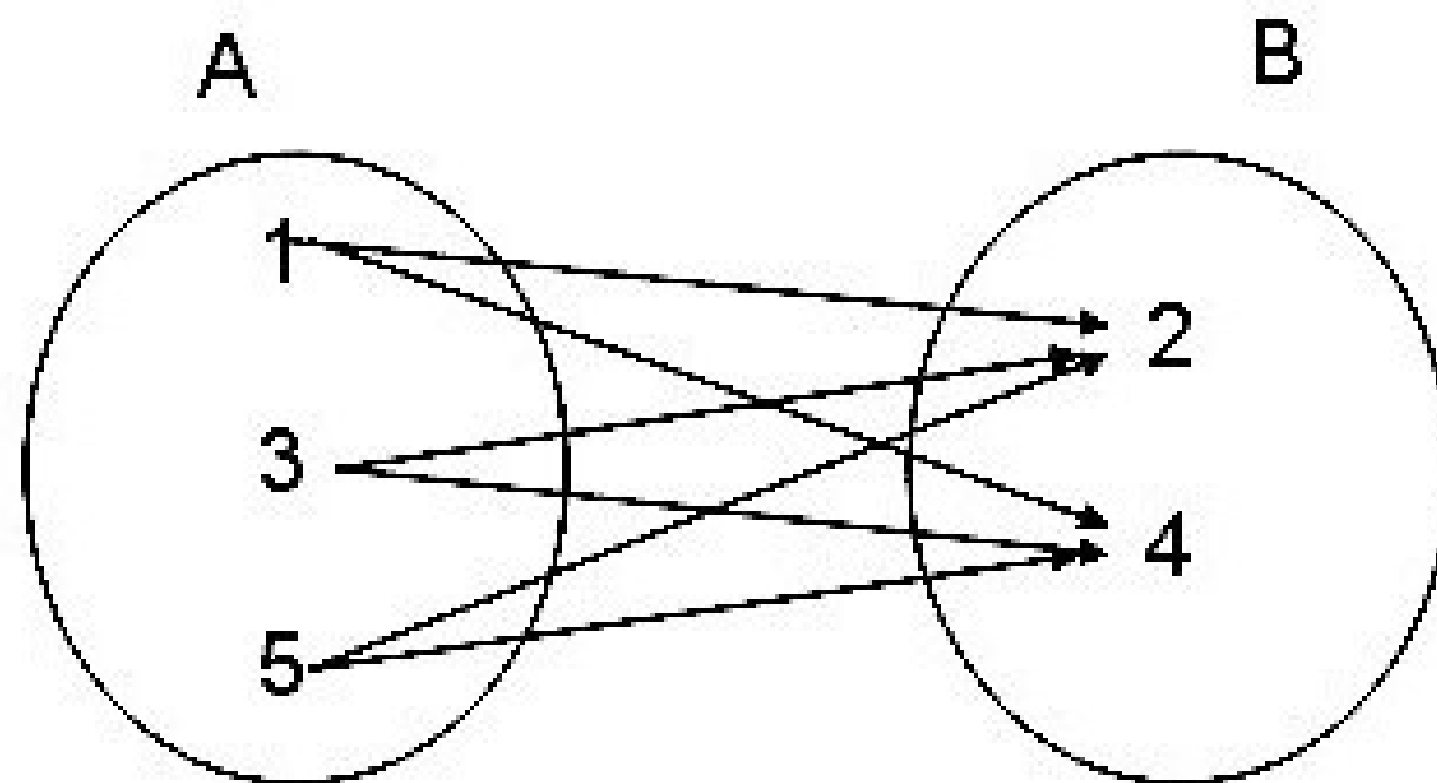
Automata Formal Languages & Logic

Cartesian Product of Sets

$A = \{1, 3, 5\}$ and $B = \{2, 4\}$ then,
 $A \times B = \{ (a, b) : a \in A \text{ and } b \in B \}$

$A \times B = \{ (1,2), (1,4), (3,2), (3,4), (5,2), (5,4) \}$

Arrow Diagram of $A \times B$:



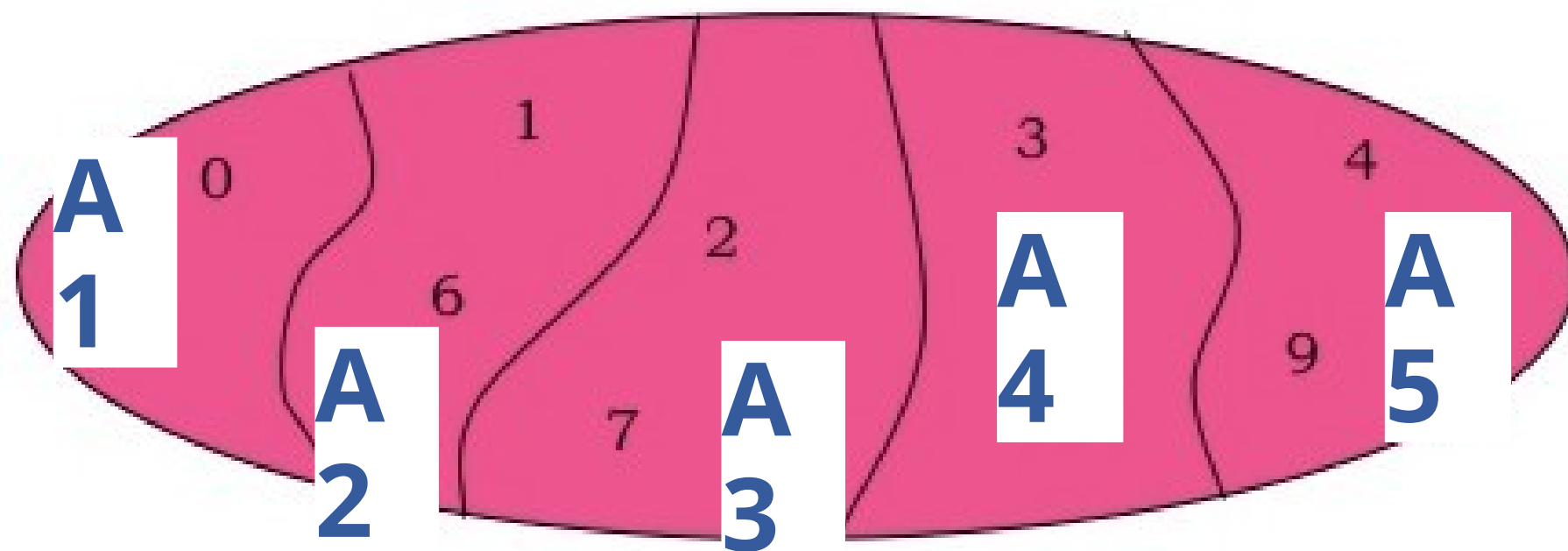
1	→	2
1	→	4
3	→	2
3	→	4
5	→	2
5	→	4

Automata Formal Languages & Logic

Partition of a Set

A partition of a set is a grouping of its elements into non-empty subsets, in such a way that every element is included in exactly one subset.

For example the regions A_1, A_2, A_3, A_4, A_5 form partitions of Set A as shown below



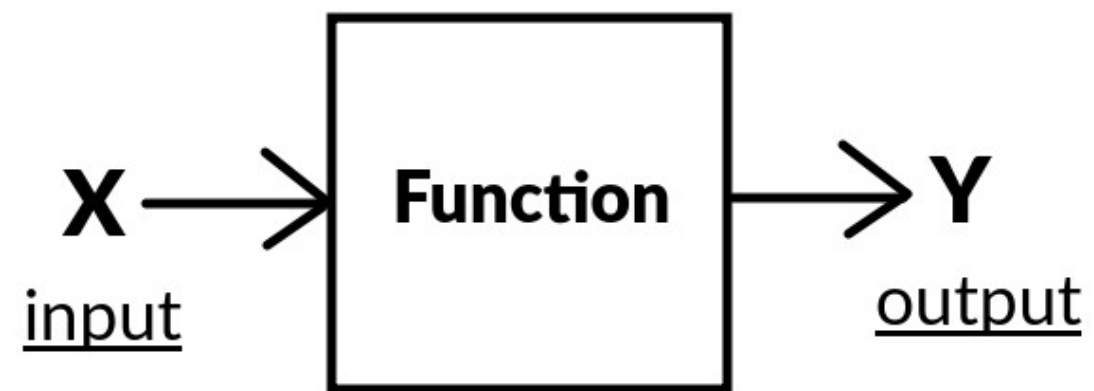
A_1, A_2, A_3, A_4, A_5 are disjoint subsets of A .

Automata Formal Languages & Logic Mathematical Preliminaries

1. Sets

2. Functions & Relations

Automata Formal Languages & Logic Functions

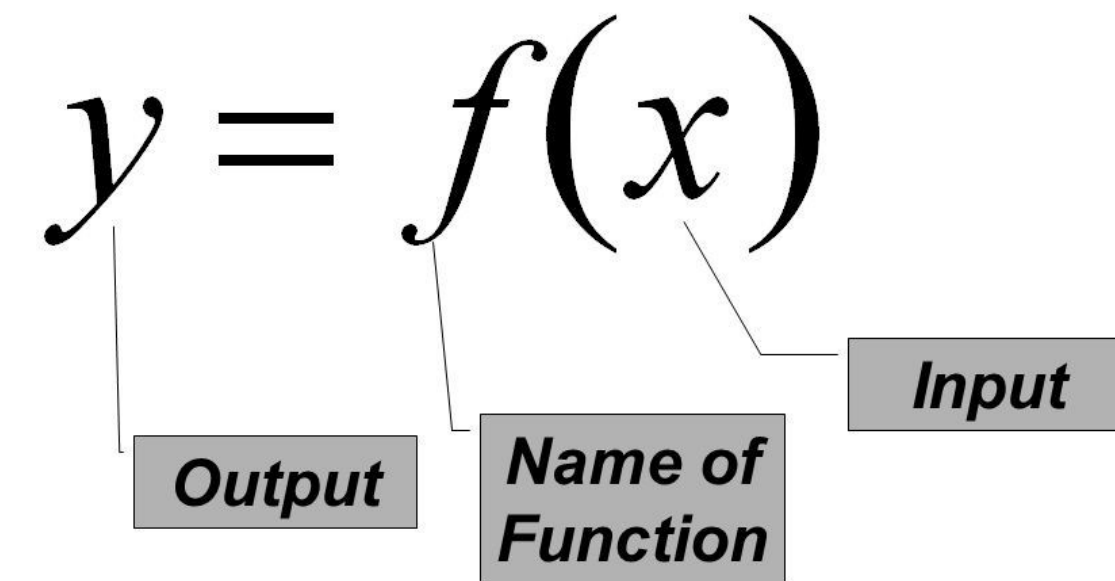


Example :

$$f(x) = \{x + 4, 0 < x < 5\}$$

$$f(2) = 6 = y$$

Function Notation



The equation $y = f(x)$ is shown with labels in grey boxes connected by lines to the corresponding parts of the notation: "Output" points to y , "Name of Function" points to f , and "Input" points to x .

Automata Formal Languages & Logic **Functions**

Set of possible inputs to a function is called its domain.

Range of a Function

:

Set of possible outputs from a function is called its range.

Notation :

$$f: D \rightarrow R$$

E

x

a

m

Domain = {1,2,3,4}

Range = {5,6,7,8}

e

$f: \{1,2,3,4\} \rightarrow \{5,6,7,8\}$

:

f

(

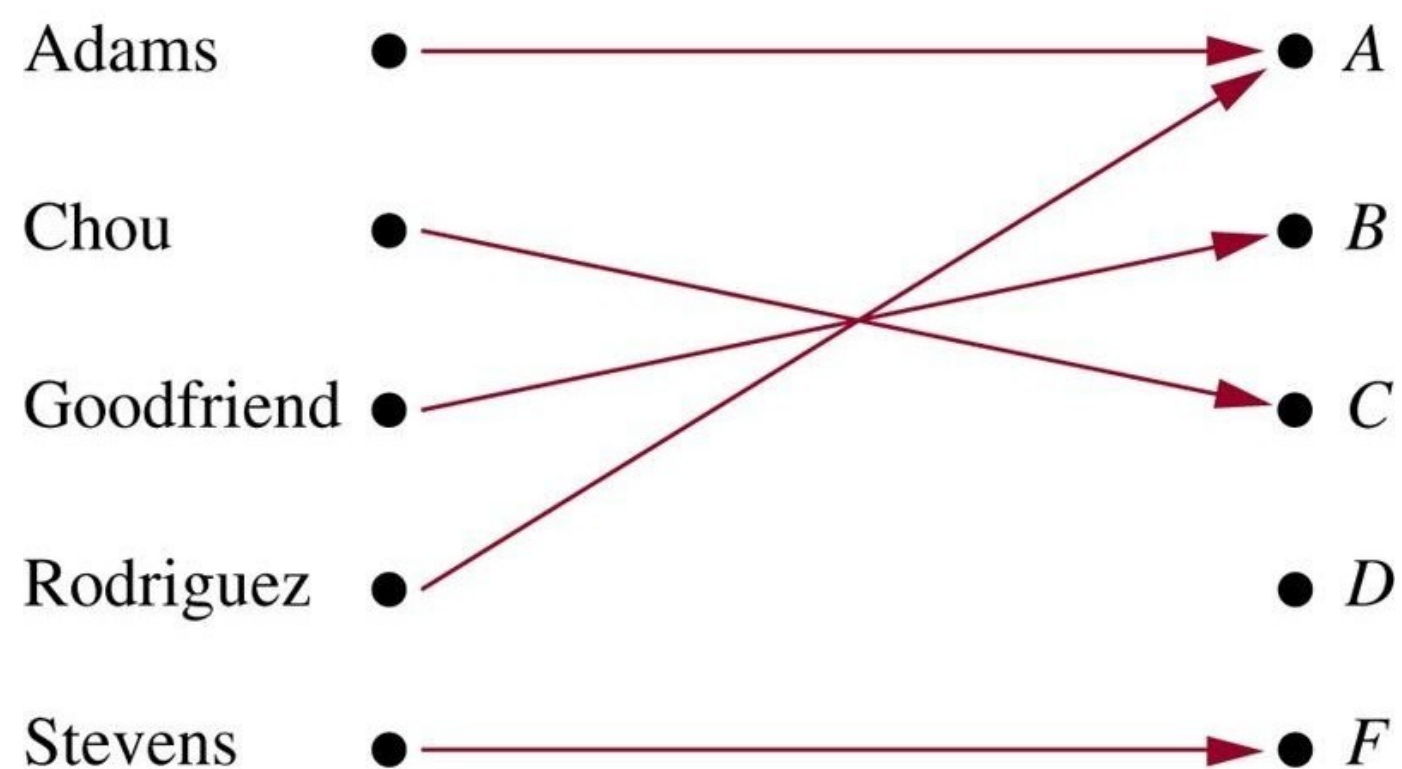
x

)

Automata Formal Languages & Logic **Functions**

- A function f may not necessarily use all the elements of the Range

$f: \text{Student} \rightarrow \text{Grades}$



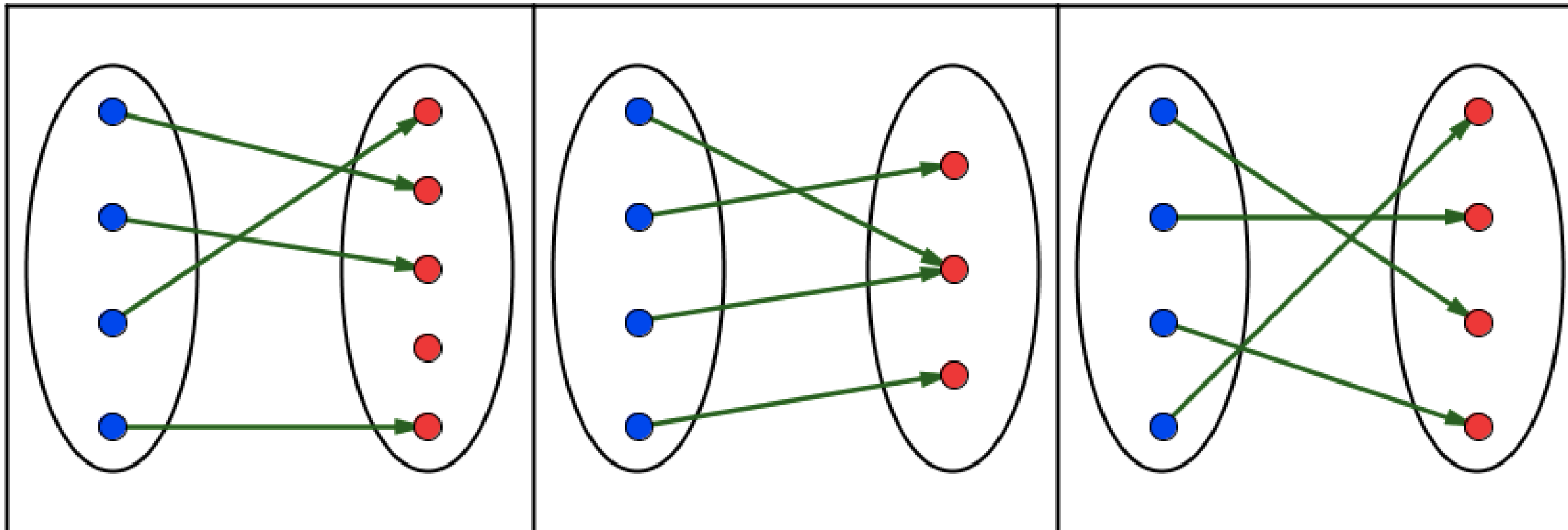
Domain of f : Set of Students
Range

of $f: \{A, B, C, F\}$

Automata Formal Languages & Logic

Types of Functions

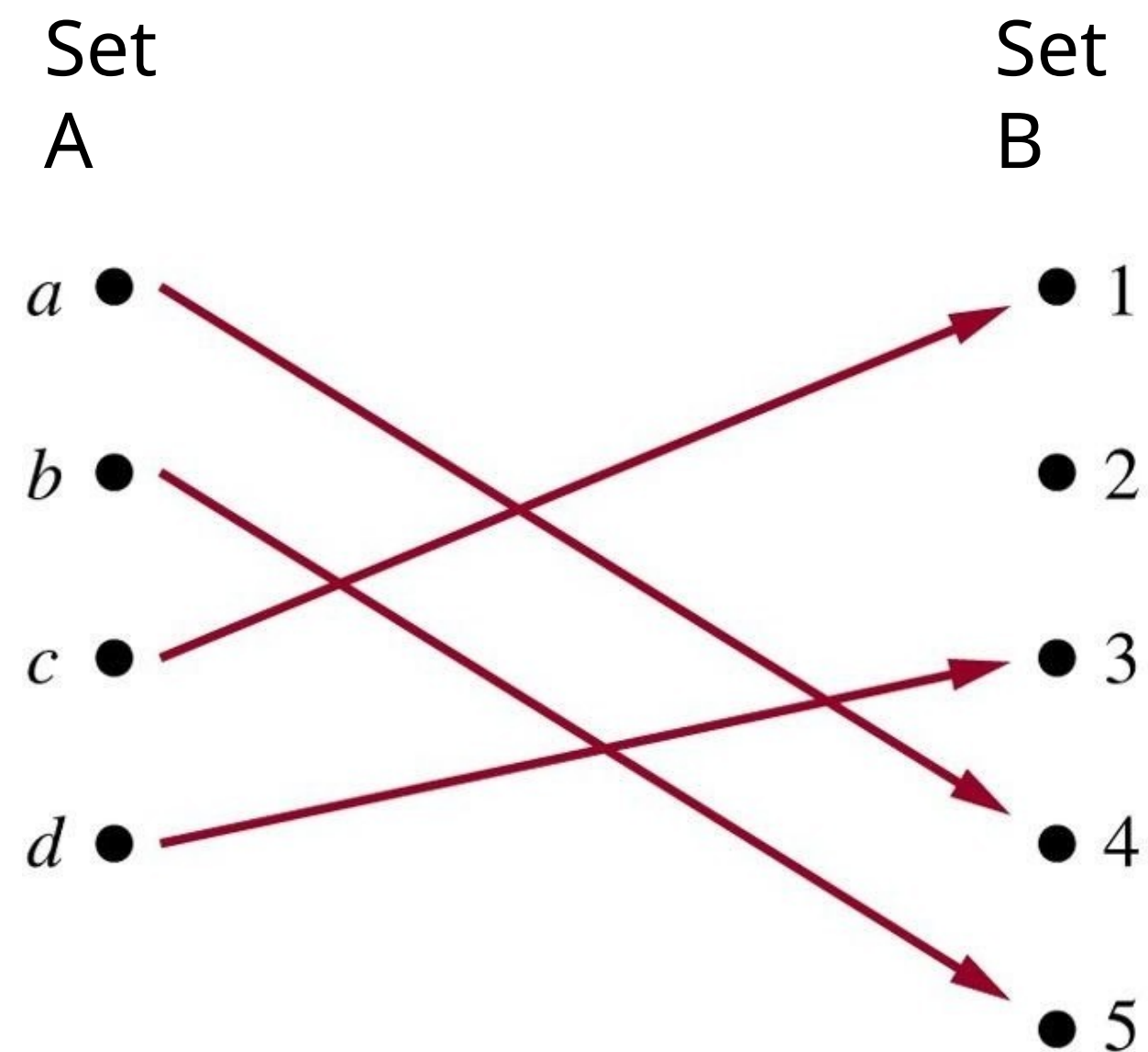
- 1) One-to-One (Injective) function
- 2) Onto (Surjective) function
- 3) One-to-One Correspondence (Bijective) function



Automata Formal Languages & Logic

Types of Functions

1) One-to-One (Injective) function - A function $f: A \rightarrow B$ is One to One if for each element of A there is a distinct element of B .

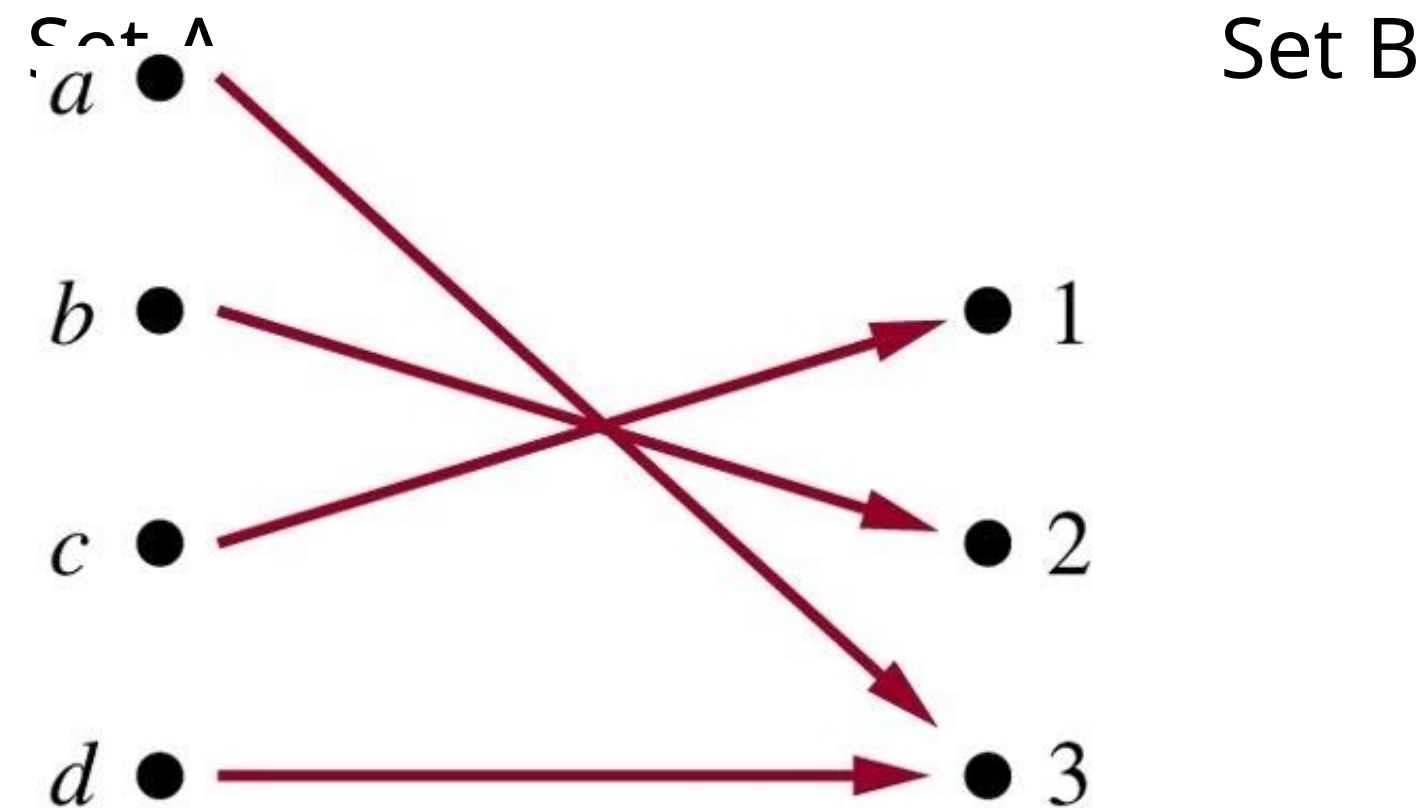


Image(a) = 4
Pre-image(4) =
 a

Automata Formal Languages & Logic

Types of Functions

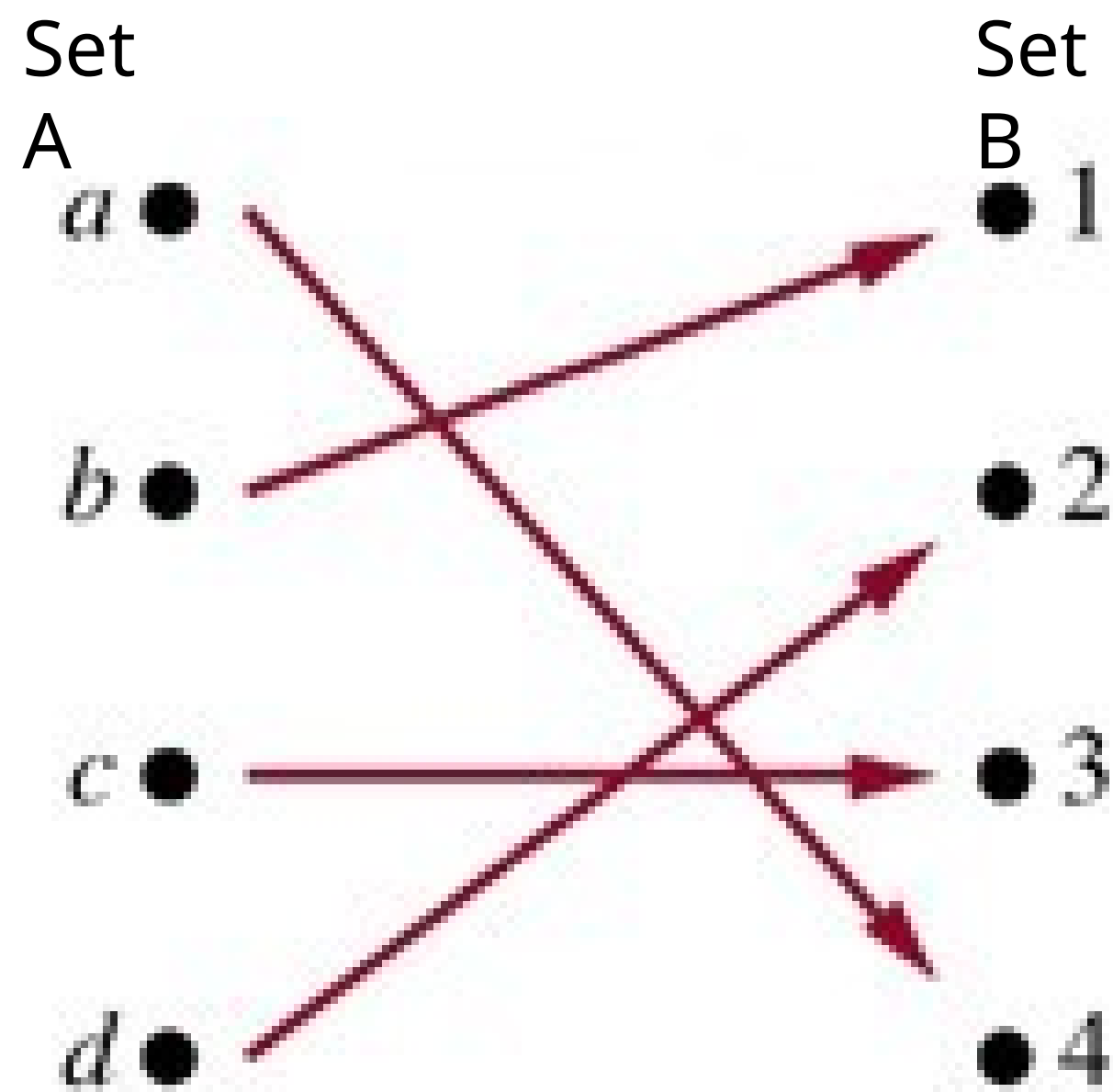
2)Onto (Surjective) function - all elements in the Range must be used(mapped)



Automata Formal Languages & Logic

Types of Functions

3) One-to-One Correspondence (Bijective) function - Both one-to-one and onto.



Automata Formal Languages & Logic **Relations**

A relation between two sets is a collection of ordered pairs (x,y) , containing one object from each set. Here x is from first set and y is from second set.

The set of the first components of each ordered pair is called the domain and the set of the second components of each ordered pair is called the range.

A function is a special relation in which each possible input maps to exactly one output value.

Example

:
 $R = \{(1,2), (2,4), (3,6)\}$

Domain is

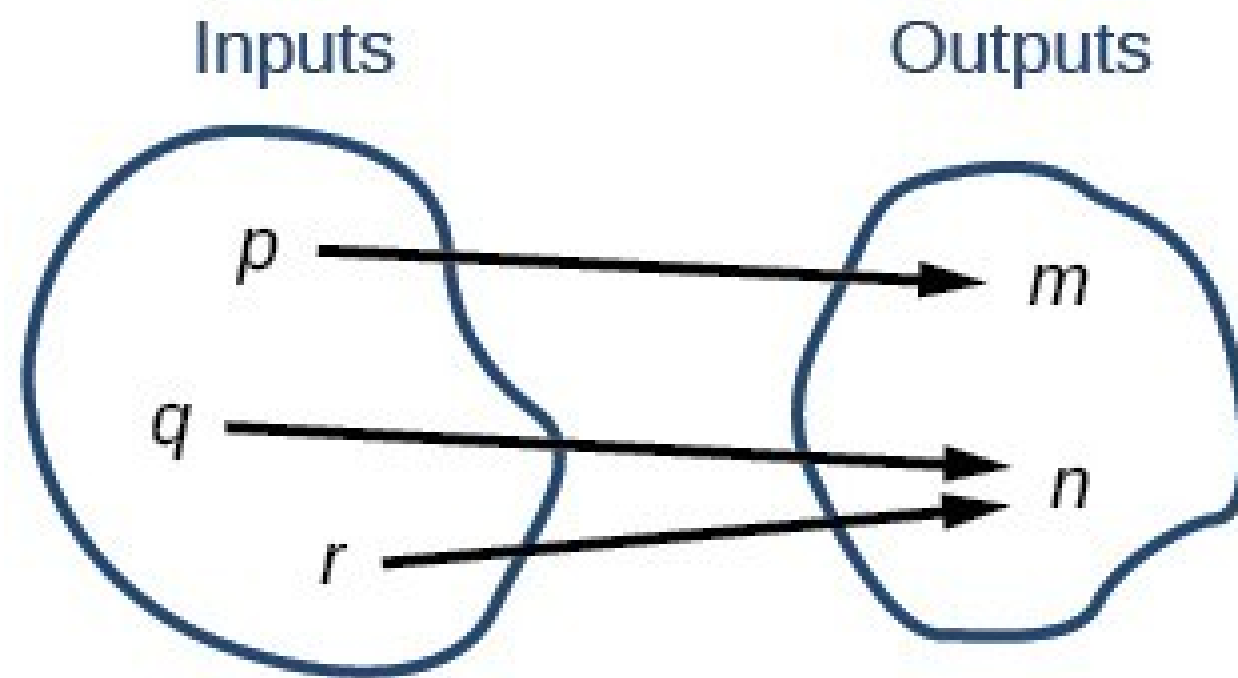
$\{1,2,3\}$

Range is $\{2,4,6\}$

Automata Formal Languages & Logic

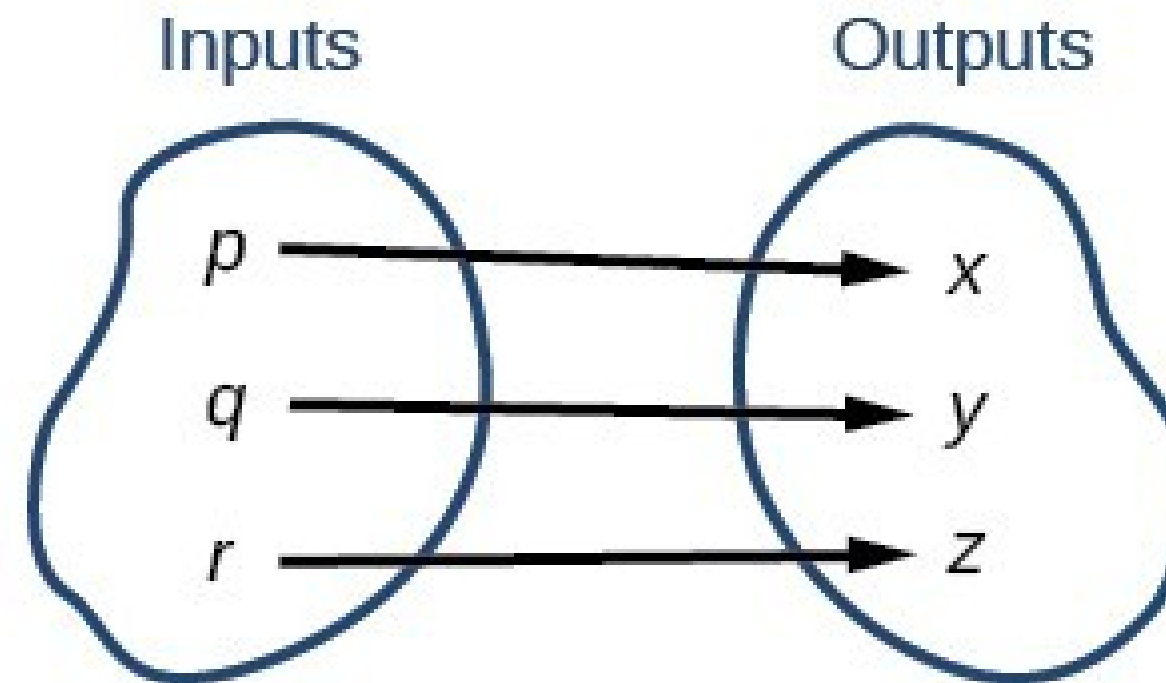
Is this Relation a function?

Relation is a Function



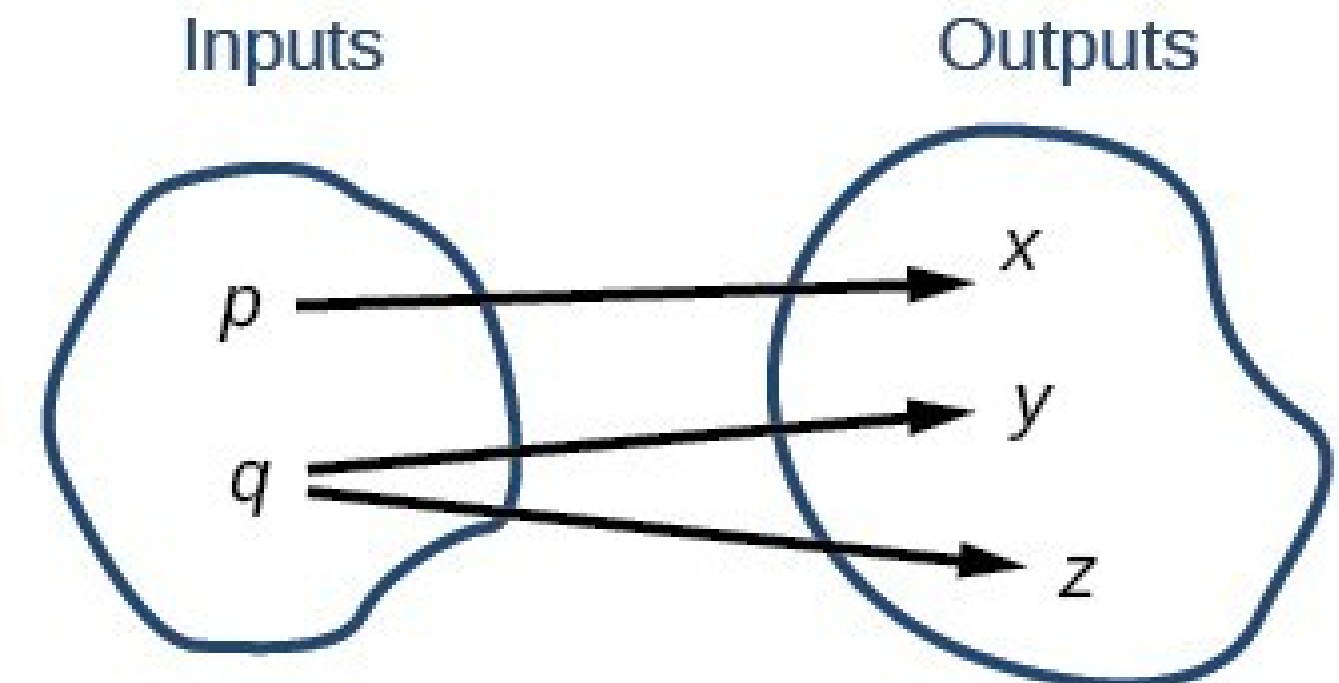
(a)

Relation is a Function



(b)

Relation is NOT a Function



(c)

Automata Formal Languages & Logic

Binary Relations

A binary relation is a relationship between two sets. Formally, binary relation over sets X and Y is a subset of the Cartesian product $X \times Y$;

Element a is related to b by R is denoted by aRb .

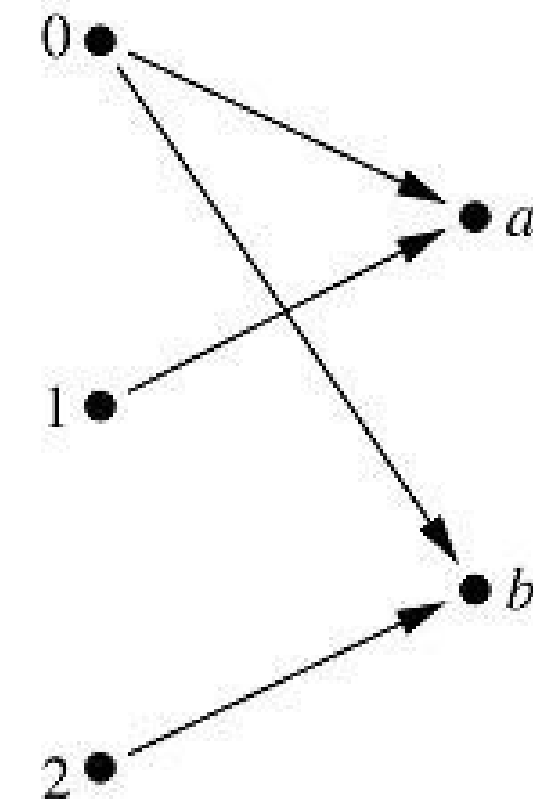
aRb denotes $(a, b) \in$

R Eg:

$A = \{0, 1, 2\}$

$B = \{a, b\}$

$A \times B = \{ (0, a), (1, a), (0, b), (1, b), (2, a), (2, b) \}$ R
 $= \{ (0, a), (1, a), (0, b), (2, b) \} \subseteq A \times B$



R	a	b
0	×	×
1	×	
2		×

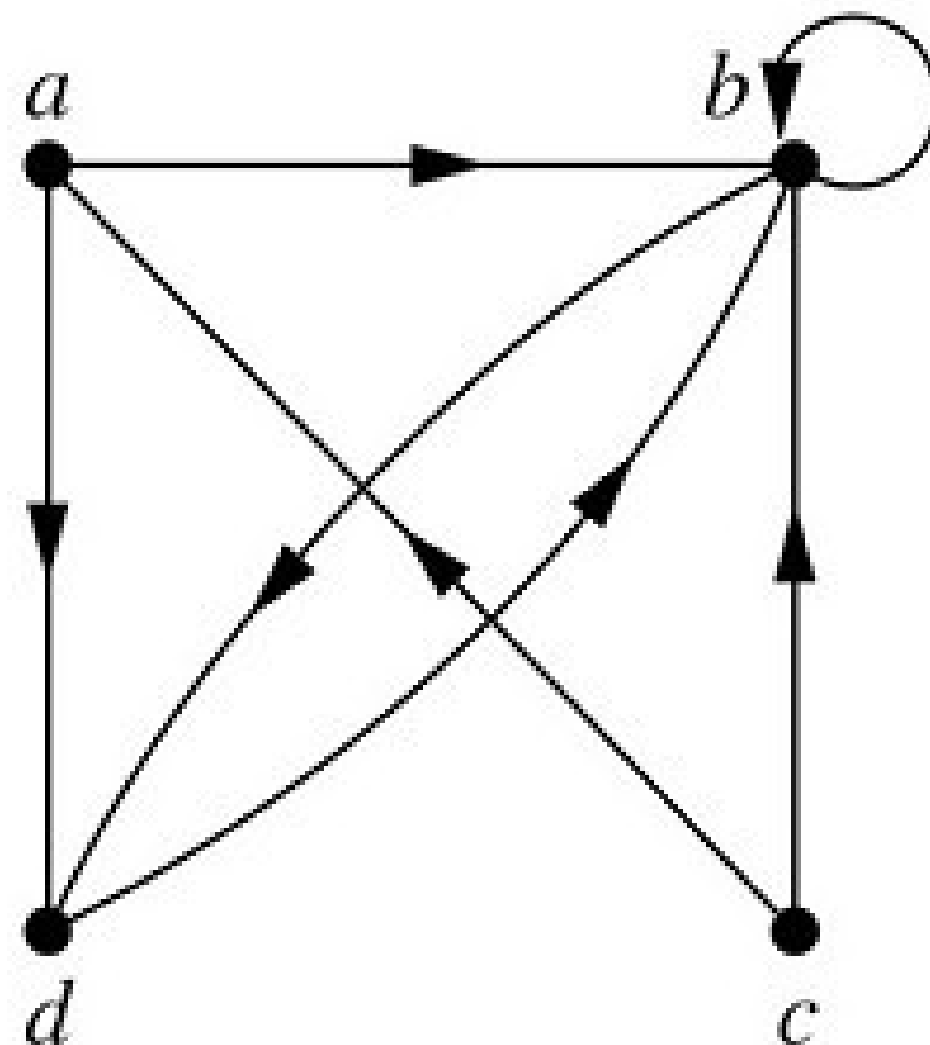
Automata Formal Languages & Logic

Representing a Relation

Representing a Relation :

- 1) Matrix form
- 2) Directed Graph (Digraph)

	a	b	c	d
a	0	1	0	1
b	0	1	0	1
c	1	1	0	0
d	0	1	0	0



Example :

Set $A = \{a, b, c, d\}$

Relation R on Set A is a relation from A to A ($\subseteq A \times A$)

$R = \{(a, d), (a, b), (b, b), (b, d), (c, b), (c, a), (d, b)\}$

Automata Formal Languages & Logic

Properties of a Relation

A relation R on a set A ($A = \{1, 2, 3, 4\}$) is

Reflexive : iff $(a,a) \in R$ for every element $a \in A$.

$R = \{(1, 1), (2, 2), (3, 3), (4, 4), (1, 2)\}$ is reflexive

Symmetric : iff $(b,a) \in R$ whenever $(a,b) \in R$, for all $a,b \in A$.

$R = \{(1, 2), (2, 1), (1, 4), (4, 1), (3, 3)\}$ symmetric

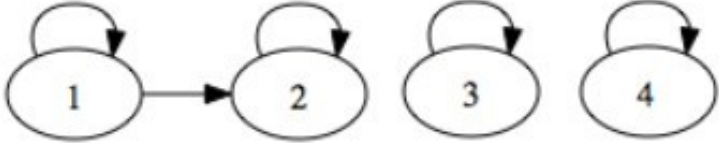
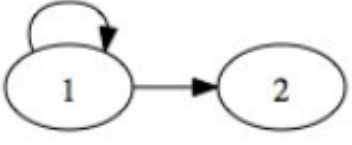
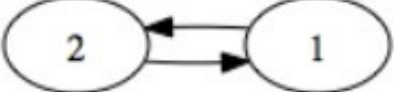
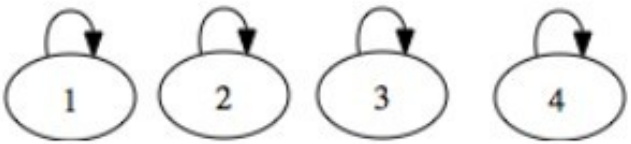
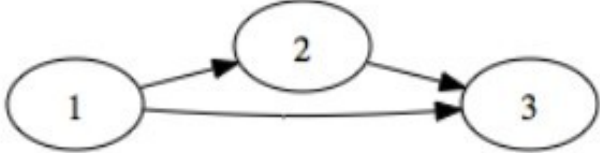
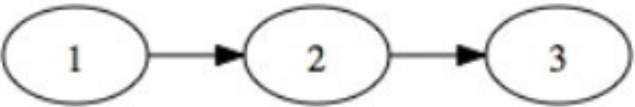
Transitive : iff $(a,c) \in R$ whenever $(a,b) \in R$ and $(b,c) \in R$, for all $a,b,c \in R$.

$R = \{(1,2), (2,1), (1,1)\}$

Automata Formal Languages & Logic

Properties of a Relation

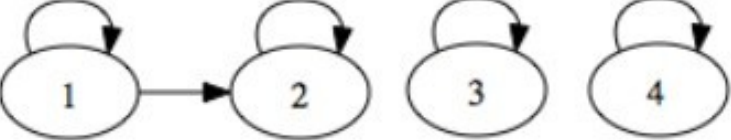
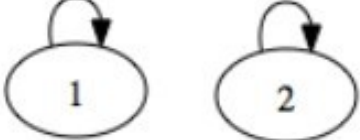
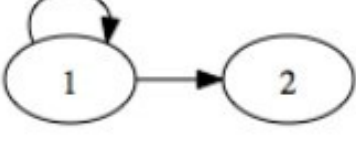
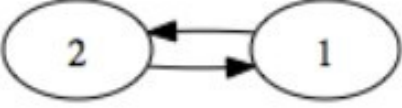
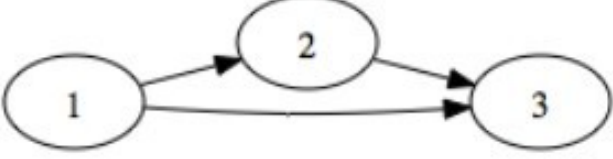
Practice

Relation	Reflexive	Symmetric	Transitive
			
			
			
			
			
			
			
			

Automata Formal Languages & Logic

Properties of a Relation

Practice

Relation	Reflexive	Symmetric	Transitive
	Y	N	Y
	Y	Y	Y
	N	N	Y
	N	Y	N
	Y	Y	Y
	N	N	Y
	N	N	N
	N	N	N

Automata Formal Languages & Logic

Equivalence Relations

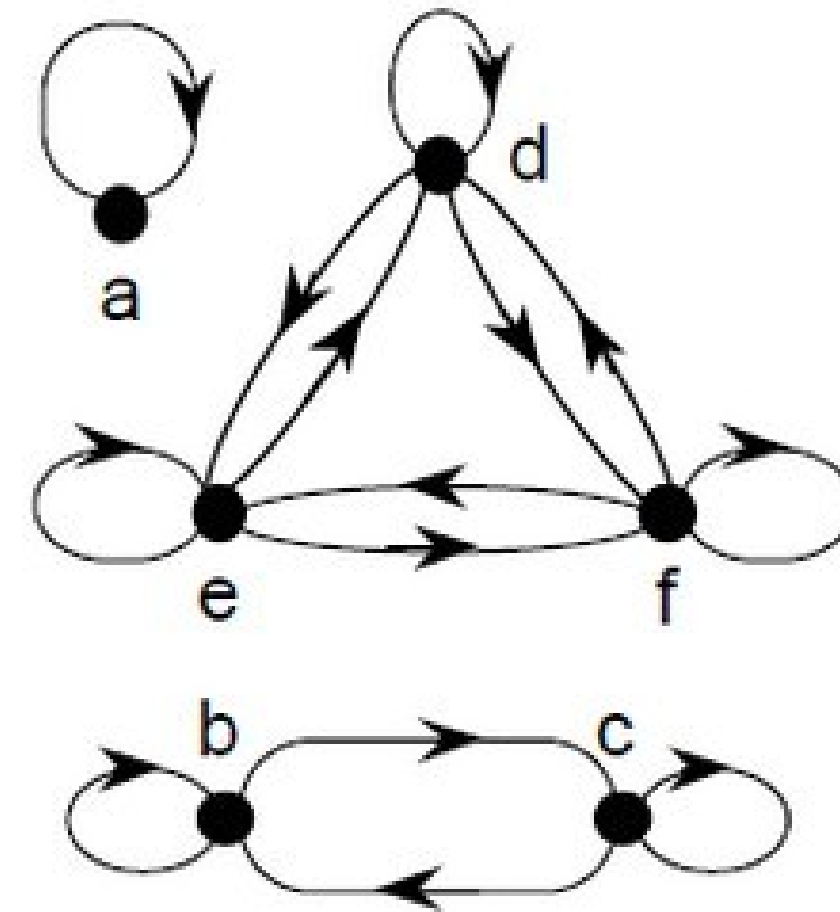
Denoted by $x \equiv y$

A relation R on a set A is called an equivalence relation if it is reflexive, symmetric, and transitive.

Example: Let $S = \{a, b, c, d, e, f\}$.

Relation R on set S be

$\{(a, a), (b, b), (b, c), (c, b), (c, c), (d, d), (d, e), (d, f), (e, d), (e, e), (e, f), (f, d), (f, e), (f, f)\}$



	a	b	c	d	e	f
a	1					
b		1	1			
c		1	1			
d				1	1	1
e				1	1	1
f				1	1	1

Automata Formal Languages & Logic

Equivalence Classes

Let R be an equivalence relation on a set A .

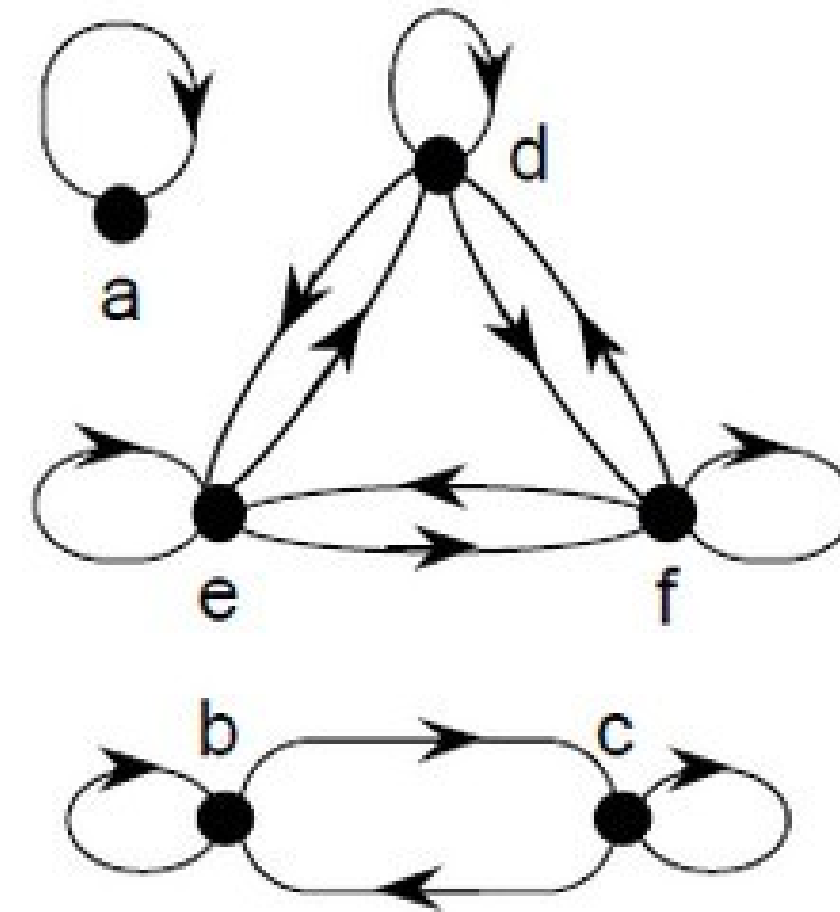
The set of all the elements that are related to an element 'a' of A is called equivalence class of 'a'.

Denoted by $[a]_R$ or $[a]$ when the relation is implicit.

Example :

$[b] = \{b, c\}$

$[d] = \{d, e, f\}$



	a	b	c	d	e	f
a	1					
b		1	1			
c		1	1			
d				1	1	1
e				1	1	1
f				1	1	1

Automata Formal Languages & Logic

Equivalence Classes

MULTIPLE CHOICE QUESTIONS:

1. Which of the following statements is true about the power set $P(S)$ of a finite set S with n elements?

- A) $|P(S)| = n$
- B) $|P(S)| = 2n$
- C) $|P(S)| = 2^n$
- D) $|P(S)| = n^2$

2. Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$. Which of the following is not a subset of $A \cup B$?

- A) $\{1, 3, 5\}$
- B) $\{2, 4, 6\}$
- C) $\{1, 2, 3, 4, 5, 6\}$
- D) $\{1, 2, 7\}$

Automata Formal Languages & Logic

Equivalence Classes

MULTIPLE CHOICE QUESTIONS:

3.
Consider the relation $R = \{(a, a), (b, b), (c, c), (a, b), (b, a)\}$ on the set $S = \{a, b, c\}$. Which property does R not satisfy?

- A) Reflexive
- B) Symmetric
- C) Transitive
- D) None of the above

4.
If $f : \{1, 2, 3\} \rightarrow \{a, b, c\}$ is a function such that $f(1) = a, f(2) = b, f(3) = a$, which type of function is f ?

- A) Injective but not surjective
- B) Surjective but not injective
- C) Both injective and surjective
- D) Neither injective nor surjective

Automata Formal Languages & Logic

Equivalence Classes

MULTIPLE CHOICE QUESTIONS:

Let $S = \{a, b, c, d\}$. Consider the relation R on S defined as $R = \{(x, y) \mid x, y \in S \text{ and } x \neq y\}$. Which of the following is true about R ?

- A) R is reflexive and symmetric but not transitive
- B) R is symmetric but neither reflexive nor transitive
- C) R is transitive but not symmetric
- D) R is an equivalence relation

2. Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be defined by

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n + 1 & \text{if } n \text{ is odd} \end{cases}$$

Which of the following statements is **correct** about the function f ?

- A) f is injective but not surjective
- B) f is surjective but not injective
- C) f is neither injective nor surjective
- D) f is both injective and surjective



THANK

YOU

Preet Kanwal

Department of Computer Science &
Engineering preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

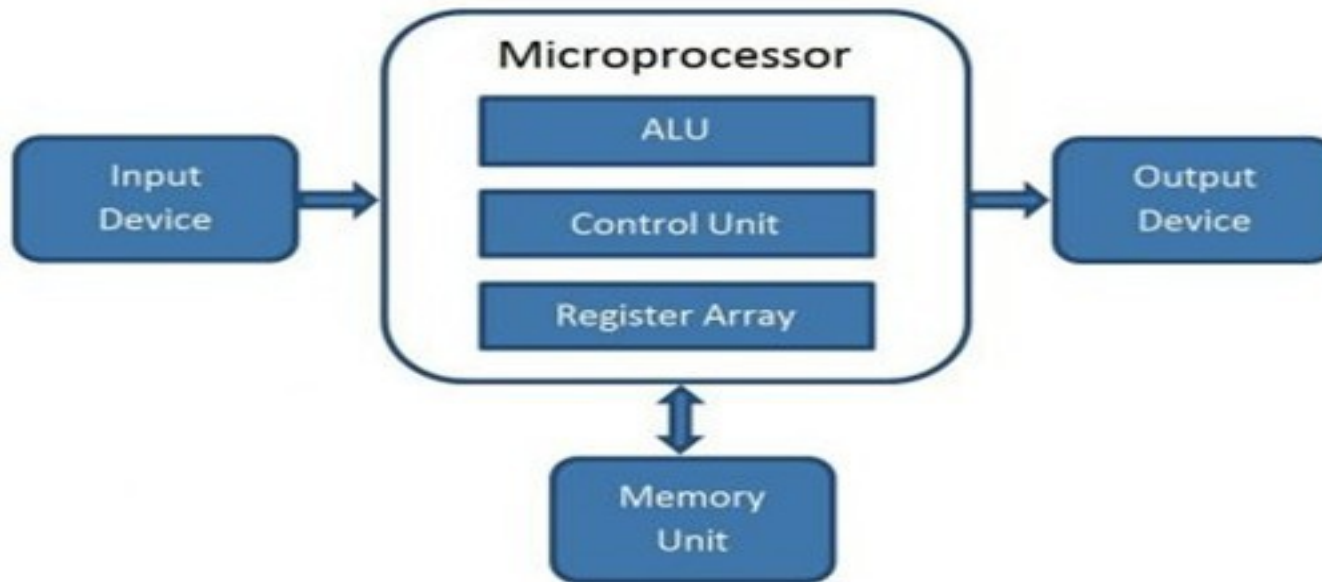
Unit 1 - Finite State Machines

Preet Kanwal

Department of Science & Engineering

Automata Formal Languages & Logic

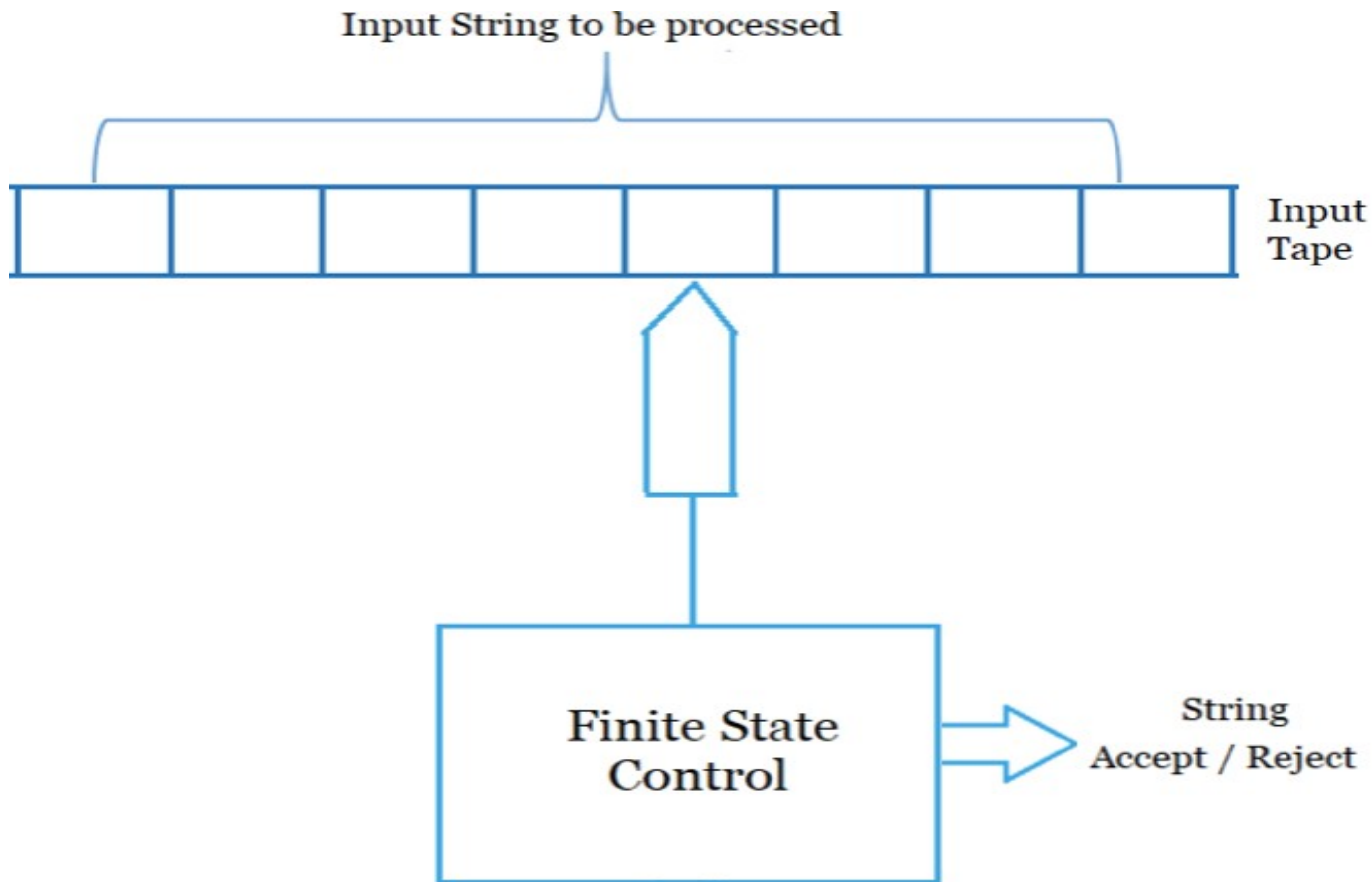
Unit 1 - Finite state Machines



Computer

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

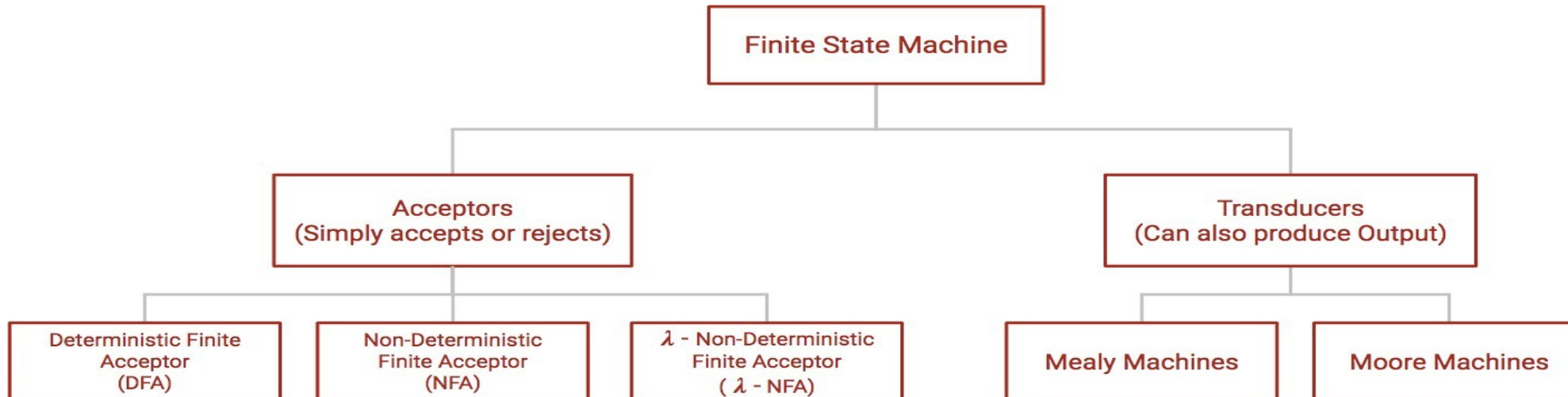


**Finite State Machines
or
Finite State Automata
or
Finite Automata**

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Classification of Finite State Machines



Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Alphabet (Σ)

String (w)

Empty String (ϵ or λ)

Length of a String ($|w|$)

Power of an Alphabet (Σ^i)

Kleens Closure or Kleens Star (Σ^*)

Kleens Plus (Σ^+)

Language (L)

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Alphabet - Finite Set of Symbols, denoted by Σ

Example :

- Binary Alphabet - $\{0, 1\}$
- English Alphabet - $\{a, b, c, \dots z\}$
- ASCII

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

String - Denoted by w is a finite sequence of symbols from the alphabet Σ .

Example :

- Empty String - $\{\epsilon\}$ or $\{\lambda\}$

Lets say, $\Sigma = \{0, 1\}^*$ then, various strings that can be generated are :

$\epsilon,$ $0,$ $1,$ $01,$ $10,$ $11,$ $00,$
 $101, 111,$ $110,$

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Length of a String - It is the number of symbols in a String w . Denoted as $|w|$.

Example :

- Empty String - $\{\epsilon\}$ or $\{\lambda\}$ has length 0
- If $w = 0100$ then, $|w| = 4$

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Power of an Alphabet - Denotes
by Σ^i the

set of strings of length i .

Example :

if $\Sigma = \{0, 1\}$ then,

$\Sigma^0 = \{\lambda\}$ set of strings of length 0

$\Sigma^1 = \{0, 1\}$ set of strings of length 1

$\Sigma^2 = \{00, 01, 10, 11\}$ set of strings of length 2

.....

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Kleene Closure(Kleene Star) -

Denoted by Σ^* is the set of strings of length ≥ 0

i.e. $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

Kleene Plus - Denoted is the
by Σ^+ set of

strings of length > 0

i.e. $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Basic Notations :

Language - Denoted by L is the set of strings obtained from Σ^* .

Language L is a subset of Σ^* ,

$$L \subseteq \Sigma^*$$

Example :

Language over $\Sigma = \{0, 1\}$
containing strings that contain equal number of
0's and 1's.

Automata Formal Languages & Logic Unit 1 - Finite state Machines

Basic Notations :

Language - Finite or Infinite Example :

Finite Language

L = set of string over $\Sigma = \{0, 1\}$ such that $|w| = 2$

$L = \{00, 01, 10, 11\}$

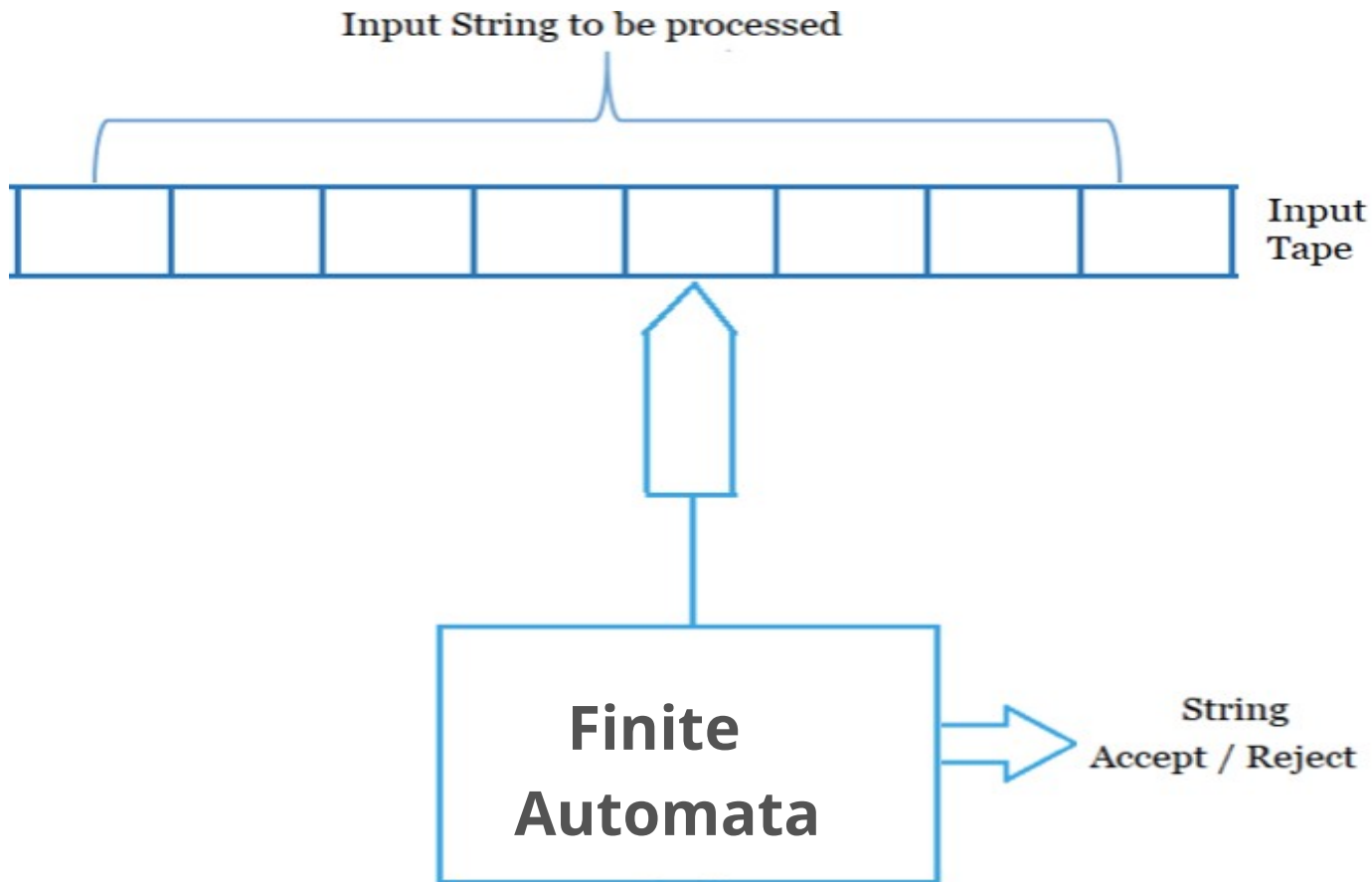
Infinite Language

L = set of string over $\Sigma = \{0, 1\}$ such that $|w| > 2$ $L = \{00, 101, 010,$

011, 100, 110, 001, 111,
1100, }

Automata Formal Languages & Logic

Unit 1 - Finite state Machines



For Infinite languages we need a finite representation which we can store in memory and is able to do the same kind of check

Automata Formal Languages & Logic **Introduction**

What is finite Automata?

A finite automaton (FA) is a **simple idealized machine used to recognize patterns within input taken from some character set (or alphabet) C** . The job of an FA is to accept or reject an input depending on whether the pattern defined by the FA occurs in the input. A finite automaton consists of: a finite set S of N states.

Automata Formal Languages & Logic

Introduction

What is formal language?

In automata theory, a formal language is a **set of strings of symbols drawn from a finite alphabet**. A formal language can be specified either by a set of rules (such as regular expressions or a context-free grammar) that generates the language, or by a formal machine that accepts (recognizes) the language.

Formal Language is defined by Grammar with 4 Tuples:

$G = (V, T, P, S)$ V = Variables, T = Terminals

P = Productions S = Start Symbol

Automata Formal Languages & Logic

Introduction

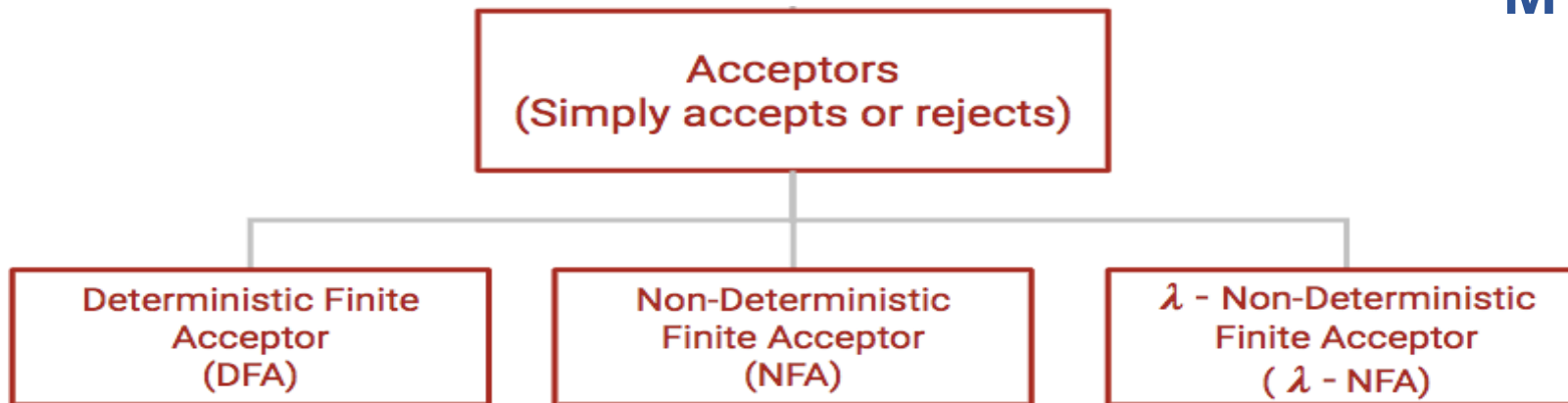
What do you mean by automata theory?

Automata theory is the study of abstract machines and automata, as well as the computational problems that can be solved using them. It is a theory in theoretical computer science. The word automata comes from the Greek word αὐτόματος, which means "self-acting, self-willed, self-moving".

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

M - Machine/Automata



5-tuple or quintuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q - Set of States (finite)

Σ - Set of Input Symbols

δ - Transition Function

q_0 - Start State

F - Set of Final States

$$F \subseteq Q$$

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Summary :

- An **automaton** is an idealized mathematical computing machine.
- A **language** is a set of strings.
- The automata we study will accept as input a string and (attempt to) determine whether that string is contained in a particular language.
- A **finite automaton** is a simple type of mathematical machine for determining whether a string is contained within some language.

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

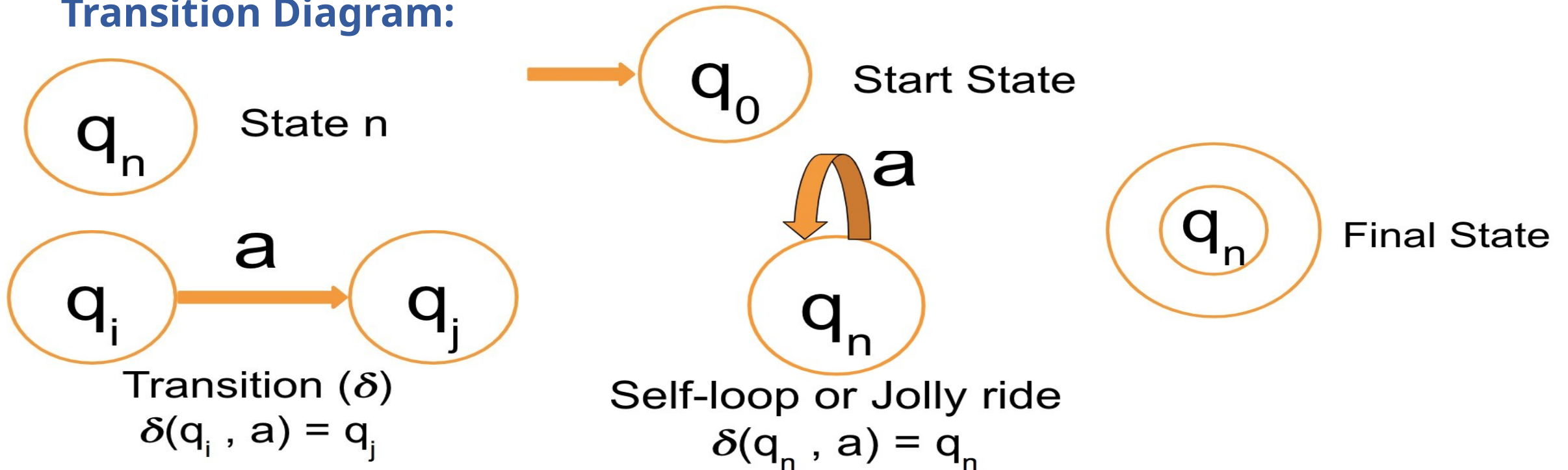
Machine M can be defined in 3 different ways :

- 1) Transition Diagram**
- 2) Transition Table**
- 3) Transition Function**

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

Each finite automaton consists of a set of *states* connected by *transitions*. Symbols used in Constructing a Finite Automata as Transition Diagram:



Automata Formal Languages & Logic

Unit 1 - Finite state Machines

MULTIPLE CHOICE QUESTIONS:

1. How many components are there in the formal definition (tuple) of a finite automaton?

- A) 3
- B) 4
- C) 5
- D) 6

2. The transition function of a finite automaton maps:

- A) Inputs and outputs
- B) Current state and input symbol
- C) Current state and output symbol
- D) Inputs and next state

Automata Formal Languages & Logic

Unit 1 - Finite state Machines

MULTIPLE CHOICE QUESTIONS:

3. Which of the following best describes the language recognized by a finite automaton?

- A) The set of input symbols
- B) The set of states
- C) The set of final states
- D) The set of all strings accepted by the automaton

4. What is the Kleene star (Σ^*) of an alphabet Σ ?

- A) The set of all strings of length 1 over Σ
- B) The set of all strings of length 0 over Σ
- C) The set of all strings of any finite length (including the empty string) over Σ
- D) The set of all possible alphabets



THANK YOU

Department of Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Approach to Construct a DFA that recognises a language L:

Step I : Enumerate Strings in the language:

- * Specify the minimal String
- * Enumerate Strings in the order of increasing length
- * Discover a Pattern

Step II : Draw a DFA skeleton of the Automata(Machine) based on the Pattern Discovered.

Step III : Complete the DFA

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Transition Function for a DFA

$$\delta : Q \times \Sigma \rightarrow Q$$

- For each state in the DFA, there must be exactly one transition defined for each symbol in Σ .
- This is the “deterministic” part of DFA
- At every point in the computation, there is exactly one choice that can be made.

M - Machine/Automata
Q - Set of States (finite)
 Σ - Set of Input Symbols
 δ - Transition
Function q_0 - Start
State
F - Set of Final States

$$F \subseteq Q$$

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Acceptance by a FA :

- A finite automaton does not accept as soon as it enters an accepting state.
- A finite automaton accepts if it ends in an accepting state.

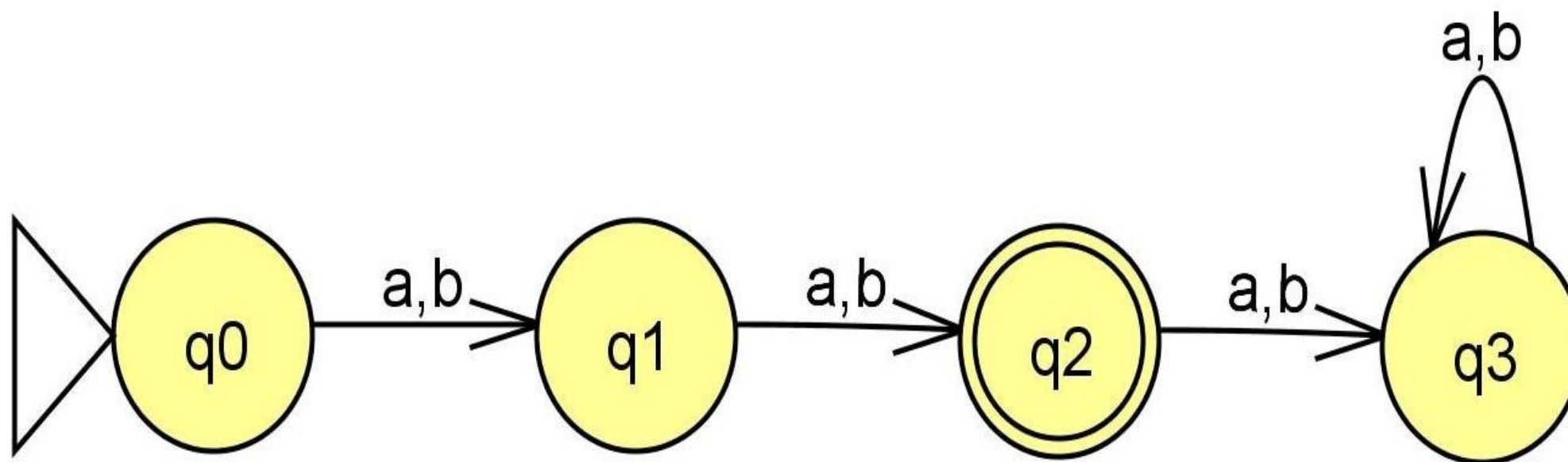
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1:

Construct DFA for the Language of strings of length 2 , over $\Sigma = \{a,b\}$.

Solution :



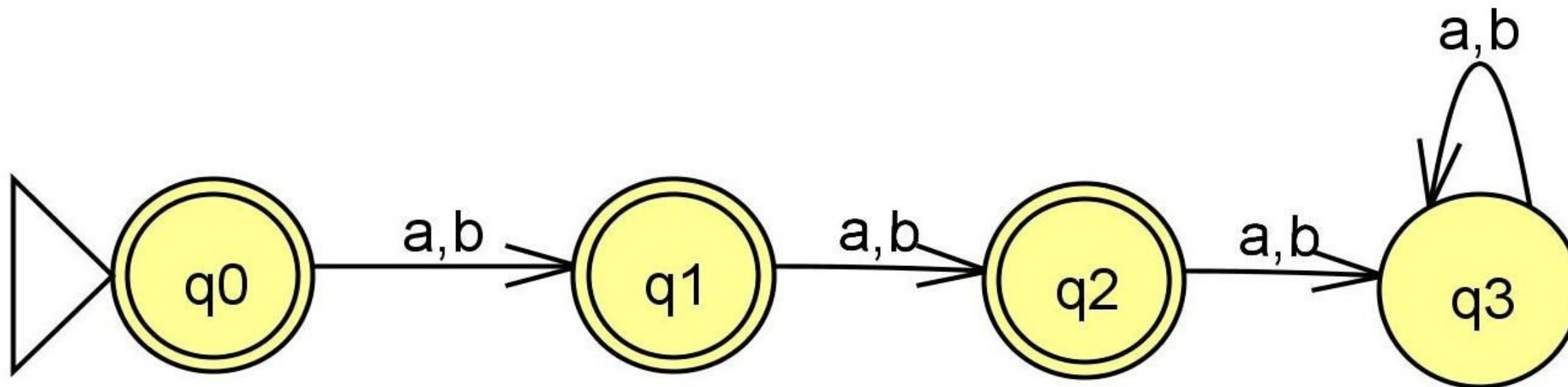
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 2:

Construct DFA for the language of strings of length ≤ 2 , over $\Sigma = \{a, b\}$.

Solution •



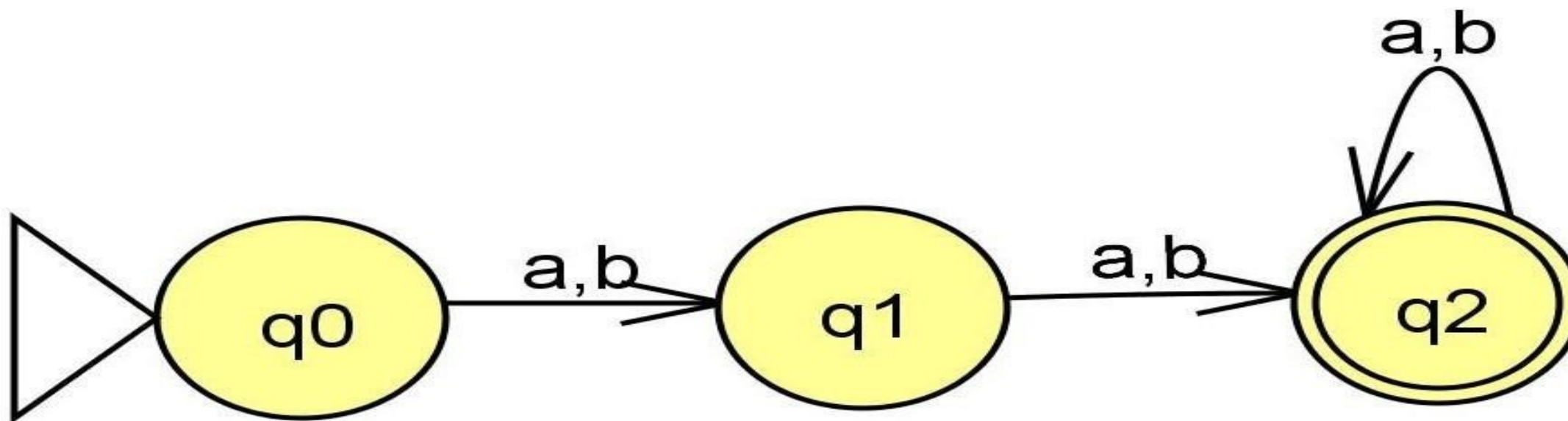
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 3:

Construct DFA for the language of strings of length ≥ 2 , over $\Sigma = \{a, b\}$.

Solution :



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

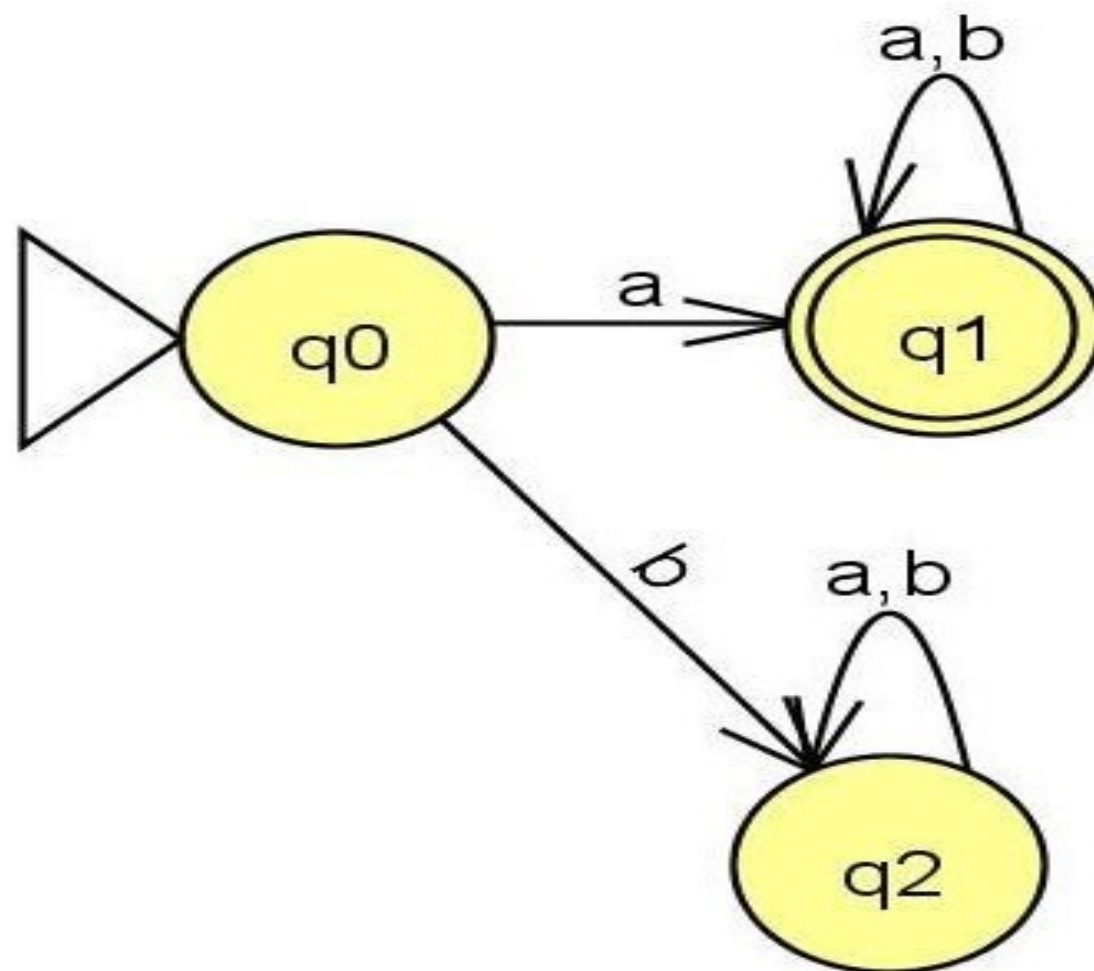
Example 4:

Construct DFA for the language of strings which start with a ,over

Σ

$=\{a,b\}$.

Solution :



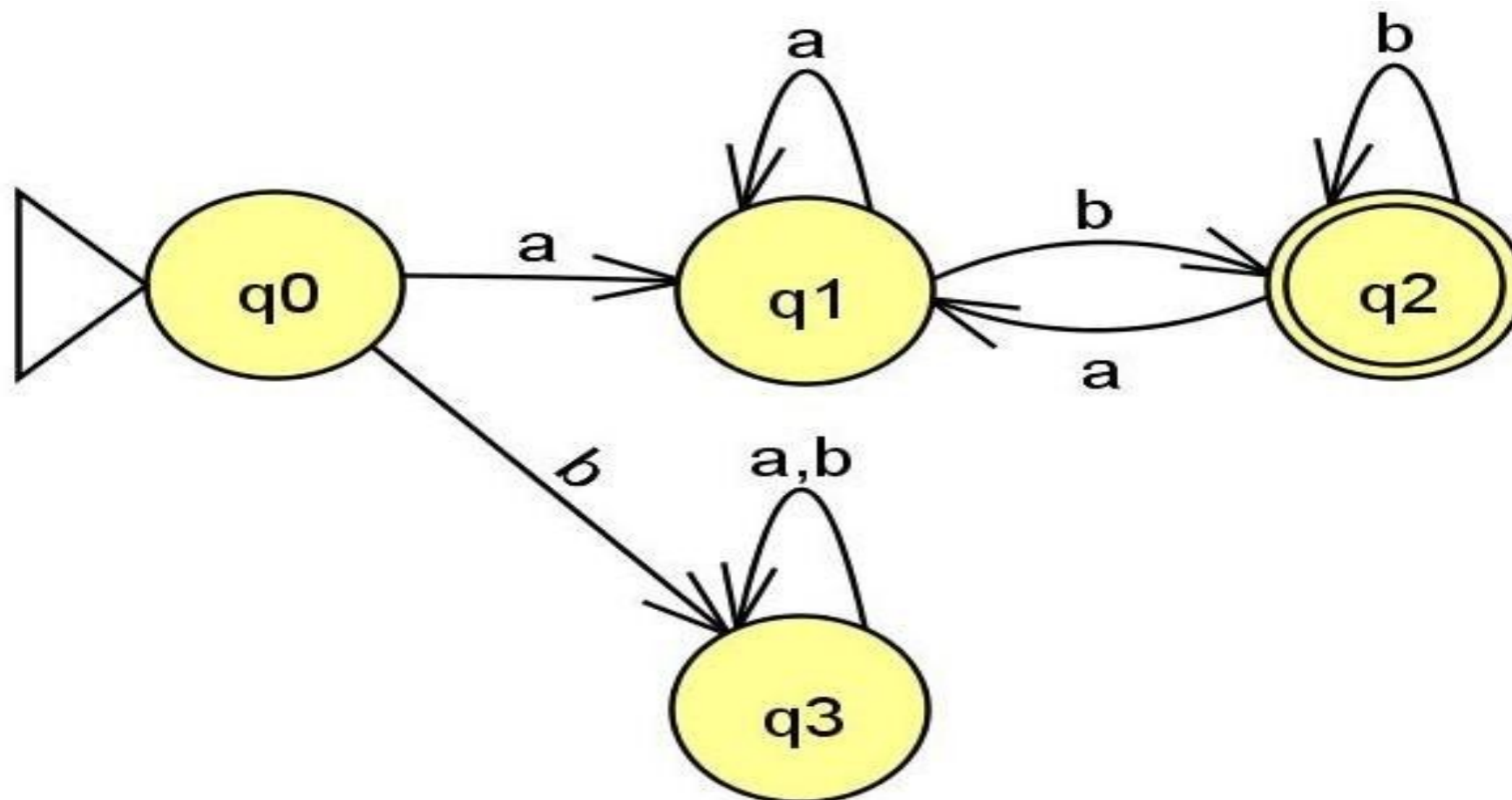
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 5:

Construct DFA for the language with strings which start with a and end in b over $\Sigma = \{a,b\}$.

Solutic



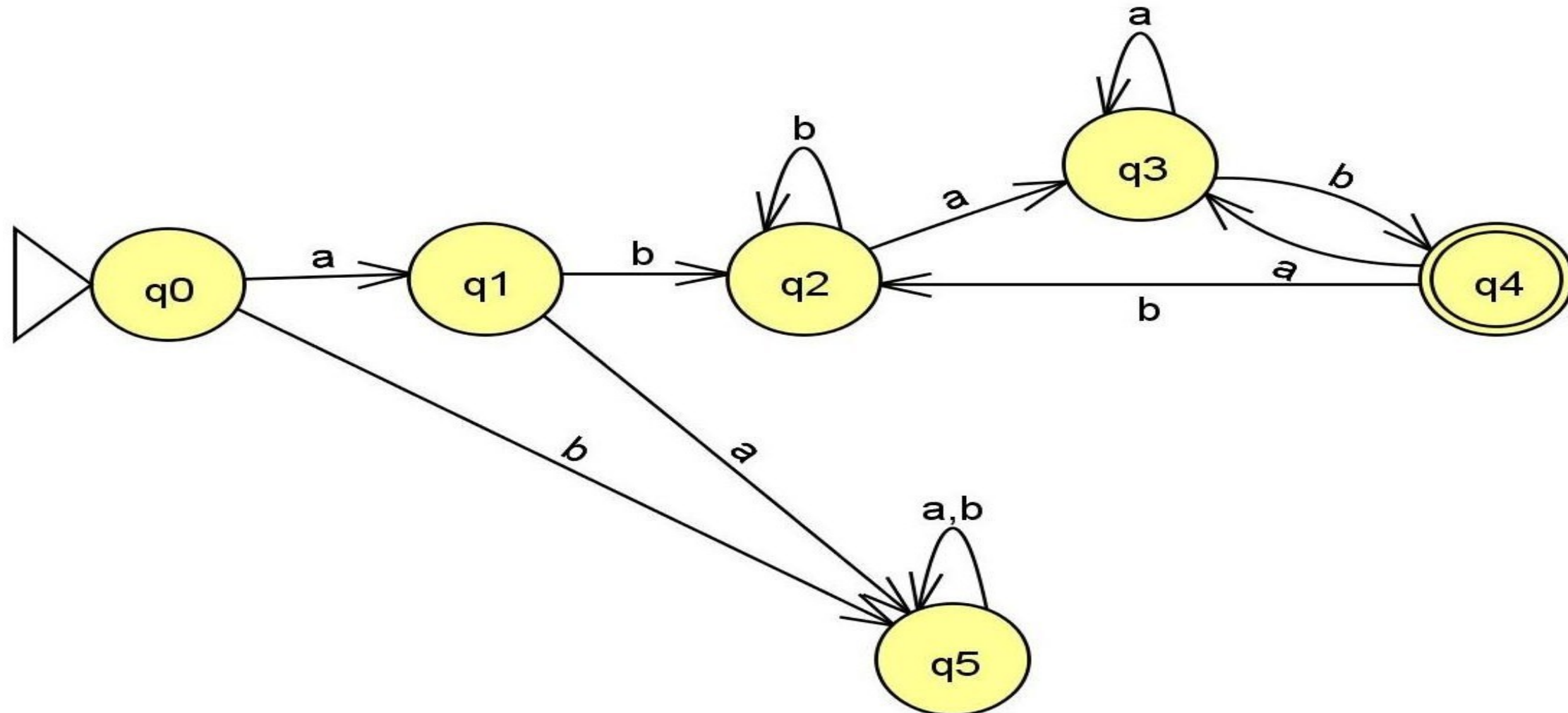
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 6:

The language with strings over $\Sigma = \{a,b\}$ where every string starts with ab and ends in ab , over $\Sigma = \{a,b\}$.

Solution:



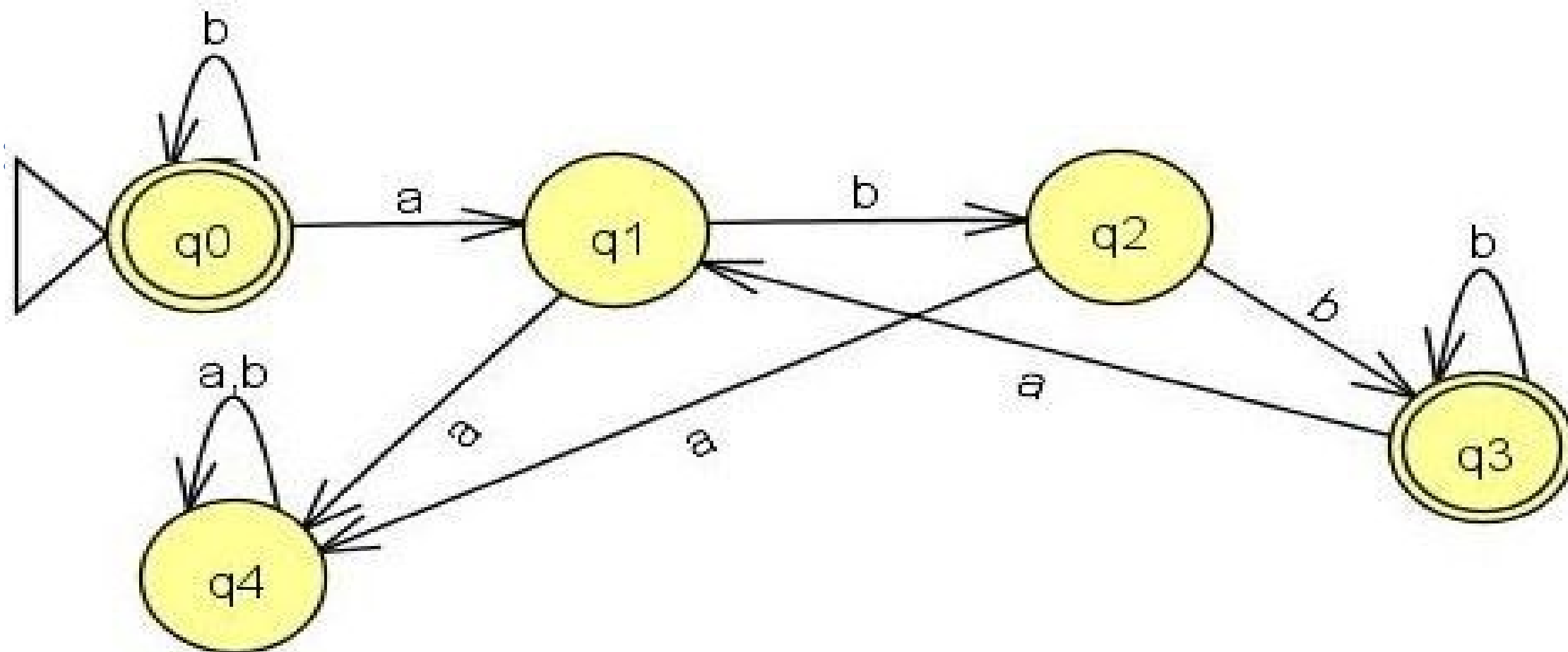
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 7:

The language consists of strings over $\Sigma = \{a, b\}$ where every a is followed by bb .

Solution :



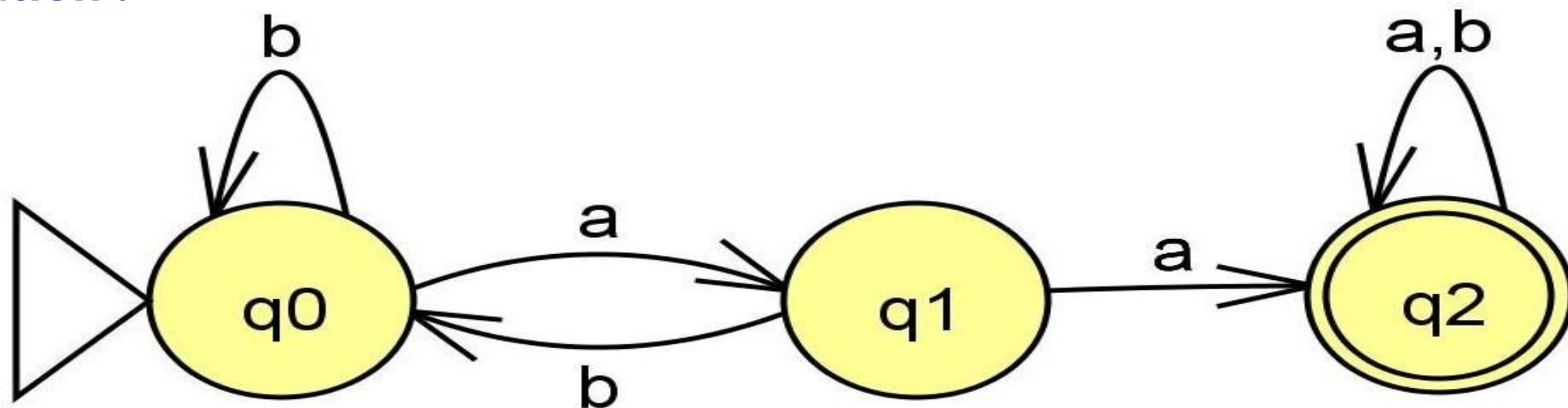
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 8:

The language of strings over $\Sigma = \{a,b\}$ where every string must contain "aa" as the substring.

Solution :



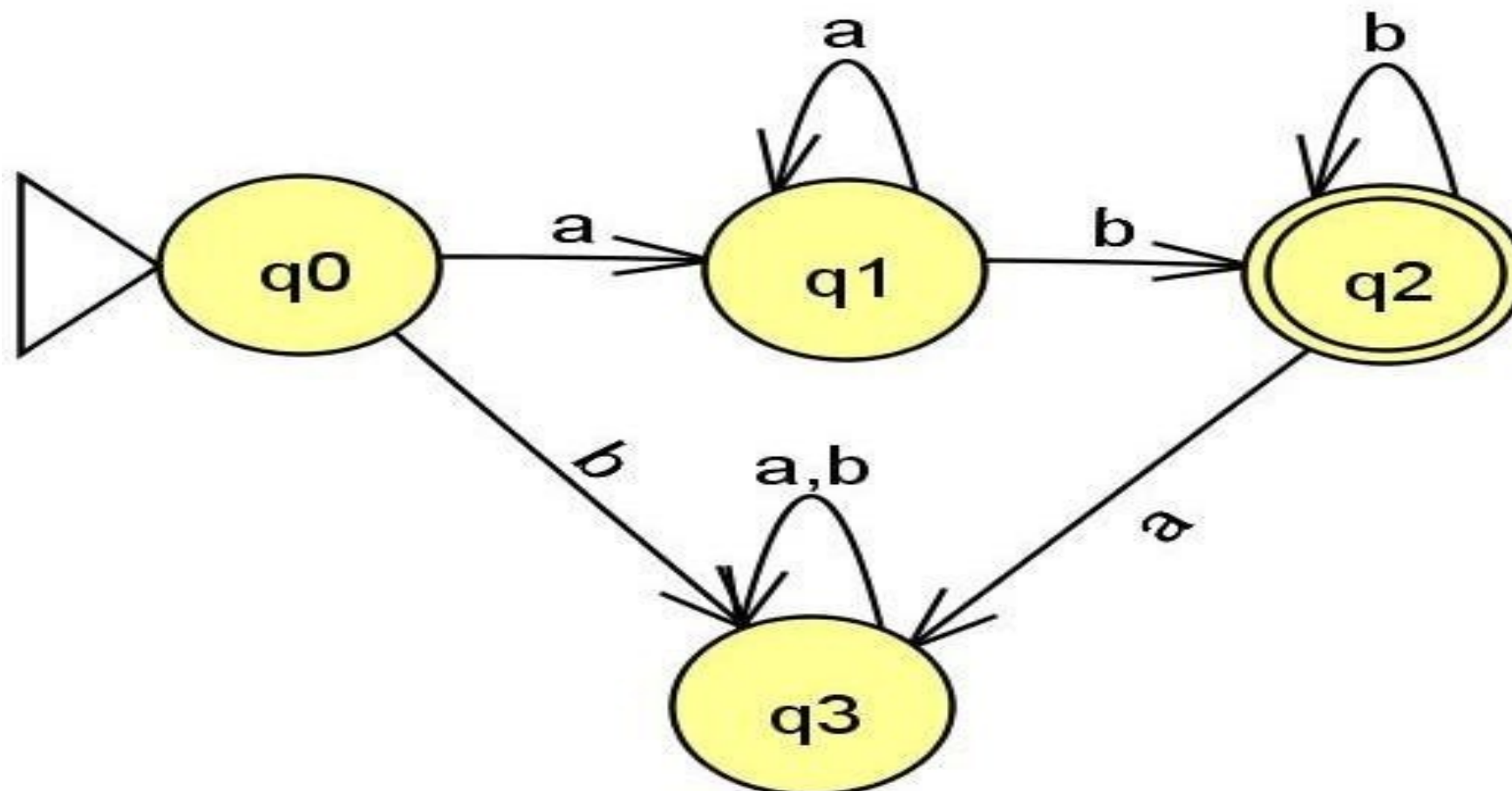
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 9:

Construct DFA for the the language of strings over Σ of the
 $\Sigma = \{a,b\}$ form $a^n b^m \mid n,m \geq 1$.

Solution :



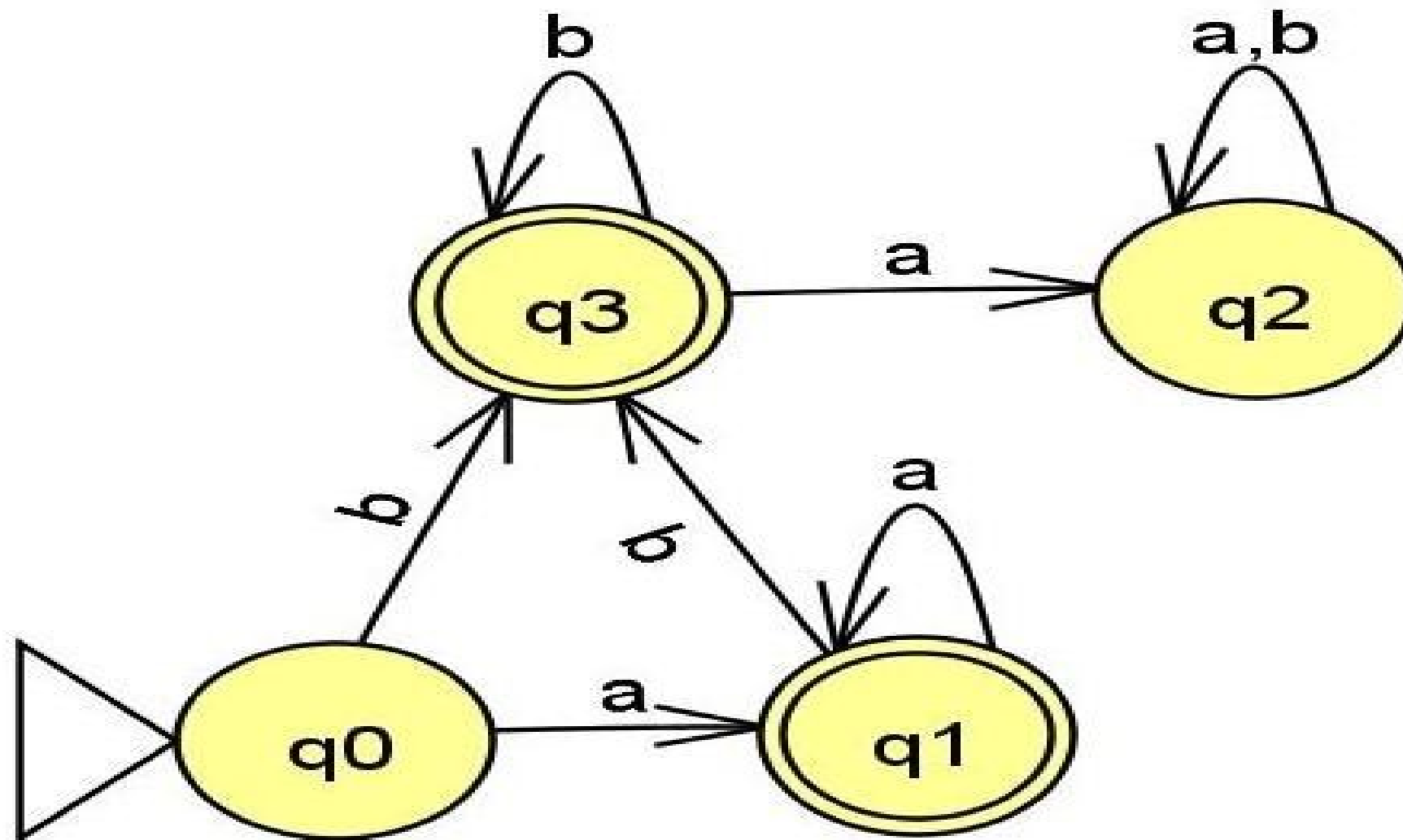
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 10:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ of the form $a^n b^m \mid n, m \geq 0$

Solution :



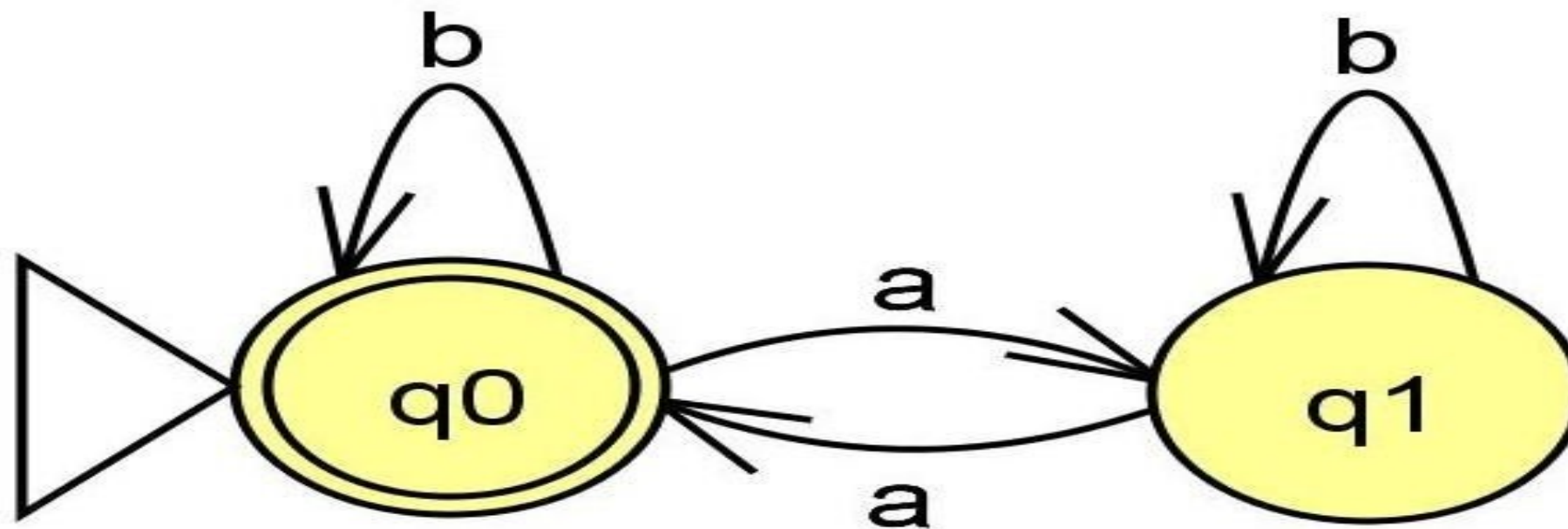
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 11:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 2 = 0$.

Solution :



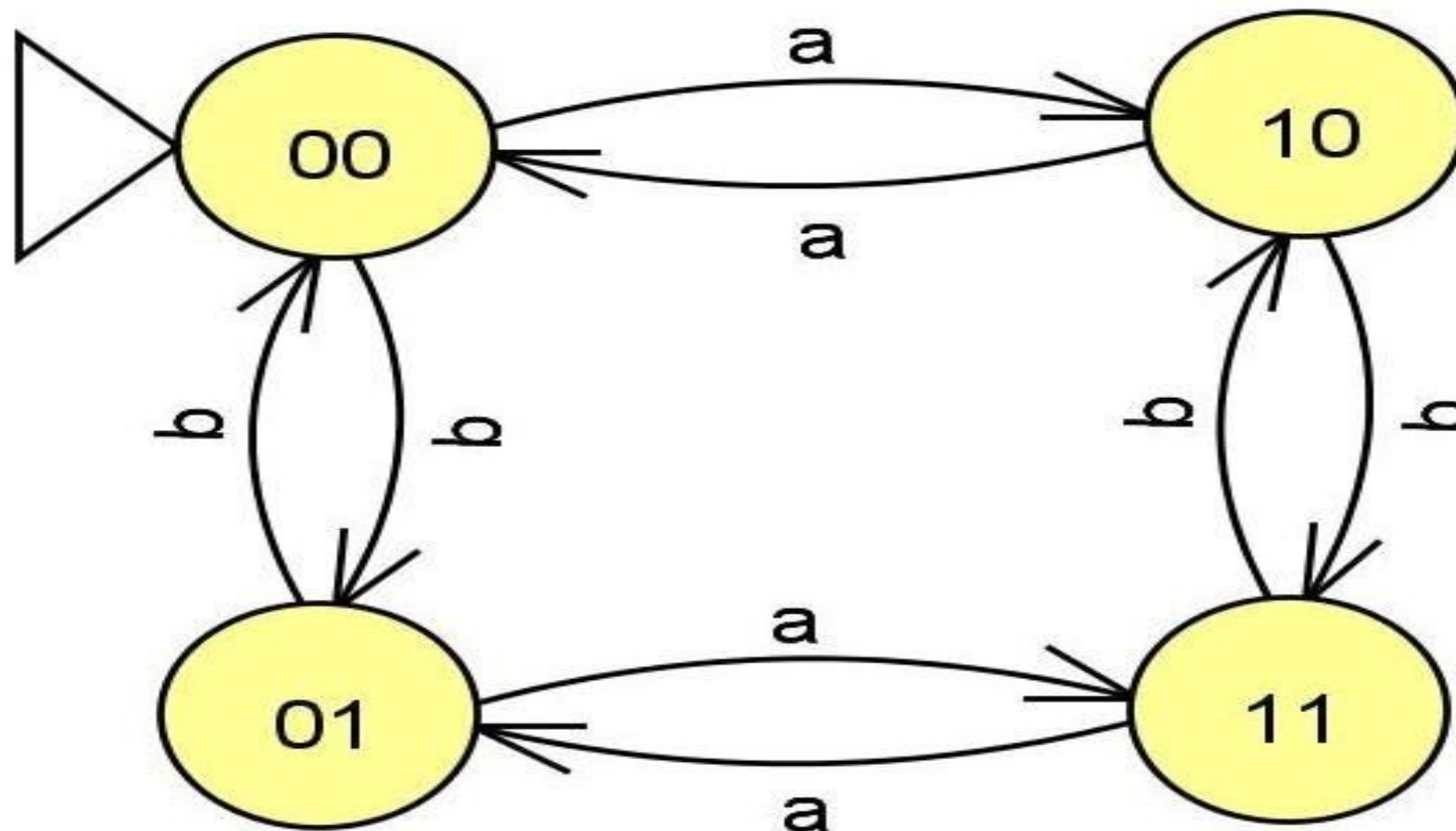
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 12:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 2 = 0$ and $n_b(w) \bmod 2 = 0$

Solution :



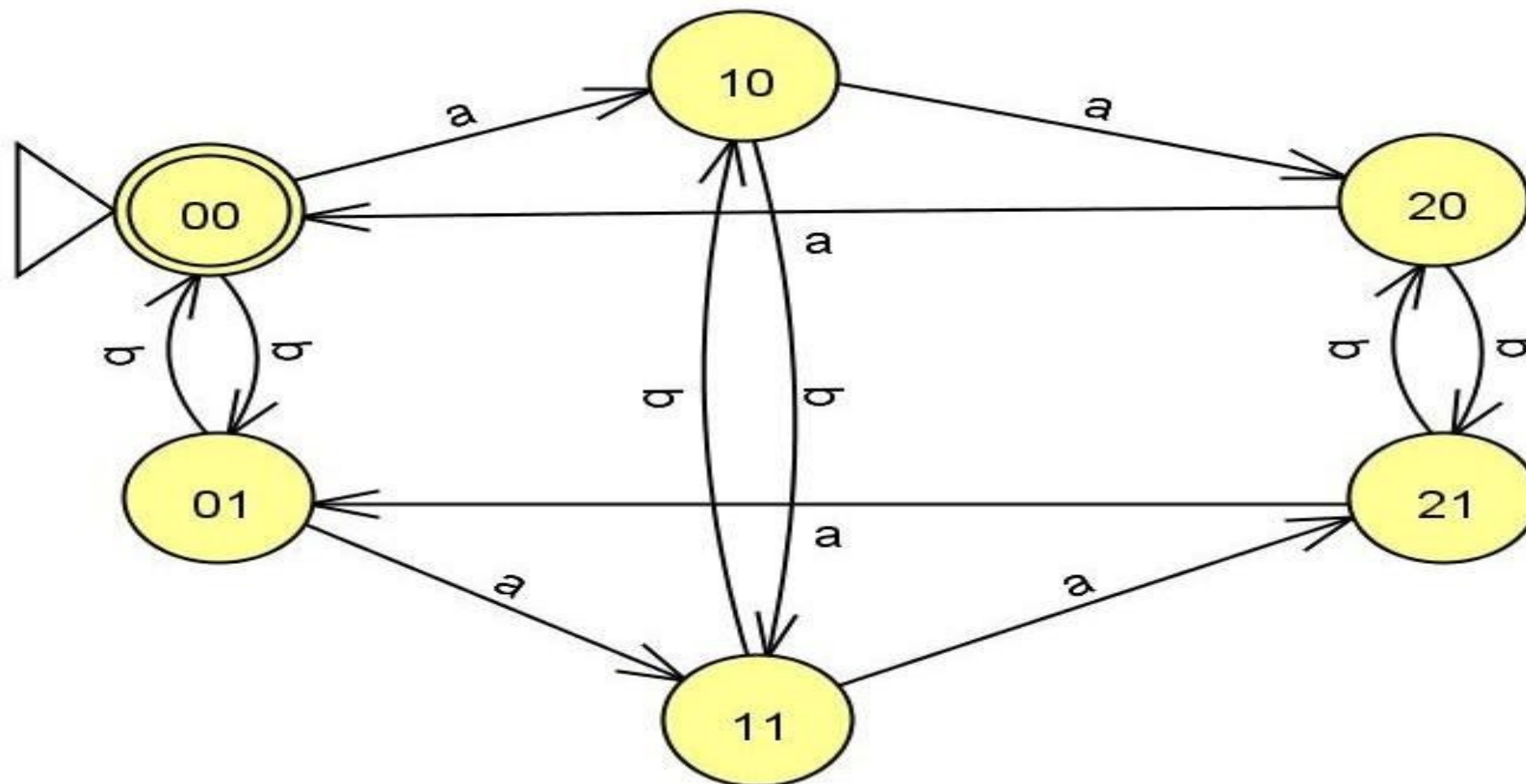
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 13:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 0$ and $n_b(w) \bmod 2 = 0$.

Solution :



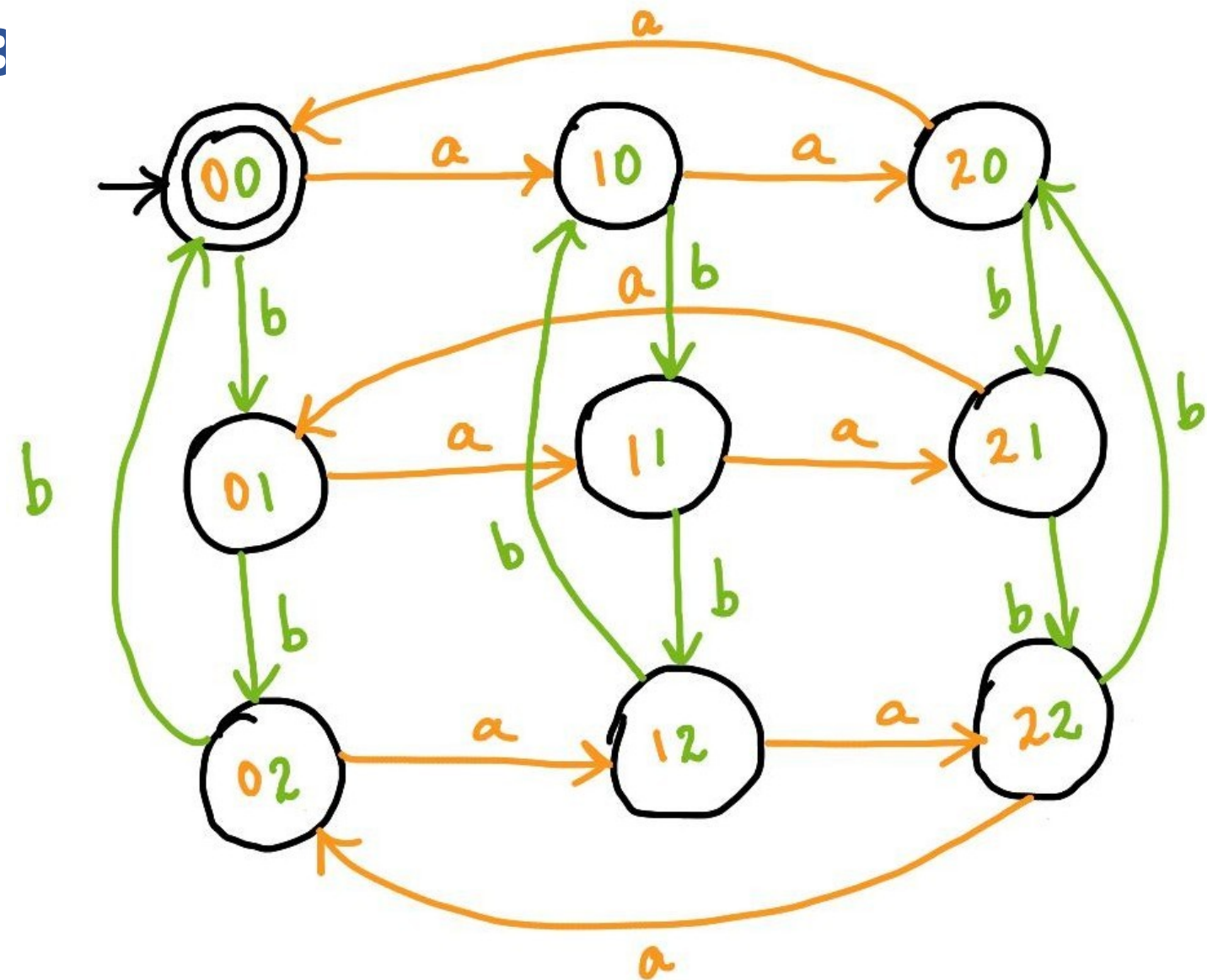
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 14:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 0$ and $n_b(w) \bmod 3 = 0$

Solution :



Automata Formal Languages and Logic

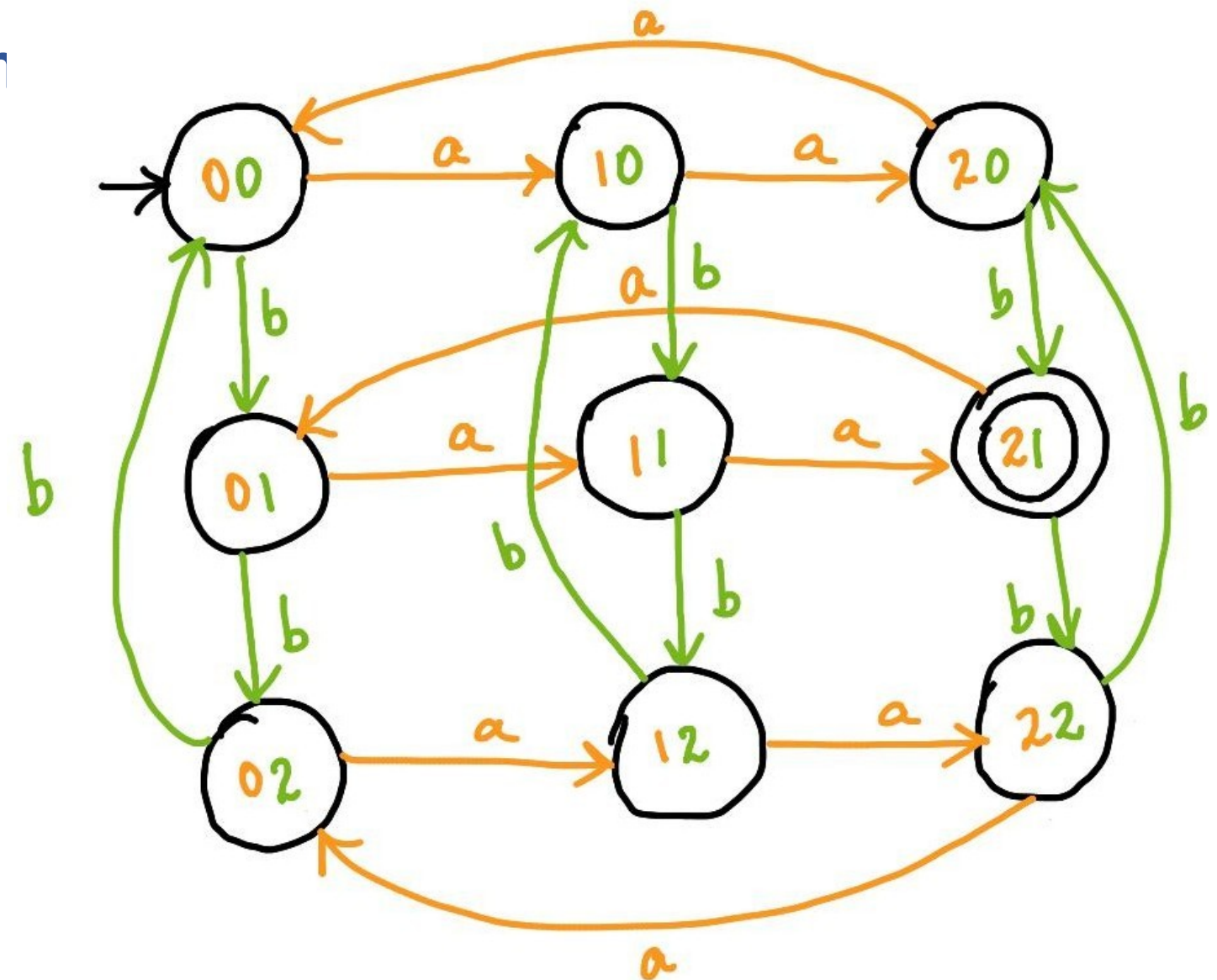
Unit 1 - Deterministic Finite Acceptor/Automata

Example 15:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$

where, $n_a(w) \bmod 3 = 2$ and $n_b(w) \bmod 3 = 2$

Solution :



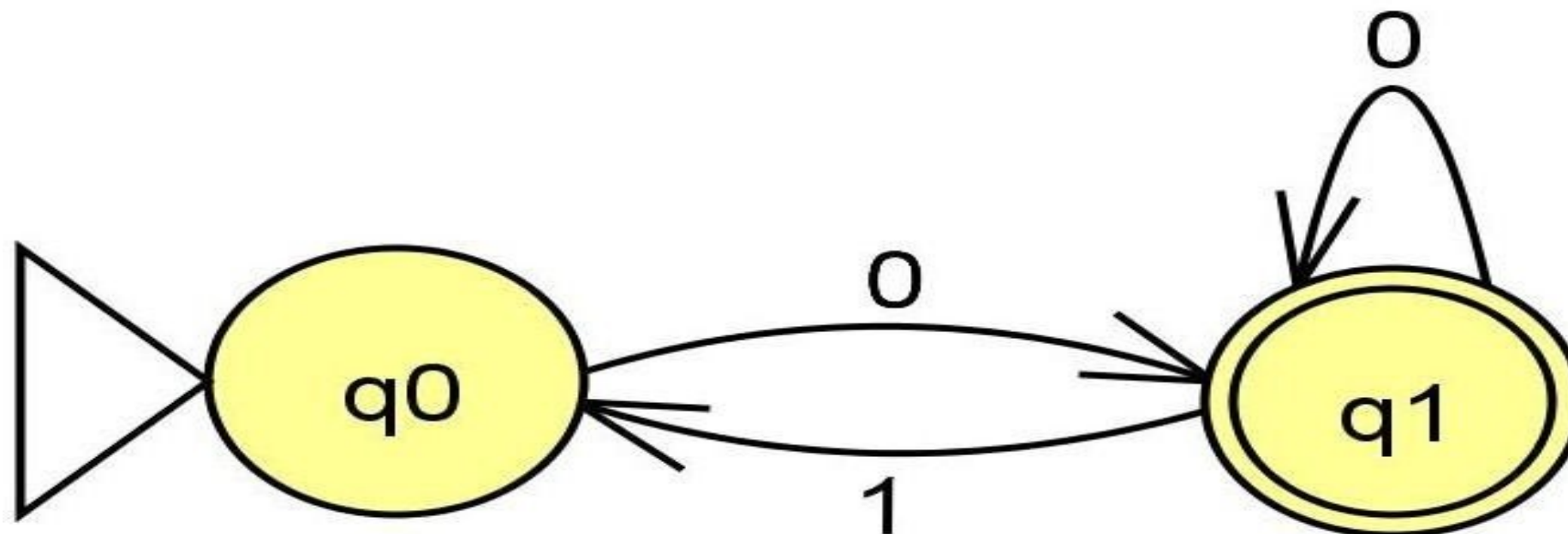
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 16:

Construct DFA for binary number divisible by 2 ($w \bmod 2 = 0$).

Solution :



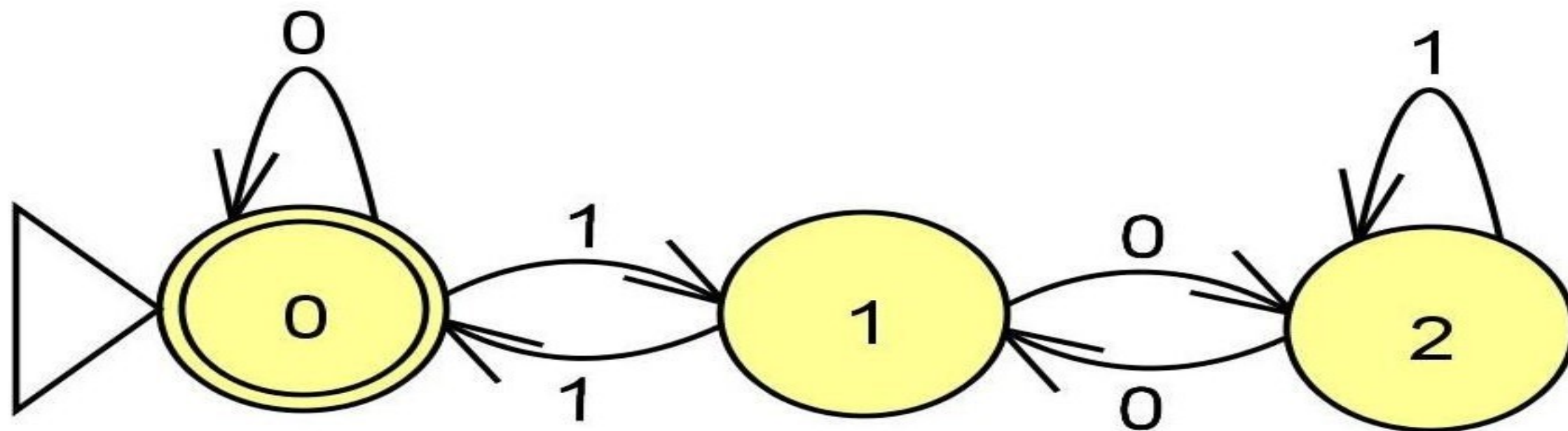
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 17:

Construct DFA for binary number divisible by 3 ($w \bmod 3 = 0$).

Solution :



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

MULTIPLE CHOICE QUESTIONS:

For the language of strings over $\Sigma = \{a, b\}$ that start with 'a' and end with 'b', which of the following strings is accepted by the DFA for this language?

- A) abb
- B) baa
- C) bab
- D) aba

In a DFA, what is the minimum number of states required to recognize all strings of length exactly 2 over $\Sigma = \{a, b\}$?

- A) 2
- B) 3
- C) 4
- D) 5



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

MULTIPLE CHOICE QUESTIONS:

Let $L = \{x \in \{a, b\}^* \mid \text{the number of } a\text{'s in } x \text{ is divisible by 2 but not by 3}\}$. What is the minimum number of states required in a DFA that accepts L ?

- A) 2
- B) 3
- C) 6
- D) 5

4. Which of the following statements about the transition function δ in a DFA is true?

- A) δ is not defined for every state and input symbol
- B) δ can map a state and input symbol to multiple next states
- C) δ maps each state and input symbol to exactly one next state
- D) δ is only defined for the start state

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

MULTIPLE CHOICE QUESTIONS:

Consider the regular expression $(a + b)^*b(a + b)$ over the alphabet $\{a, b\}$. What is the minimum number of states required in a DFA that accepts the language defined by this regular expression?

- A) 2
- B) 3
- C) 4
- D) 5



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Prof. Divyaprabha K N

Department of Computer Science & Engineering

ACK: Prof. Kavitha K N

Automata Formal Languages & Logic

Unit 1- Minimization of DFA

Divyaprabha K N

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

What is minimization of DFA?

In automata theory (a branch of theoretical computer science), DFA minimization is the task of transforming a given deterministic finite automaton (DFA) into an equivalent DFA that has a minimum number of states. Here, two DFAs are called equivalent if they recognize the same regular language .(definition from wikipedia)

Unit 1 - Deterministic Finite Acceptor/Automata

Why one must reduce the number of states in DFA?

When constructing NFA from regular expression and converting the resultant NFA to DFA using subset construction method, the resulting DFA may not always be the smallest one with less number of states accepting the same class of languages. It is always desirable to minimize the automata where the implementation of the minimized DFA will be more efficient in terms of space and speed of execution.

Note: The minimal equivalent DFA is unique up to naming of the states

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Algorithm to Minimize/reduce the number of states in DFA

Table filling Algorithm:

Let the DFA be $D=(Q, \Sigma, \delta, q_0, F)$,

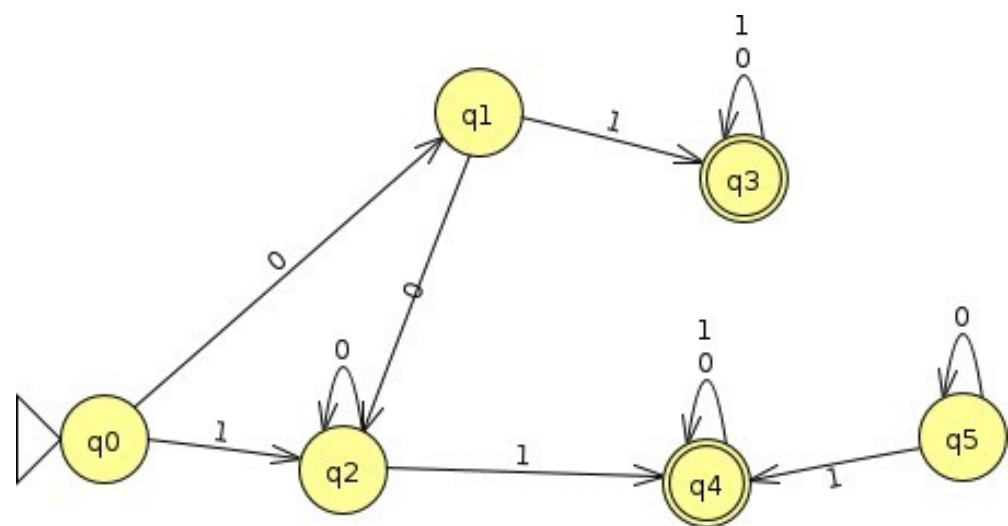
The table filling algorithm for minimizing the given DFA is as follows:

- 1) Construct a table (triangular matrix table) of pairs (x,y) where x and y are the states in DFA, $x,y \in Q$, initially all the entries in the table are unmarked
- 2) Mark the pair (x,y) if $x \in F$ and $y \notin F$ and the other way around. This marked pairs are called distinguishable pairs.
- 3) Repeat the following step up till you make an entire pass of the table and no new pair gets marked:
If (x,y) is unmarked and there exists a symbol ' $a \in \Sigma$ ' such that a pair $\{\delta(x,a), \delta(y,a)\}$ is marked then mark the pair (x,y)
- 4) After completion, $x=y$, if and only if the pair (x,y) is not marked and that pair can be merged as considered as a single state. And this pair is called indistinguishable pair

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:



Transition table:

	0	1
q0	q1	q2
q1	q2	q3
q2	q2	q4
q3	q3	q3
q4	q4	q4

To minimize the above DFA,
 Identify the state which is not reachable from the start state and remove that state. In the above DFA state q5 is not reachable from start state and should be removed. Then apply table filling algorithm for the DFA resultant DFA(DFA after removing the state not reachable from the start state).

Construct a table of pairs as shown below:

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:

While constructing the table , start the horizontal cells from state 0 to last but one state , in this example you can start form state q0 and end with state q3, Start the vertical cells from second state and end with last state , in this example start with state q1 and end with state q5. you can follow any order but it is advisable to go in increasing order for easier understanding.

Step 1:

q1				
q2				
*q3				
*q4				
	q0	q1	q2	* q3

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:

Table after 1st iteration:

Mark all distinguishable pairs: i.e, Mark all final and non final states(step 2:) since q0,q1,q2 are non final states and q3 and q4 are final states we can quickly enter 'X' in 6 cells.

q1				
q2				
*q3	X	X	X	
*q4	X	X	X	
	q0	q1	q2	* q3

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:

Next apply 3rd rule and try and see whether the remaining cells can be marked,

$\{q_0, q_1\}$:

$$\{\delta(q_0, 0), \delta(q_1, 0)\} = \{q_1, q_2\}$$

The resulting pair is not marked.

$$\{\delta(q_0, 1), \delta(q_1, 1)\} = \{q_2, q_3\}$$

The resulting pair $\{q_2, q_3\}$ is marked, so we can mark the pair $\{q_1, q_2\}$ as a distinguished pair.

$\{q_0, q_2\}$:

$$\{\delta(q_0, 0), \delta(q_2, 0)\} = \{q_1, q_2\}$$

The resulting pair is not marked

$$\{\delta(q_0, 1), \delta(q_2, 1)\} = \{q_2, q_4\}$$

The resulting pair $\{q_2, q_4\}$ is marked, so we can mark the pair $\{q_0, q_2\}$ as a distinguished pair.

$\{q_1, q_2\}$:

$$\{\delta(q_1, 0), \delta(q_2, 0)\} = \{q_2\} \text{ can't mark}$$

$$\{\delta(q_1, 1), \delta(q_2, 1)\} = \{q_3, q_4\} \text{ can't mark as the resulting pair is both final state.}$$

$$\{q_3, q_3\} \text{ can't be marked as distinguishable pair as both the states are final states}$$

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Example 1: Minimize the following DFA:

Final Table

q1	X			
q2	X			
*q3	X	X	X	
*q4	X	X	X	
	q0	q1	q2	* q3

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:

The cell corresponding to the pairs which are not marked are indistinguishable pairs of final state and non-final states and can be merged as a single final state and single non-final state. Pair $\{q_1, q_2\}$ can be merged as a single non-final state and pair $\{q_3, q_4\}$ can be merged as single final state

The resulting DFA will have 3 states after merging they are $\{q_1, q_2\}$, $\{q_3, q_4\}$, q_0 The transition table of the minimized DFA :

$$\delta(q_0, 0) = q_1$$

$$\delta(q_0, 1) = q_2$$

$$= \{q_1, q_2\}$$

$$\delta(\{q_1, q_2\}, 0) = q_2 \text{ (} q_1, q_2 \text{)}$$

$$\delta(\{q_1, q_2\}, 1) = \{q_3, q_4\}$$

$$\delta(\{q_3, q_4\}, 0) = \{q_3, q_4\}$$

$$\delta(\{q_3, q_4\}, 1) = \{q_3, q_4\}$$

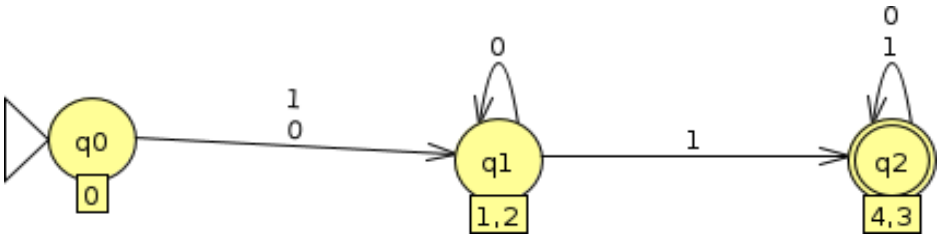
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 1: Minimize the following DFA:

	0	1
q0	{q1,q2}	{q1,q2}
{q1,q2}	{q1,q2}	{q3q4}
{q3,q4}	{q3q4}	{q3q4}

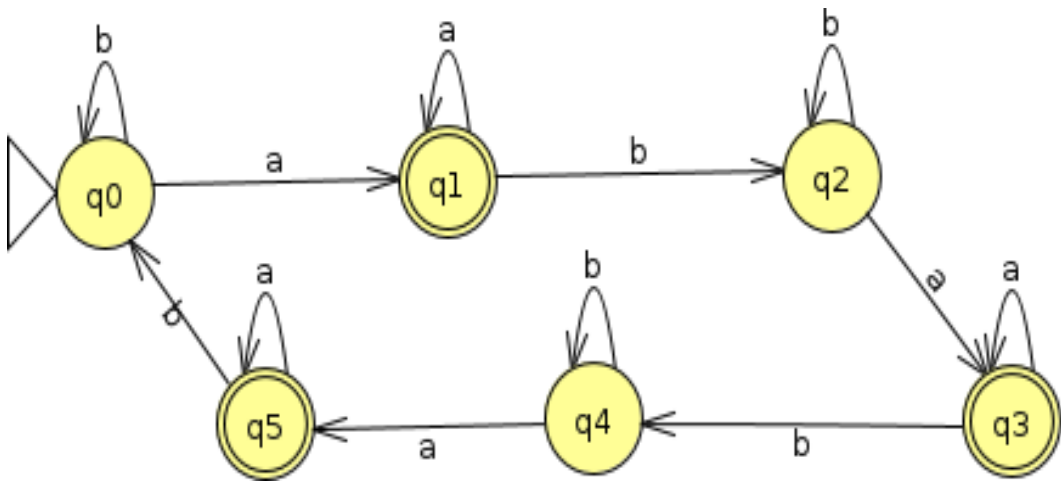
The final minimized DFA state diagram:



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 2: Minimize the following DFA:



TRANSITION TABLE:

	a	b
q0	q1	q0

q1	q1	q2
q2	q3	q2
12 q3	q3	q4
q4	q5	q4
q5	q5	q0

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 2: Minimize the following DFA:

Final table:

All the states are reachable from the start state. Construct the table using table filling algorithm
 Table after filling the final and non final pairs is given below:

*q1	X				
q2		X			
*q3	X		X		
q4		X		X	
*q5	X		X		X
	q0	*q1	q2	*q3	q4

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 2: Minimize the following DFA:

Next apply 3rd rule and try and see whether the remaining cells can be marked,

$\{q_0, q_2\}$:

$$\{\delta(q_0, a), \delta(q_2, a)\} = \{q_1, q_3\}$$

The resultant pair is not marked

$$\{\delta(q_0, b), \delta(q_2, b)\} = \{q_0, q_2\}$$

The resultant pair is not marked

So the pair $\{q_0, q_2\}$ can't be marked as distinguishable pair

$\{q_0, q_4\}$:

$$\{\delta(q_0, a), \delta(q_4, a)\} = \{q_1, q_5\} \text{ Resultant pair is not marked}$$

$$\{\delta(q_0, b), \delta(q_4, b)\} = \{q_0, q_4\}$$

Resultant pair is not marked

So the pair $\{q_0, q_4\}$ can't be marked as distinguishable pair

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 2: Minimize the following DFA:

{q1,q5}:

$\{\delta(q1,a), \delta(q5,a)\} = \{q1,q5\}$ Resultant pair is not marked

$\{\delta(q1,b), \delta(q5,b)\} = \{q2,q0\}$ Resultant pair is not marked

So the pair {q1,q5} can't be marked as distinguishable pair

{q1,q3}:

$\{\delta(q1,a), \delta(q3,a)\} = \{q1,q3\}$ Resultant pair is not marked

$\{\delta(q1,b), \delta(q3,b)\} = \{q2,q4\}$ Resultant pair is not marked

So the pair {q1,q3} can't be marked as distinguishable pair

So the states, {q0,q4,q2} can be merged into single state and {q1,q3,q5} can be merged as another state.

Automata Formal Languages and Logic

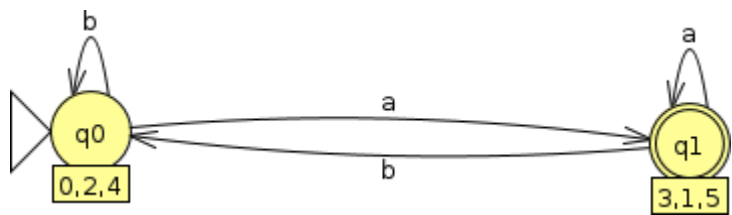
Unit 1 - Deterministic Finite Acceptor/Automata

Example 2: Minimize the following DFA:

The transition table for the minimized DFA:

	a	b
{q0q2q4}	{q1q3q5}	{q0q2q4}
{q1q3q5}	{q1q3q5}	{q0q2q4}

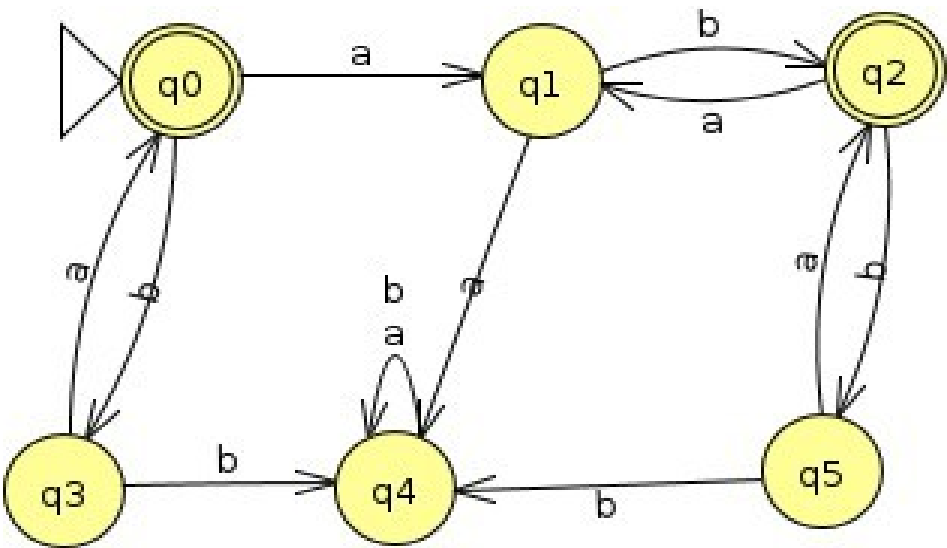
Minimized DFA state diagram:



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 3 : Minimize the following DFA



Transition table:

	a	b
->*q0	q1	q3
q1	q4	q2
*q2	q1	q5
q3	q0	q4
q4	q4	q4
q5	q2	q4

The table after applying table filling algorithm

q1	X				
q2		X			
q3	X	X	X		
q4	X	X	X		
q5	X	X	X		X
	q0	q1	q2	q3	q4

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example 3 : Minimize the following DFA

The 4 states after applying table filling algorithm are: {q0q2}, { q3,q5}, q1,q4

The minimized DFA transition table:

The states are renamed as:

{q0q2}=q3

{q3q5}= q1 Q1 = q1

q0=q4

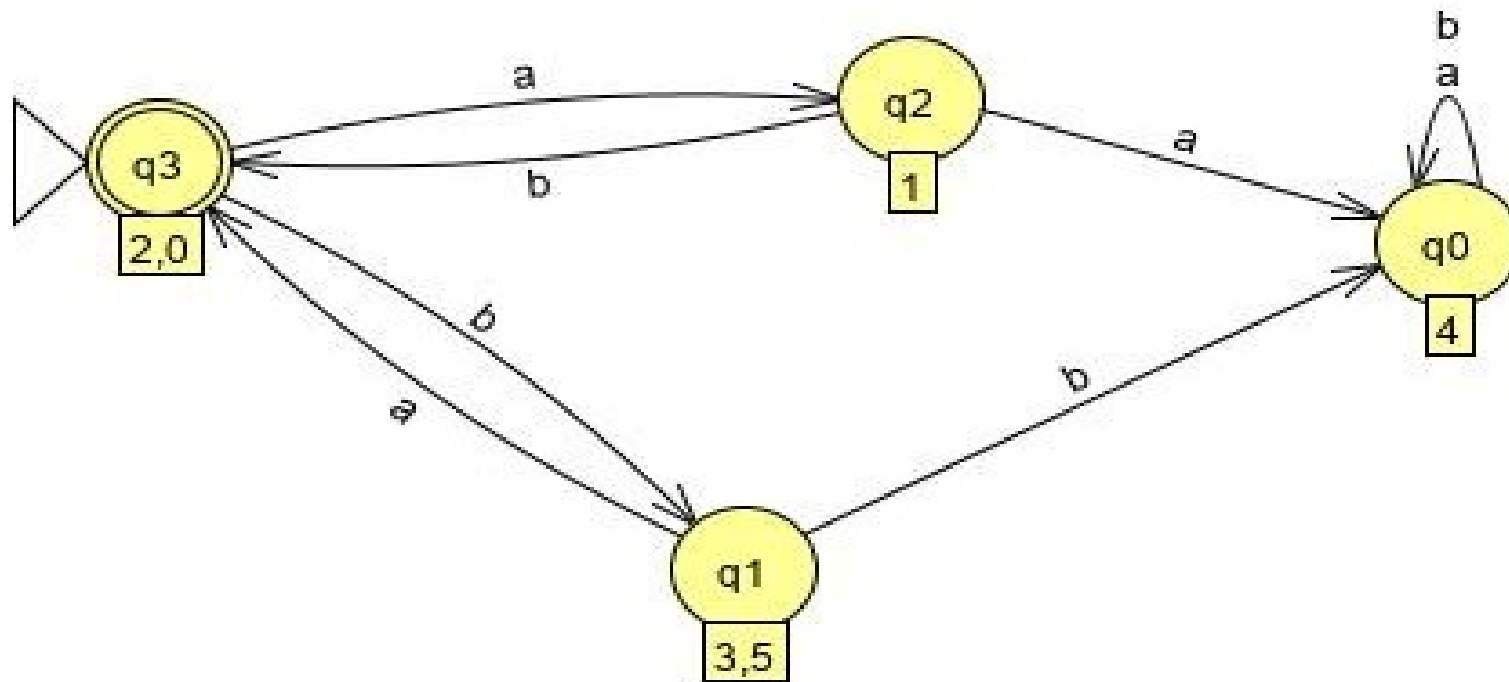
	a	b
->*q3	q2	q1
q2	q0	q3
q1	q3	q0
q0	q0	q0

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

Example : Minimize the following DFA

The minimized DFA transition diagram::





THANK YOU

Divyaprabha K N

Department of Computer Science & Engineering

divyaprabha@pes.edu



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 1 - Non-Deterministic Finite Automata

NFA

Transition Function for a NFA

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

- A model of computation is nondeterministic if the computing machine may have multiple decisions that it can make at one point.
- If there is at least one choice that leads to an accepting state, the machine will accept the input string.

Automata Formal Languages and Logic

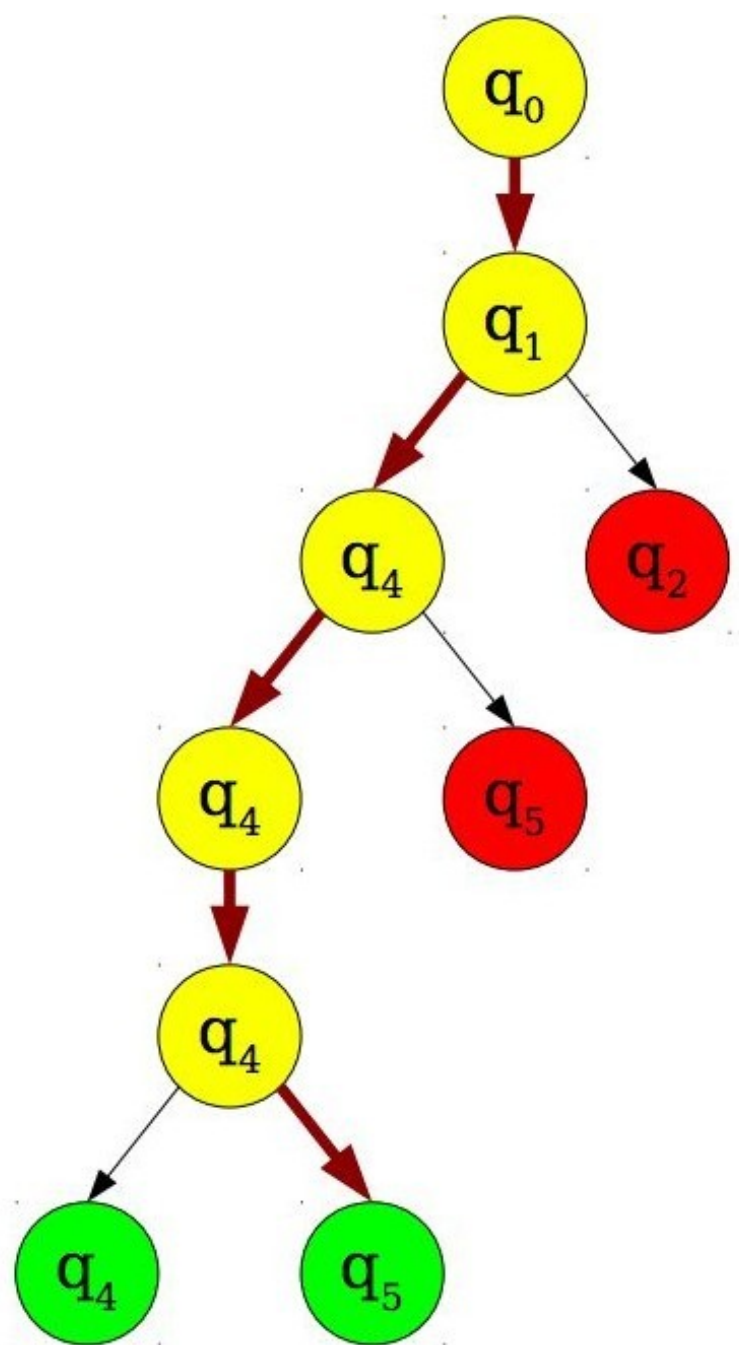
Unit 1 - Non-Deterministic Finite Automata

NFA

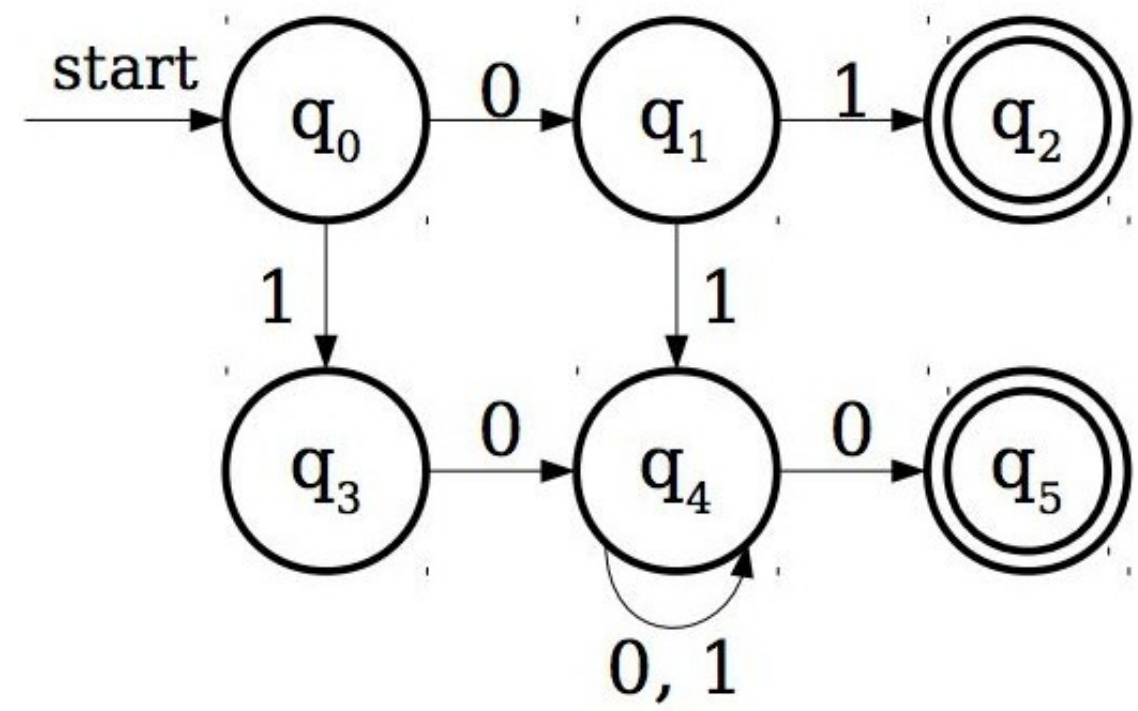
- Can have missing transitions or multiple transitions done on the same input symbol.
- Structurally similar to a DFA, but represents a fundamental shift in how we'll think about computation.
- Computation in an NFA looks like a Tree

Automata Formal Languages and Logic

Unit 1 - Non-Deterministic Finite Automata



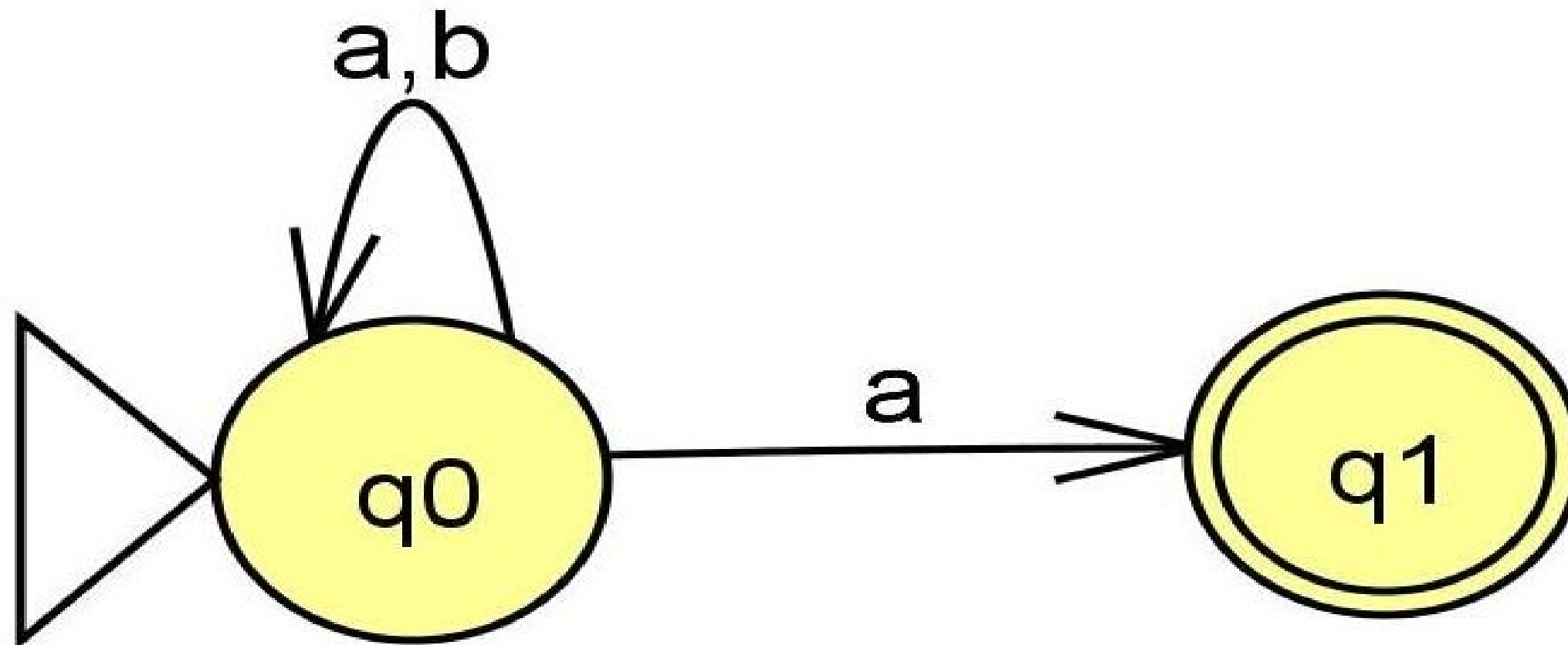
0 1 0 1 0



Unit 1 - Non-Deterministic Finite Automata

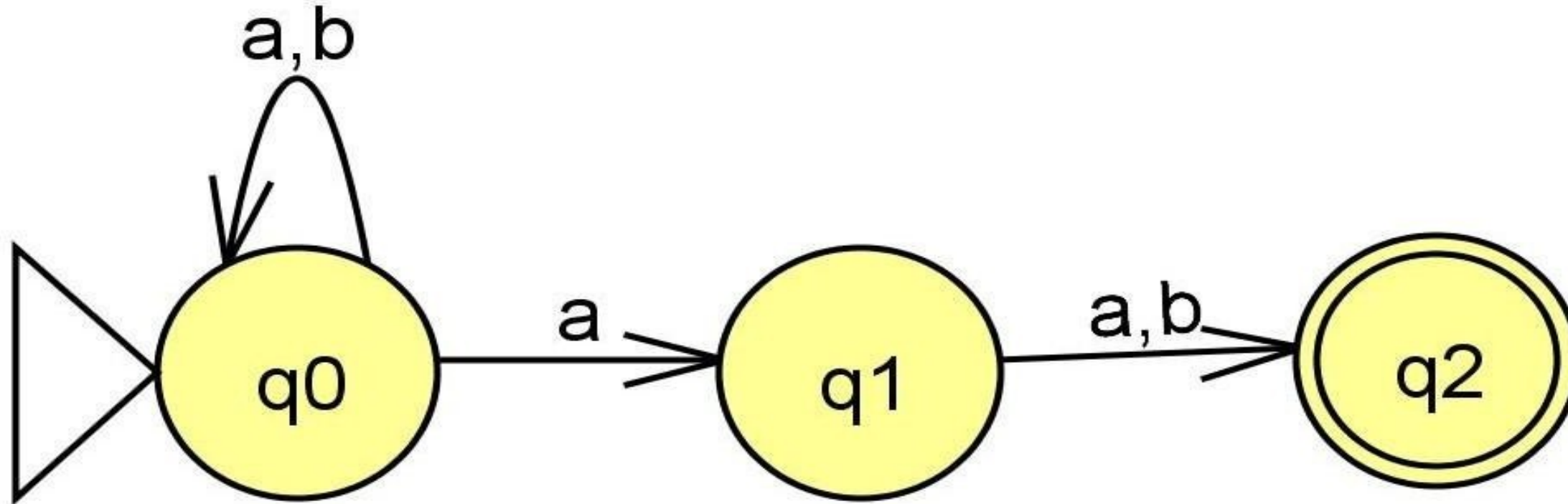
Example 1: Construct NFA for the language of string of a's and b's that end in a.

Solution:



Example 1: Construct NFA for the language of string of a's and b's ,where the second symbol from RHS is 'a'.

Solution:



Automata Formal Languages and Logic

Unit 1 - λ -Non-Deterministic Finite Automata

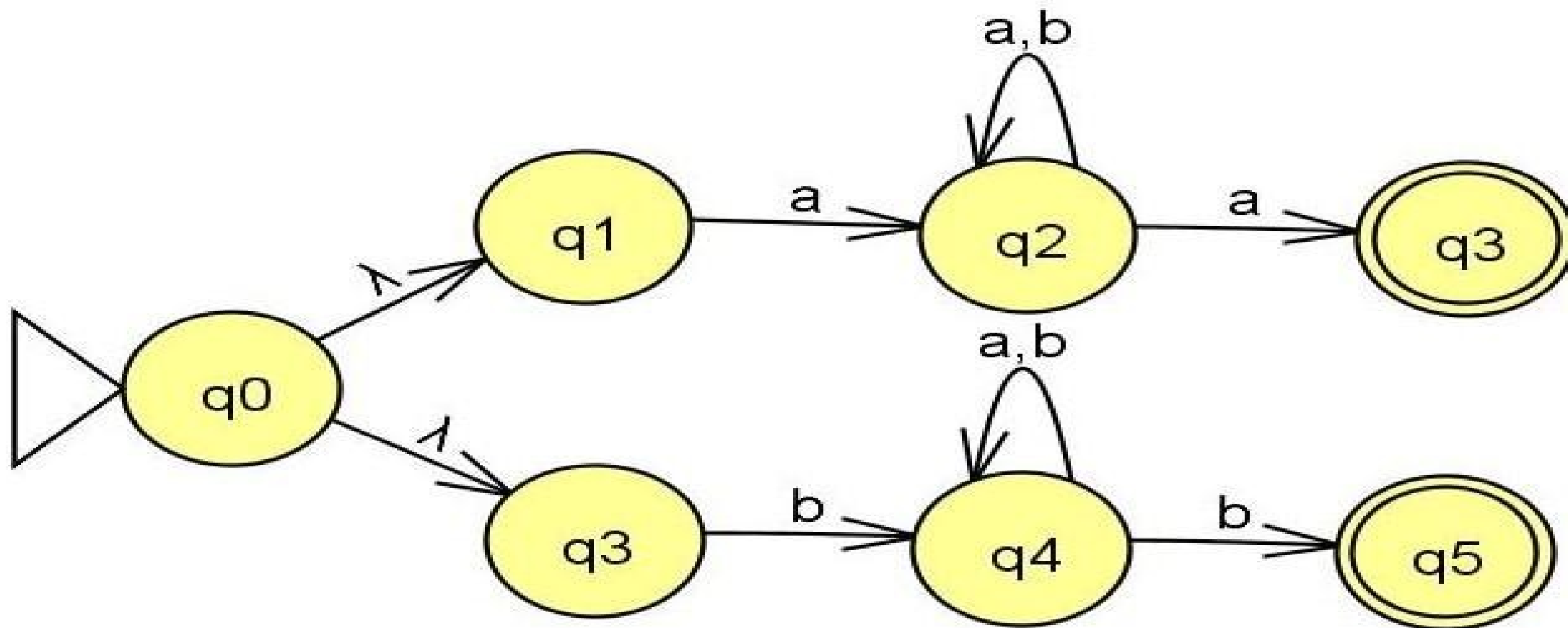
Transition Function for a λ -NFA

$$\delta : Q \times (\Sigma \cup \lambda) \rightarrow 2^Q$$

An NFA may follow any number of λ -transitions at any time without consuming any input.

Example 1: Construct λ -NFA for language of strings of a's and b's that start and end with the same symbol.

Solution:



Unit 1 - NFAs and DFAs

A model of computation is deterministic if at every point in the computation, there is exactly one choice that can be made.

- The machine accepts if that series of choices leads to an accepting state.
- A model of computation is nondeterministic if the computing machine may have multiple decisions that it can make at one point.
- The machine accepts if any series of choices leads to an accepting state.

Unit 1 - NFAs and DFAs

Any language that can be accepted by a DFA can be accepted by an NFA.

- NFAs and DFAs are finite automata; there can only be finitely many states in an NFA or DFA.
- DFA's, NFA's, and ϵ -NFA's all accept exactly the same set of languages: the regular languages.
- The NFA types are easier to design and may have exponentially fewer states than a DFA.
- But only a DFA can be implemented!

Automata Formal Languages and Logic

Unit 1 - NFAs and DFAs

MULTIPLE CHOICE QUESTIONS:

1. In an NFA, the transition function δ maps:

- A) A state and an input symbol to a single state
- B) A state and an input symbol to multiple states
- C) Two states to an input symbol
- D) An input symbol to a state

2. Which of the following statements is true about the relationship between NFA and DFA?

- A) Every NFA can be converted to an equivalent DFA
- B) Every DFA can be converted to an equivalent NFA
- C) Both A and B
- D) Neither A nor B

Automata Formal Languages and Logic

Unit 1 - NFAs and DFAs

MULTIPLE CHOICE QUESTIONS:

3. Which of the following can use an empty string (ϵ) transition?

- A) DFA only
- B) NFA only
- C) Both DFA and NFA
- D) Neither DFA nor NFA

Let N be an NFA with n states. What is the maximum possible number of states in the equivalent DFA constructed from N using the subset construction method?

- A) n
- B) 2^n
- C) $n!$
- D) n^2



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Unit 1 - Conversion of NFA to DFA

NFA $\rightarrow$ DFA (Subset Construction)

- The initial state is the start state, plus all states reachable from the start state via λ -transitions (Called λ -closure).
- Transition from a state S on character a is found by following all possible transitions on a for each state in S , then taking the set of states reachable from there by λ -transitions.
- Accepting states are any set of states where some state in the set is an accepting state.

Unit 1 - Conversion of NFA to DFA

NFA $\rightarrow$ DFA (Subset Construction)

- In converting an NFA to a DFA, the DFA's states correspond to set of NFA states.
- In the worst-case, the construction can result in a DFA that is exponentially larger than the original NFA .
- Minimization of a DFA ensures that the resulting DFA (after minimization) has the least possible states. The advantages of having a minimal DFA are: Faster Execution: The more the number of states the more time the DFA will take to process a string, hence minimization ensures faster execution.

Unit 1 - Conversion of NFA to DFA

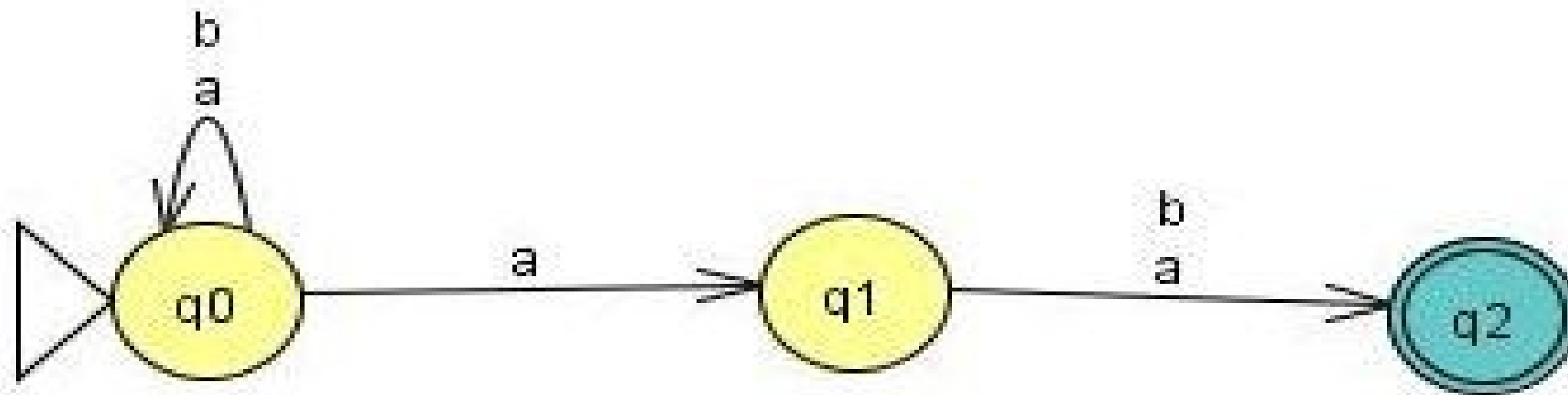
NFA $\rightarrow$ DFA (Subset Construction)

Given an NFA with states Q , inputs Σ , transition function δ_N , state state q_0 , and final states F , construct equivalent DFA with:

- * States 2^Q (Set of subsets of Q).
- * Inputs Σ .
- * Start state $\{q_0\}$.
- * Final states = all those with a member of F .
- The DFA states have names that are sets of NFA states.
- But as a DFA state, an expression like $\{p,q\}$ must be read as a single symbol, not as a set.

Unit 1 - Conversion of NFA to DFA

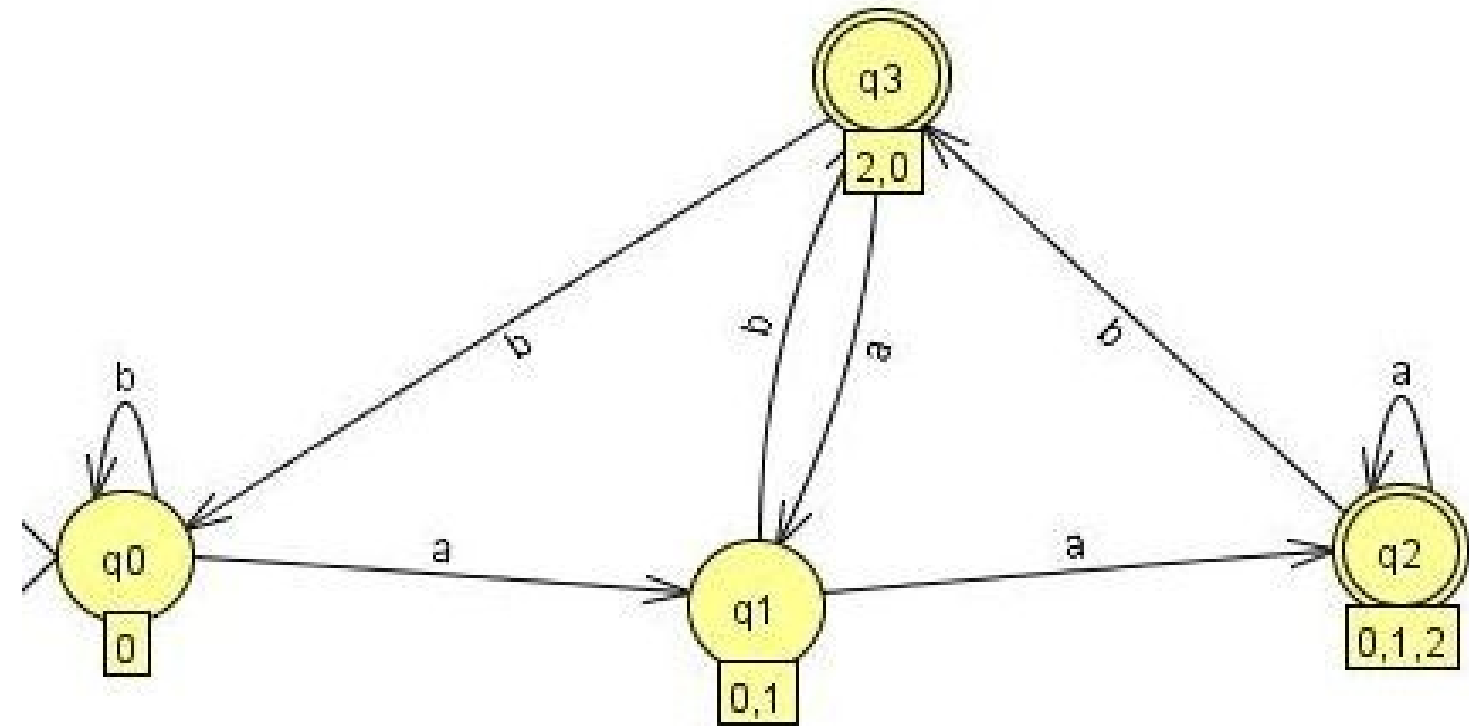
Example 1: Convert the following NFA, $\Sigma=\{a,b\}$, $L=\{\text{Strings where the second symbol from RHS is a}\}$ to DFA.



Unit 1 - Conversion of NFA to DFA

Solution:

	a	b
q0	{q0,q1}	q0
{q0q1}	{q0,q1,q2}	{q0,q2}
{q0,q1,q2}	{q0,q1,q2}	{q0,q2}
{q0,q2}	{q0q1}	q0

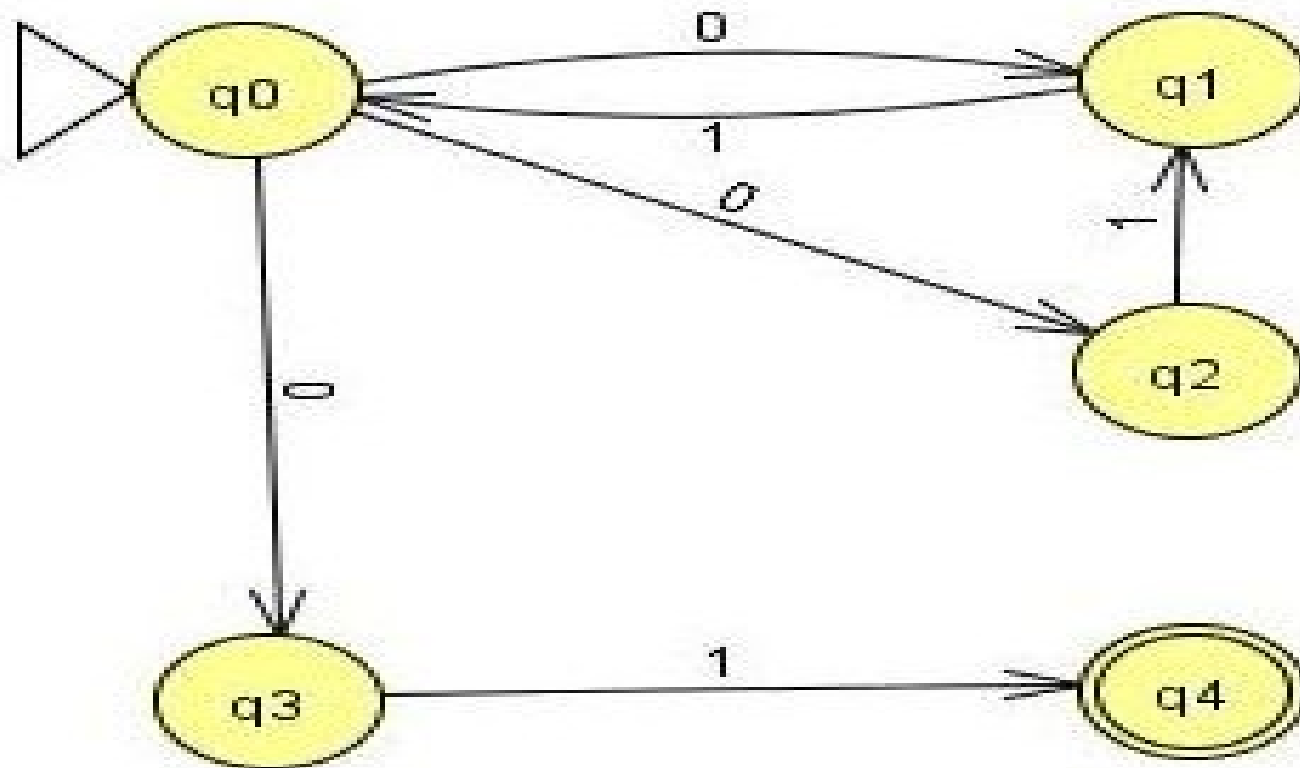


States= [{q0}, {q0q1},{q0q1q2},{q1q2}]

Can rename the states as $q0=q0$, $\{q0q1\}=q1$, $\{q0,q1,q2\}=q2$, $\{q0,q2\}=q3$

Unit 1 - Conversion of NFA to DFA

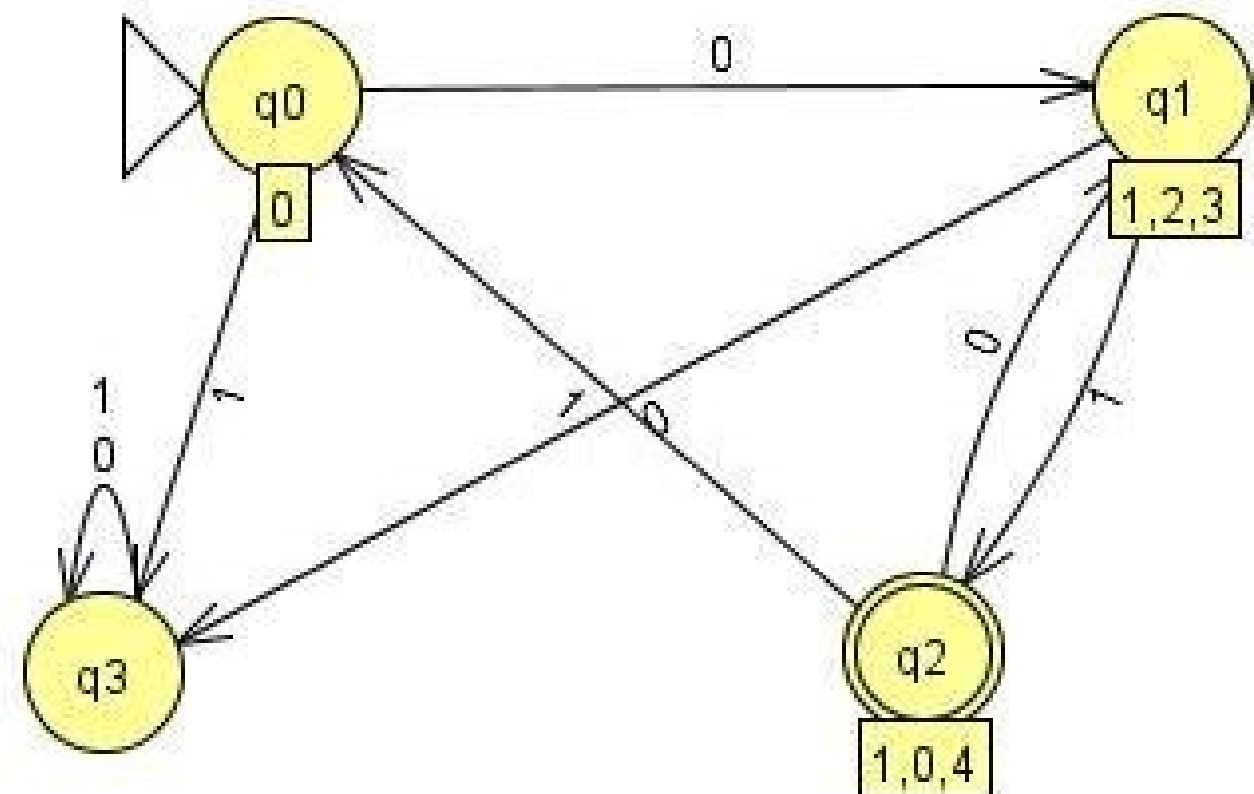
Example 2: Convert the following NFA to DFA.



Unit 1 - Conversion of NFA to DFA

Solution:

	0	1
---> q0	{q1,q2,q3}	Φ
{q1,q2,q3}	Φ	{q0,q1,q4}
*{q0,q1,q4}	{q1,q2,q3}	q0
Φ	Φ	Φ

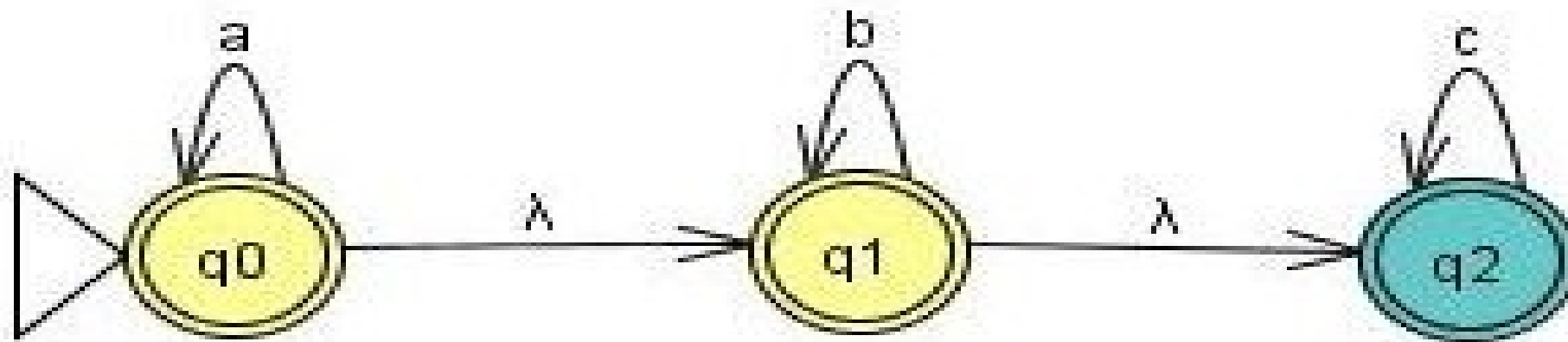


States: [$\{q0\}, \{q0q1q2\}, \{q0q1q4\}, \Phi$]

Rename the states as $q0=q0, q0q1q2=q1, q0q1q4=q2, \Phi=q3$

Unit 1 - Conversion of NFA to DFA

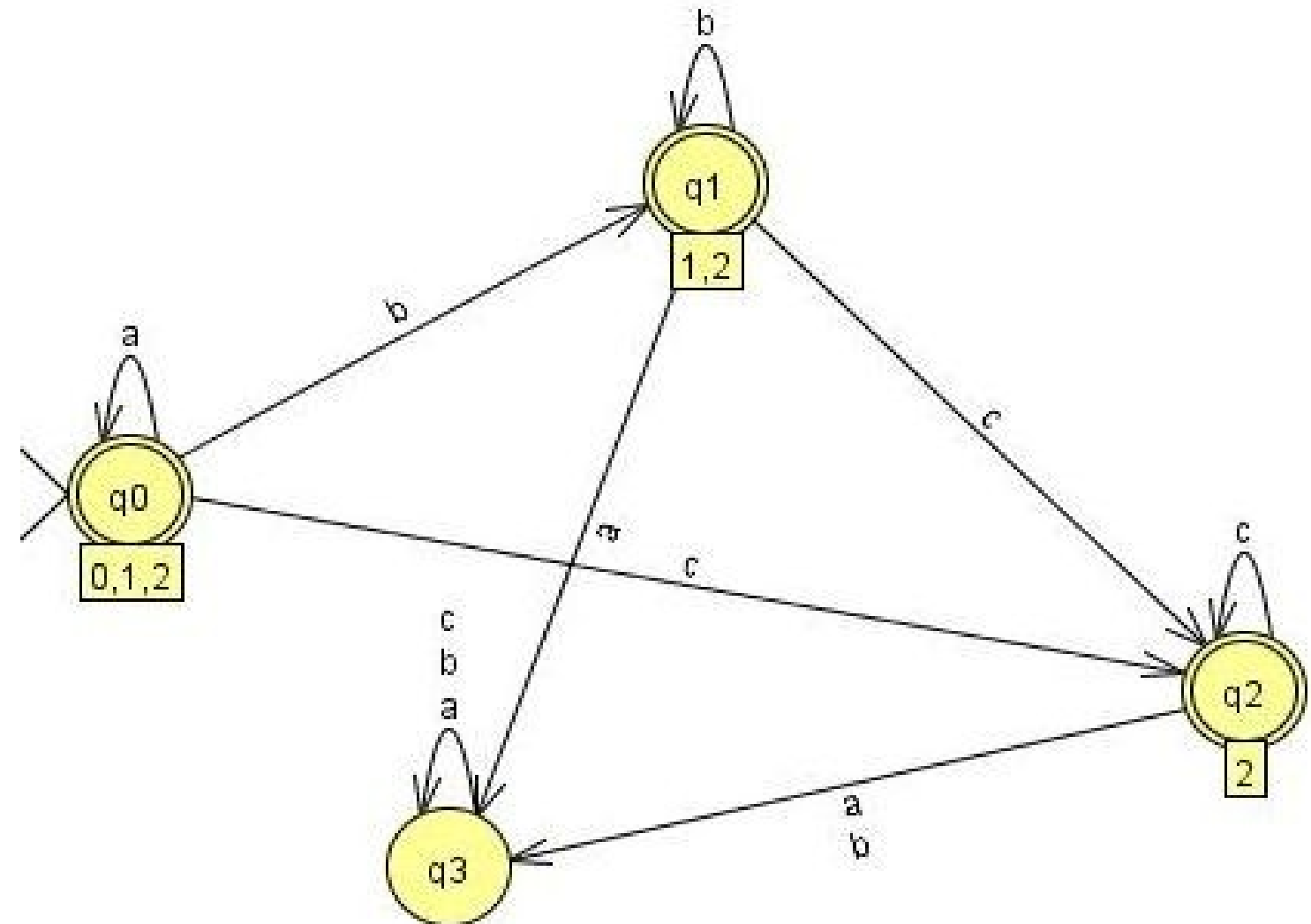
Example 3: Convert the following λ -NFA $L=\{a^n b^m c^k, n,m,k \geq 0\}$ to DFA.



Unit 1 - Conversion of NFA to DFA

Solution:

	a	b	c
$\rightarrow^* \{q_0q_1q_2\}$	$\{q_0q_1q_2\}$	$\{q_1q_2\}$	q_2
$\{q_1q_2\}$	Φ	$\{q_1q_2\}$	q_2
q_2	Φ	Φ	q_2
Φ	Φ	Φ	Φ

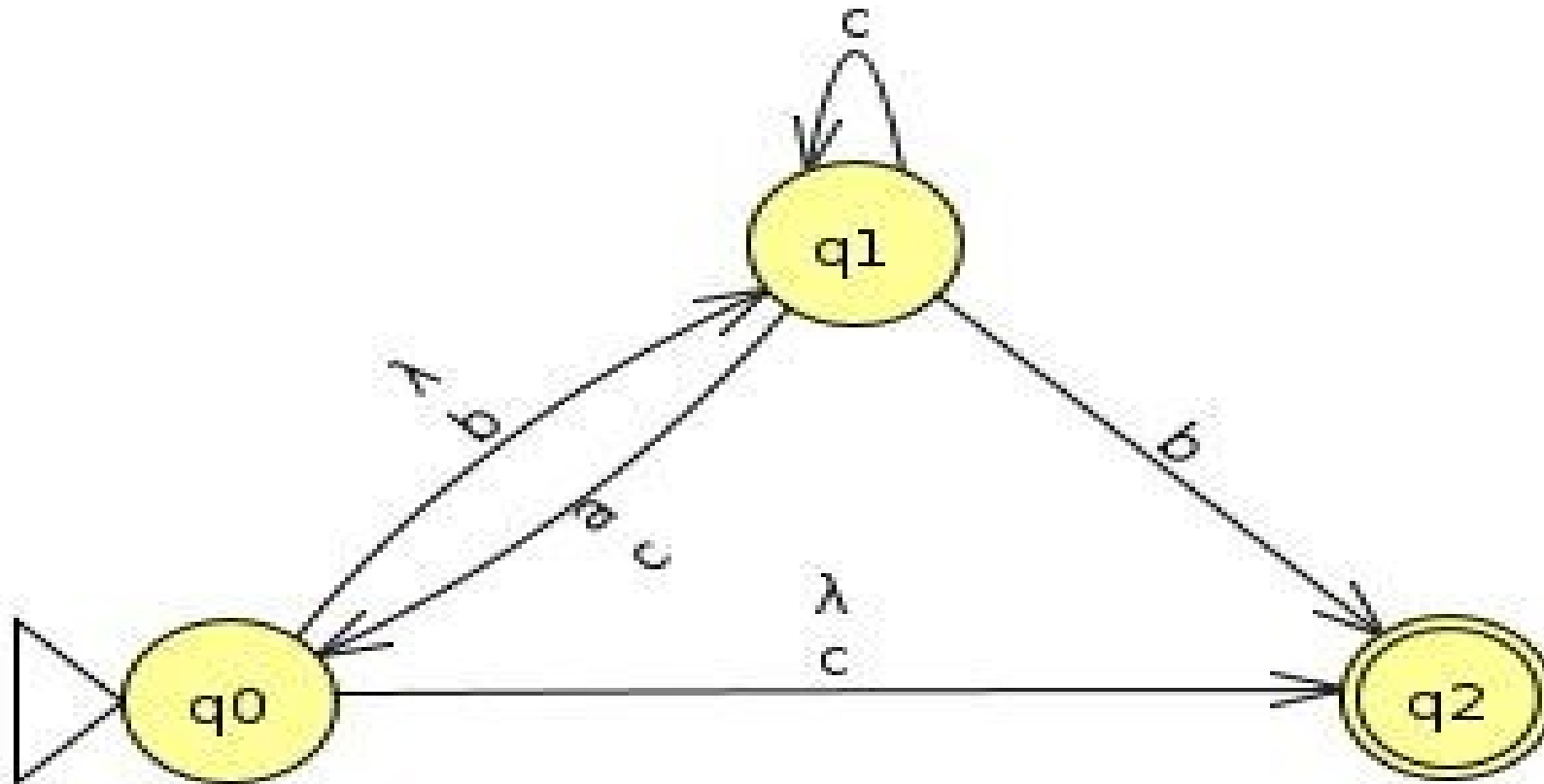


Rename the states as:

$\{q_0q_1q_2\}$ as q_0 , $\{q_1q_2\}$ as q_1 , q_2 as q_2 ,

Unit 1 - Conversion of NFA to DFA

Example 4: Convert the following λ -NFA to DFA.



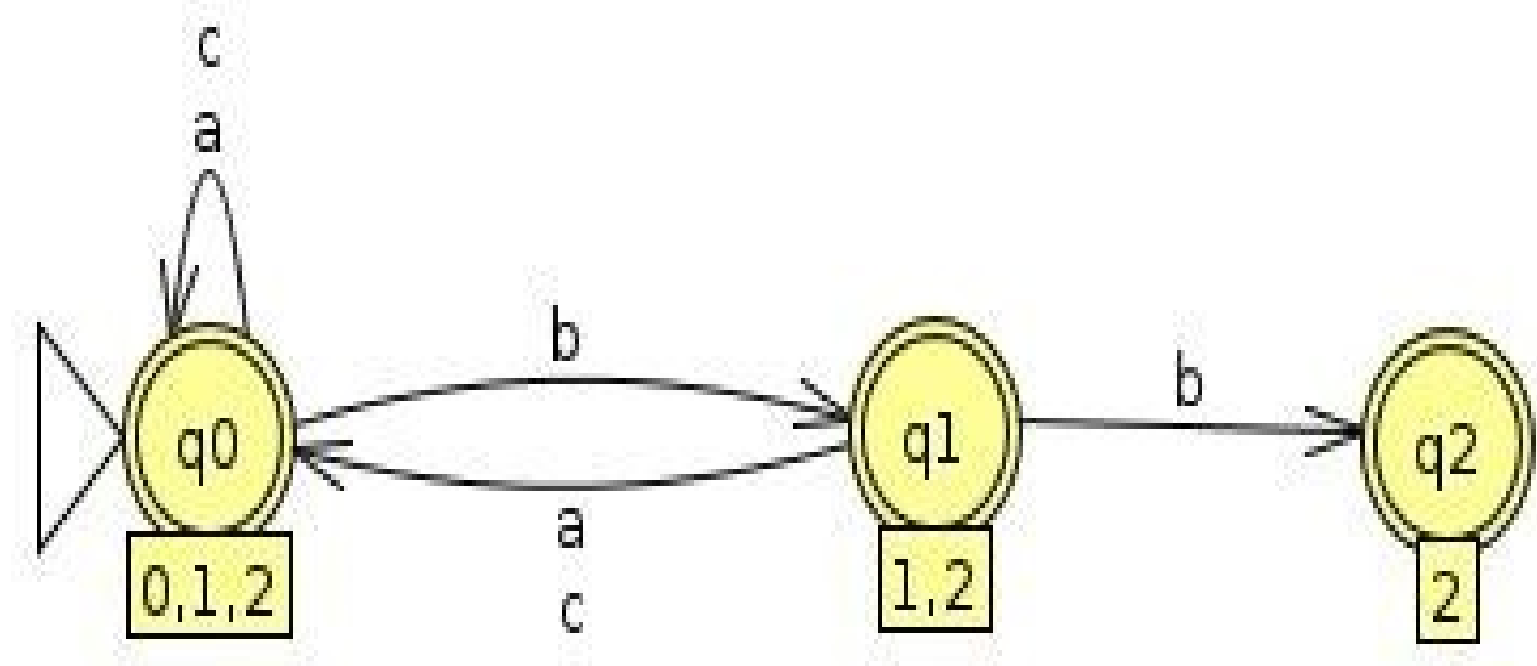
Unit 1 - Conversion of NFA to DFA

Solution:

Transition Table

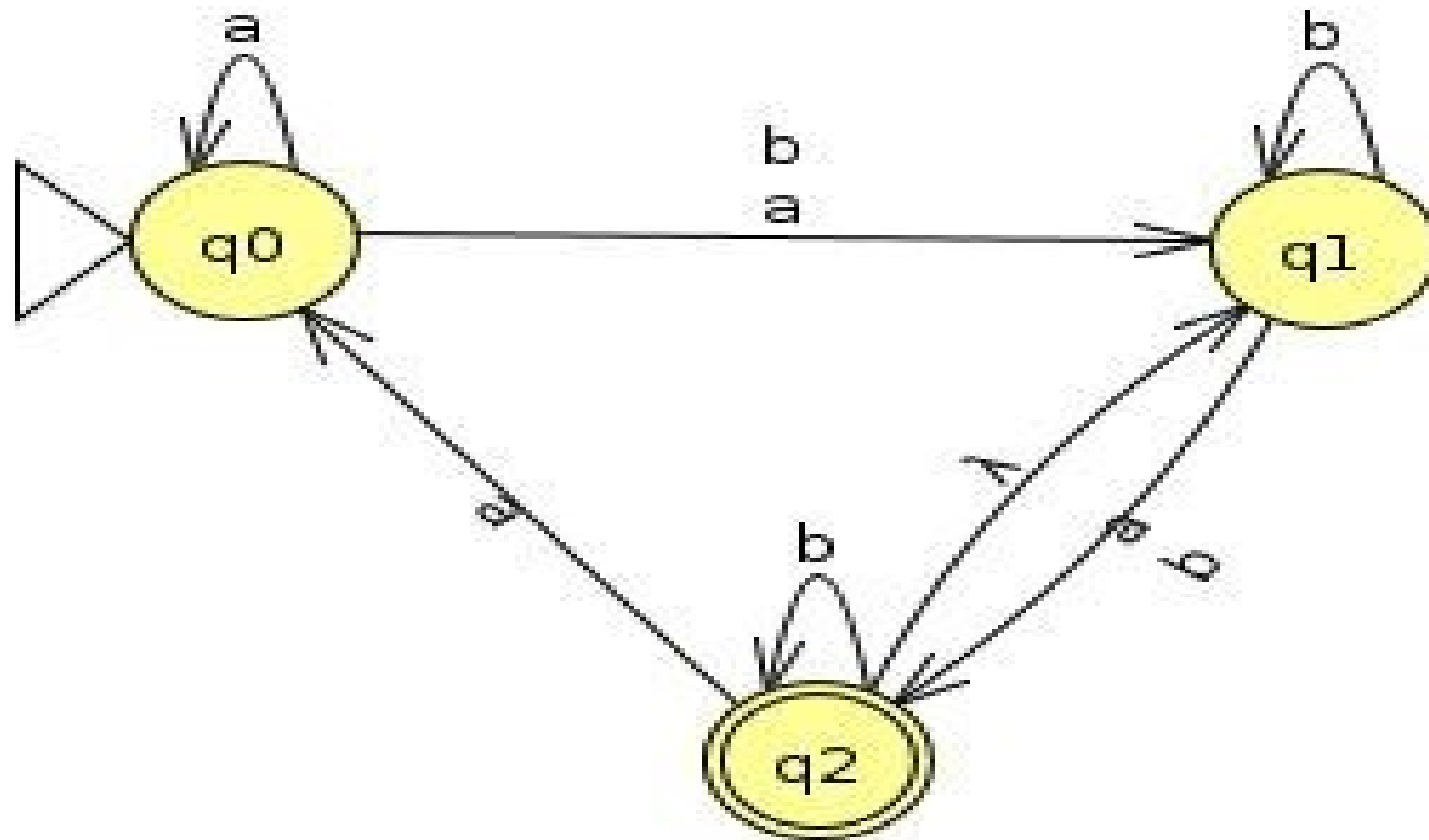
	a	b	c
$\rightarrow * \{q_0q_1q_2\}$	$\{q_0q_1q_2\}$	$\{q_1q_2\}$	$\{q_0q_1q_2\}$
$* \{q_1q_2\}$	$\{q_0q_1q_2\}$	q_2	Φ
$*q_2$	Φ	Φ	Φ
Φ	Φ	Φ	Φ

DFA



Unit 1 - Conversion of NFA to DFA

Example 5: Convert the following λ -NFA to DFA.



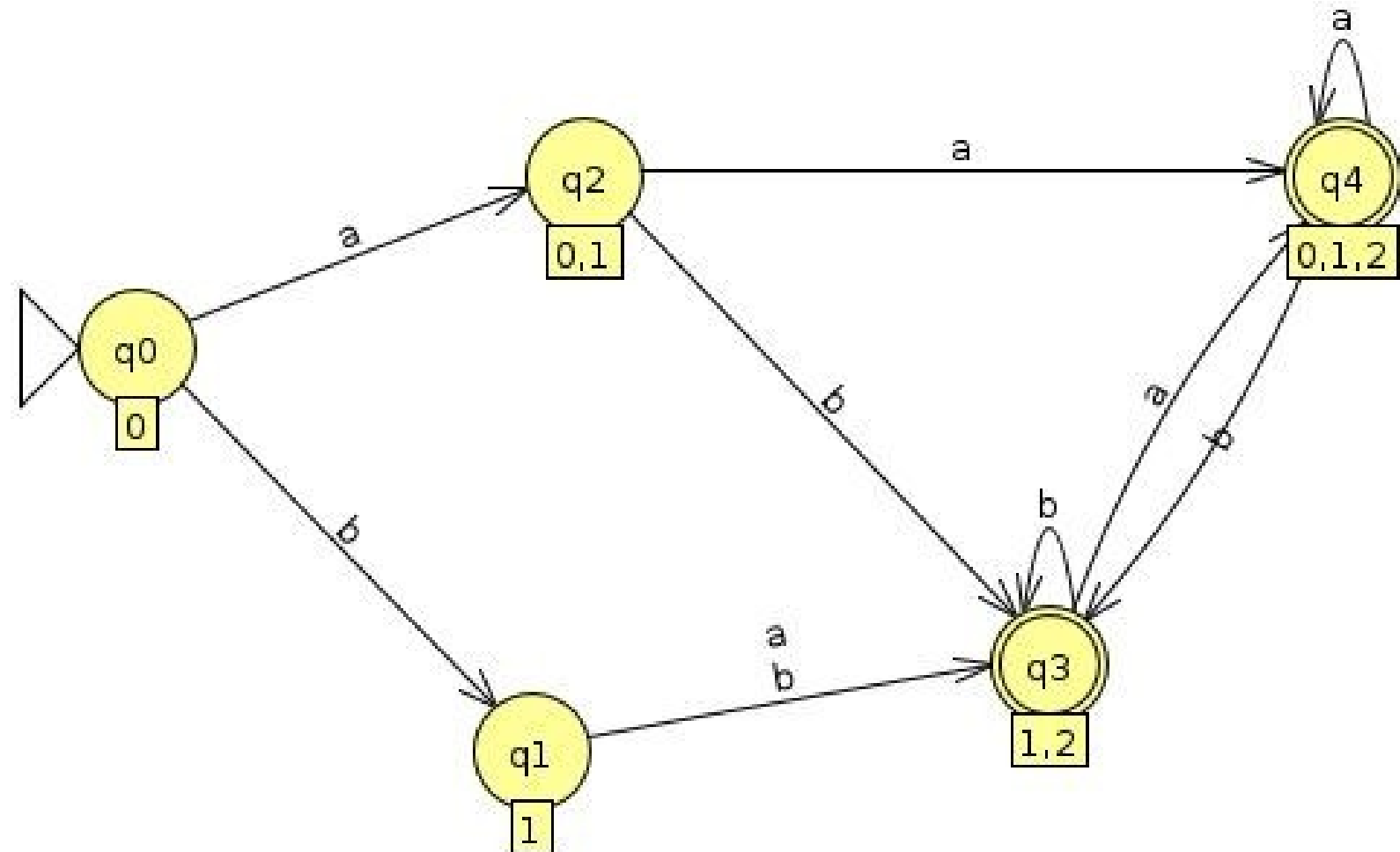
Unit 1 - Conversion of NFA to DFA

Solution:

Transition Table

	a	b
$\rightarrow q_0$	$\{q_0q_1\}$	q_1
q_1	$\{q_1q_2\}$	$\{q_1q_2\}$
$\{q_0q_1\}$	$\{q_0q_1q_2\}$	$\{q_0q_2\}$
$\{q_1q_2\}$	$\{q_0q_1q_2\}$	$\{q_1q_2\}$
$\{q_0q_1q_2\}$	$\{q_0q_1q_2\}$	$\{q_1q_2\}$

DFA



Automata Formal Languages and Logic

Unit 1 - Conversion of NFA to DFA

Multiple Choice Questions

1.

Which of the following statements about the subset construction method for NFA to DFA conversion is correct?

- A) Each DFA state corresponds to a single NFA state.
- B) The initial DFA state is always the set of all NFA states.
- C) The DFA states correspond to subsets of NFA states.
- D) The number of DFA states is always equal to the number of NFA states.

2.

During the conversion of an NFA with λ -transitions to a DFA, what is the main purpose of computing the λ -closure of a state?

- A) To find unreachable states
- B) To identify all accepting states
- C) To determine all states reachable from a given state using only λ -transitions
- D) To minimize the number of transitions

Unit 1 - Conversion of NFA to DFA

Multiple Choice Questions:

3.

In the subset construction method, which of the following is true about the accepting states of the resulting DFA?

- A) Only if all NFA states in the subset are accepting
- B) If at least one NFA state in the subset is accepting
- C) Only if the subset contains the initial NFA state
- D) Only if the subset is the full set of NFA states

4.

What is a potential drawback of the subset construction method when converting an NFA to a DFA?

- A) The DFA may have fewer states than the NFA
- B) The DFA may become non-deterministic
- C) The DFA can be exponentially larger than the NFA
- D) The DFA cannot accept the same language as the NFA

Automata Formal Languages and Logic

Unit 1 - Conversion of NFA to DFA

Multiple Choice Questions:

Given an NFA with n states, the subset construction method can theoretically produce up to 2^n DFA states. However, in practice, not all 2^n states are always realized in the resulting DFA.

Which of the following best explains why?

- A) Only subsets containing the initial state are considered valid DFA states.
- B) Only subsets that are reachable from the start state via valid input strings are constructed as DFA states.
- C) The subset construction method always constructs all 2^n states, but some are removed during minimization.
- D) Only subsets containing accepting states are included in the DFA.

Answers:

1. C

2. C

3. B

4. C

5. B



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Unit 1 - Regular Expression

Regex is an algebraic way to describe regular languages.

Regular Expression

Atom

- A Character/symbol
- ϕ ($L = \{ \}$)
- λ ($L = \{ \lambda \}$)

Operator

- (R)
- R^*
- $R_1.R_2$ or R_1R_2
- $R_1 + R_2$ or $R_1 | R_2$

Example

$a \quad b^* \quad c \quad + \quad d$

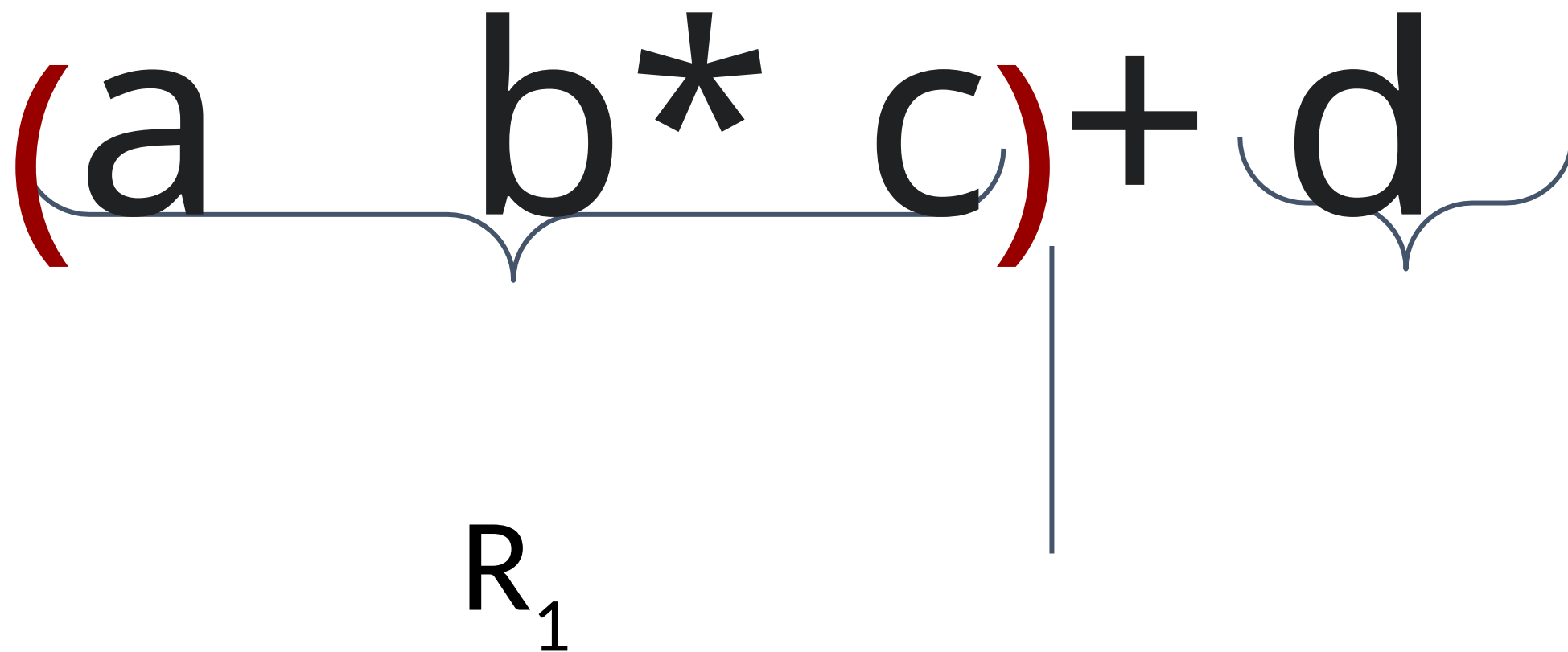
Automata Formal Languages and Logic

Unit 1 - Example of Regular Expressions

Example

$(a b^* c) + d$

R_1



Unit 1 - Example of Regular Expressions

Example

(a b*

c) + d

Starting with
an 'a'

Followed by 0 or
more no. of b's

Ending with
a 'c'

Automata Formal Languages and Logic

Unit 1 - Example of Regular Expressions

Example

(a b*

String that
contains only d

c) + d

Starting with
an 'a'

Ending with
a 'c'

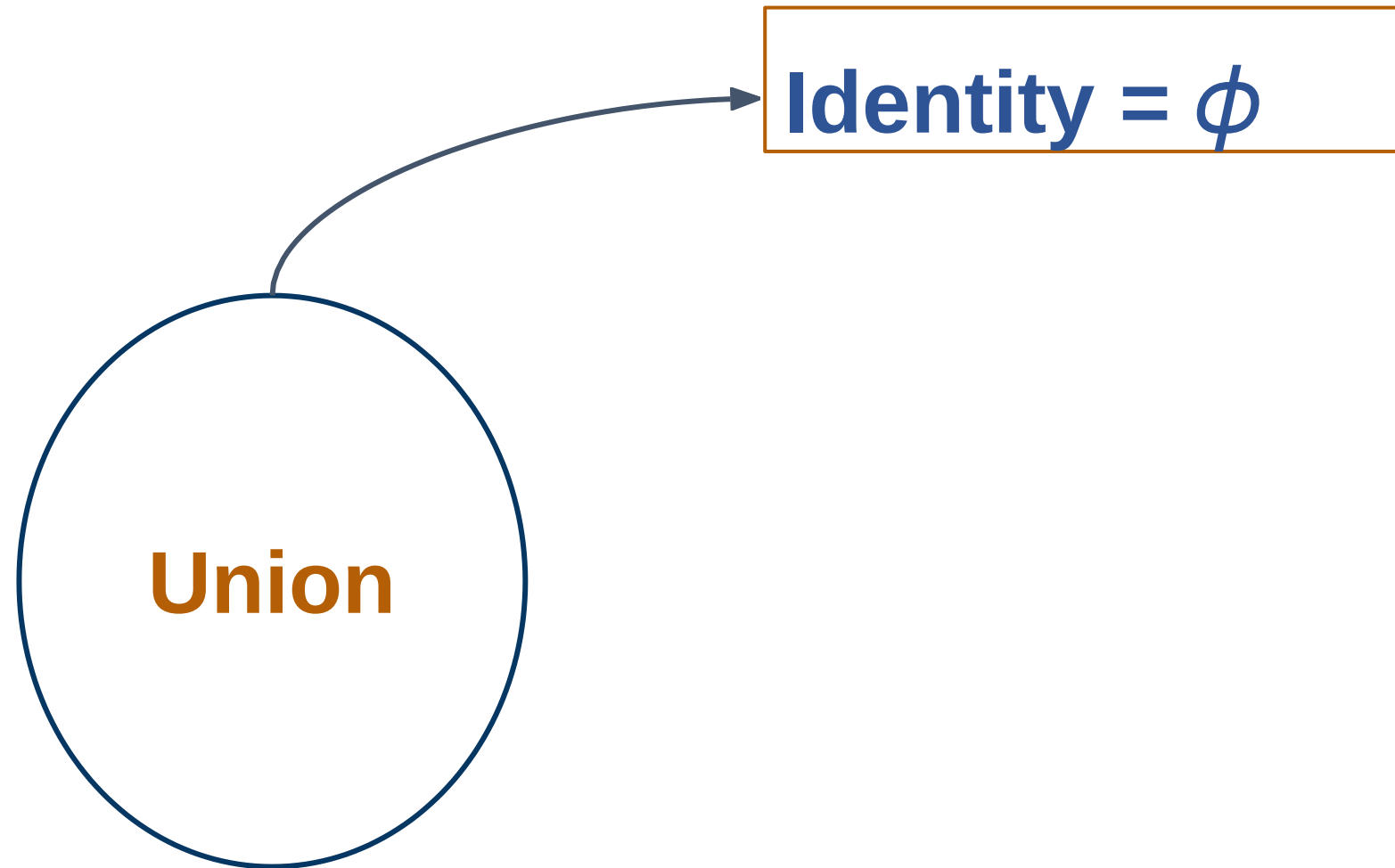
Followed by 0 or
more no. of b's

Unit 1 - Example of Regular Expressions

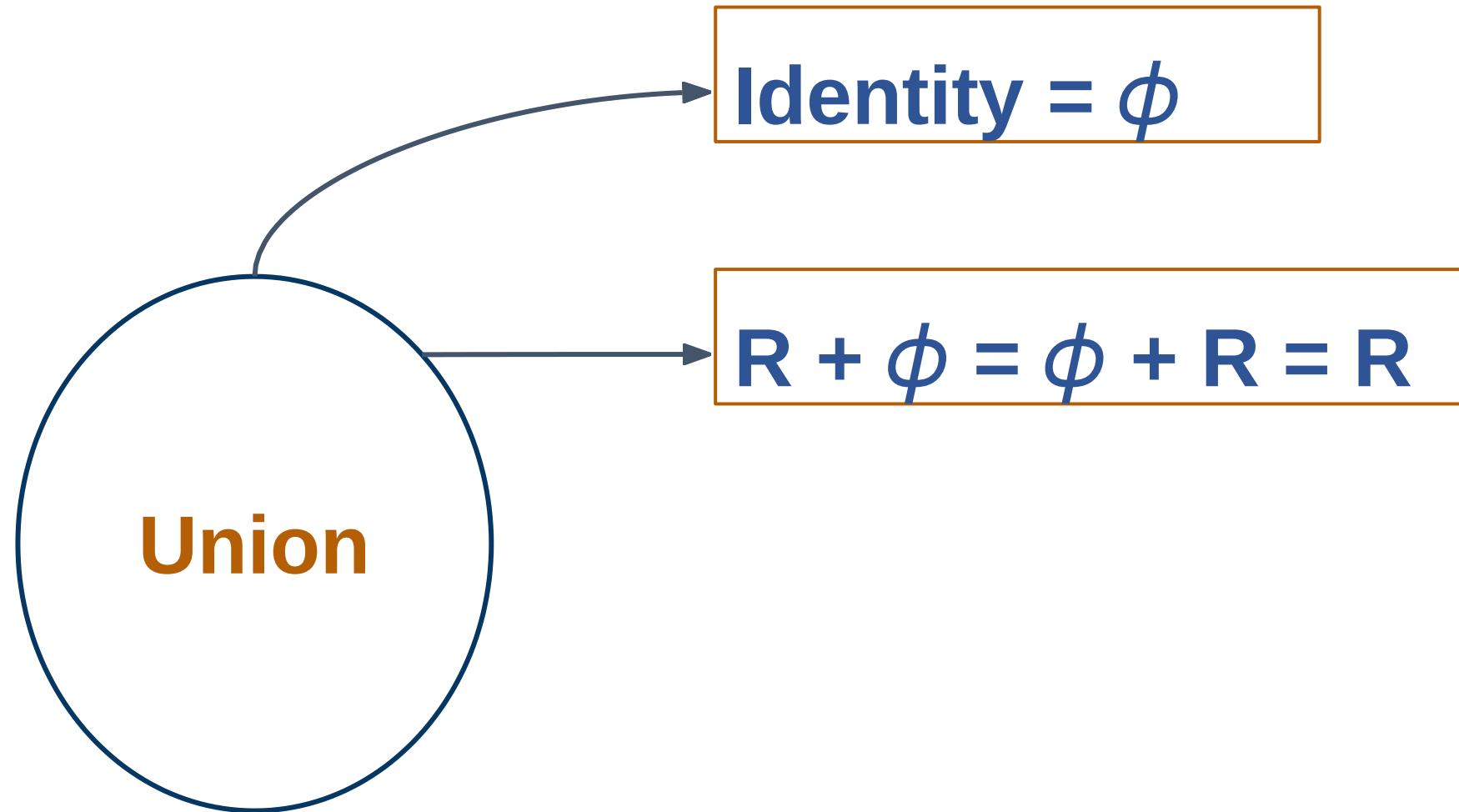
Example

$$(a \quad b^* \quad c) + d$$
$$L = \{ ac, \\ abc, \\ abbb\dots c, \\ d \}$$

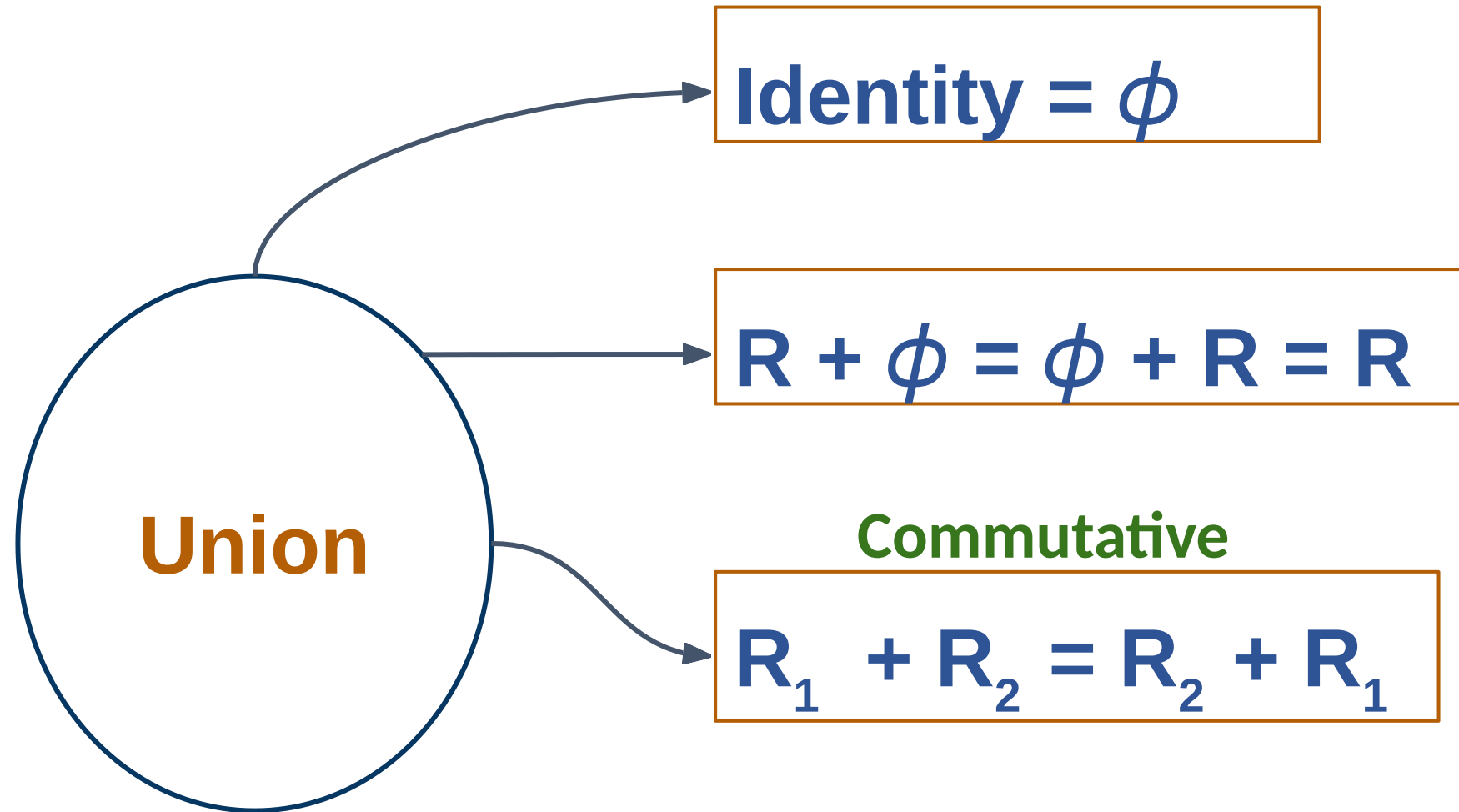
Unit 1 - Algebraic Laws of Regular Expressions



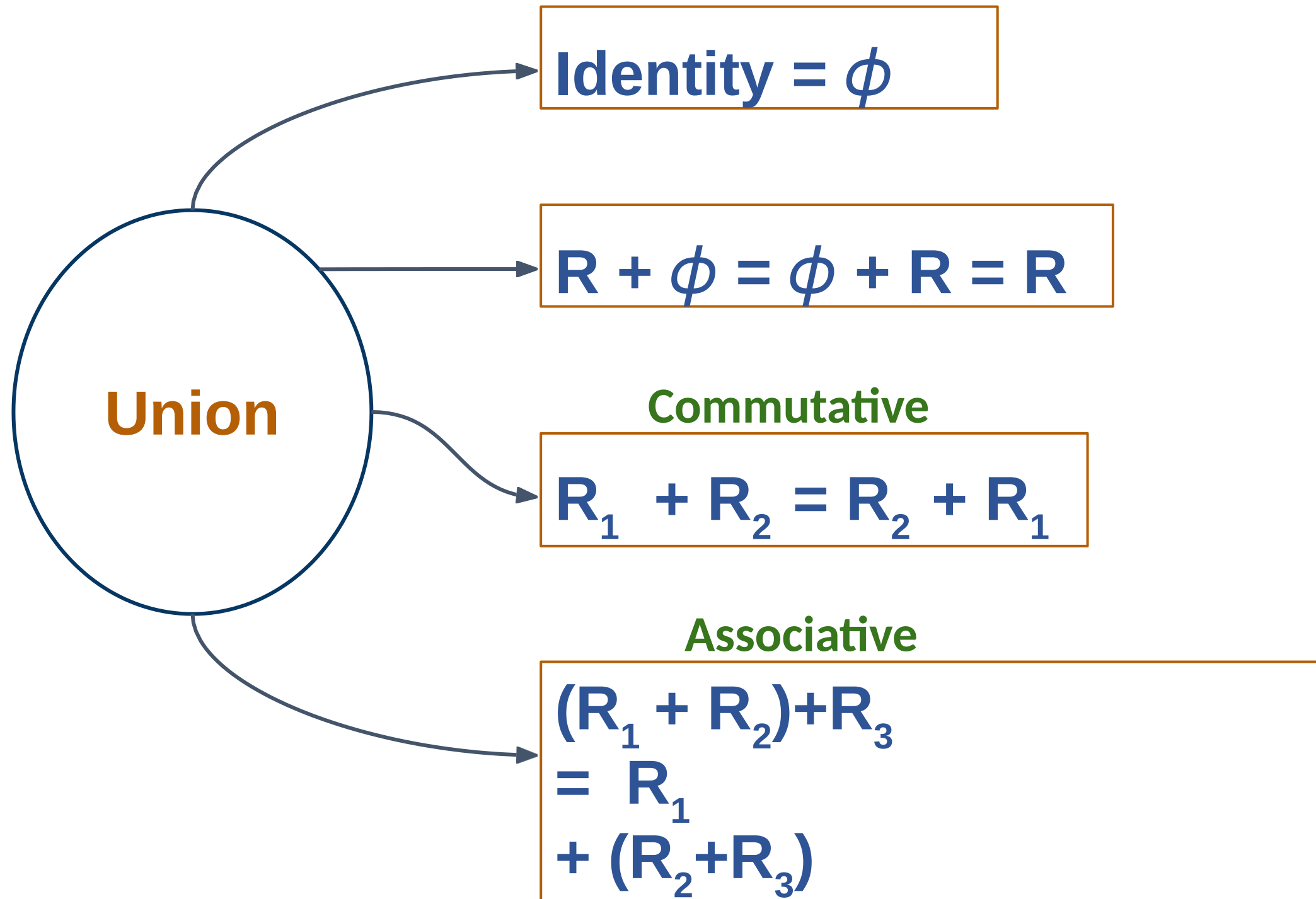
Unit 1 - Algebraic Laws of Regular Expressions



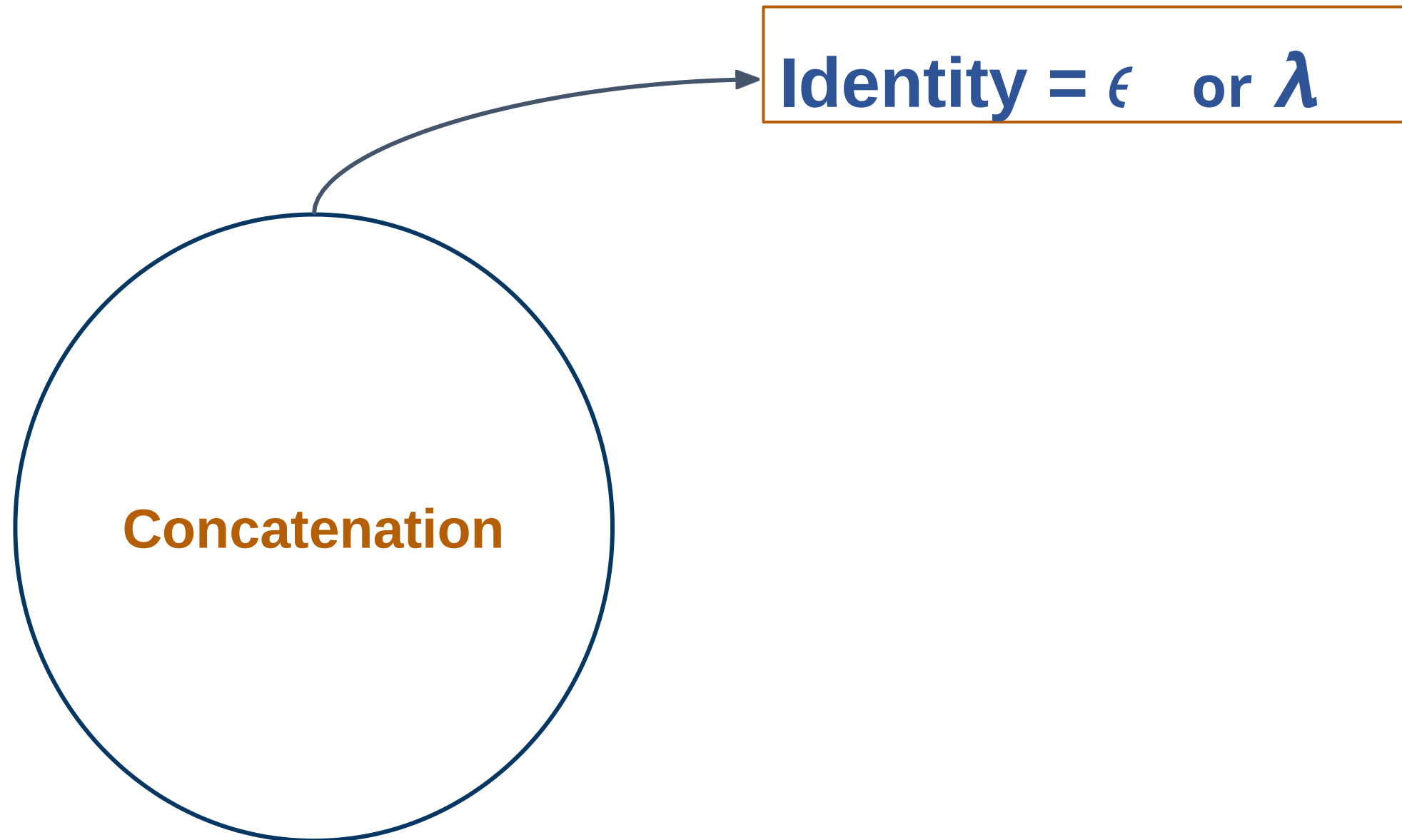
Unit 1 - Algebraic Laws of Regular Expressions



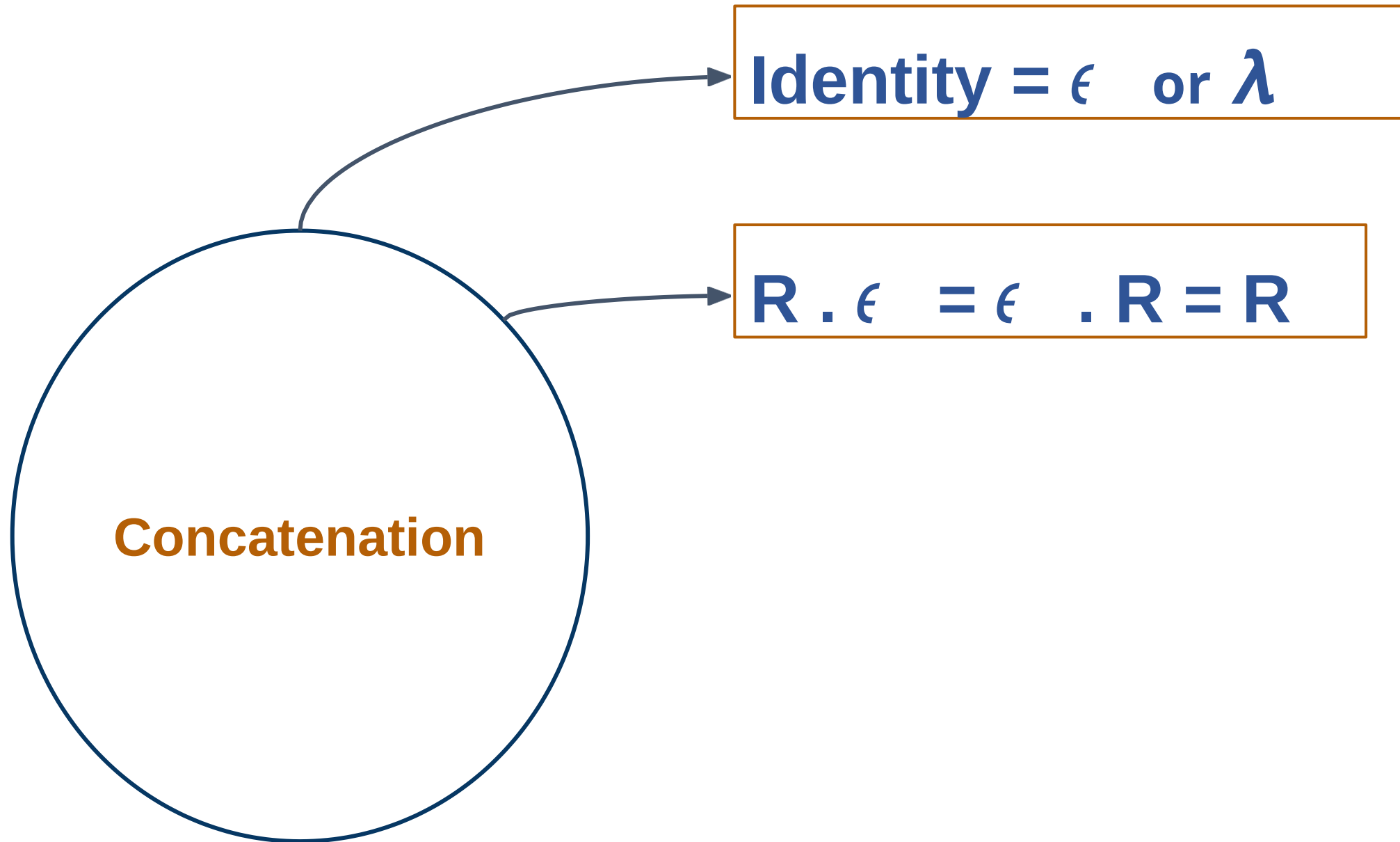
Unit 1 - Algebraic Laws of Regular Expressions



Unit 1 - Algebraic Laws of Regular Expressions



Unit 1 - Algebraic Laws of Regular Expressions

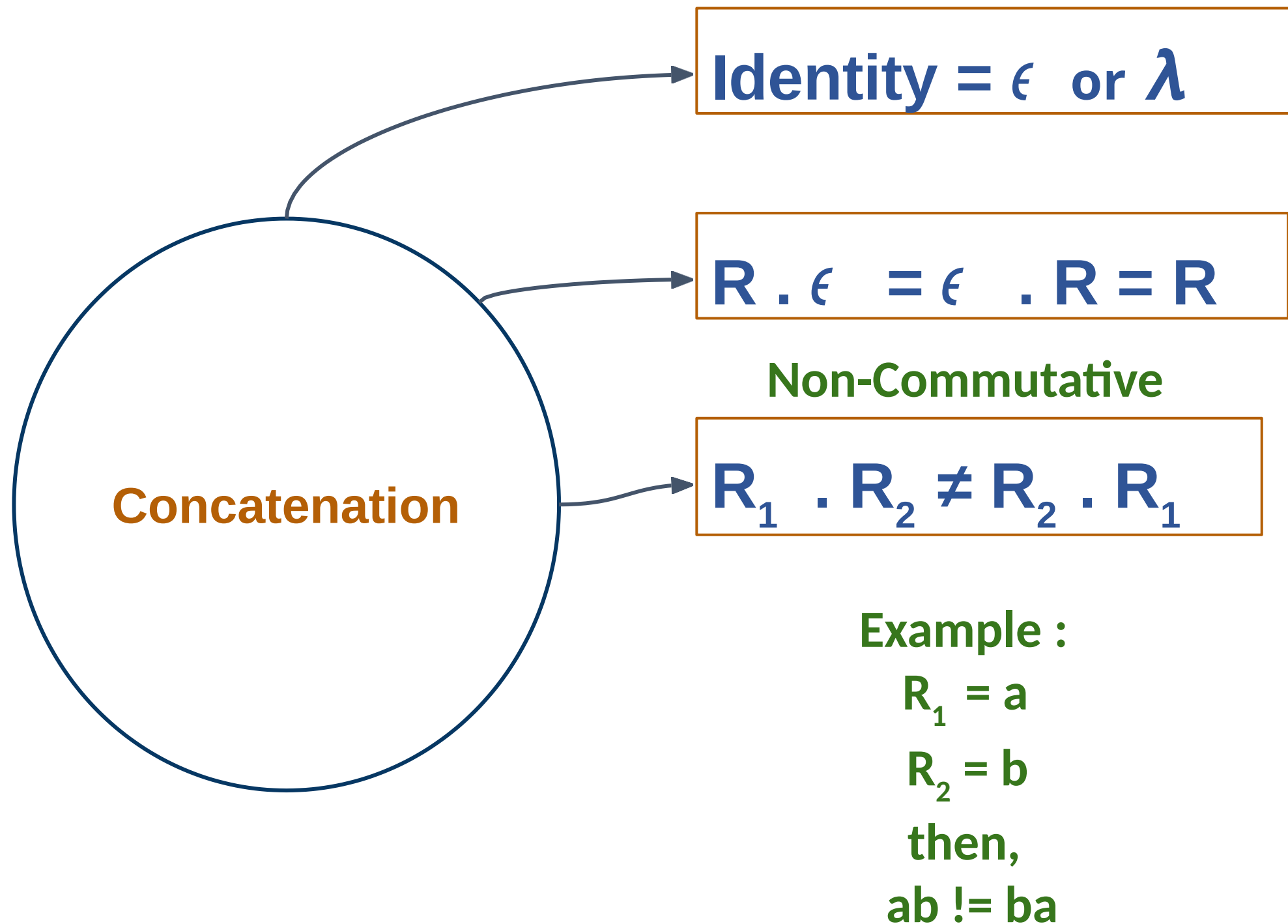


Unit 1 - Algebraic Laws of Regular Expressions

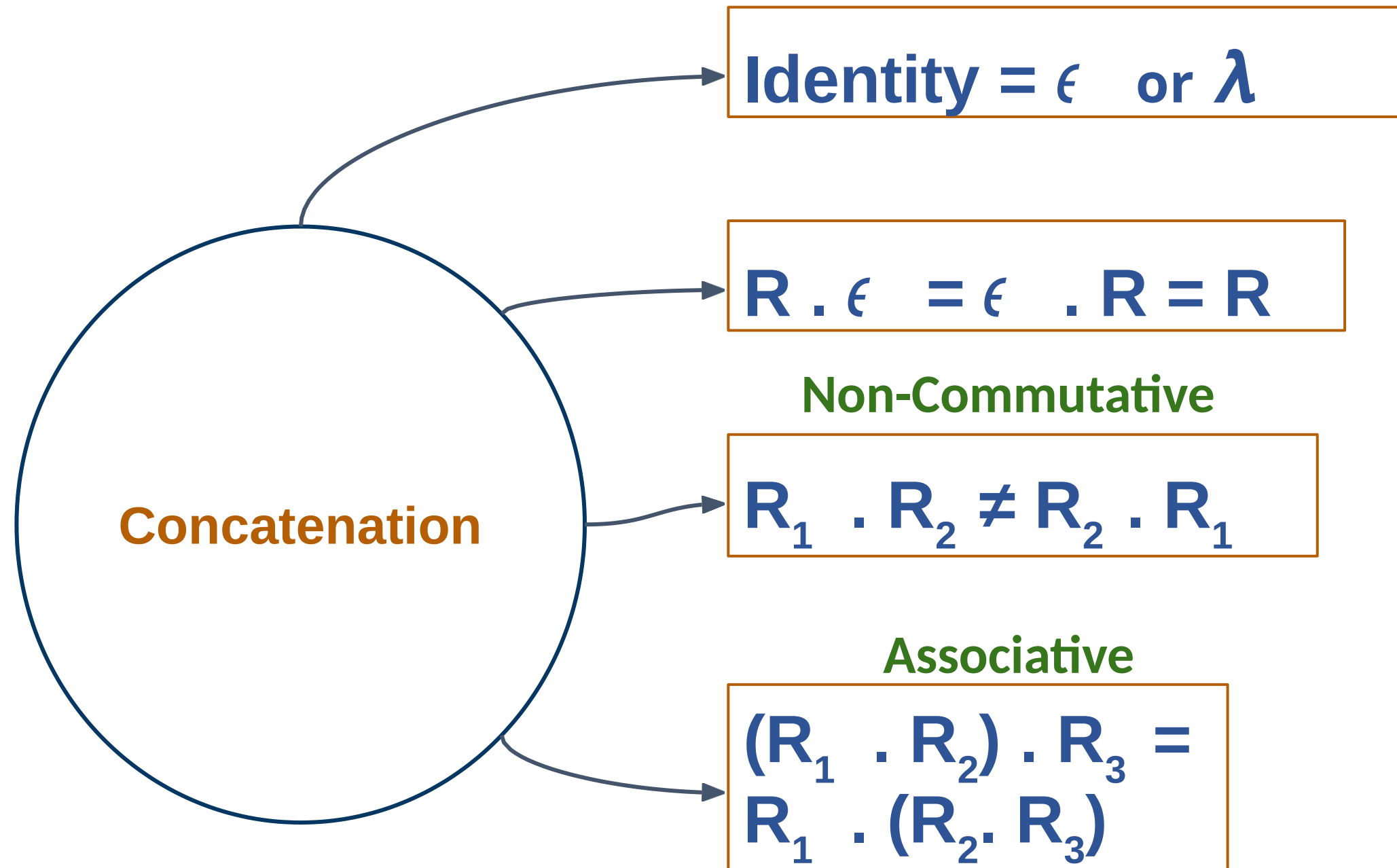
$$R \cdot \phi = \phi \cdot R = \phi$$

Let L_1, L_2 be languages, then the concatenation $L_1 \circ L_2 = \{w \mid w = xy, x \in L_1, y \in L_2\}$. If $L_2 = \emptyset$, then there is no string $y \in L_2$ and so there is no possible w such that $w = xy$. Thus for any L_1 , we'll have $L_1 \circ \emptyset = \emptyset$.

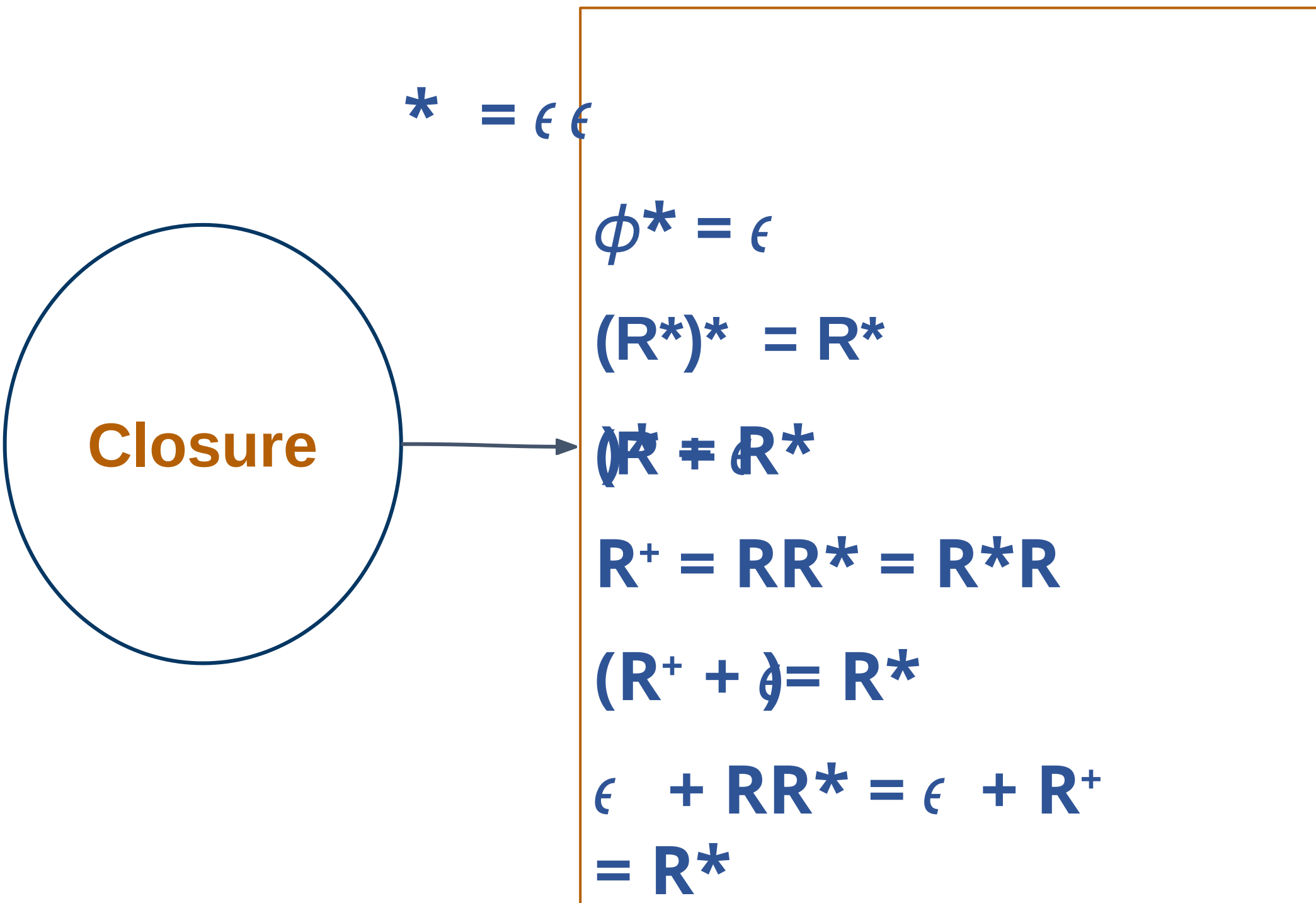
Unit 1 - Algebraic Laws of Regular Expressions



Unit 1 - Algebraic Laws of Regular Expressions



Unit 1 - Algebraic Laws of Regular Expressions



Unit 1 - Algebraic Laws of Regular Expressions

Closure

$$\begin{aligned} & (a + b)^* \\ &= (a^* + b^*)^* \\ &= (a^* b^*)^* \\ &= (a + b^*)^* \\ &= (a^* + b)^* \\ &= a^*(ba^*)^* \\ &= b^*(ab^*)^* \end{aligned}$$

Construct a Regular Expression for a given language L



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Regular Expression in Practice

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. `.`
(dot)
matches any
single
character
except
newline

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
(Star)
0 or
more
repet
itions
of
prece
ding
regex

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *

Example :

a^*

Strings

matched

:

0 or more

no. of a's

Example :

λ, a, aa, aa

$a, aaaa, aa$

$aaa....$

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +

Example :

a+

Strings

matched :

1 or more no.

of a's

Example :

a,aa,aaa,aaaa,

aaaaa....

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?

Example:

ab?c

matches

abc or ac

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .

2. *

3. +

4. ?

5. []

Character

Class

Matches any

single

character

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?
5. []

Example

[cat]

String
matche

d : c

or a

or

t

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

^ (caret) :

Example:

Example: ^abc matches "a" at the start of the string.

[^abc]: This pattern matches any character except a or b or c.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

\$ (dollar) :

Example :

- > `abc$` matches "c" at the end of a line.
- > `^$` matches the empty string.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$
8. { } (Syntax - $R\{m,n\}$)

Example of { }

$a\{0, \}$ - same as a^*

$a\{1, \}$ - same as

a^+

$a\{2,3\}$ matches "aa" or "aaa".

$a\{3\}$ matches aaa only

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$
8. { } (Syntax - $R\{m,n\}$)
9. \d - matches
any digit
between [0-9]
 - a. \d{9}
matches

Unit 1 - Regular Expression in Practice

Basic regular expressions

$(a+b)$ can also be represented as $(a|b)$ This regular expression means either a or b $L=\{a,b\}$

$(a+b)^*$ Set of strings greater than equal to 0 over the alphabet a or b $L=\{ \lambda, a, b, aa, bb, ab, ba, aab, \dots \}$

$(a+b)^0$ String of length 0 $L=\{\epsilon\} = \Sigma^0$

$(a+b)^1$ Set of all strings with length 1 $L=\{a,b\} = \Sigma^1$

$(a+b)^2$ Set of all strings of length 2 $(a+b)(a+b)$ or $L=\{aa, ab, ba, bb\} = \Sigma^2$

Unit 1 - Regular Expression in Practice

Obtain regular expression for the following over the alphabet $\Sigma\{a,b\}^*$:

1) Strings ending with ab

The string should end with ab, and it can start with any number of a's and b's in any order of any length

RE= $(a+b)^*ab$

2) Strings starts with ab

The string should start with ab, and it can end with any number of a's and b's in any order of any length

RE= $ab(a+b)^*$

3) Strings starting and ending with ab over $(a+b)^*$

RE = _____

Unit 1 - Regular Expression in Practice

Exercise : Obtain regular expression for the following over the alphabet $\Sigma\{a,b\}^*$:

1. String contains ab
2. String should end with ab or ba
3. Strings that start and end with same symbol
4. Strings that start and end with different symbol
5. String where the 3rd symbol from the left (from starting) is 'a'(LHS)
6. String where the 2nd symbol from the right (from ending) is a(RHS)
7. String where at least one 'a' in the first three symbols and at least one 'b' in the last two Symbols.
8. Strings contain at least one a and at least one b.

Unit 1 - Regular Expression in Practice

Exercise Solution : Obtain regular expression for the following over the alphabet $\Sigma\{a,b\}^*$:

1. $RE = (a+b)^* ab (a+b)^*$
2. $RE = (a+b)^* (ab+ba)$
3. $RE = a (a+b)^* a + b(a+b)^* b$
4. $RE = a(a+b)^* b + b(a+b)^* a$
5. $RE = (a+b)(a+b)a(a+b)^*$
6. $RE = (a+b)^* a (a+b)$
7. $RE = (a(a+b)(a+b)) + ((a+b) a(a+b)) + (a+b)(a+)a (a+b)^* (b(a+b)) + (a+b)b$
8. $RE = (a+b)^* a (a+b)^* b (a+b)^* + (a+b)^* b (a+b)^* a (a+b)^*$

Unit 1 - Regular Expression in Practice

Exercise : Obtain regular expression for the following over the alphabet $\Sigma\{0,1\}^*$:

1. Length of the string is exactly equal to 2 or $|w|=2$

$(0+1)(0+1)$

2. Length of the string is greater than or equal to 2 (at least 2) or $|w| \geq 2$

$(0+1)(0+1)(0+1)^*$

3. Length of the string is at most 2 or $|w| \leq 2$

Here you can have strings of length $\{0,1,2\}$

$(\lambda + a + b + aa + ab + ba + bb)$

$= (\lambda + (a+b))(\lambda + (a+b))$

Mod counter questions:

1. Length of the string is even $|w| \bmod 2 = 0$

Generate strings in multiples of 2

RE = $((a+b)(a+b))^*$

2. Length of the string is odd

3. Length of the string $|w| \bmod 3 = 0$

4. Length of the string $|w| \bmod 3 = 2$

Unit 1 - Regular Expression in Practice

Restriction to the individual symbols $\Sigma = \{a, b\}$

1. Number of a's in the strings is equal to 2 or

$$L = \{ n_a(w) = 2, w \in \{a, b\}^* \}$$

$$\text{RE} = b^* a b^* a b^*$$

2. Number of a's in the string should be greater than or equal to 2

$$L = \{ n_a(w) \geq 2 \}$$

$$\text{RE} = \underline{\hspace{2cm}}$$

3. Number of a's in the string should be at most 2

$$L = \{ n_a(w) \leq 2 \}$$

$$\text{RE} = \underline{\hspace{2cm}}$$

Unit 1 - Regular Expression in Practice

Mod counter questions:

1. $L = \{ n_a(w) \bmod 2 = 0 \}$

$$RE = (b^* a b^* a b^*)^* + b^*$$

2. $L = \{ n_a(w) \bmod 2 = 1 \}$

$$RE = (b^* a b^* a b^*)^* b^* a b^*$$

3. $L = \{ a^n b^m : n+m \text{ is even} \}$

$$RE = (aa)^*(bb)^* + (aa)^*a(bb)^*b$$

4. $L = \{ a^{2n} b^{2m} : n, m \geq 1 \}$

The minimum string is aabb

$$RE = aa(aa)^* bb(bb)^*$$

Unit 1 - Regular Expression in Practice

Other examples:

1. $L = \{a^n b^m : n \geq 4 \text{ and } m \leq 3\}$

Number of a's is at least 4 and the number of b's is at most 3

RE= _____

2. $L = \{a^n b^m c^k : n+m \text{ is odd and } k \text{ is even}\}$

Note: odd+even=odd and even+odd=odd

RE = $((aa)^* a (bb)^* + (aa)^* (bb)^* b) (cc)^*$

3. $L = \{a^n b^m c^k : n \leq 4, m \geq 2, k \leq 2\}$

RE= _____

Unit 1 - Regular Expression in Practice

Other examples:

1. $L = \{aba, a^5, aba^7, a^{11}, aba^{13}, \dots\}$

RE = $(aba + a^5)(a^6)^*$ or $(aba + aaaaa)(aaaaaa)^*$

or RE = $(a+b)(a+b)(a+b)^*(a+b)(a+b)$

2. $L = \{VUV : U, V \in \{a,b\}^* \text{ and } |V| = 2\}$

RE = $(a+b)(a+b)(a+b)^*(a+b)(a+b)$

3. $L = \{\text{Strings of a's and b's which do not contain two consecutive a's and two consecutive b's.}\}$

RE = $(b+\lambda)ab(a+\lambda)$ or RE = $(a+\lambda)ba(b+\lambda)$

Unit 1 - Regular Expression in Practice

Other examples:

1. $L = \{aba, a^5, aba^7, a^{11}, aba^{13}, \dots\}$

RE = $(aba + a^5)(a^6)^*$ or $(aba + aaaaa)(aaaaaa)^*$

or RE = $(a+b)(a+b)(a+b)^*(a+b)(a+b)$

2. $L = \{VUV : U, V \in \{a,b\}^* \text{ and } |V| = 2\}$

RE = $(a+b)(a+b)(a+b)^*(a+b)(a+b)$

3. $L = \{\text{Strings of a's and b's which do not contain two consecutive a's and two consecutive b's.}\}$

RE = $(b+\lambda)ab(a+\lambda)$ or RE = $(a+\lambda)ba(b+\lambda)$

Unit 1 - Regular Expression in Practice

Regular expressions on binary strings $\Sigma=\{0,1\}$

- 31) If the binary string is even then the length of the string is even, and if the binary string is odd then the length of the string is odd.**

$$\text{RE} = ((0+1)(0+1))^*(0+1)0 + ((0+1)(0+1))^*1$$

Recognize pattern

- 32) $L=\{ \text{ Binary string whose decimal equivalent is twice an odd number} \}$**

2* Odd no	Decimal value	Binary string
2*1	2	10
2*3	6	110
2*5	10	1010
2*7	14	1110
2*9	18	10010

The pattern what we can observe here is all the binary strings ends in 10

$$\text{RE} = (0+1)^*(10)$$

Regular expressions on binary strings $\Sigma=\{0,1\}$

33) $L=\{\text{binary strings that do not have two consecutive zero}\}$

-> 01 as the building block:

$$\text{RE} = (1+01)^*(0+\lambda)$$

-> 10 as the building block:

$$\text{RE} = (0+\lambda)(1+10)^*$$

34) $L= \{\text{binary strings that have exactly one pair of 0's}\}$

$$\text{RE} = (1+01)^* 00 (1+10)^*$$

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

We will discuss how to construct regex for validating the following :

- **PAN Card**
- **Adhaar Card**
- **Mobile Number**
- **Date**
- **Email Address**

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

- **PAN CARD:**

The valid PAN Card number must satisfy the following conditions:

1. It should be 10 characters long.
2. The first five characters should be any upper case alphabets.
3. The next four-characters should be any number from 0 to 9.
4. The last(tenth) character should be any upper case alphabet.
5. It should not contain any white spaces.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

PAN CARD Regex - $^{[A-Z]\{5\}[0-9]\{4\}[A-Z]\$$

Input: str = "BNZAA2318J"

Output: true

Explanation:

The given string satisfies all the above mentioned conditions.

Input: str = "23ZAABN18J"

Output: false

Explanation:

The given string does not start with upper case alphabets, therefore it is not a valid PAN

Card number.
<https://www.geeksforgeeks.org/how-to-validate-pan-card-number-using-regular-expression/>

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

- **AADHAR NUMBER:**

The valid Aadhar number must satisfy the following conditions:

1. It should have 12 digits.
2. It should not start with 0 and 1.
3. It should not contains any alphabet and special characters.
4. It should have white space after every 4 digits.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

AADHAR Number Regex - `^[2-9]{1}\d{3}\s\d{4}\s\d{4}$`

Input: str = "3675 9834 6012"

Output: true

Explanation:

The given string satisfies all the above mentioned conditions. Therefore it is a valid Aadhar number.

Input: str = "3675 9834 6012 8"

Output: false

Explanation:

The given

string

contains 13

digits.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

- **INDIAN MOBILE NUMBER:**

The valid Mobile number must satisfy the following conditions:

- It is a 10 digits number
- The first digit should contain number between 6 to 9.
- The rest 9 digit can contain any number between 0 to 9.
- The mobile number can have 11 digits also by including 0 at the starting.
- The mobile number can be of 13 digits also by including +91 at the starting

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

MOBILE Number Regex - `^(0|"+91")?[6-9]\d{9}$`

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Dates in the year 2020

- It is a leap year
- Let the format be DD-MM-YYYY

Months with 31 days are : Jan, Mar, May, July, Aug, Oct, Dec
: [1, 3, 5, 7, 8, 10, 12]

Months with 30 days are : Apr, June, Sep, Nov
: [4, 6, 9, 11]

Month with 29 days : Feb
: [2]

- Year : 2020

Dates in the year 2020

Regex:

```
(  
(0[1-9])| [10-31]"-"(0[13578]| 1[02])  
|  
(0[1-9])| [10-30]"-"(0[469]| 11)  
|  
(0[1-9])| [10-29]"-"02"  
)"-""2020"
```

<https://www.sitepoint.com/demystifying-regex-with-practical-examples/>

Regular Expression in practice: Examples

Example 5: -mail addresses Validation

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Email Address:

An email is a string (a subset of ASCII characters) separated into two parts by @ symbol. a "personal_info" and a domain, that is personal_info@domain.

The length of the personal_info part may be up to 64 characters long and domain name may be up to 253 characters.

The personal_info part contains the following ASCII characters.

- Uppercase (A-Z) and lowercase (a-z) English letters.
- Digits (0-9).
- Characters ! # \$ % & ' * + - / = ? ^ _ ` { | } ~
- Character . (period, dot or fullstop) provided that it is not the first or last character and it will not come one after the other.

The domain name [for example com, org, net, in, us, info] part contains letters, digits, hyphens, and dots.

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Email Address:

Example of valid email id

- mysite@ouearth.com
- my.ownsite@ouearth.org
- mysite@you.net

Example of invalid email id

- mysite.ouearth.com [@ is not present]
- mysite@.com.my [tld (Top Level domain) can not start with dot "."]
- @you.me.net [No character before @]
- <https://www.geeksforgeeks.org/write-regular-expressions/>

$$^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)^*@[a-zA-Z0-9-]+(\. [a-zA-Z0-9-]+)^*\.$$
$$(((0-9){1,3})|([a-zA-Z]{2,3})|(com|edu|info|museum|name))\$$$

Regular Expression in practice: Examples

Example 5: -mail addresses Validation

$$^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)^*@[a-zA-Z0-9-]+(\.[a-zA-Z0-9-]+)^*\.[br/>((([0-9]\{1,3\})|([a-zA-Z]\{2,3\})|(com|edu|info|museum|name))\$$$

Regular Expression in practice: Examples

Example 2: Simple URL Validator

<https://www.geeksforgeeks.org/check-if-an-url-is-valid-or-not-using-regular-expression/>

$^((ht|f)tp(s?))\backslash:\/\backslash([0-9a-zA-Z\backslash-]+\backslash.)+[a-zA-Z]\{2,6\}(\backslash:[0-9]+\backslash)?(\backslashS^*)?\$$

Regular Expression in practice: Examples

Example 6: Ip Address validation:

Regular Expression in practice: Examples

Example 6: Ip Address validation:

```
^(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9])\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9]|0)\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9]|0)\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[0-9])$
```

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Multiple Choice Questions

Which of the following regular expressions will match any string that starts with "abc" and ends with "xyz", regardless of what is in between?

- A) `^abcxyz$`
- B) `^abc.*xyz$`
- C) `abc^.*xyz$`
- D) `^abc[xyz]$`

Which regular expression would correctly match a valid Indian mobile number that may optionally start with "+91" or "0"?

- A) `^[6-9]\d{9}$`
- B) `^(0|\+91)?[6-9]\d{9}$`
- C) `^[0-9]{10,13}$`
- D) `^91[6-9]\d{9}$`

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Multiple Choice Questions

In a regular expression, what is the primary purpose of the caret (^) character when placed at the start of a pattern?

- A) It matches any digit
- B) It matches the end of a string
- C) It matches the beginning of a string
- D) It makes the preceding character optional

Which of the following strings will NOT be matched by the regex `^[2-9]{1}\d{3}\s\d{4}\s\d{4}$` for Aadhar number validation?

- A) 3675 9834 6012
- B) 1675 9834 6012
- C) 2675 9834 6012
- D) 3675 9834 6012



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

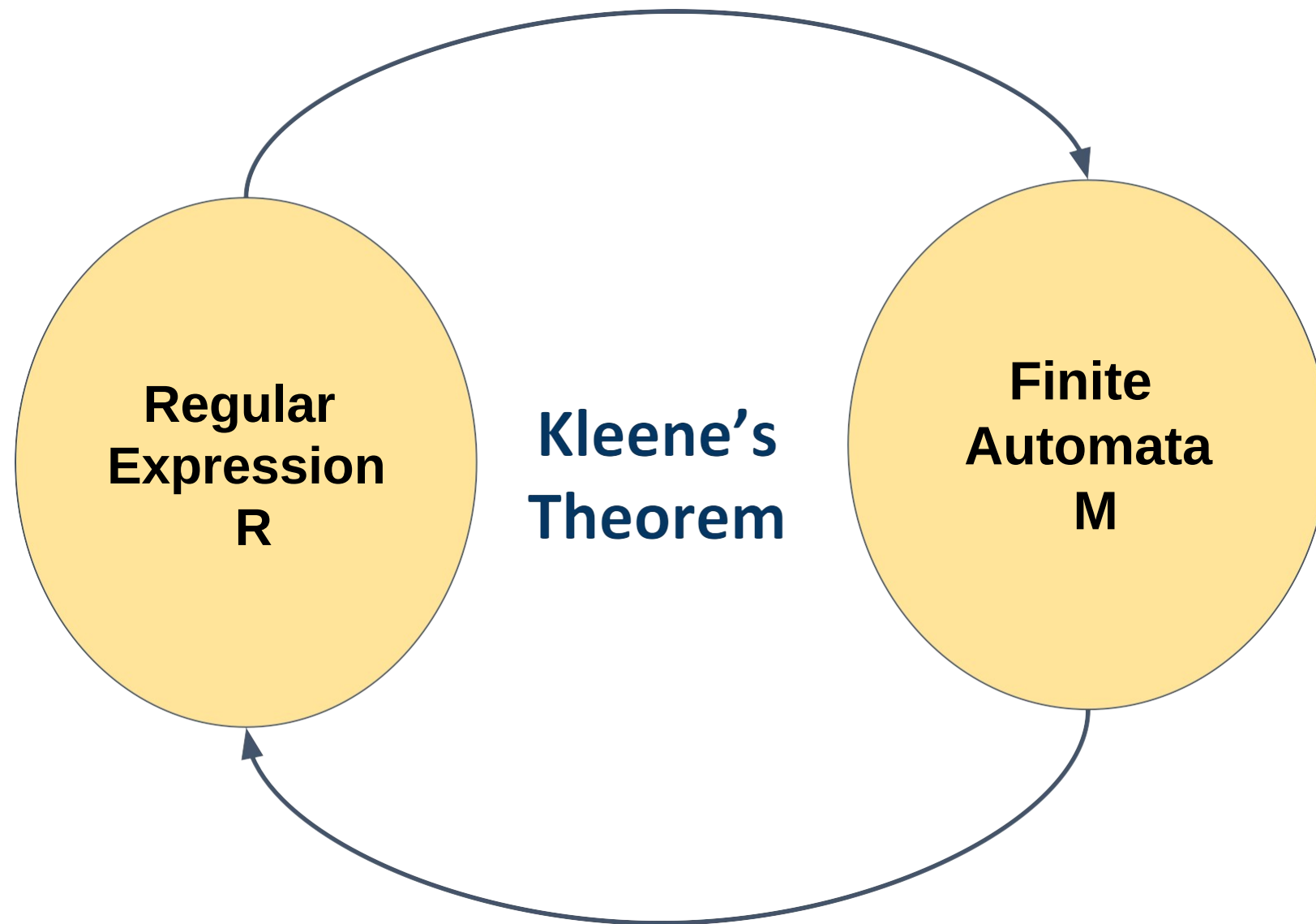
Automata Formal Languages & Logic

Unit 2

Preet Kanwal

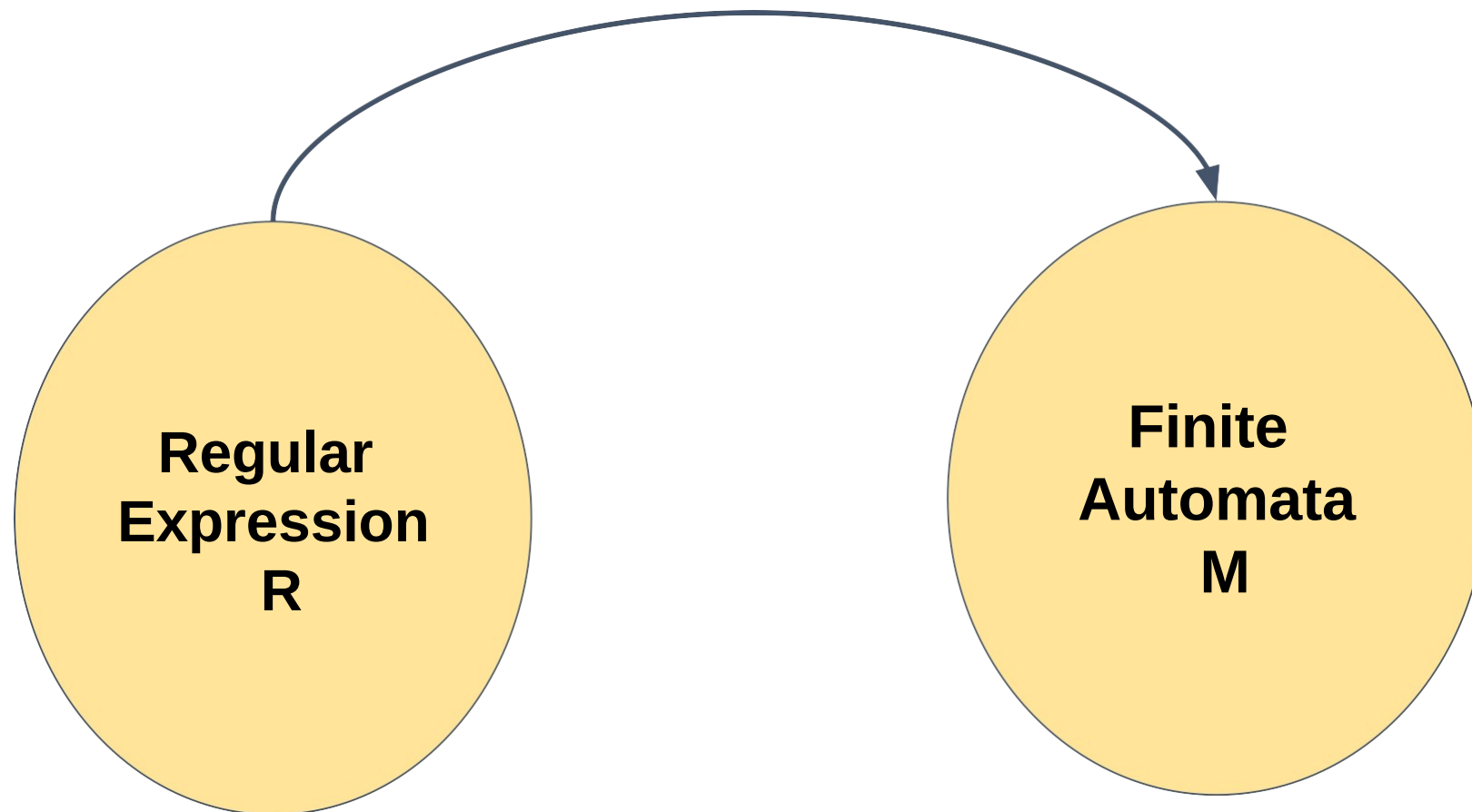
Department of Computer Science & Engineering

Thompson's Construction Algorithm

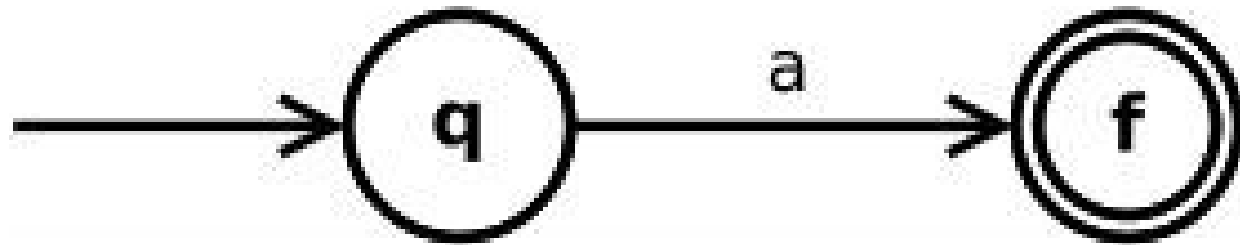
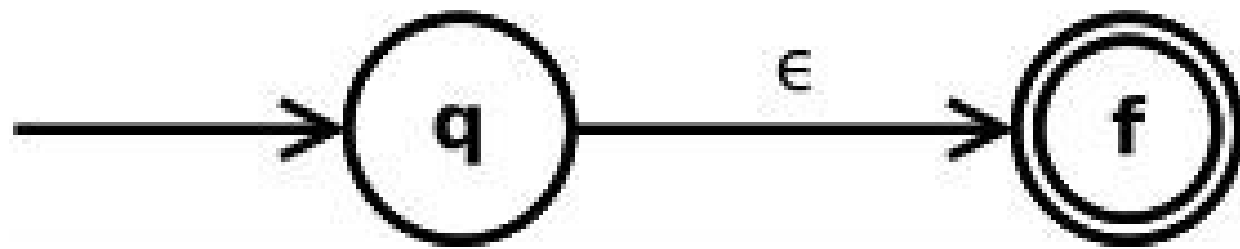


State Elimination Algorithm

Thompson's Construction Algorithm

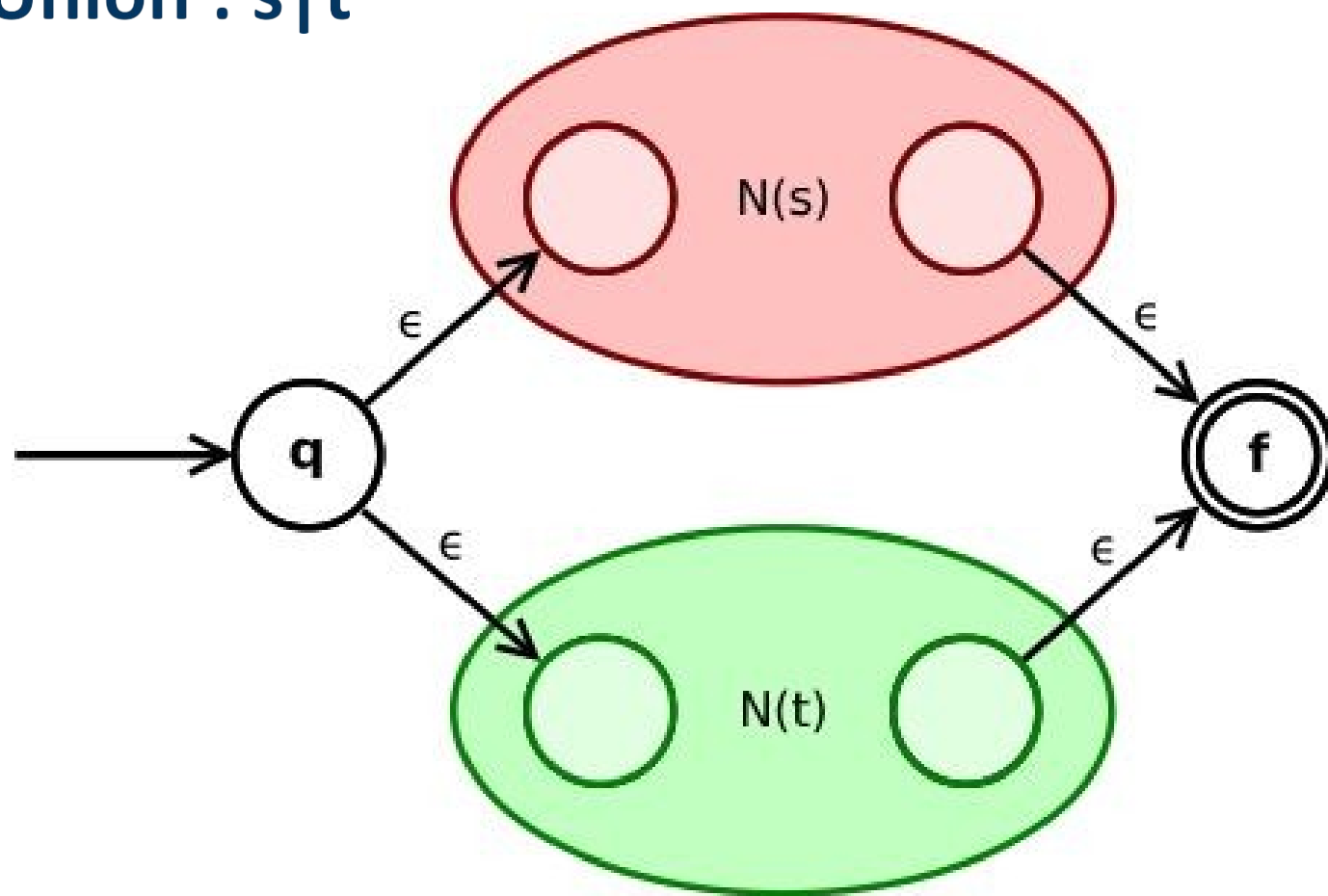


Thompson's Construction Algorithm



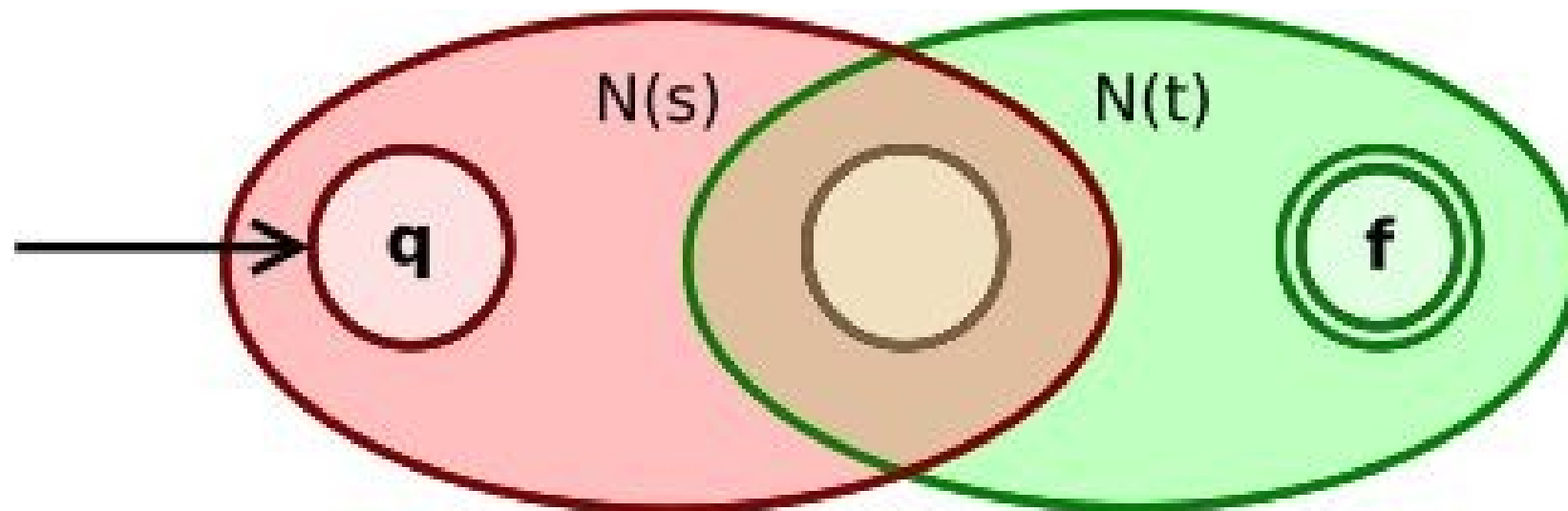
Thompson's Construction Algorithm

Union : $s|t$



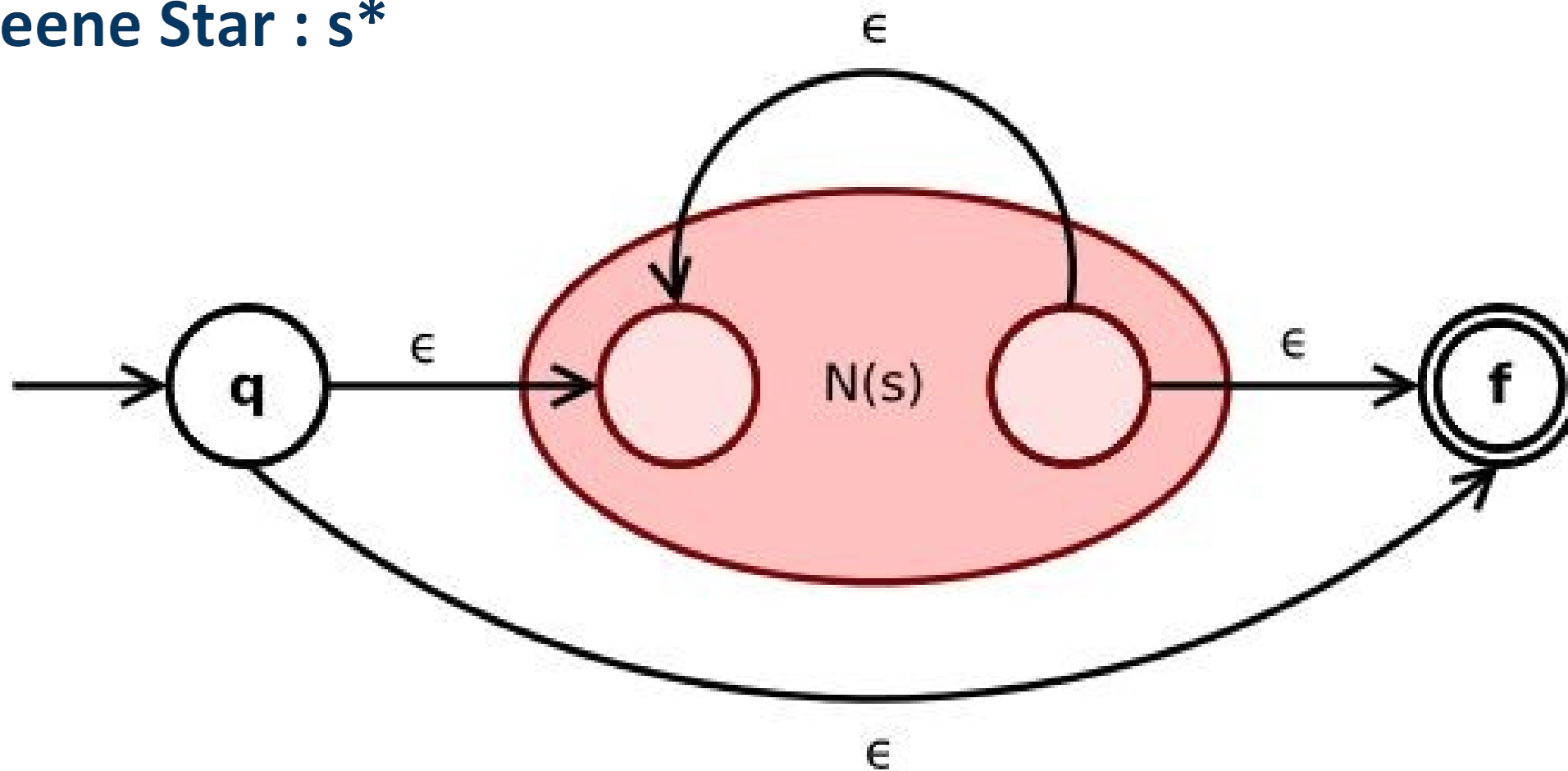
Thompson's Construction Algorithm

Concatenation : $s.t$



Thompson's Construction Algorithm

Kleene Star : s^*

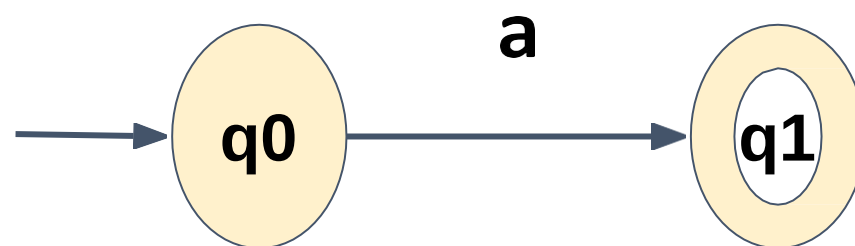


Construct Finite Automata for the following Regex :

$(ab+c)^*$

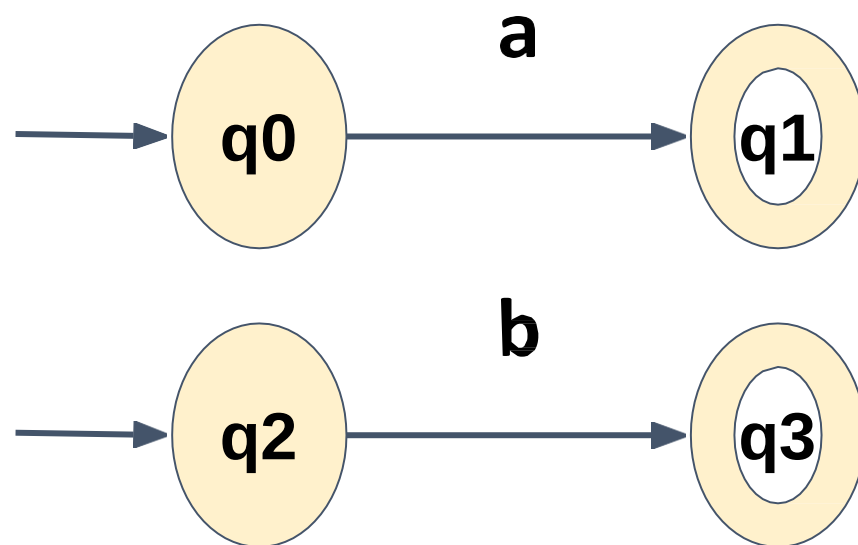
Construct Finite Automata for the following Regex :

$(ab+c)^*$



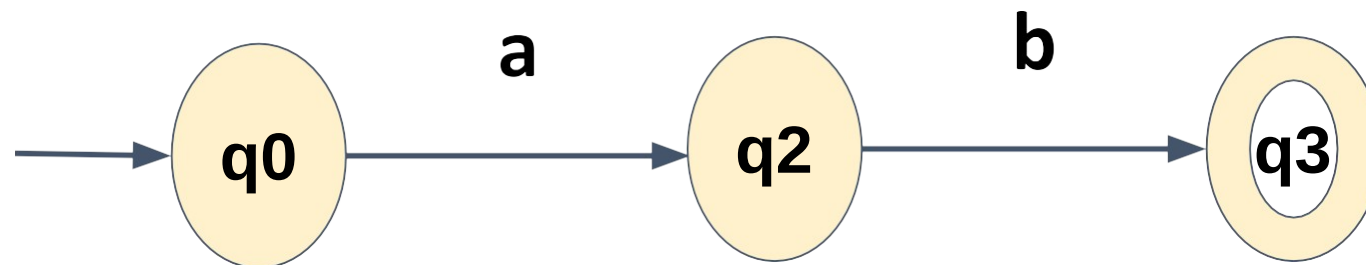
Construct Finite Automata for the following Regex :

$(ab+c)^*$



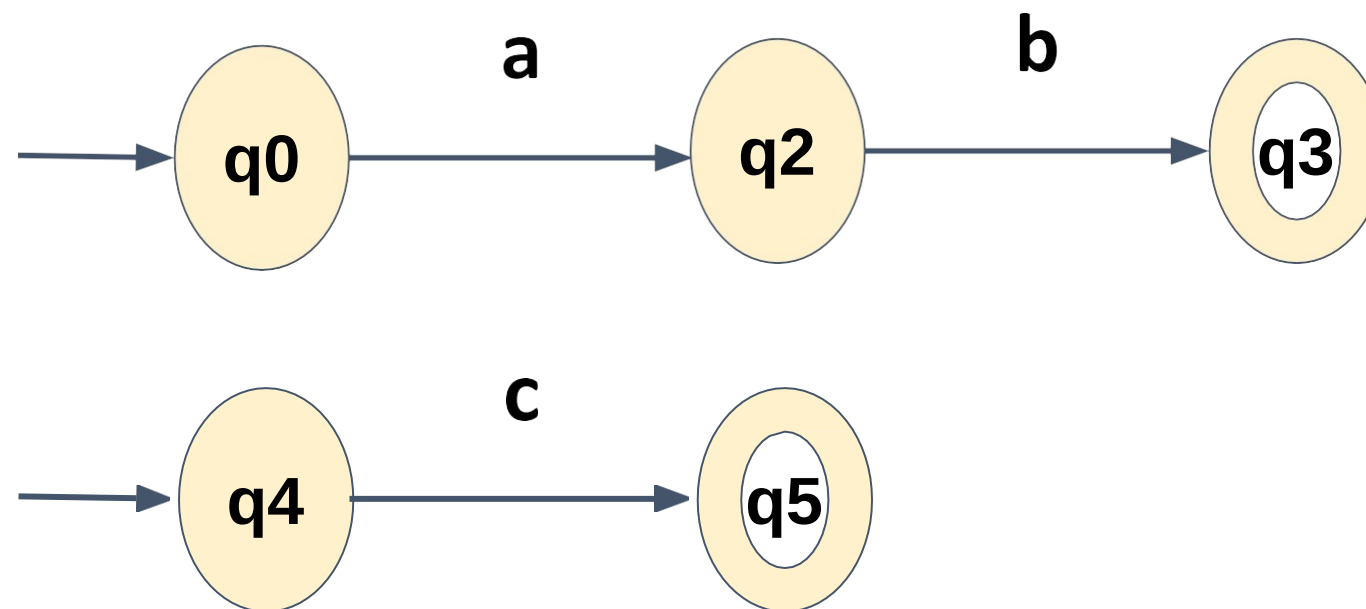
Construct Finite Automata for the following Regex :

$(ab+c)^*$



Construct Finite Automata for the following Regex :

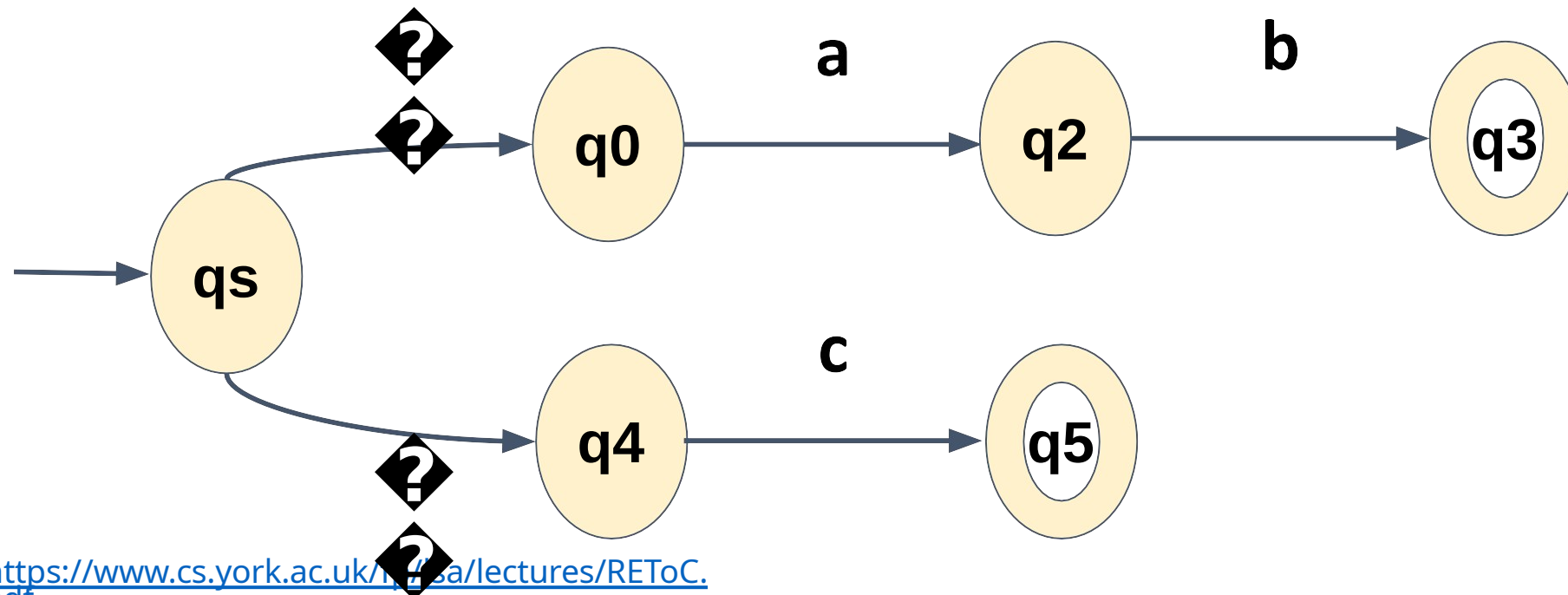
$(ab+c)^*$



Unit 2 - Regular Expression to Finite Automata

Construct Finite Automata for the following Regex :

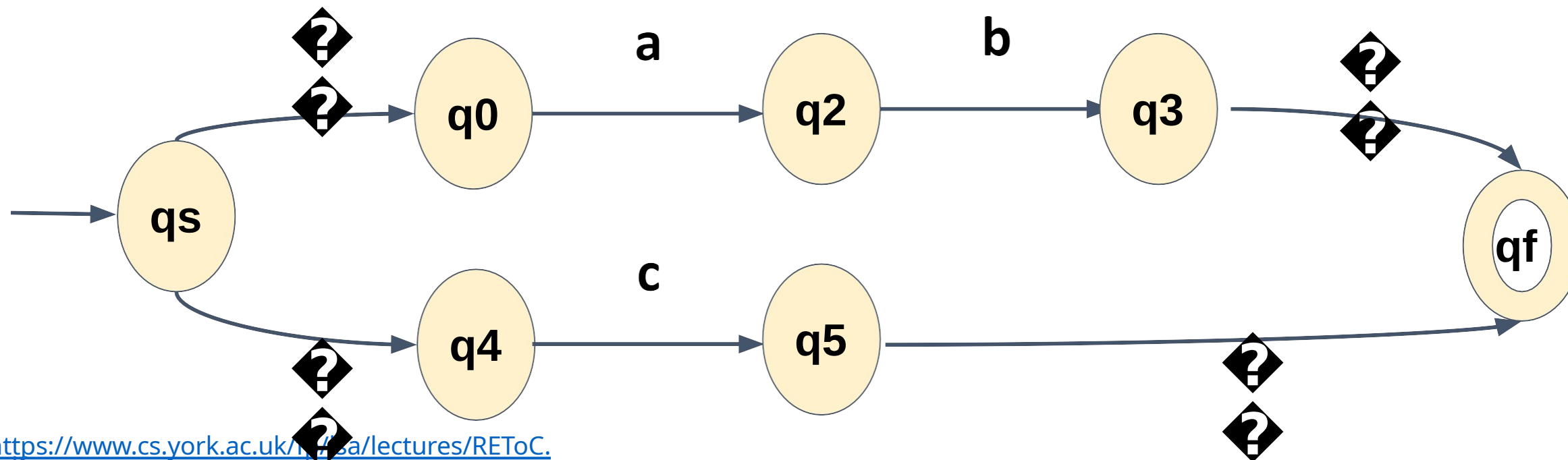
$(ab+c)^*$



Unit 2 - Regular Expression to Finite Automata

Construct Finite Automata for the following Regex :

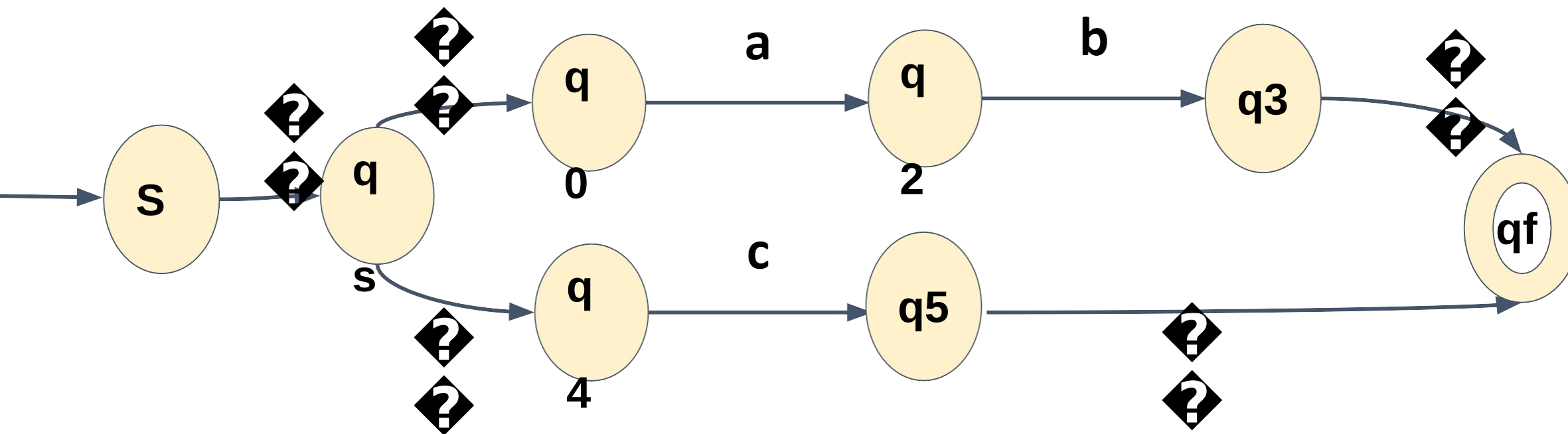
$(ab+c)^*$



Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata

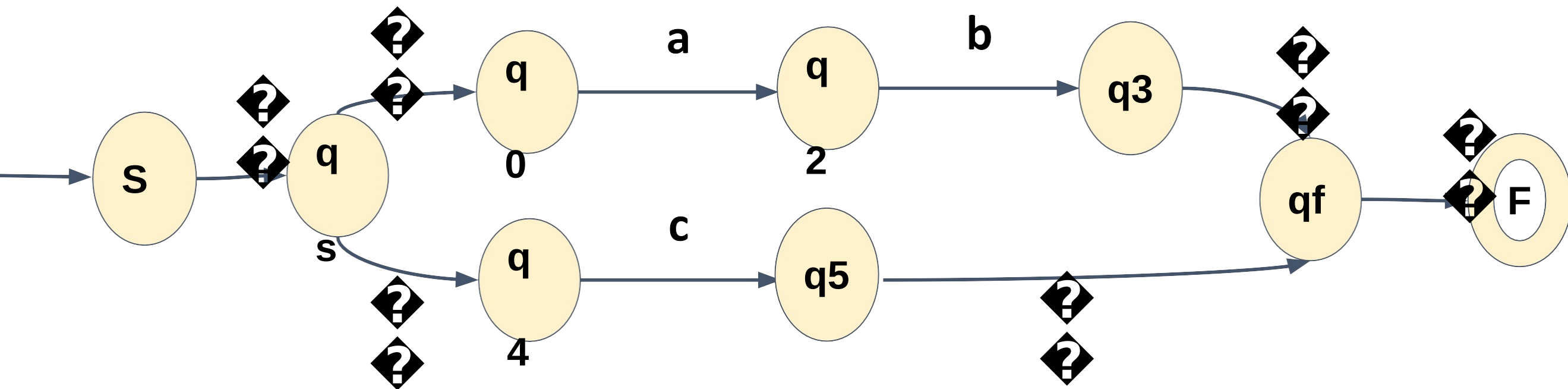
$(ab+c)^*$



Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata

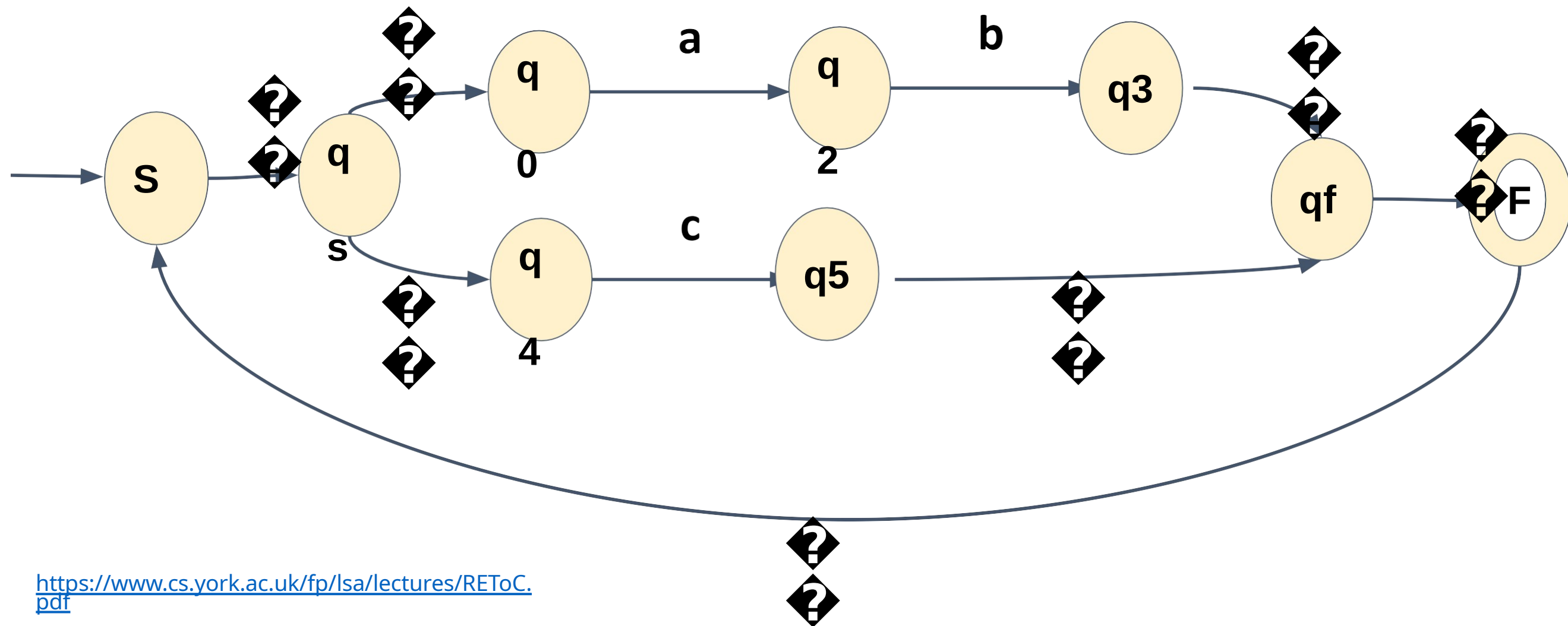
$(ab+c)^*$



Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata

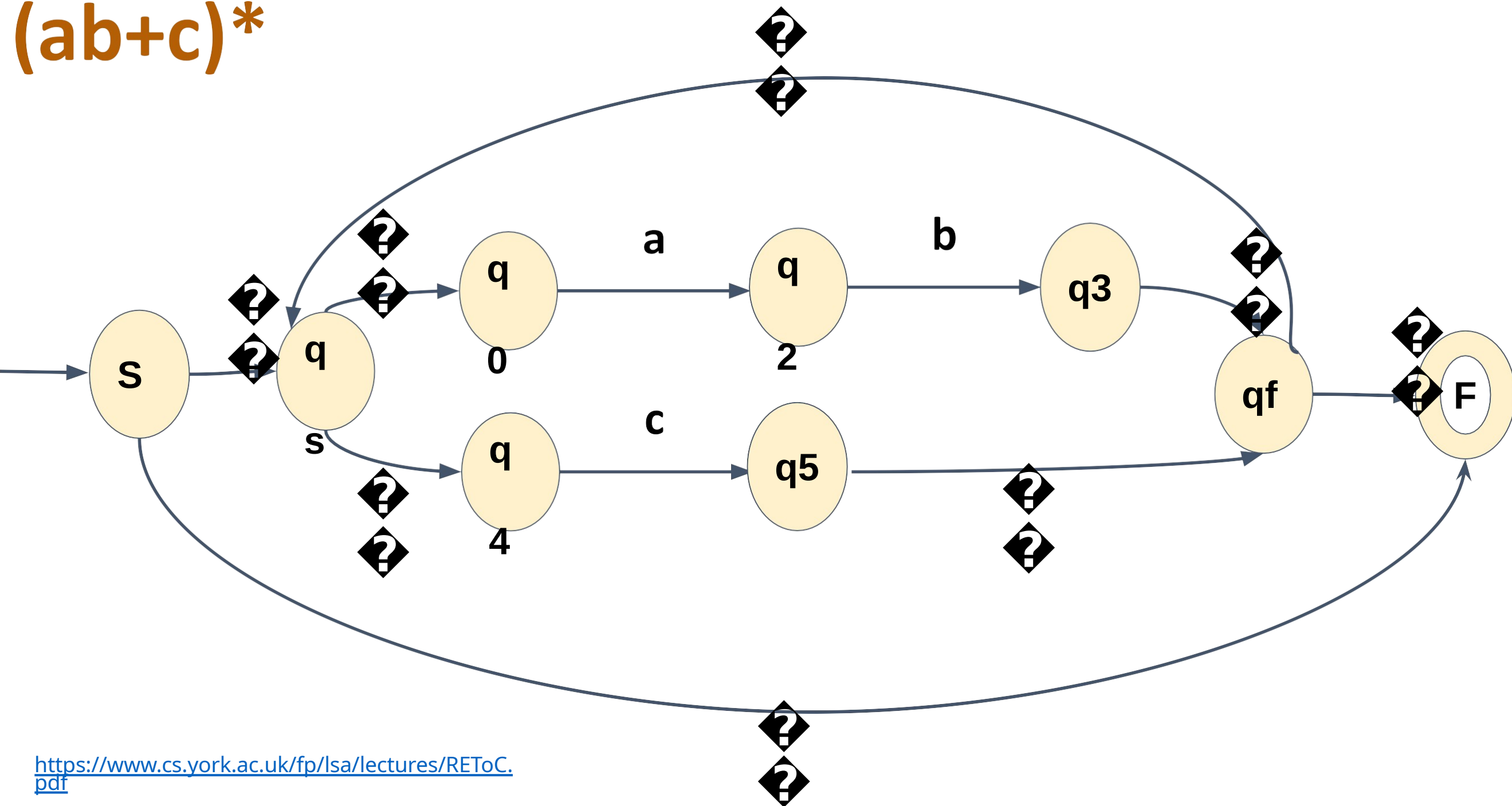
$(ab+c)^*$



Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata

$(ab+c)^*$



Construct Finite Automata for given Regex



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

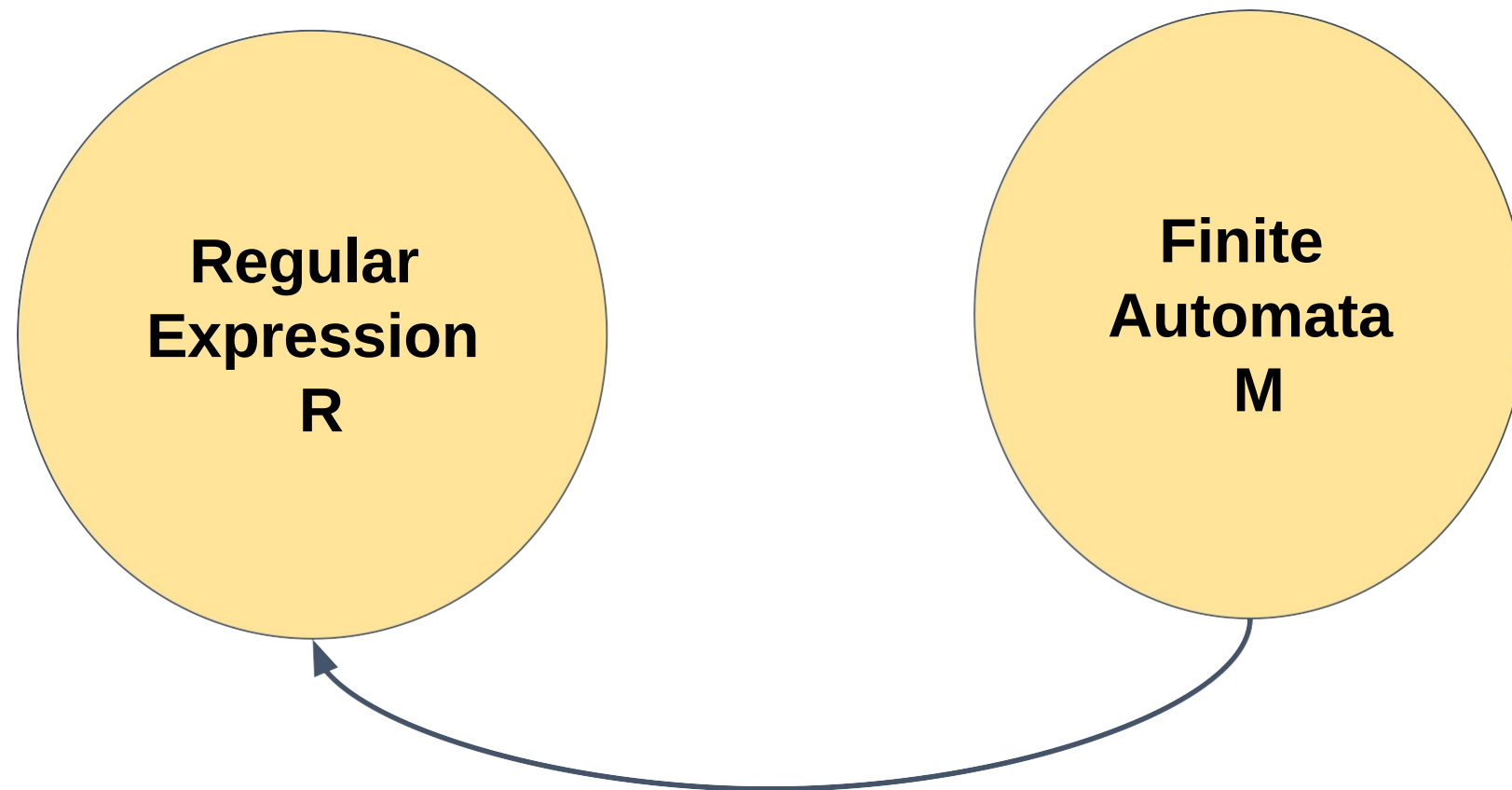
Unit 1

Preet Kanwal

Department of Mechanical Engineering

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression



State Elimination Algorithm

Automata Formal Languages and Logic

Unit 1 - Regular Grammar

State Elimination Method

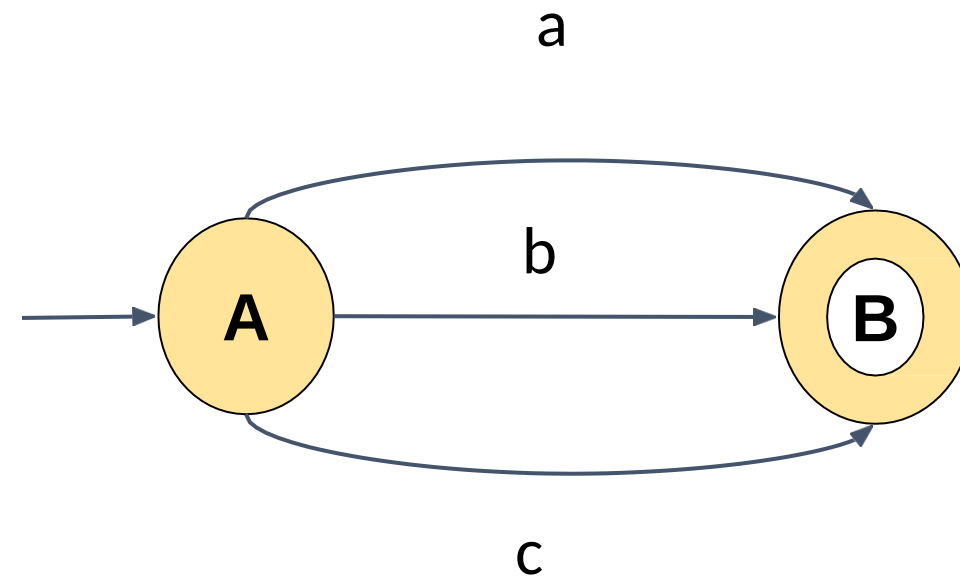
Start with an FA for the language L.

- Add a new start state q_s (if there is any incoming edge to initial state)A new Final state F is introduced (if there is an Outgoing edge from the existing Final State).
- Add ϵ -transitions from each original accepting state to q_f , then mark them as not accepting.
 - Repeatedly remove states other than q_s and q_f from the FA by “shortcutting” them until only two states remain: q_s and q_f .
- The transition from q_s to q_f is then a regular expression for the FA.

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

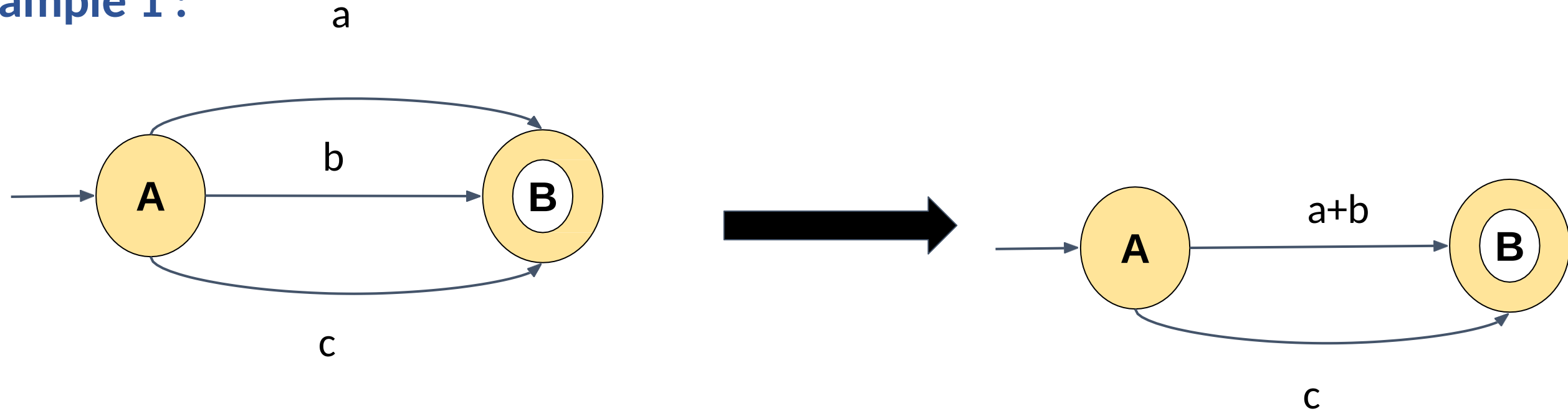
Example 1 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

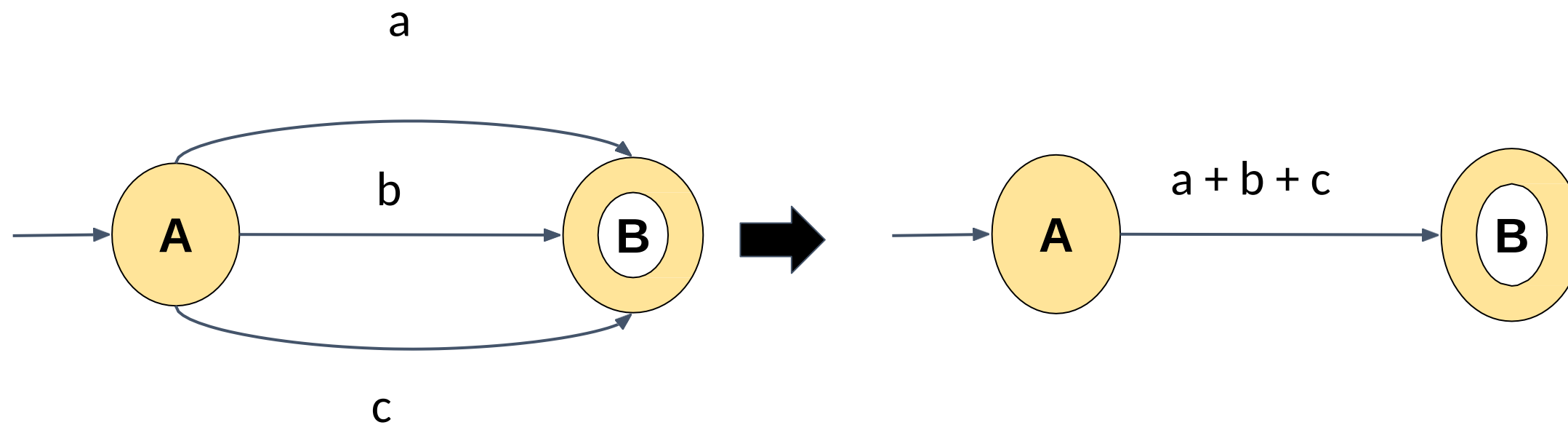
Example 1 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

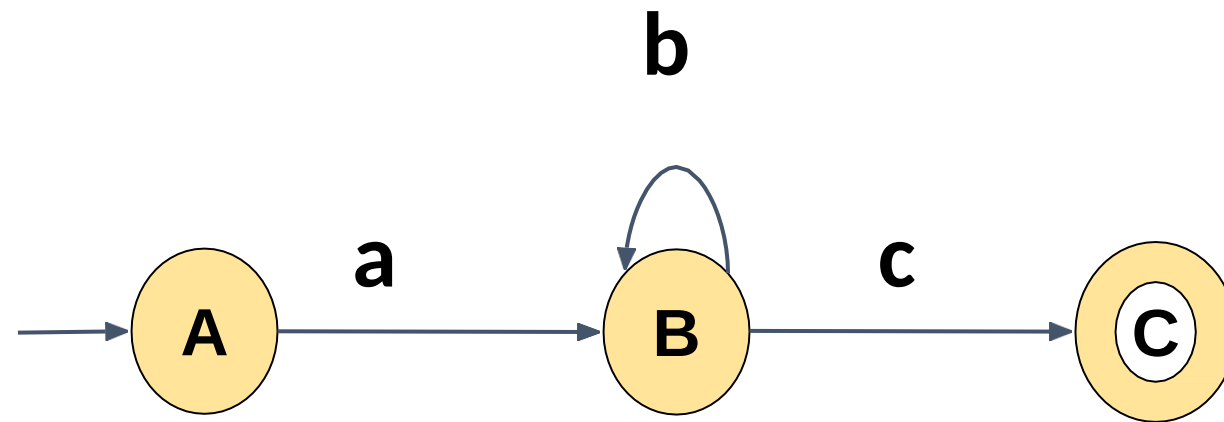
Example 1 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

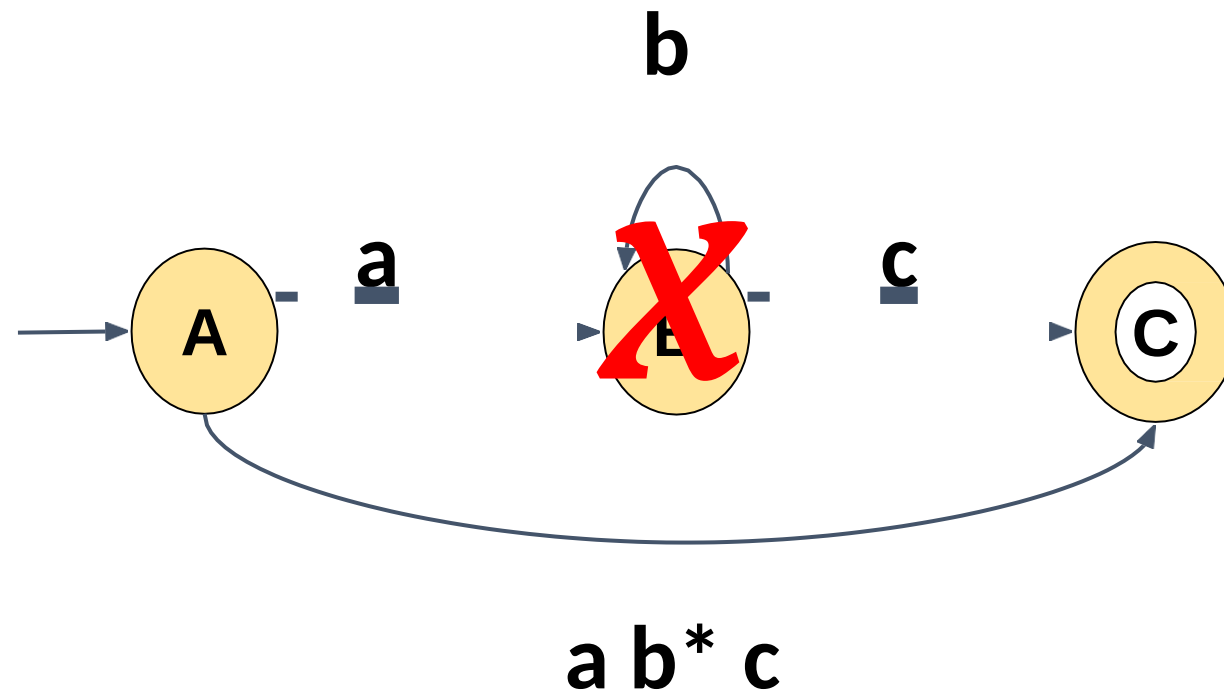
Example 2 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 2 :

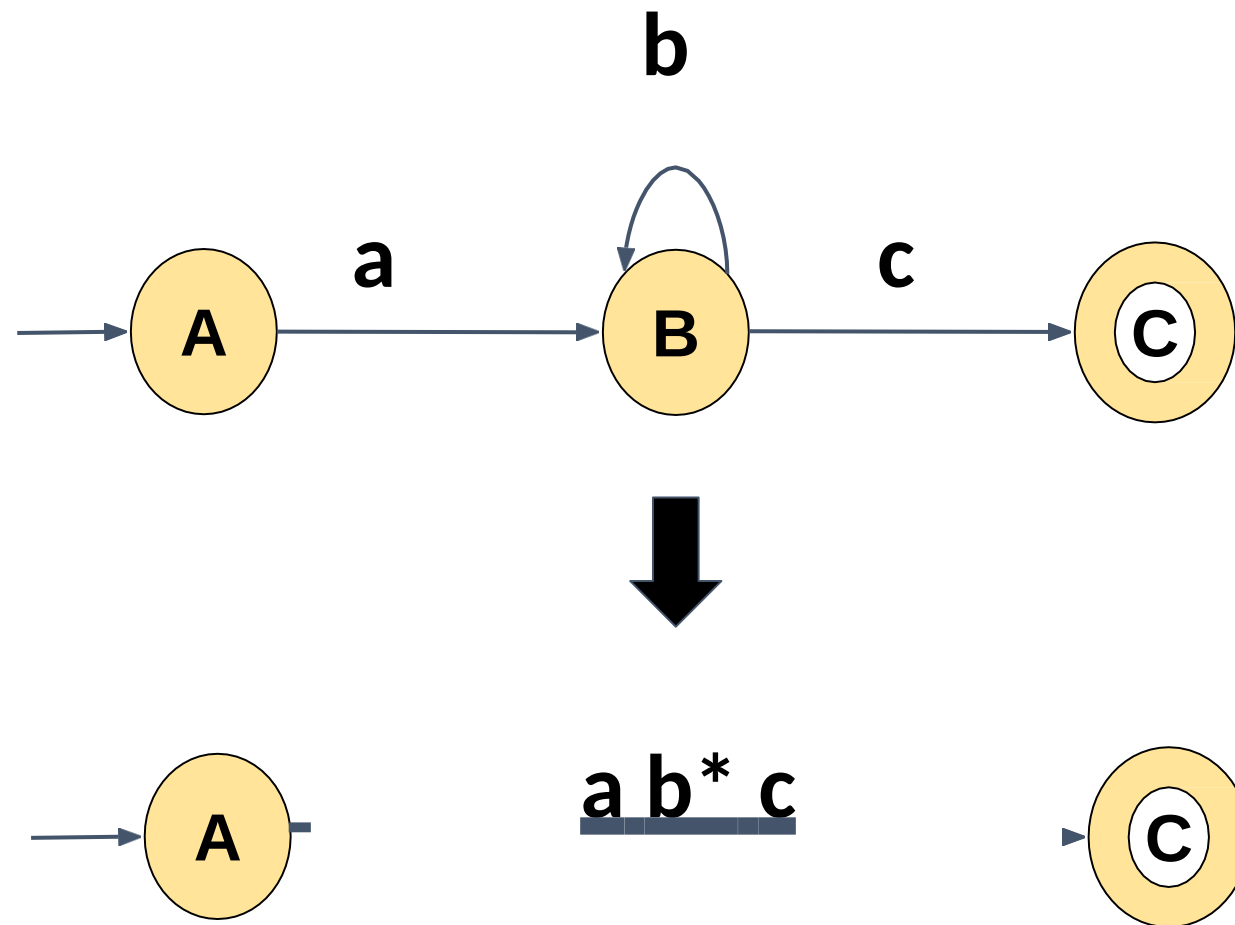


Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

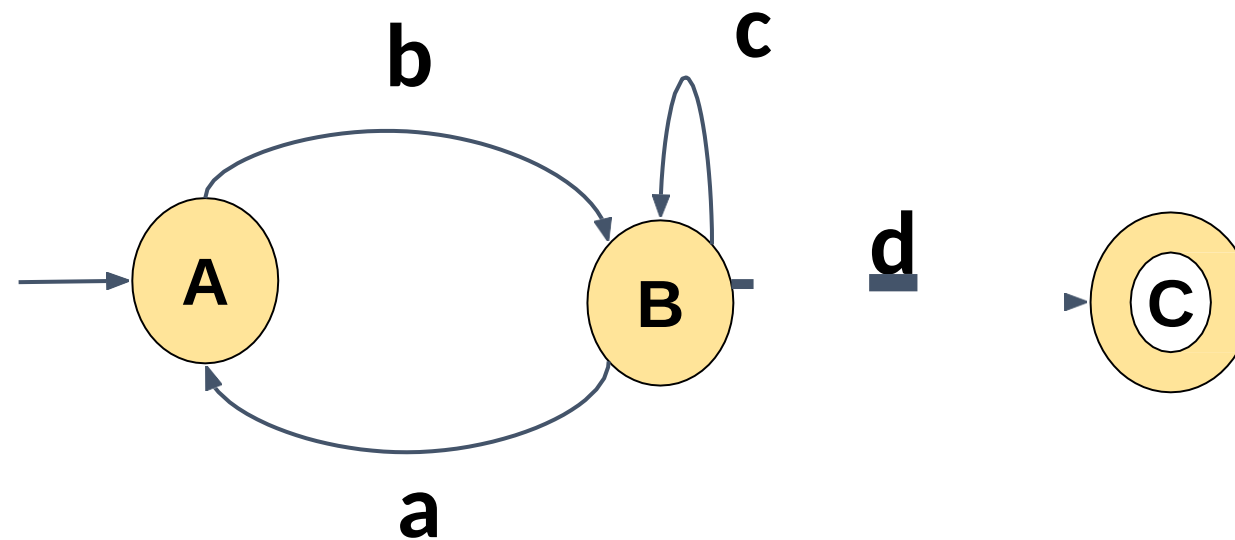
Example 2 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

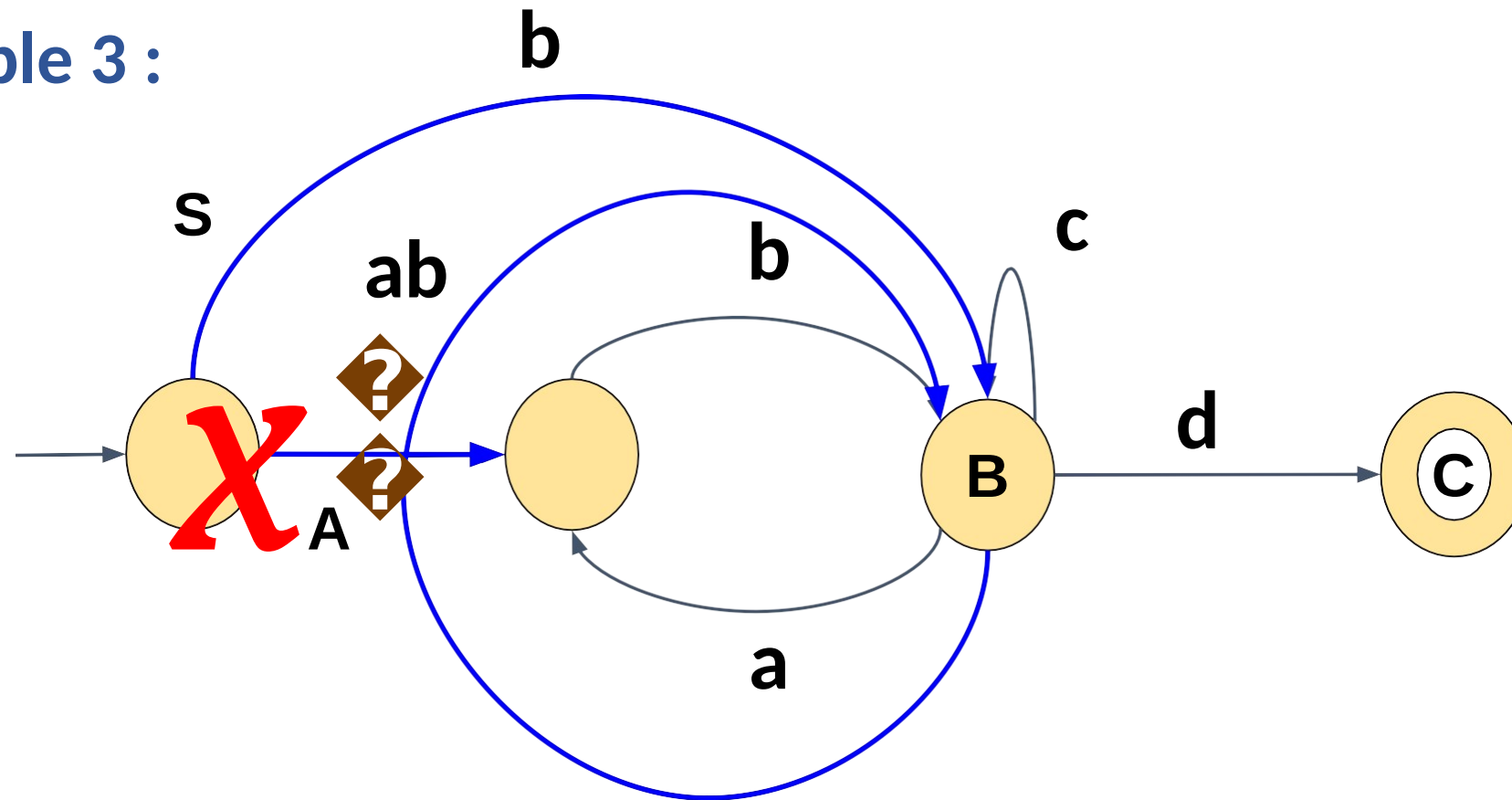
Example 3 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 3 :

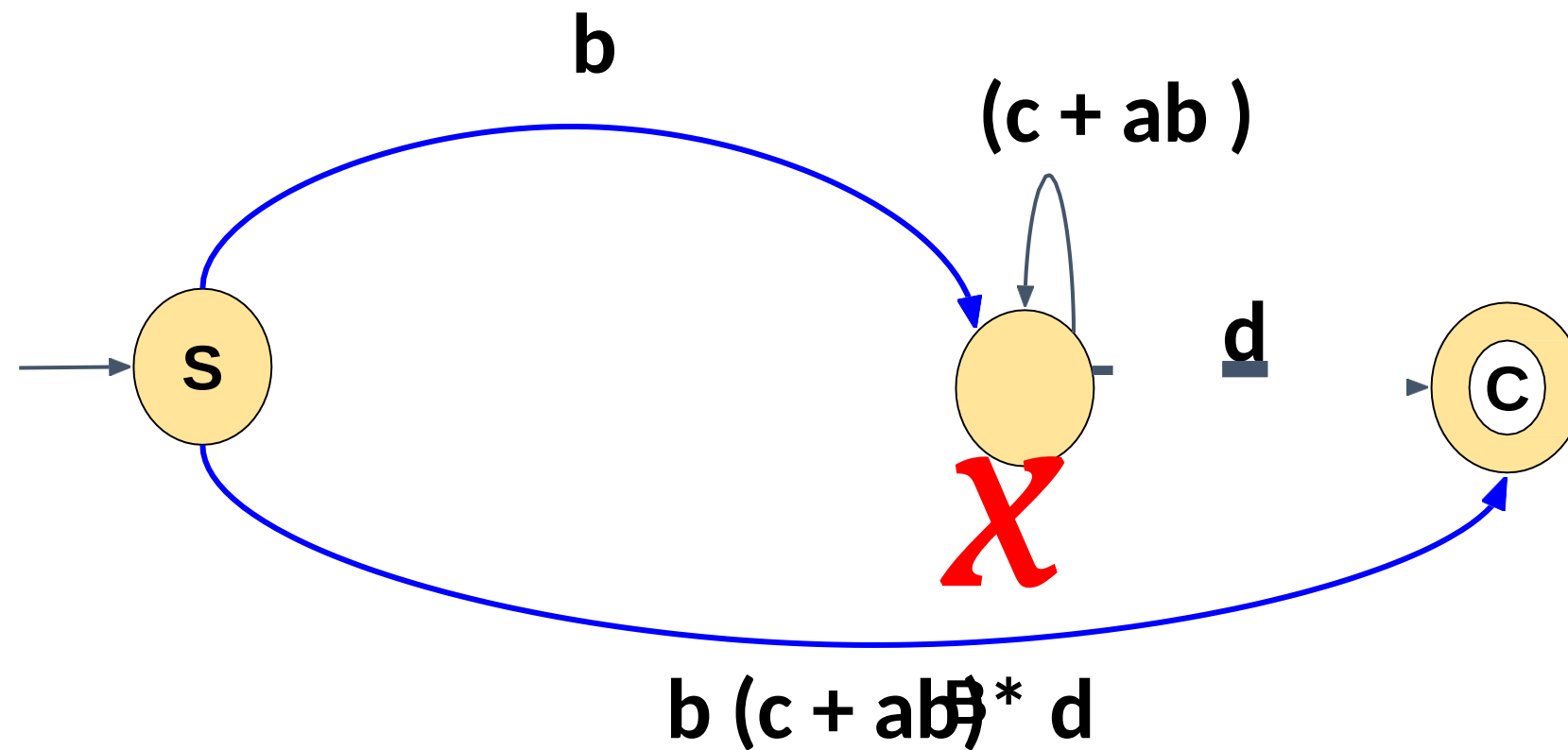


1. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 3 :

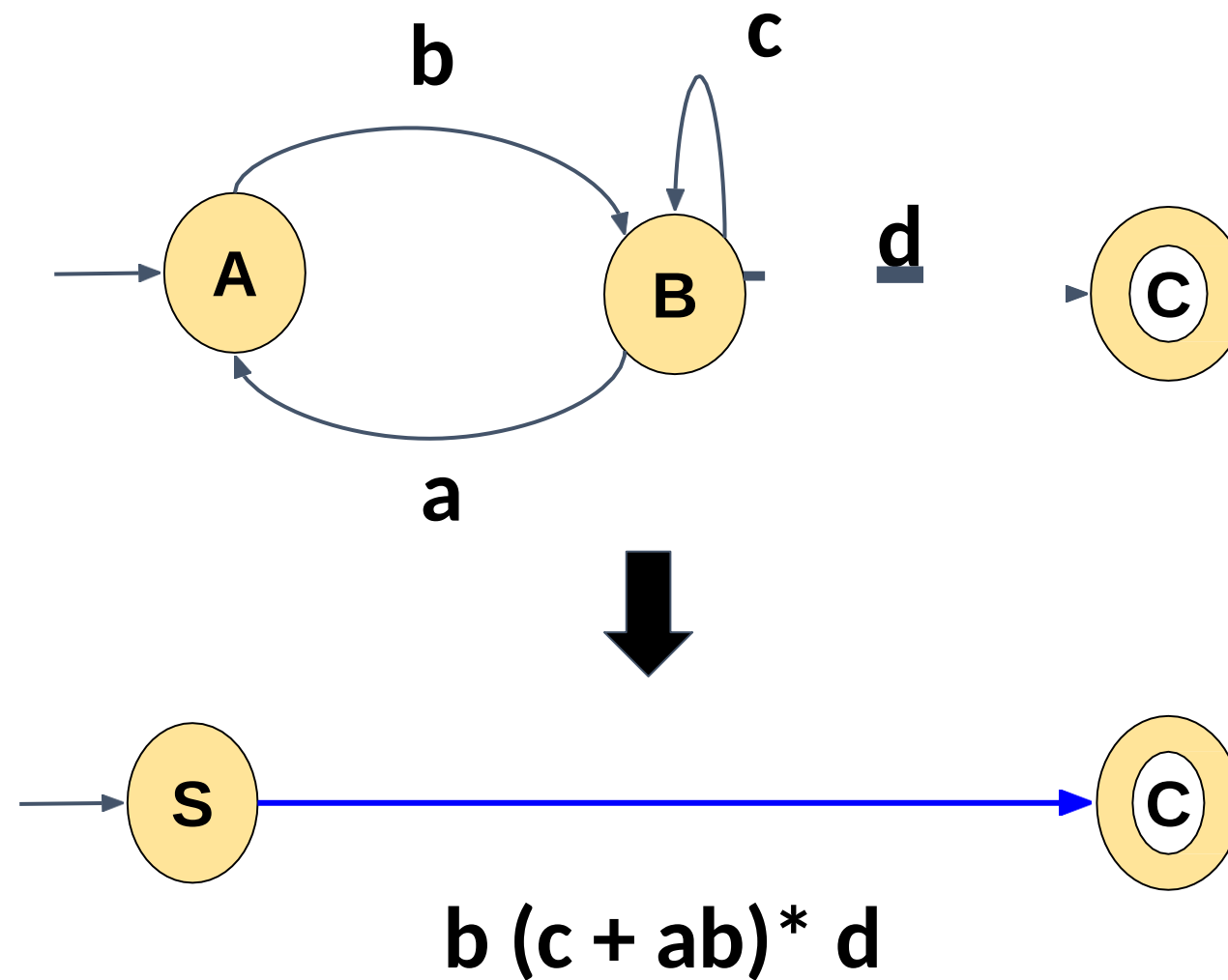


1. Eliminate A
2. Eliminate B

Automata Formal Languages and Logic

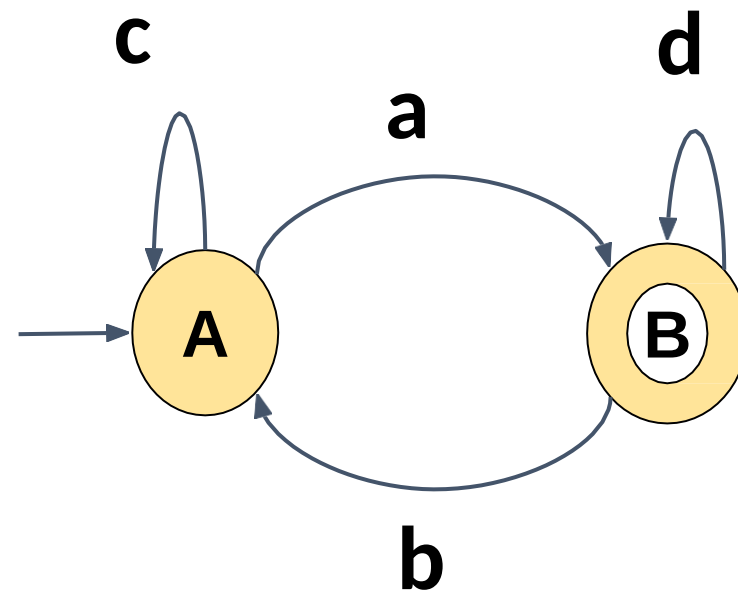
Unit 1 - Finite Automata to Regular Expression

Example 3 :



Unit 1 - Finite Automata to Regular Expression

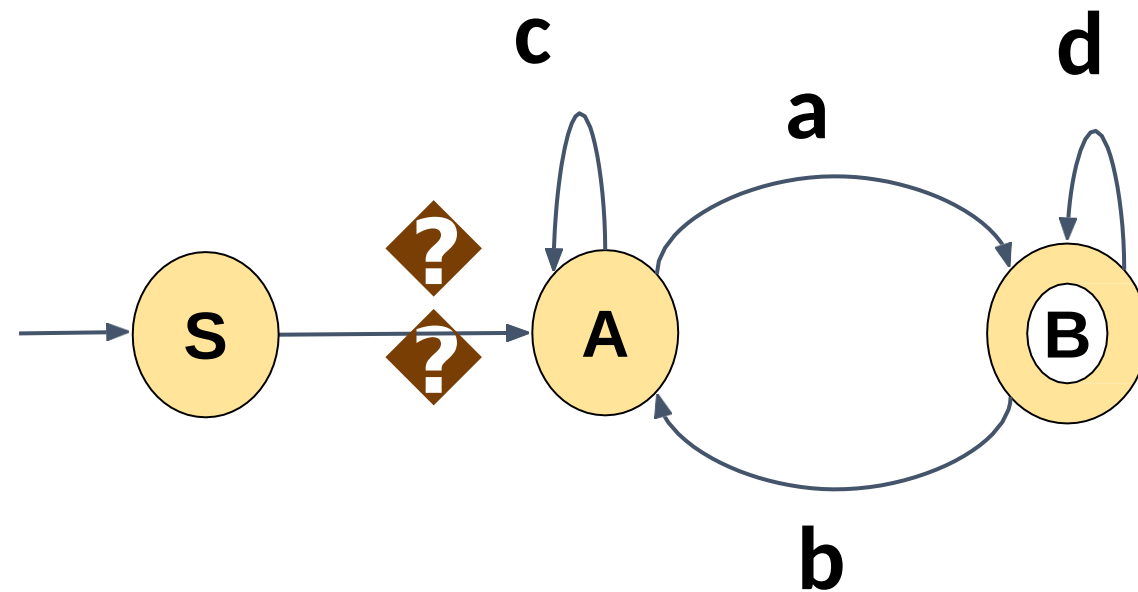
Example 4 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

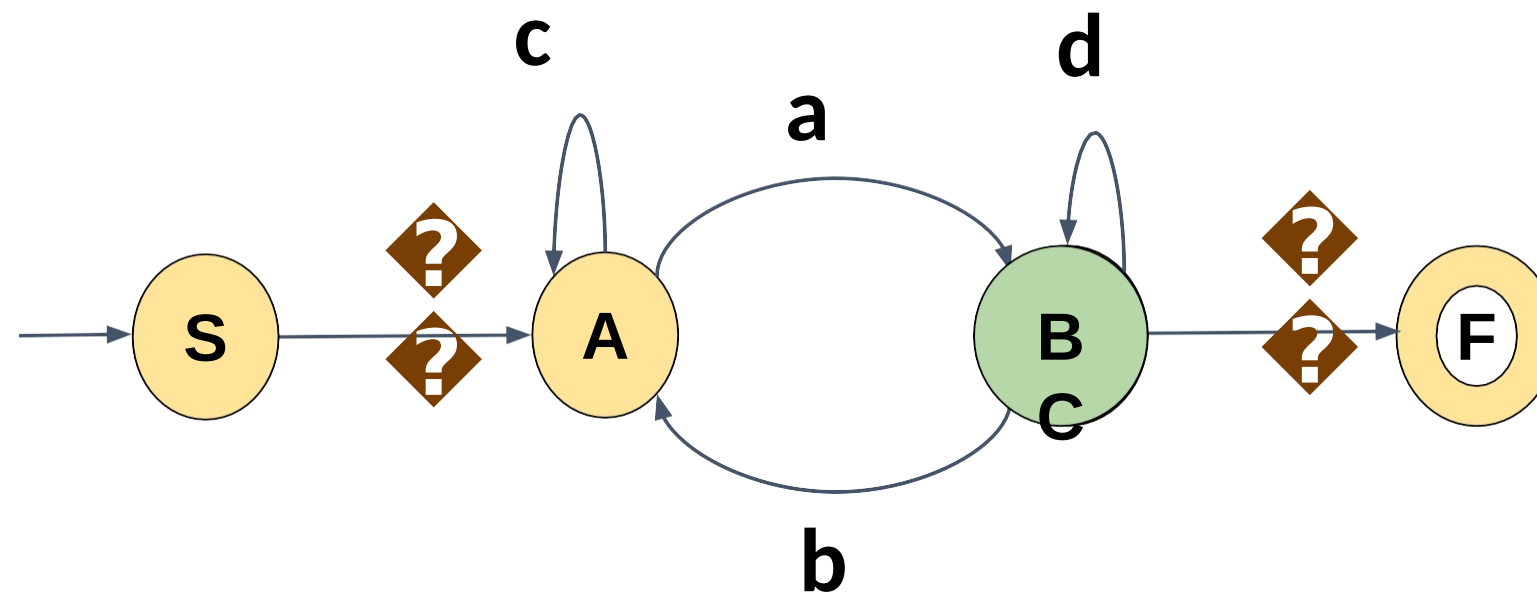
Example 4 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

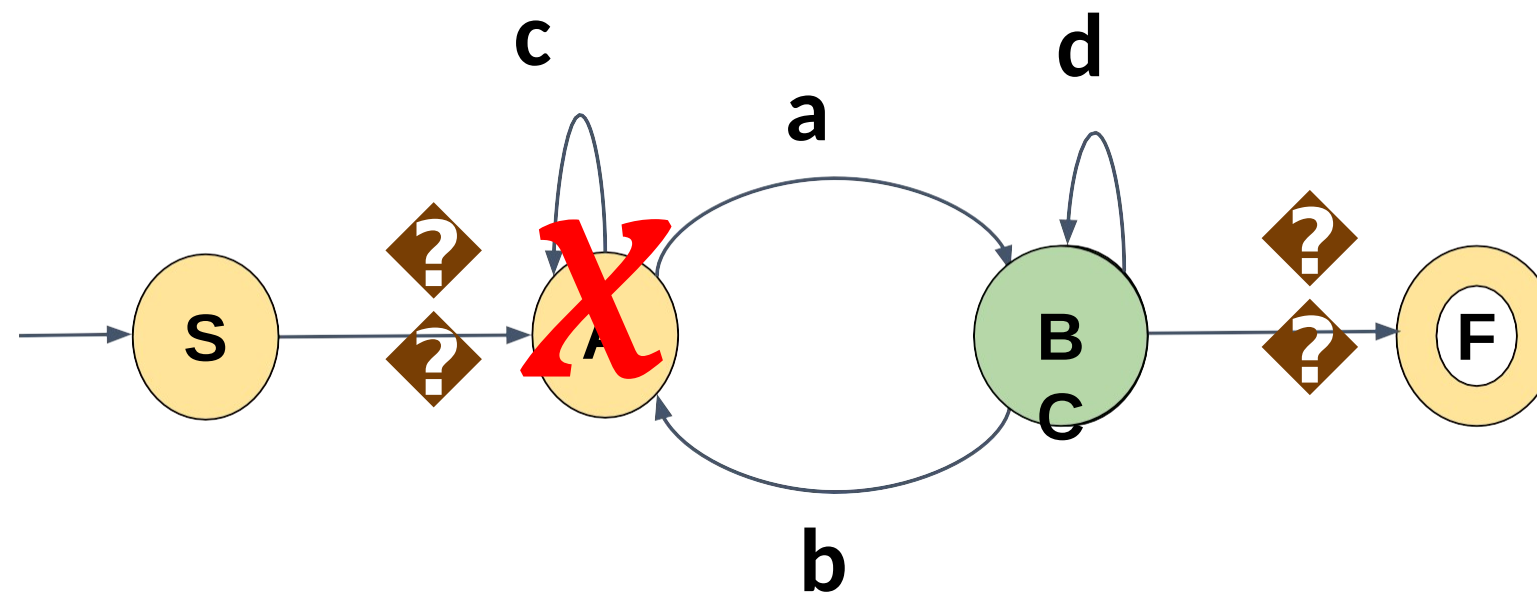


A new Final state F is introduced as there is an Outgoing edge from the existing Final State B.

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

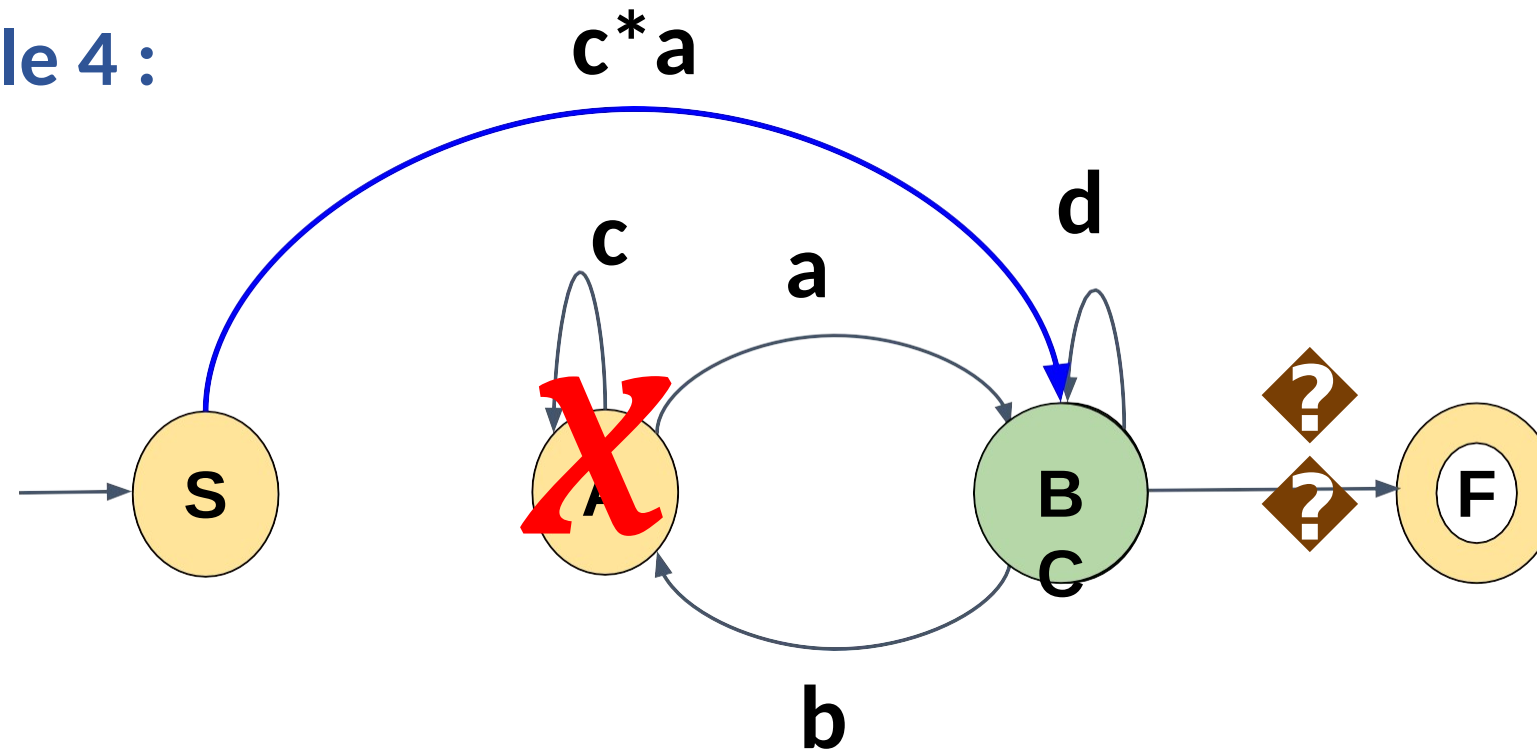


1. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

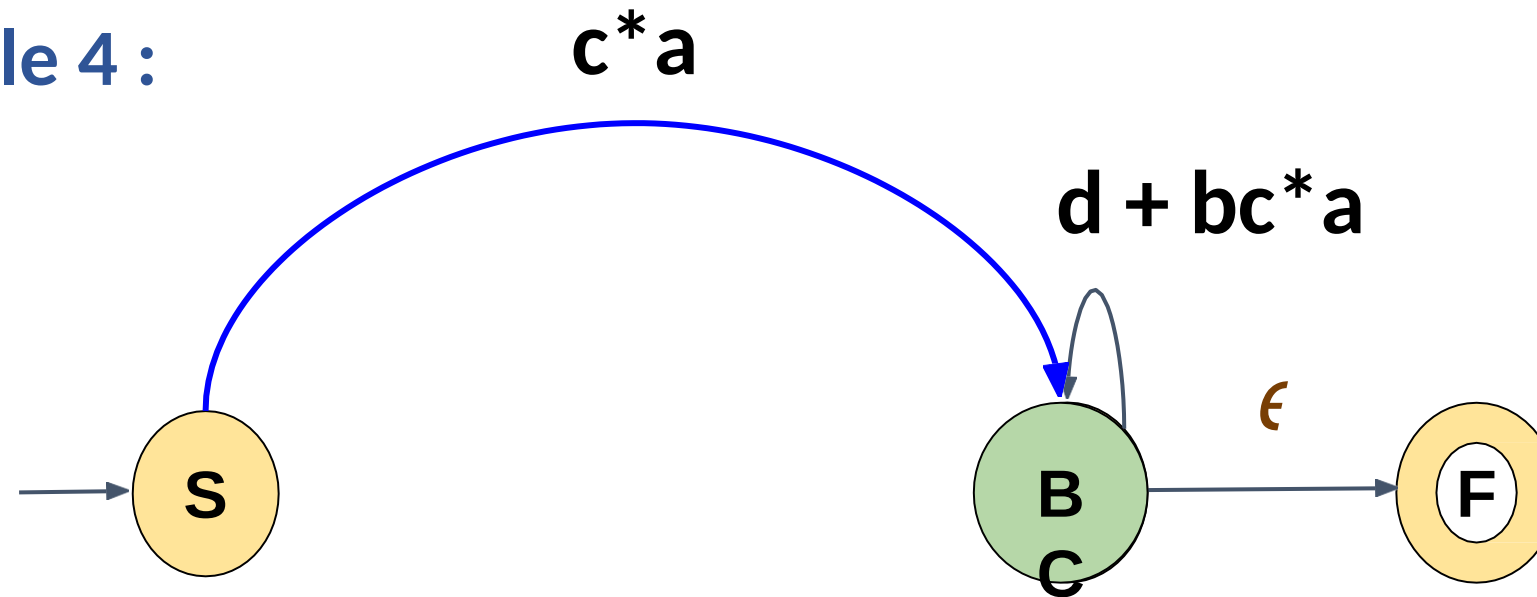


1. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

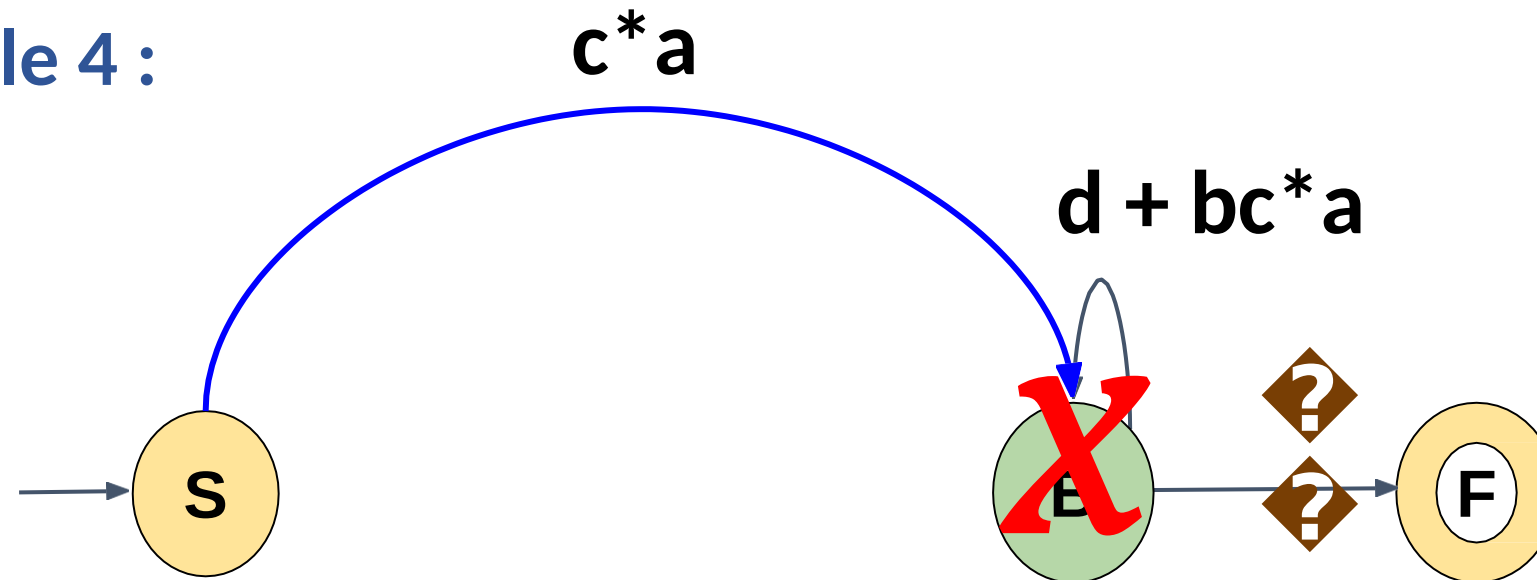


1. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

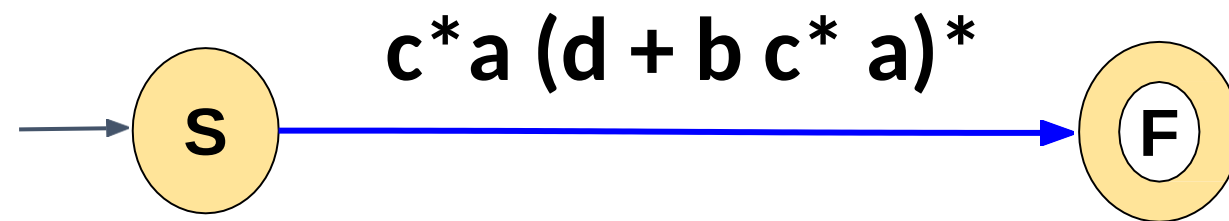


1. Eliminate A
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

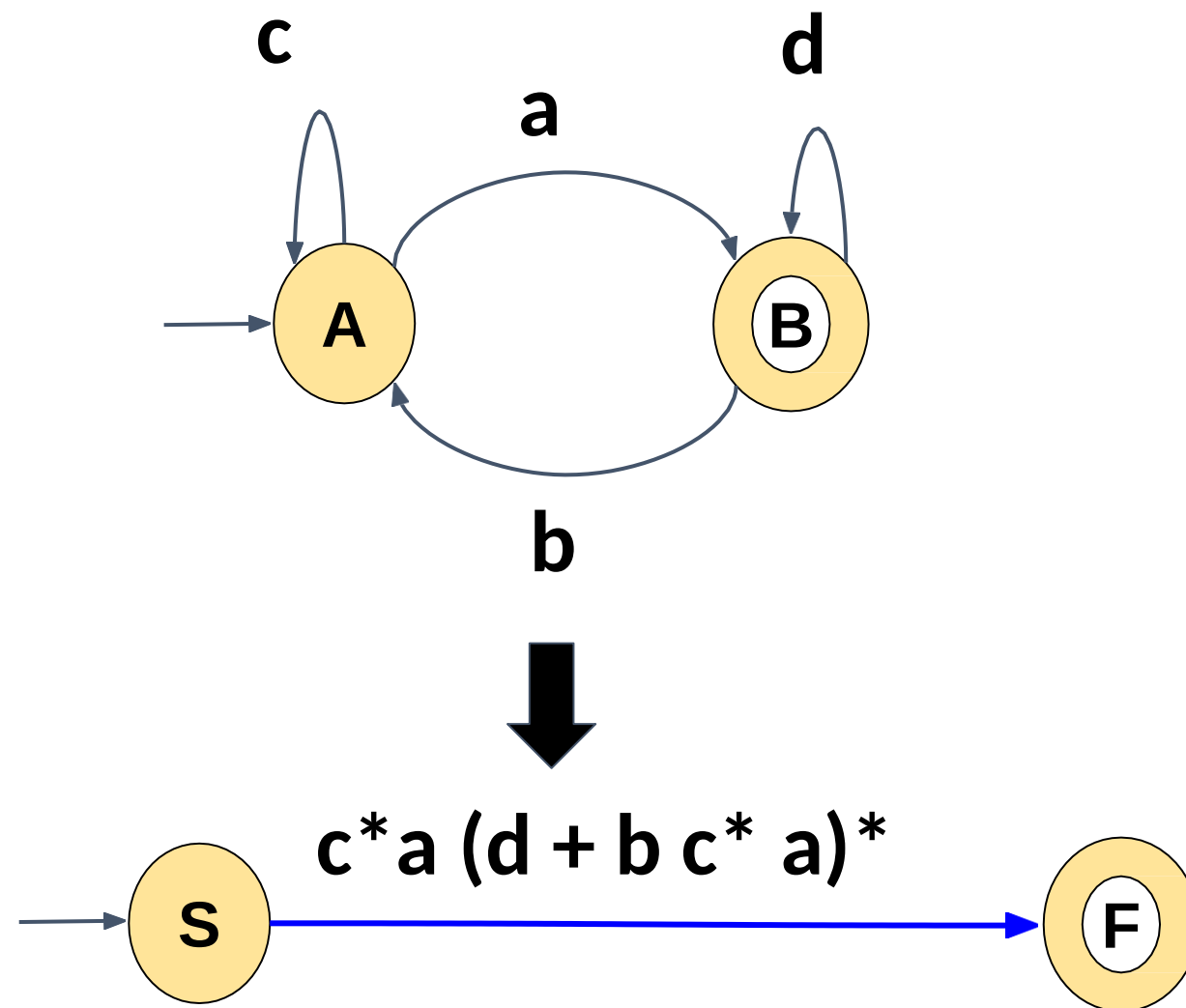


1. Eliminate A
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

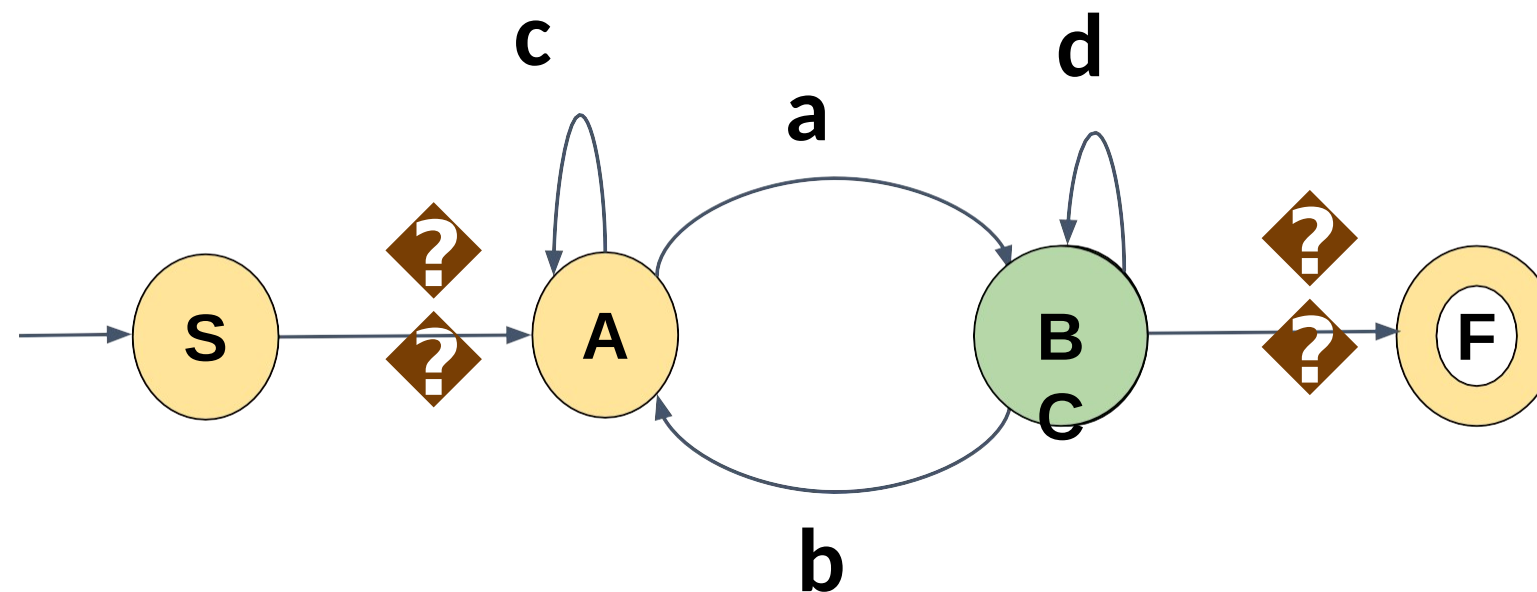
Example 4 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

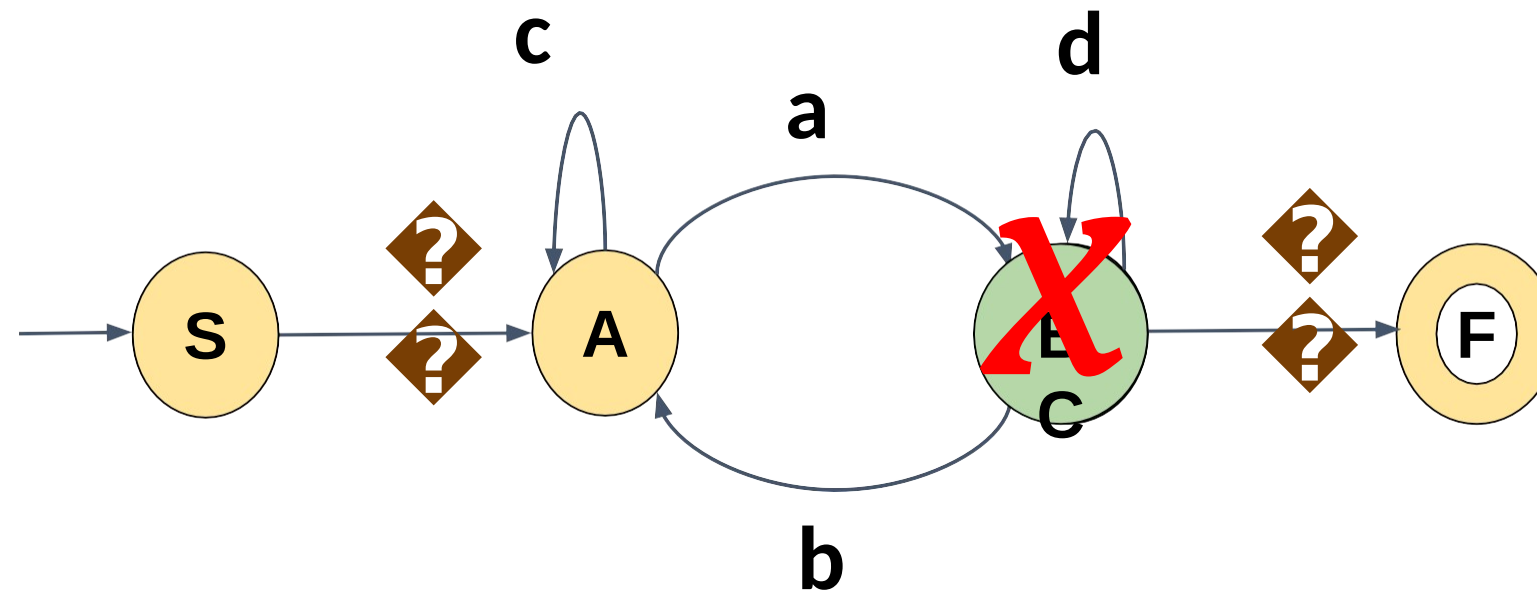
Example 4 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

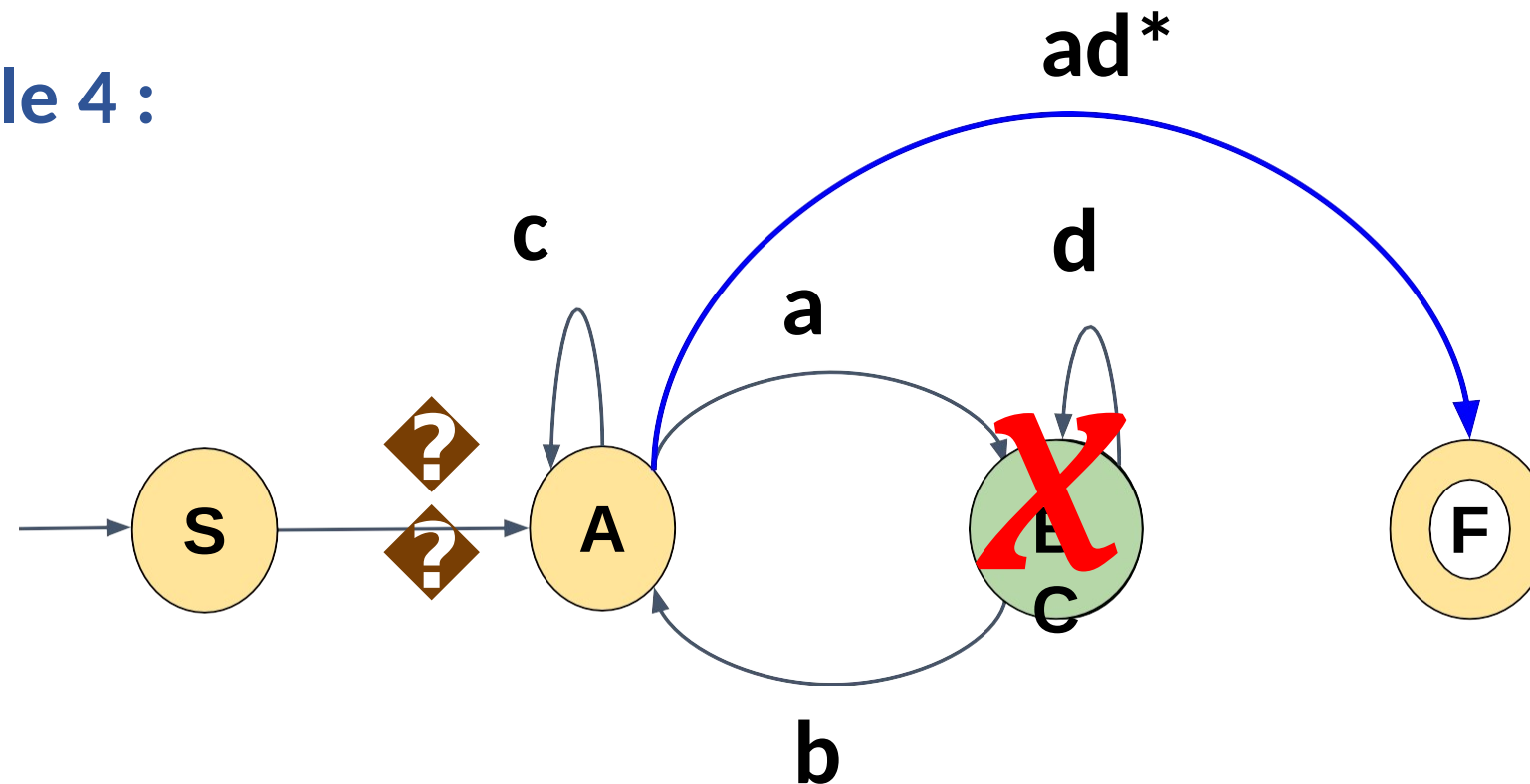


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

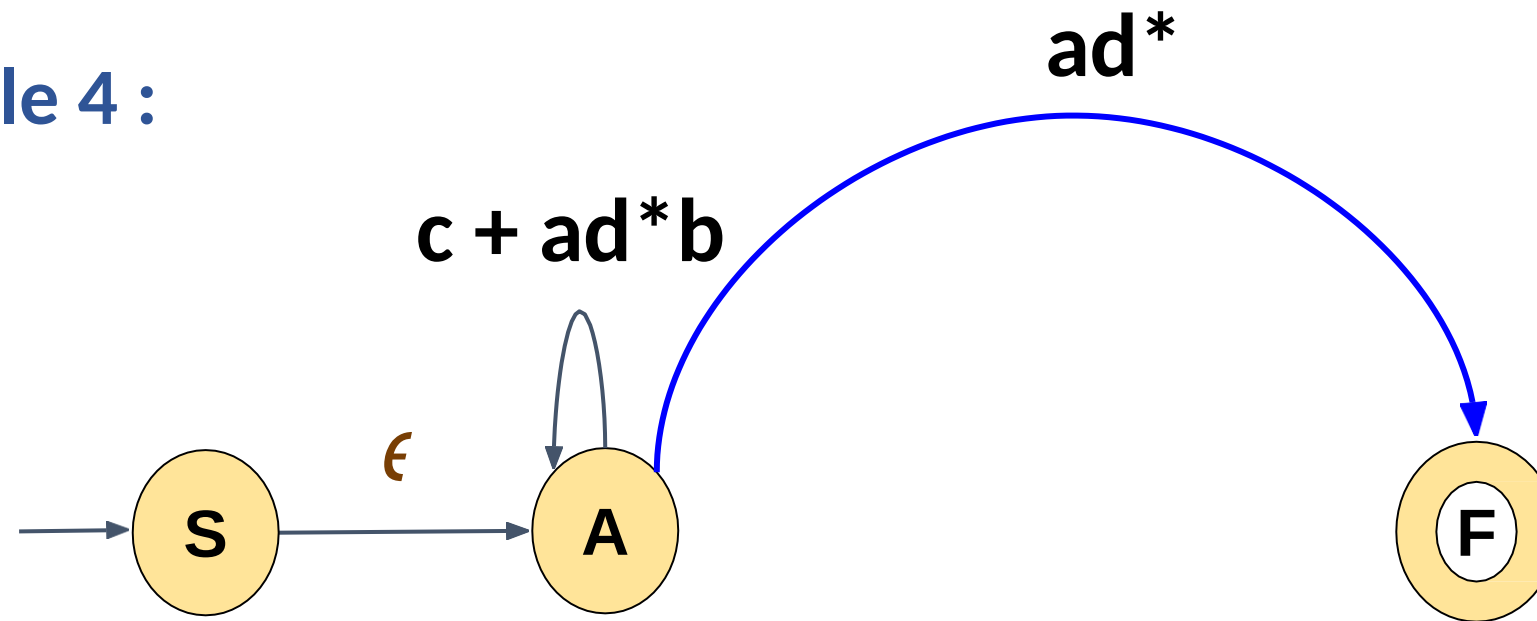


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

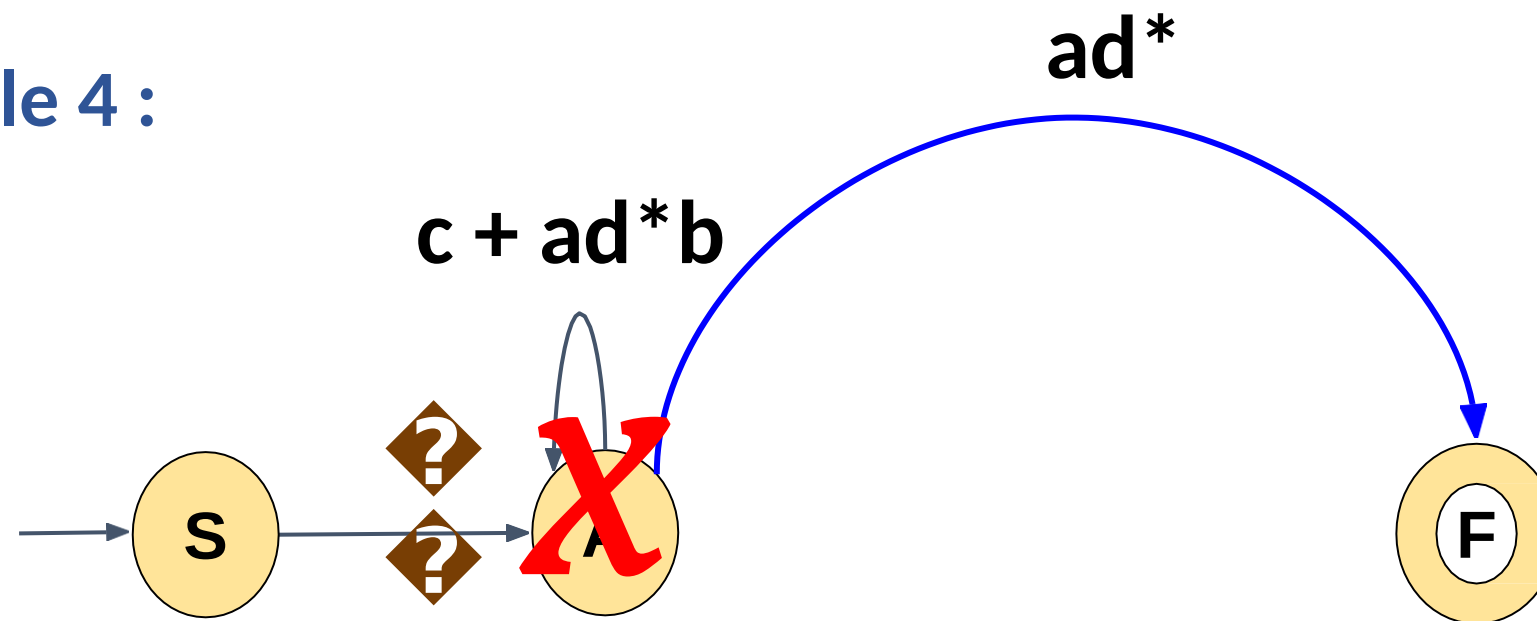


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

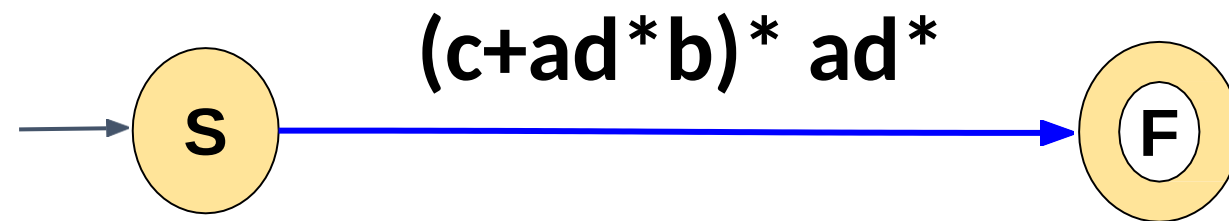


1. Eliminate B
2. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :

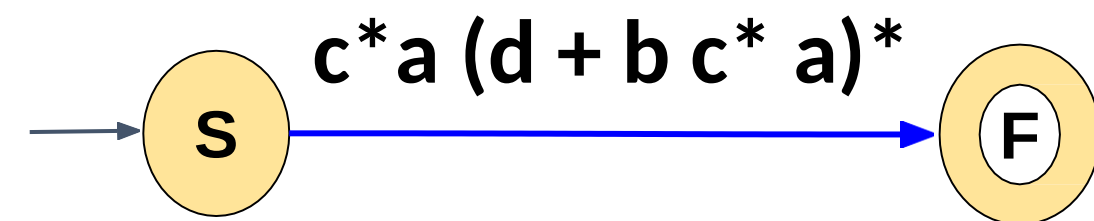
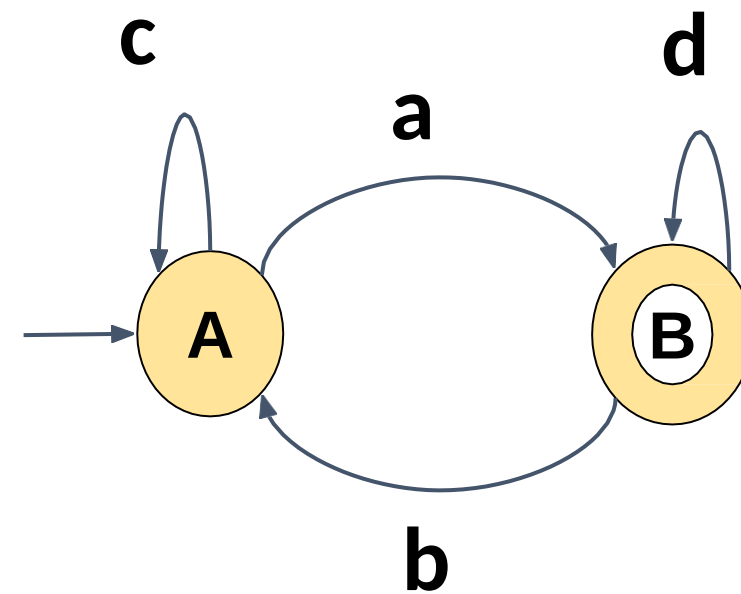


1. Eliminate B
2. Eliminate A

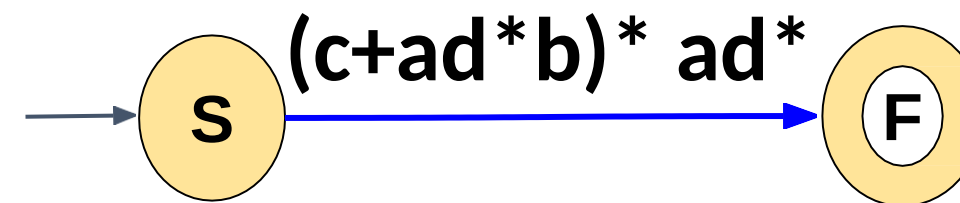
Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 4 :



1. Eliminate A
2. Eliminate B

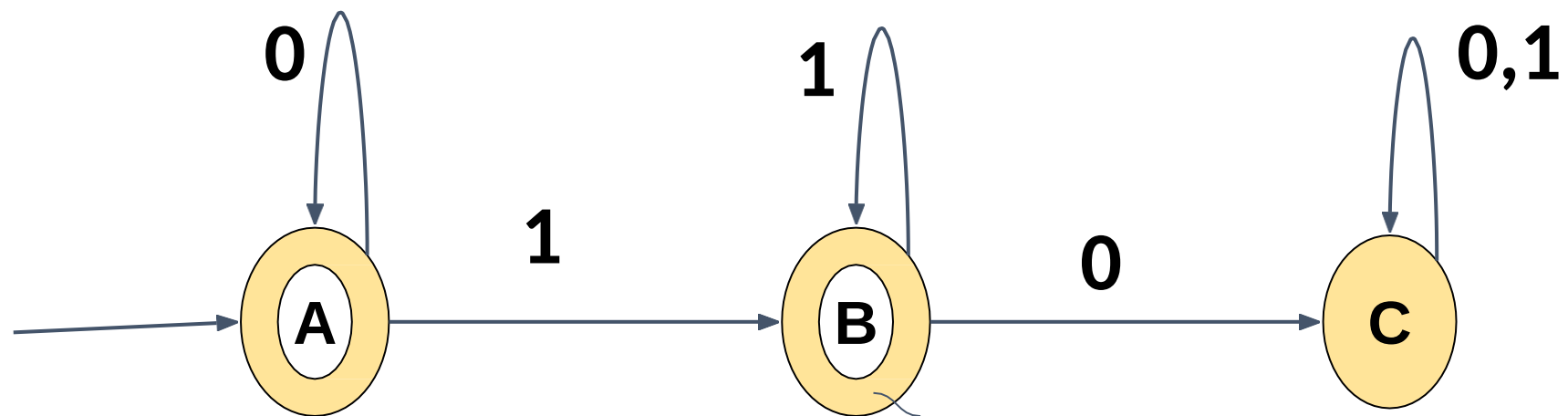


1. Eliminate B
2. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

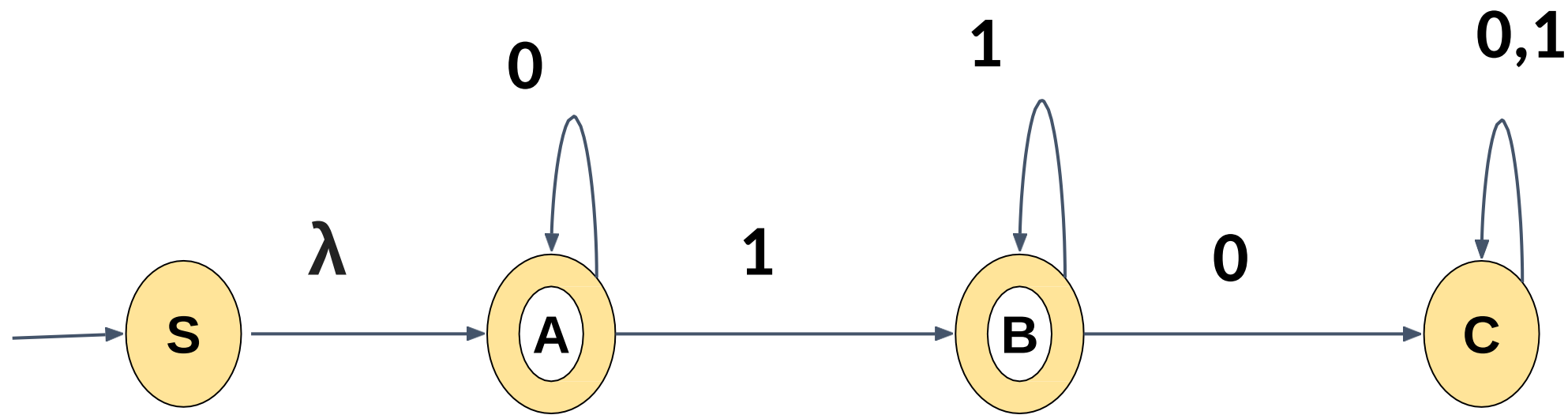
Example 5 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

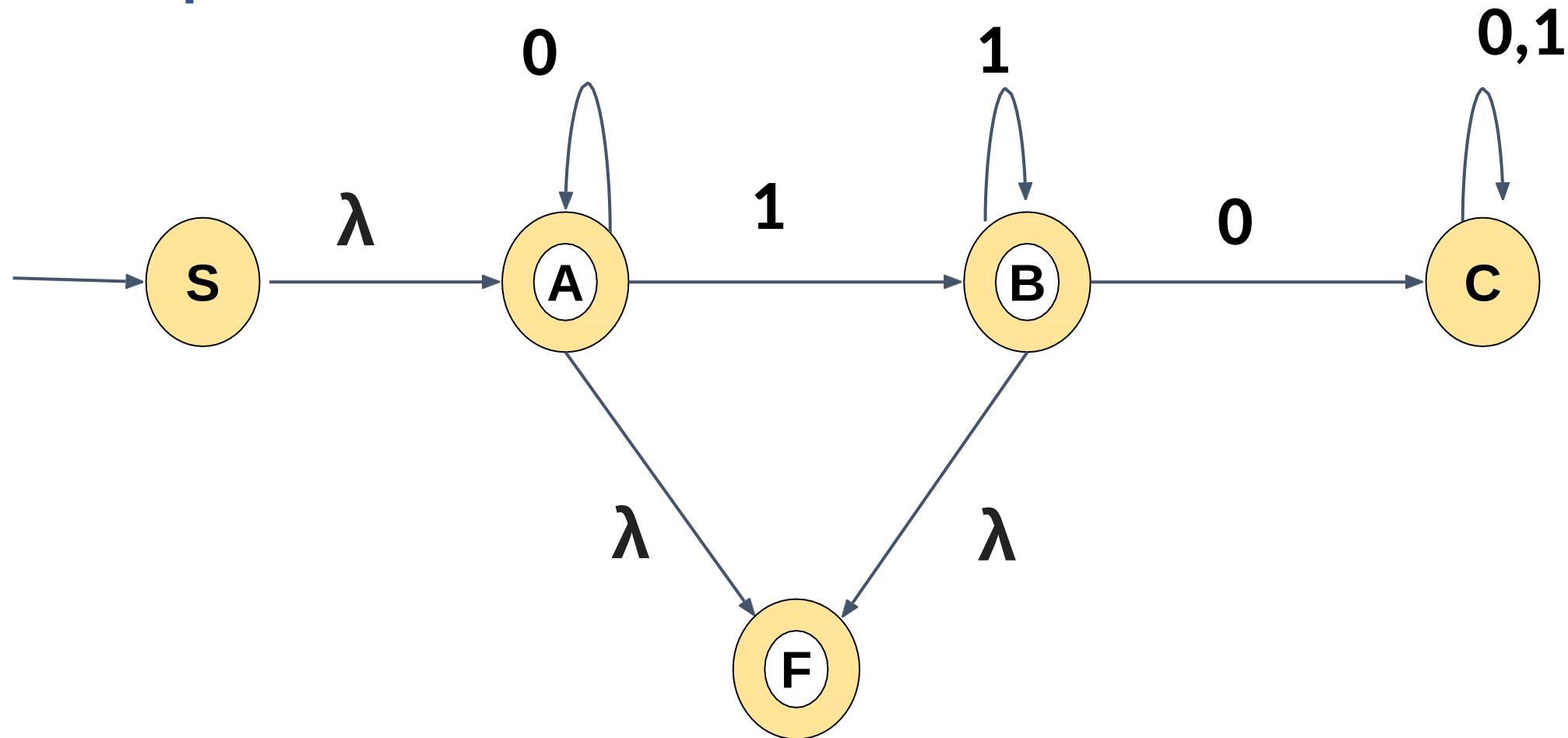


A new start state (S) is introduced as there is an incoming edge to the existing start state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

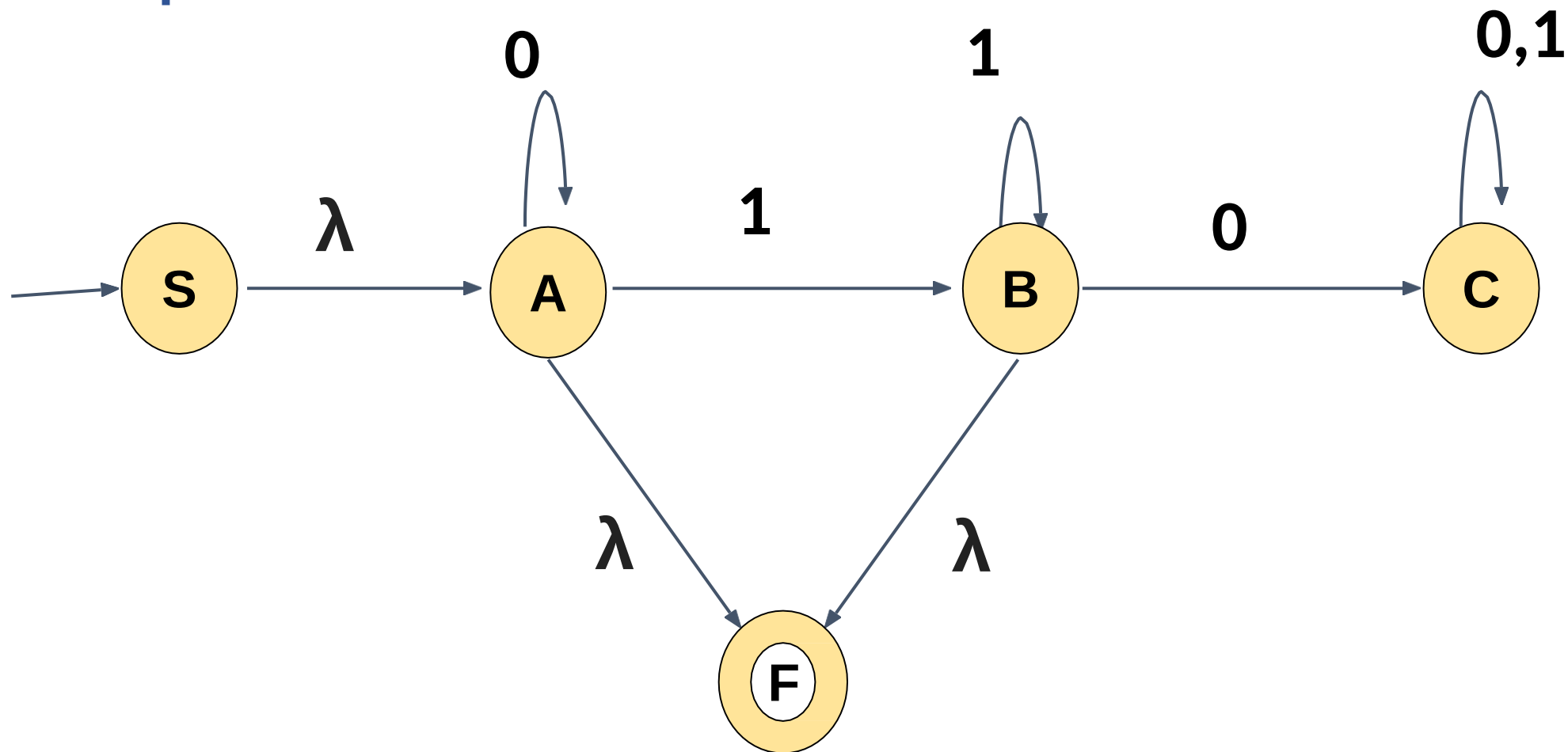


A new final state (F) is introduced as there is an outgoing edge from the existing final state and we must have a single accepting state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

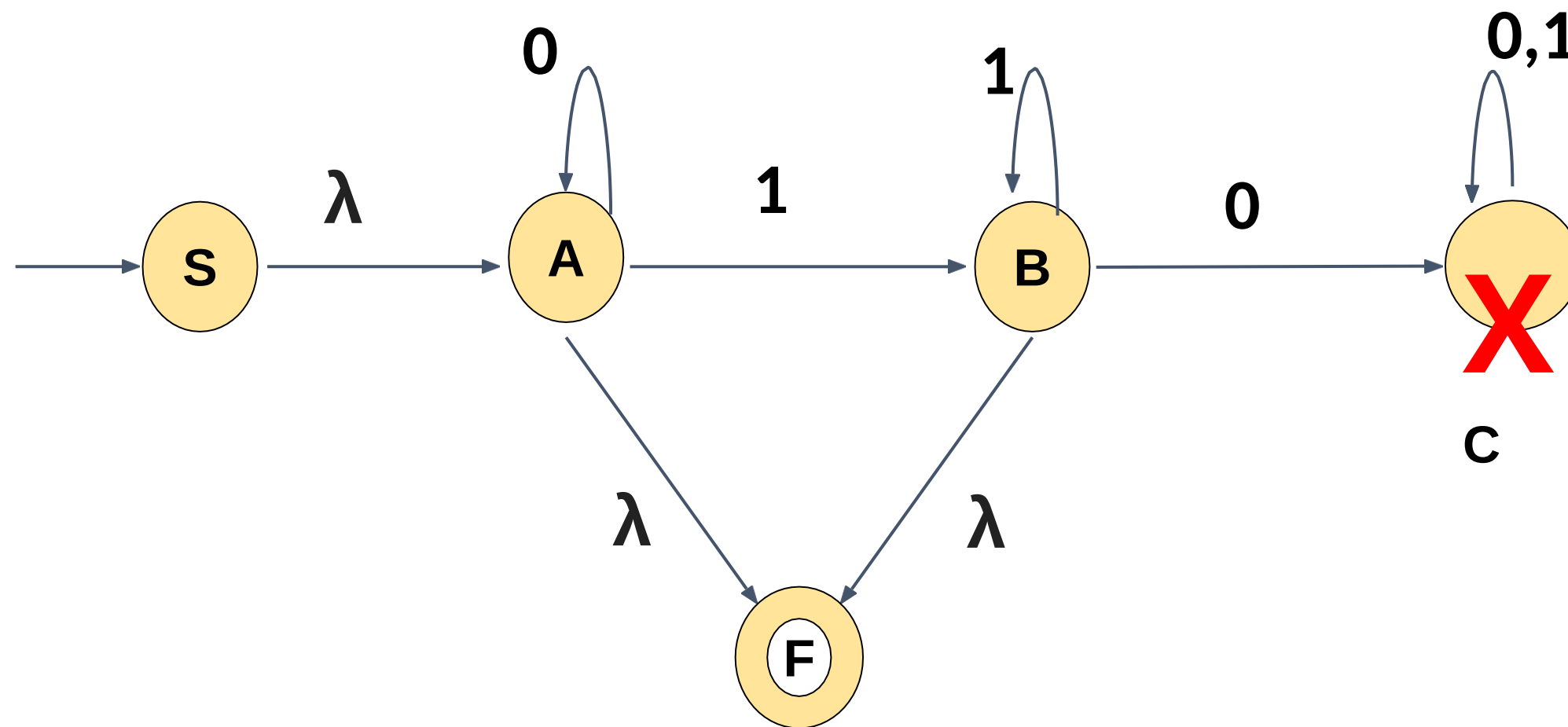


Previous final state is made as non final state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

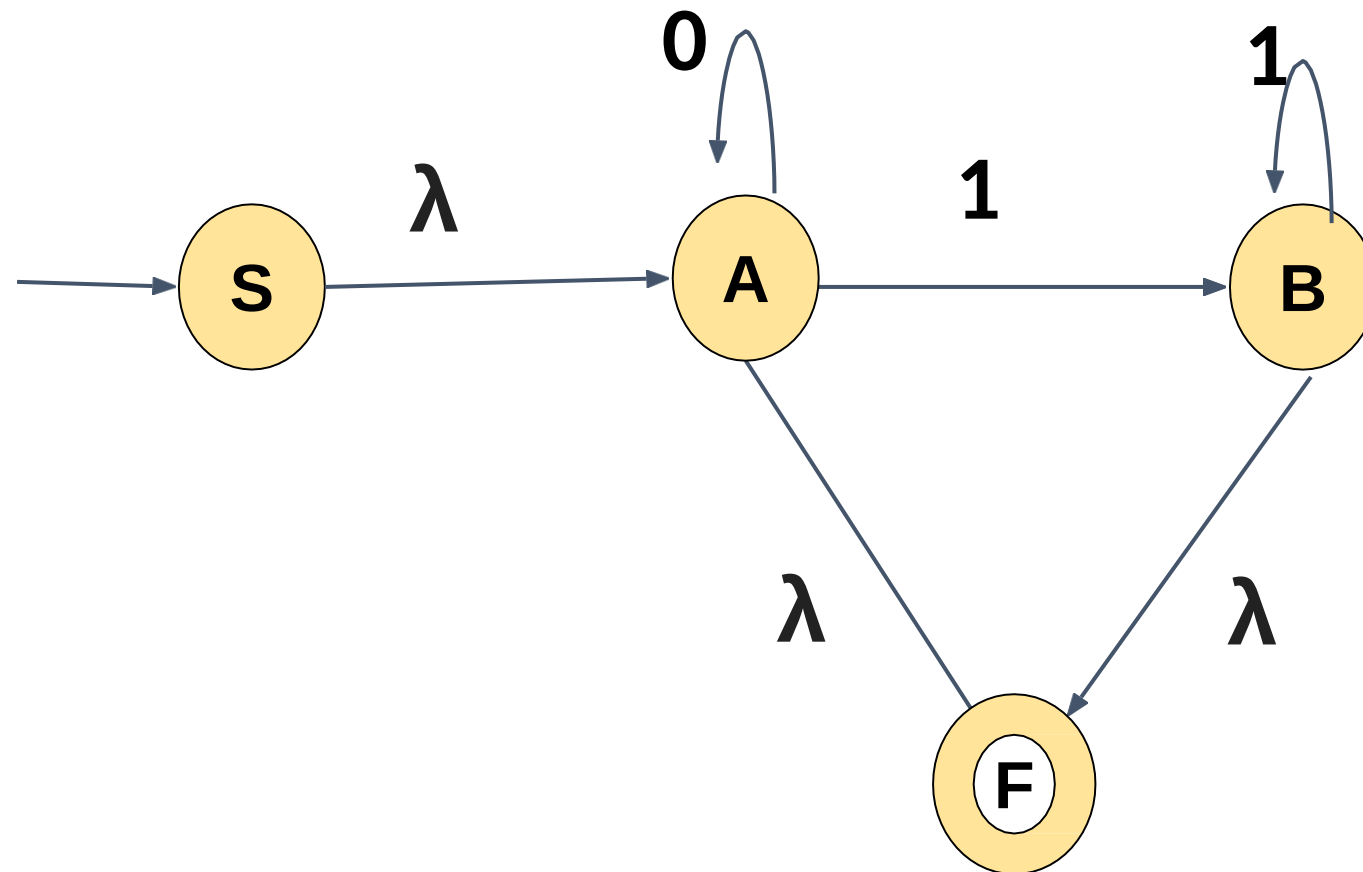


1. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

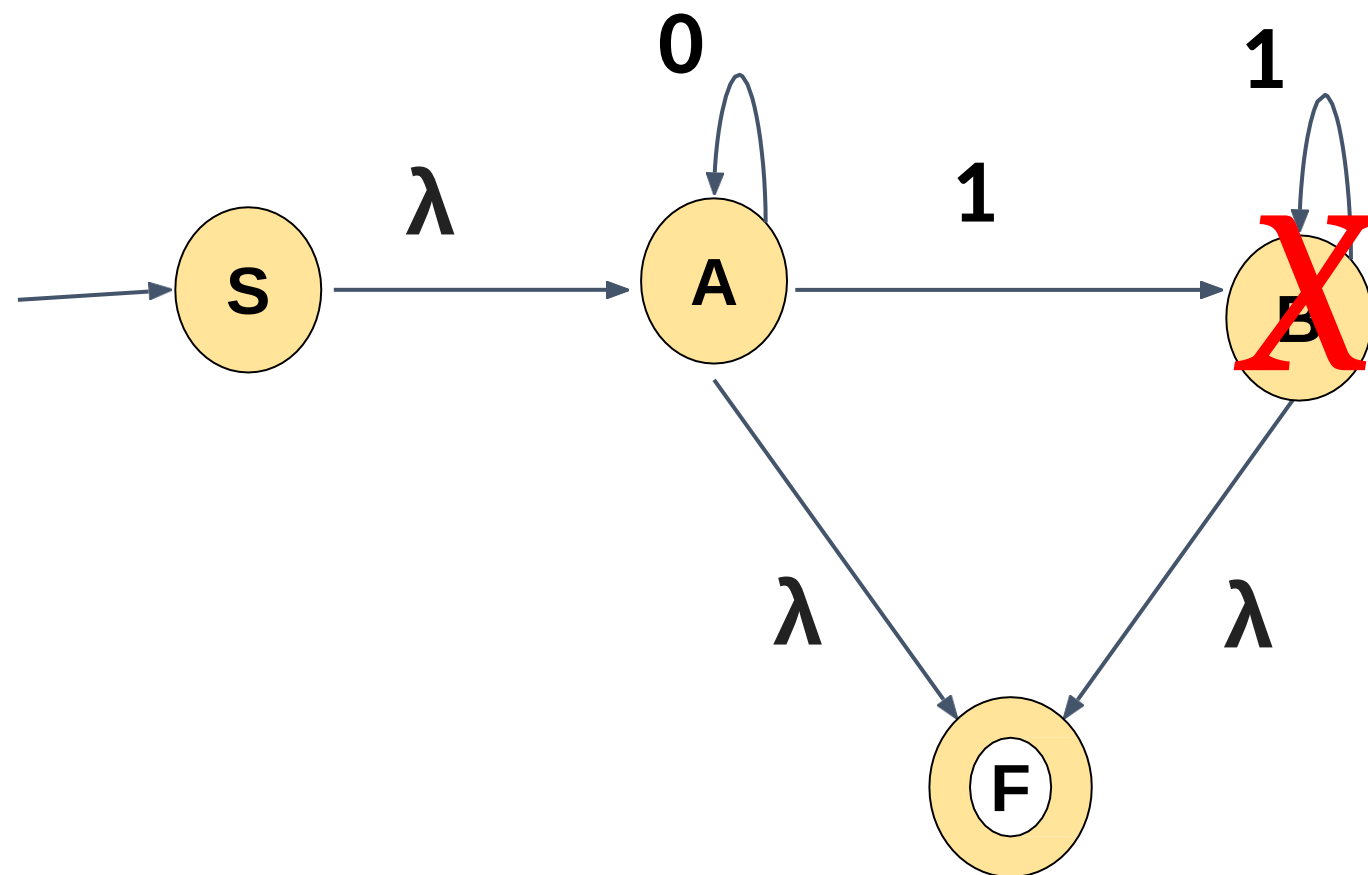


1. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

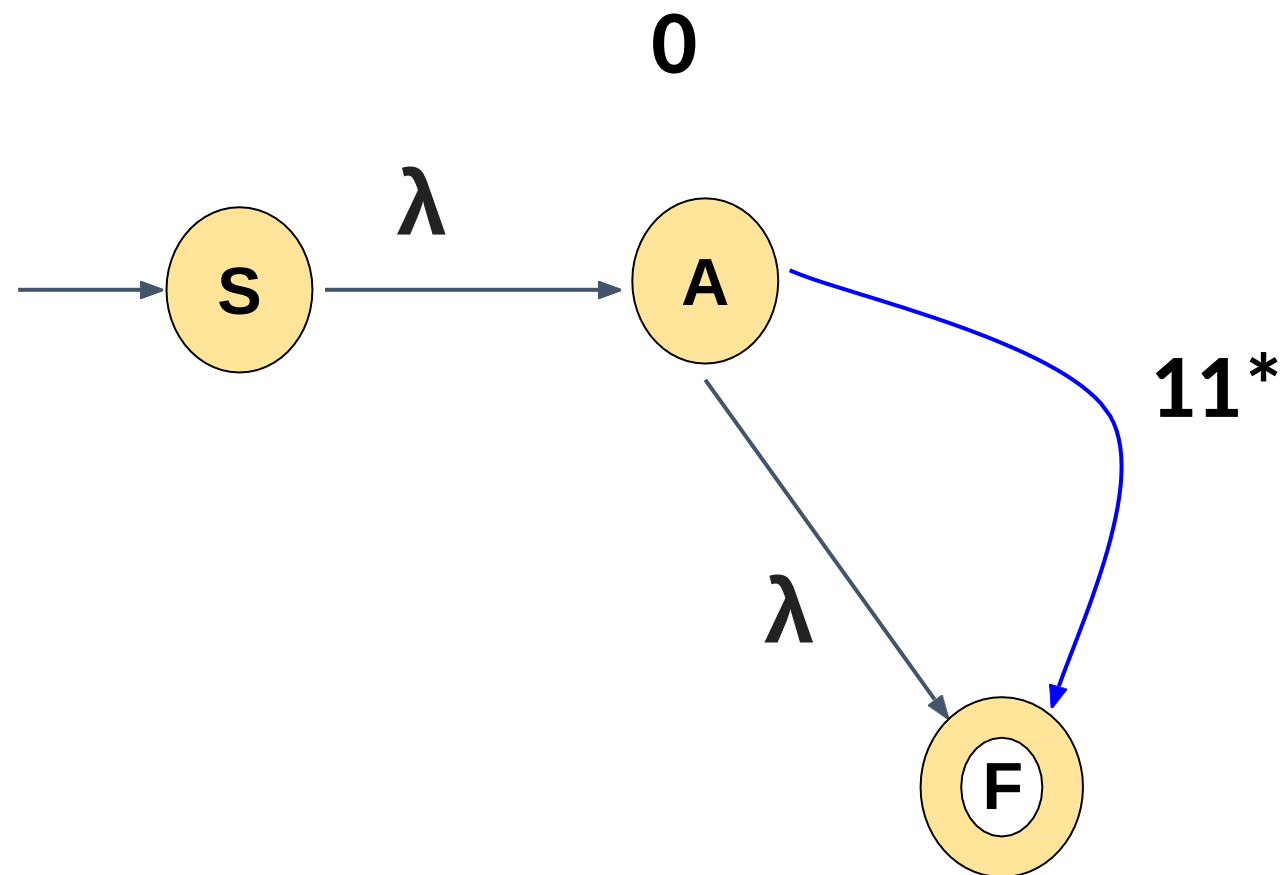


1. Eliminate C
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

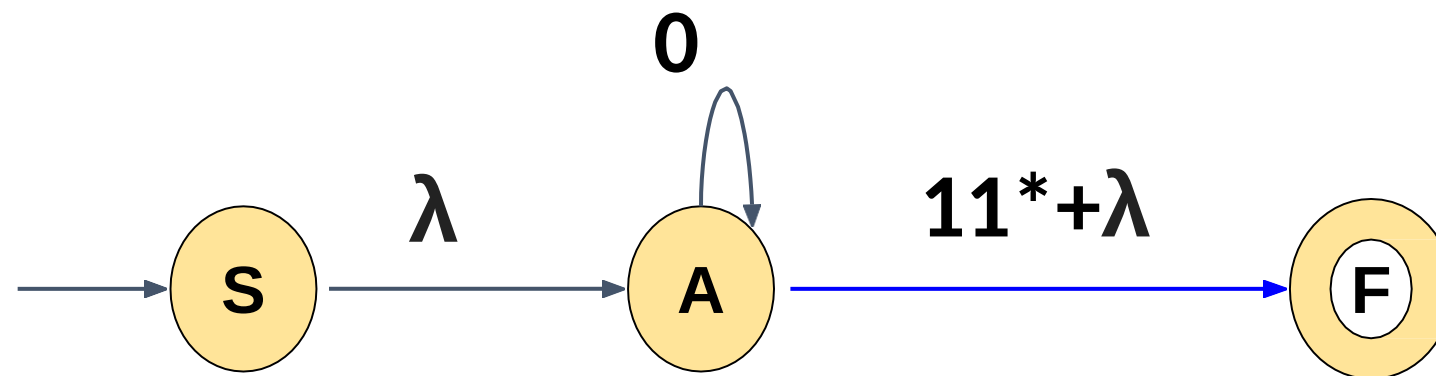


1. Eliminate C
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

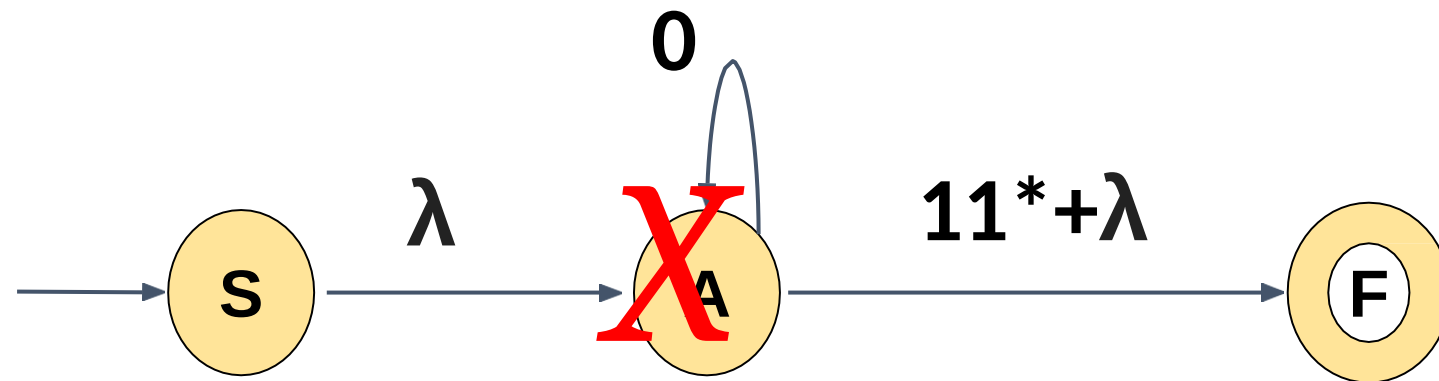


1. Eliminate C
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

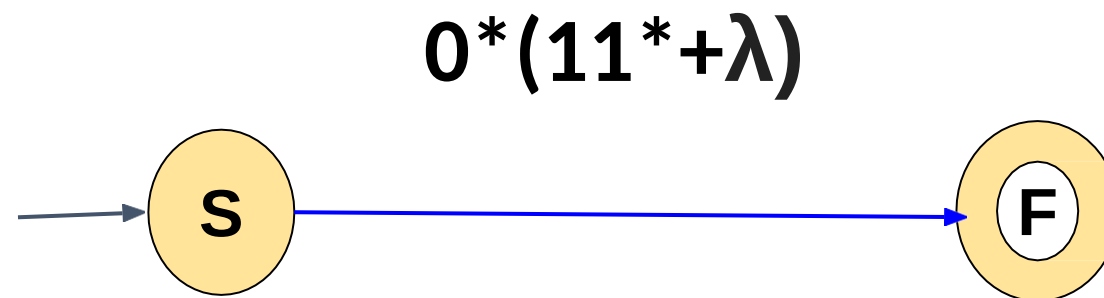


1. Eliminate C
2. Eliminate B
3. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 5 :

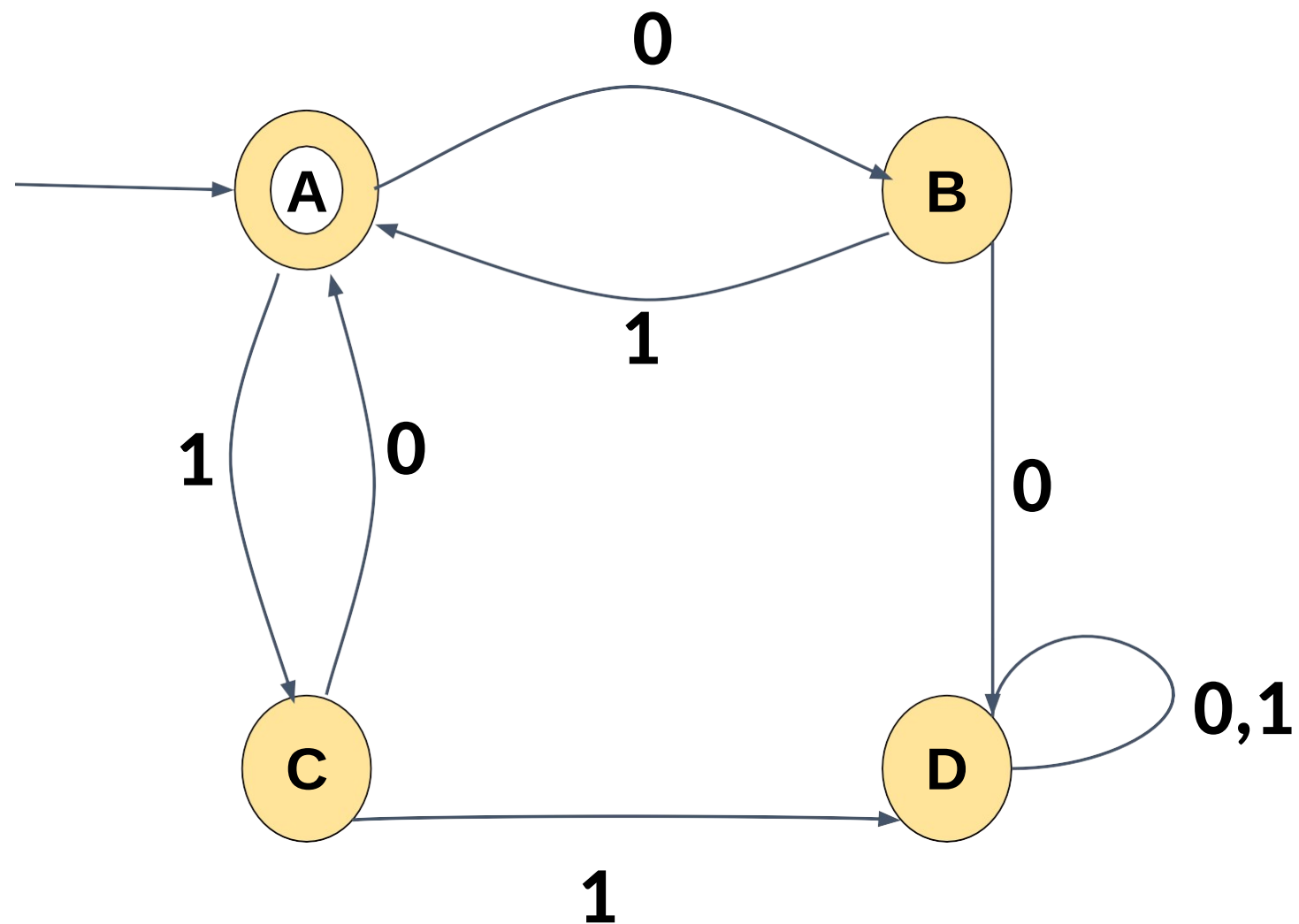


$$\begin{aligned} &= 0^*(11^*+\lambda) \\ &= 0^*(1^++\lambda) \end{aligned}$$

$$\text{RE} = 0^*1^*$$

Unit 1 - Finite Automata to Regular Expression

Example 6 :



Unit 1 - Finite Automata to Regular Expression

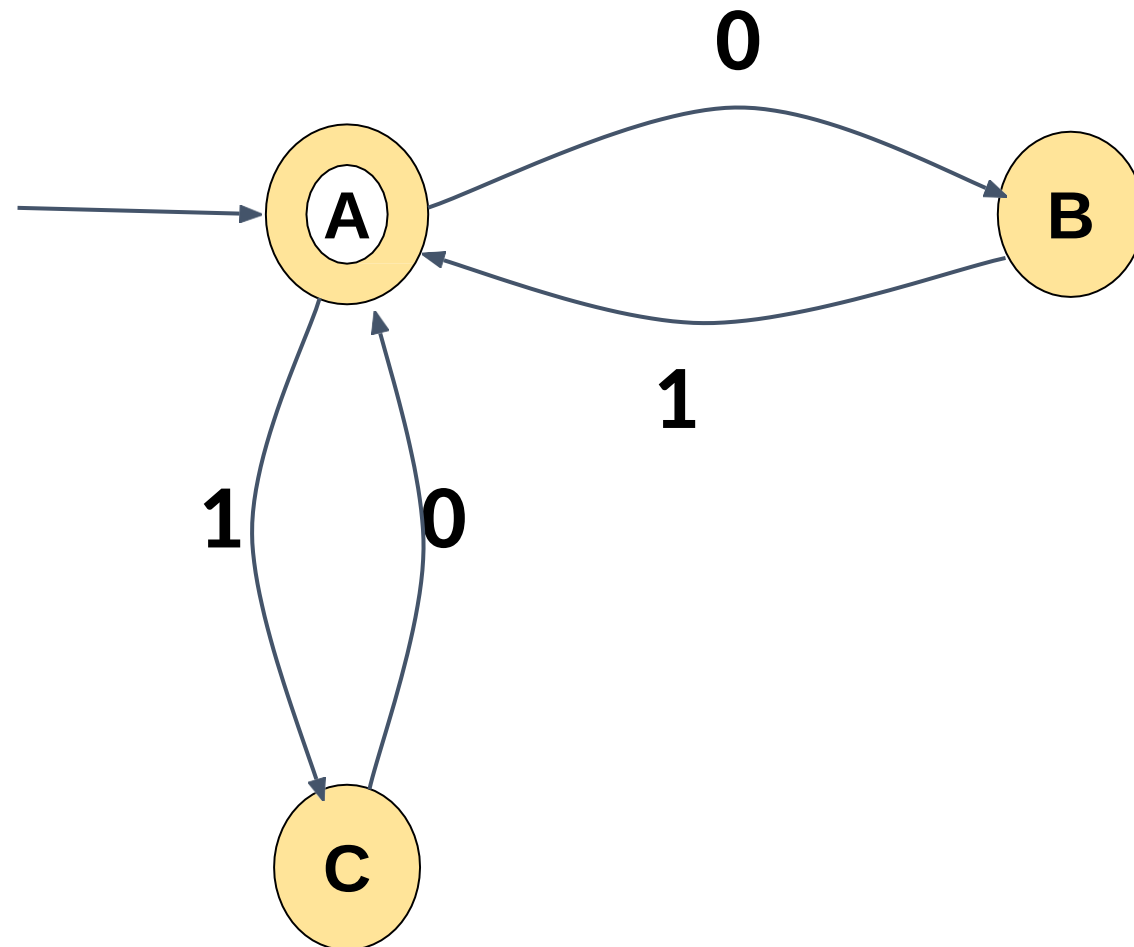
```
graph TD; Start(( )) --> A(((A))); A -- 0 --> B((B)); B -- 1 --> A; A -- 1 --> C((C)); C -- 0 --> A; B -- 0 --> D(((D))); C -- 1 --> D; D -- "0,1" --> D; style A stroke-width:4px; style D stroke:#f00,stroke-width:4px; style Start fill:none,stroke:none;
```

- # 1. Eliminate D (Dead state)

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

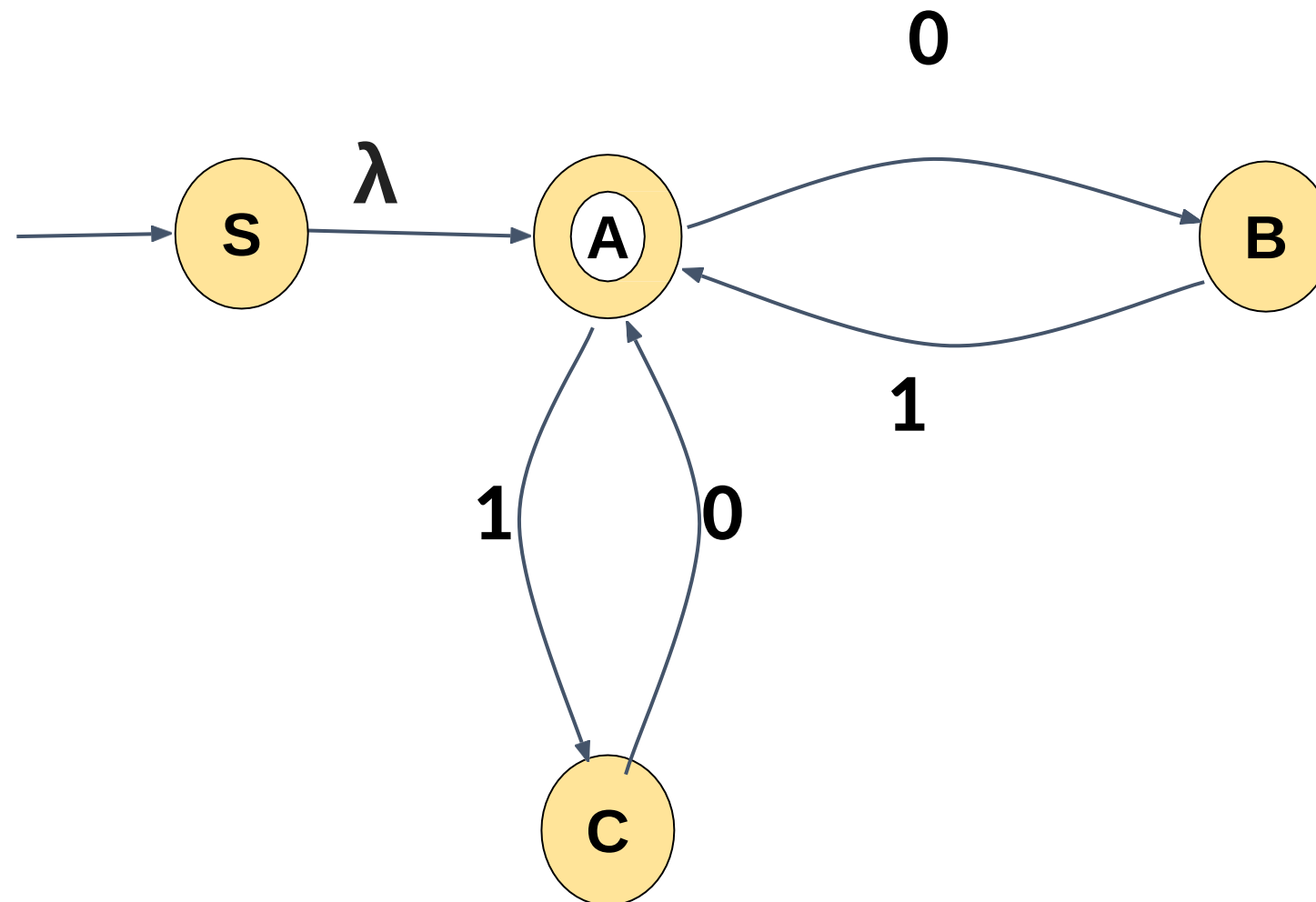
Example 6 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

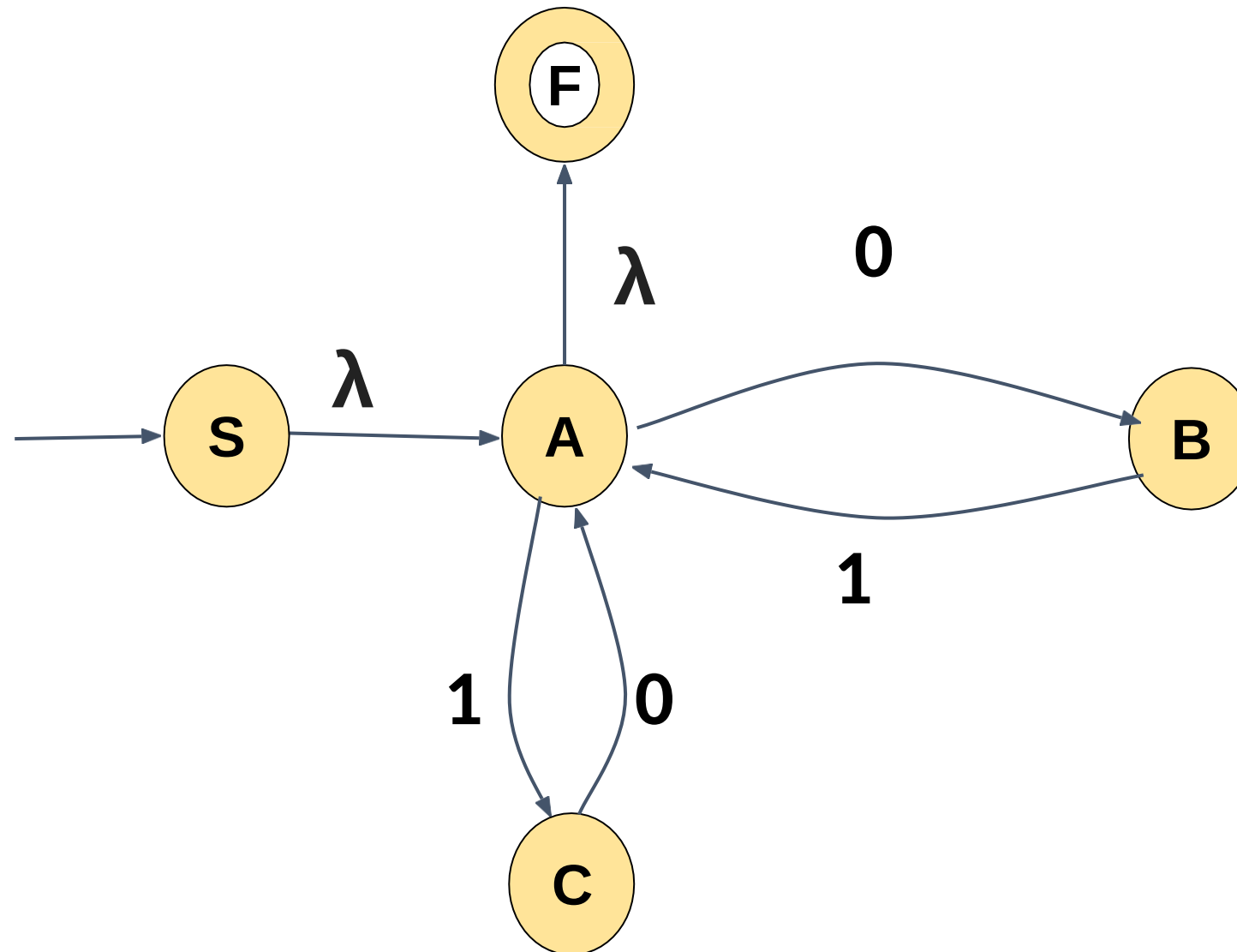


A new start state (S) is introduced as there is an incoming edge to the existing start state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

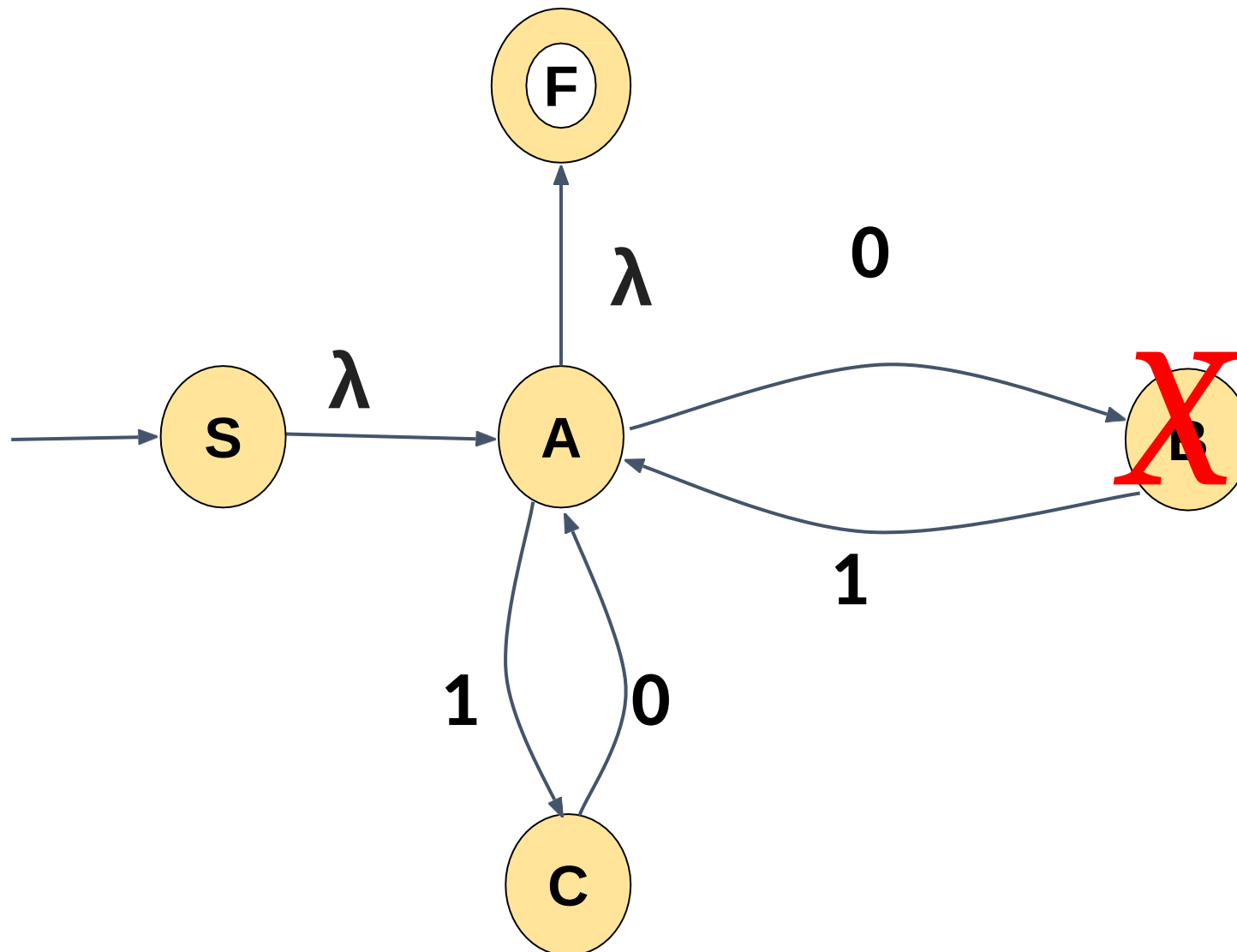


A new final state (F) is introduced as there is an outgoing edge from the existing final state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

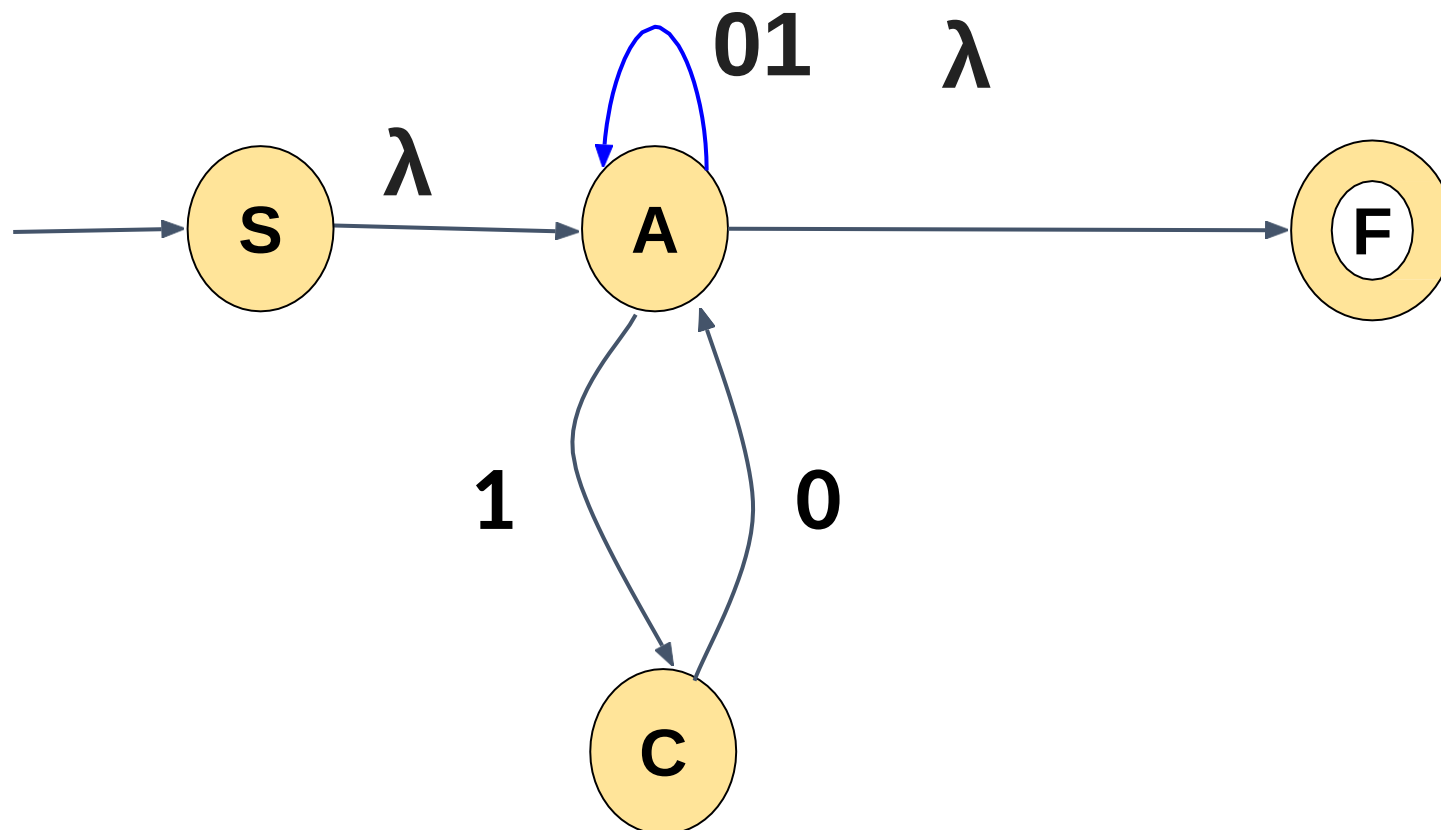


1. Eliminate D
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

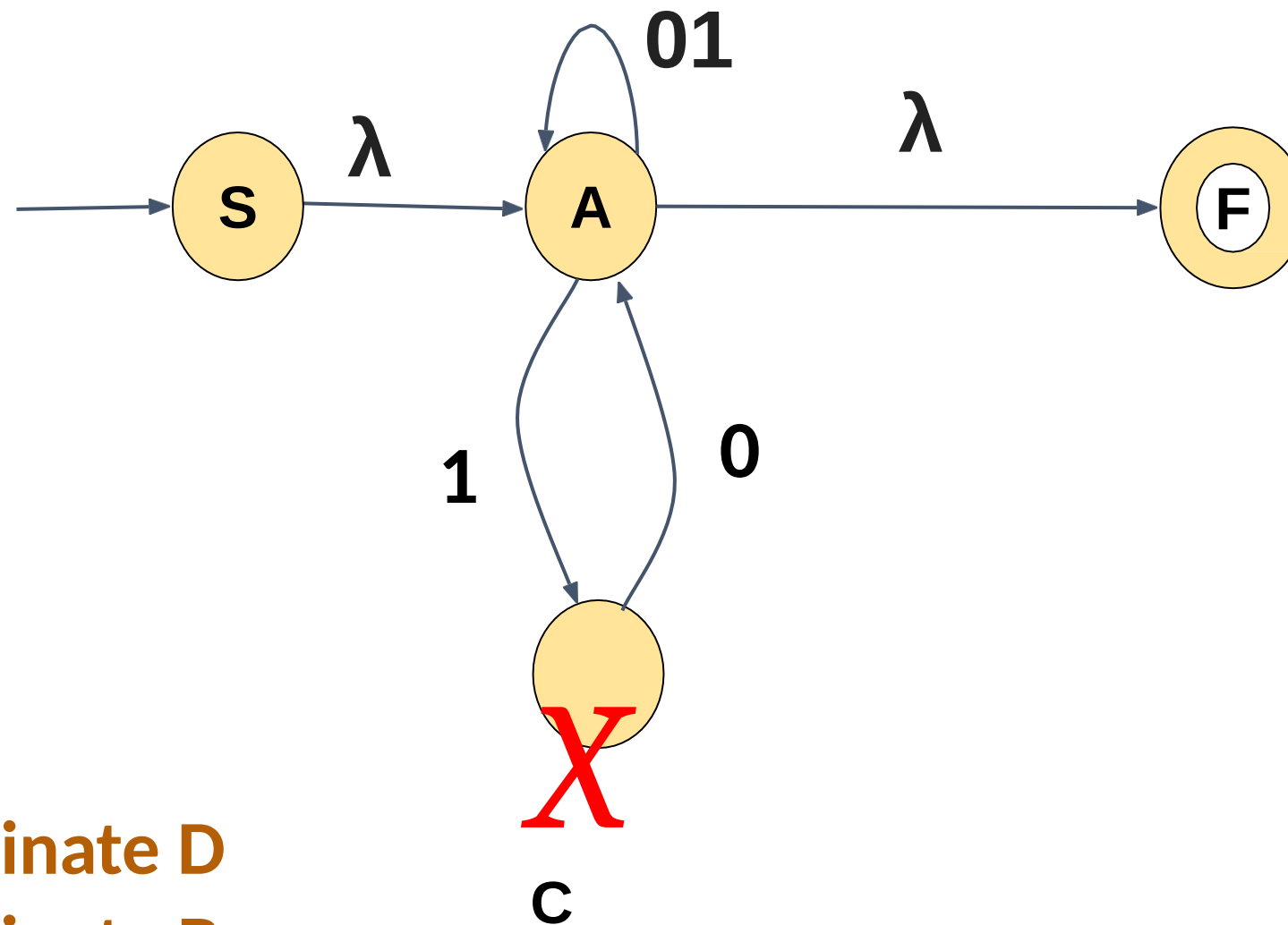


1. Eliminate D
2. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

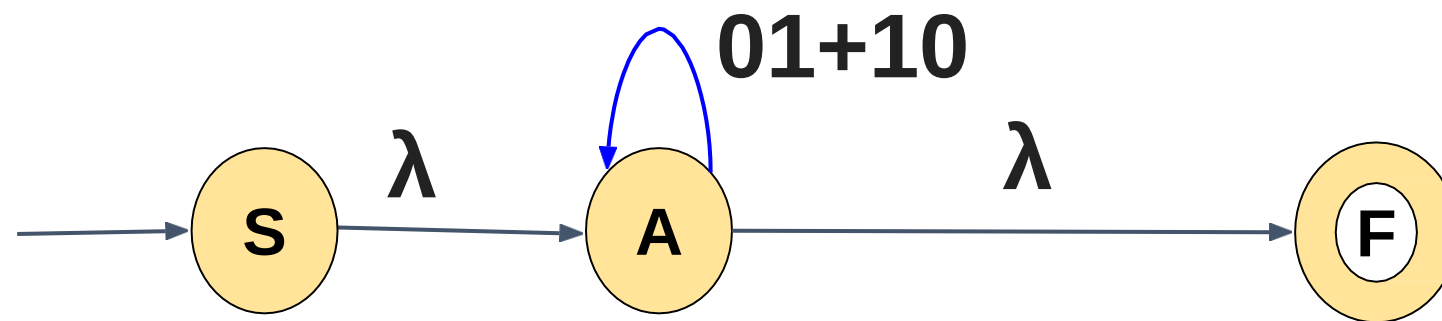


1. Eliminate D
2. Eliminate B
3. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

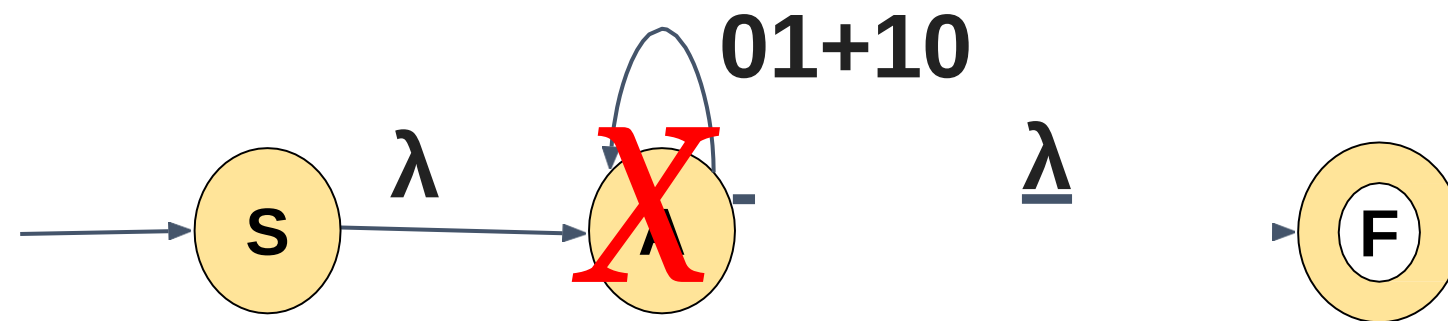


1. Eliminate D
2. Eliminate B
3. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 6 :

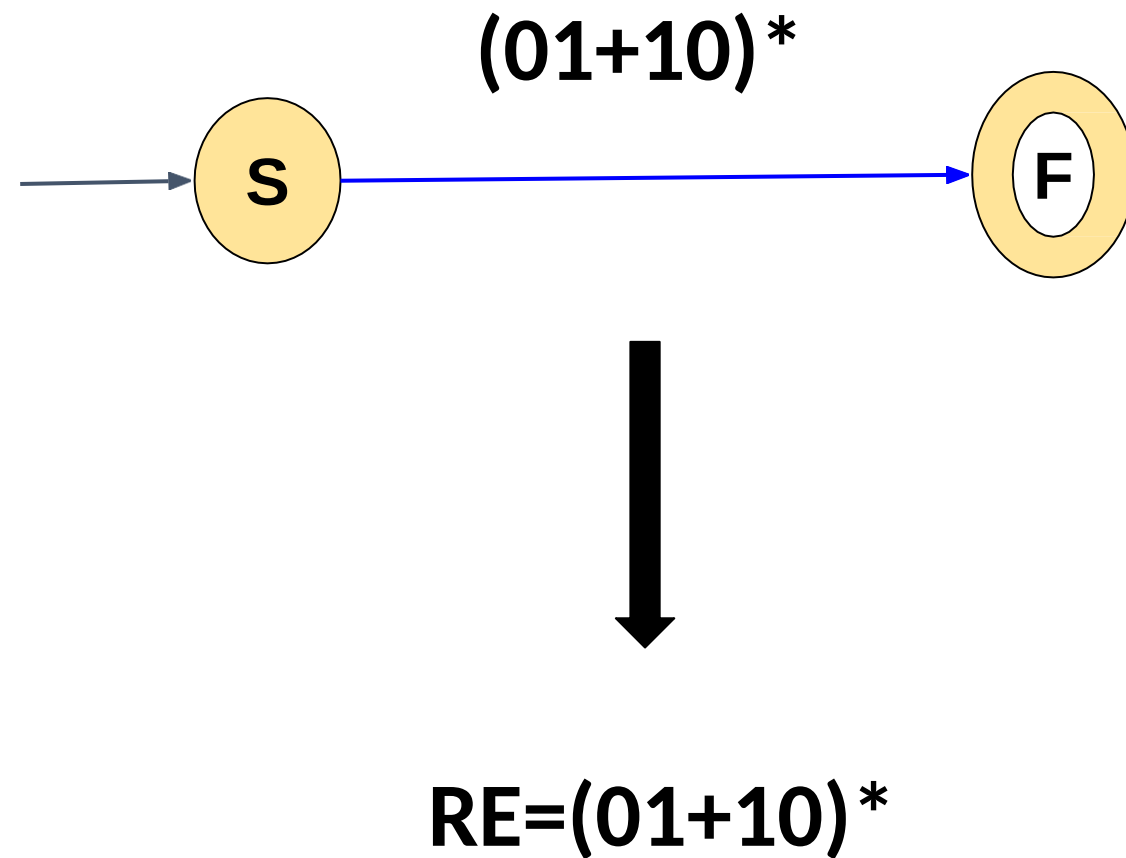


1. Eliminate D
2. Eliminate B
3. Eliminate C
4. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

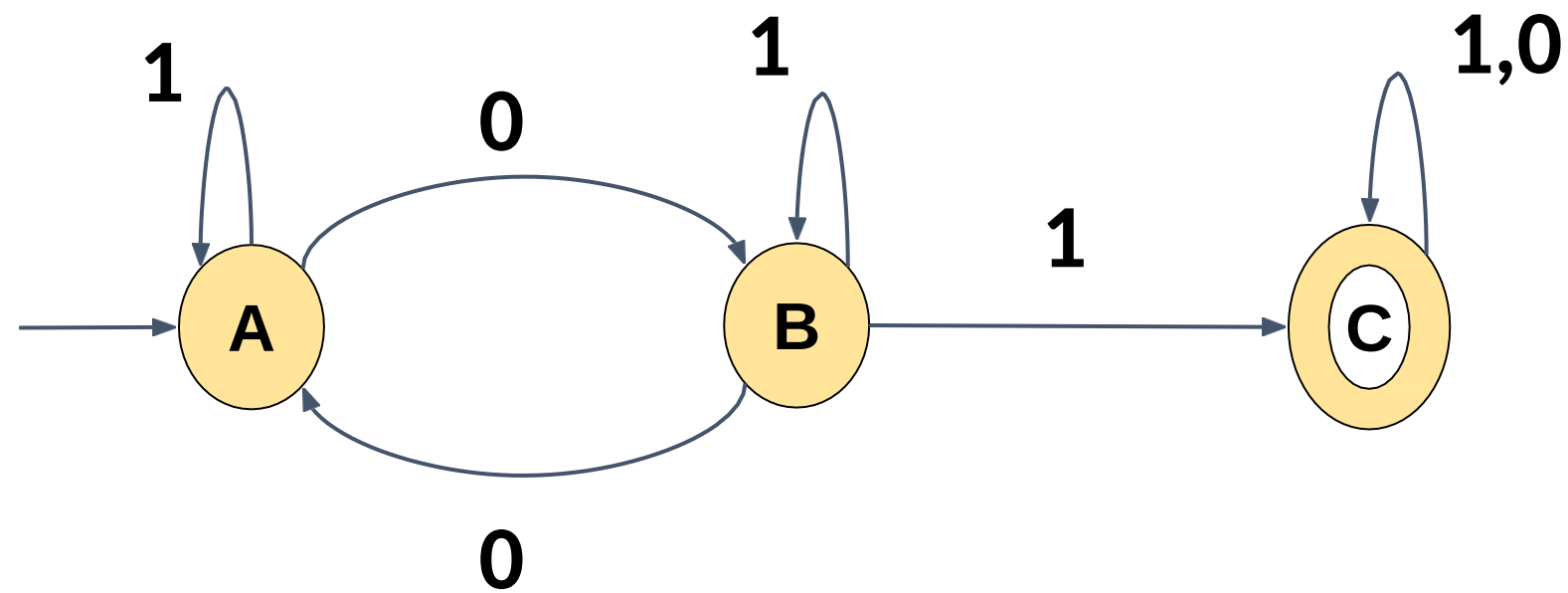
Example 6 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

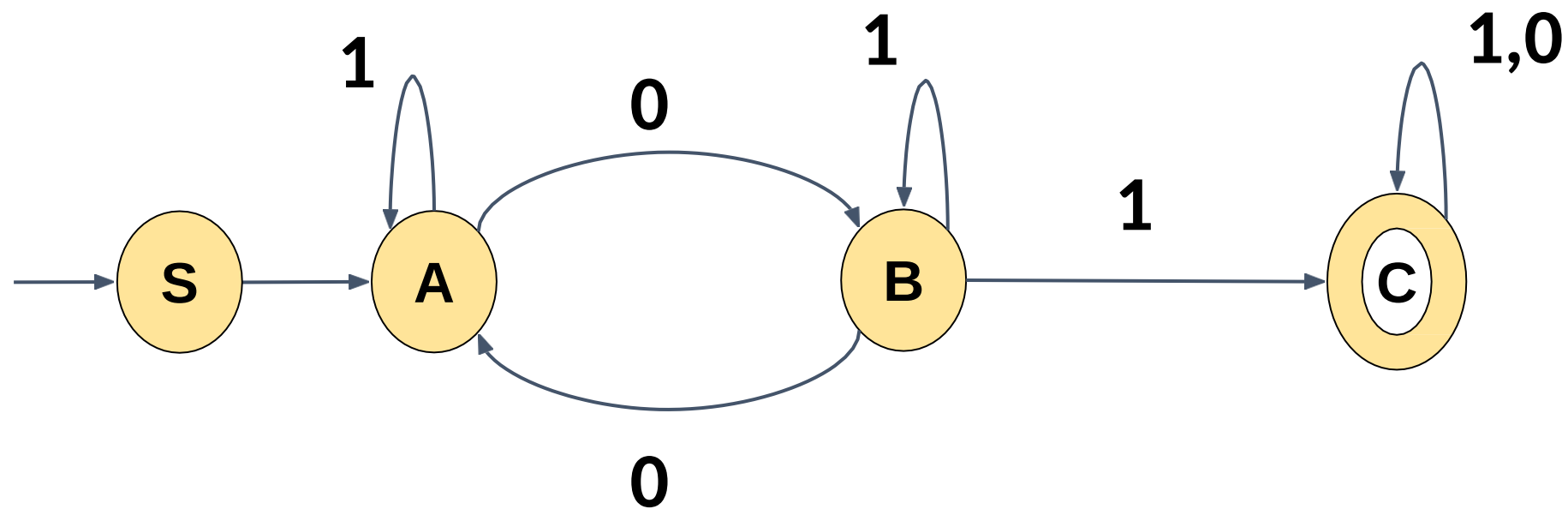
Example 7 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

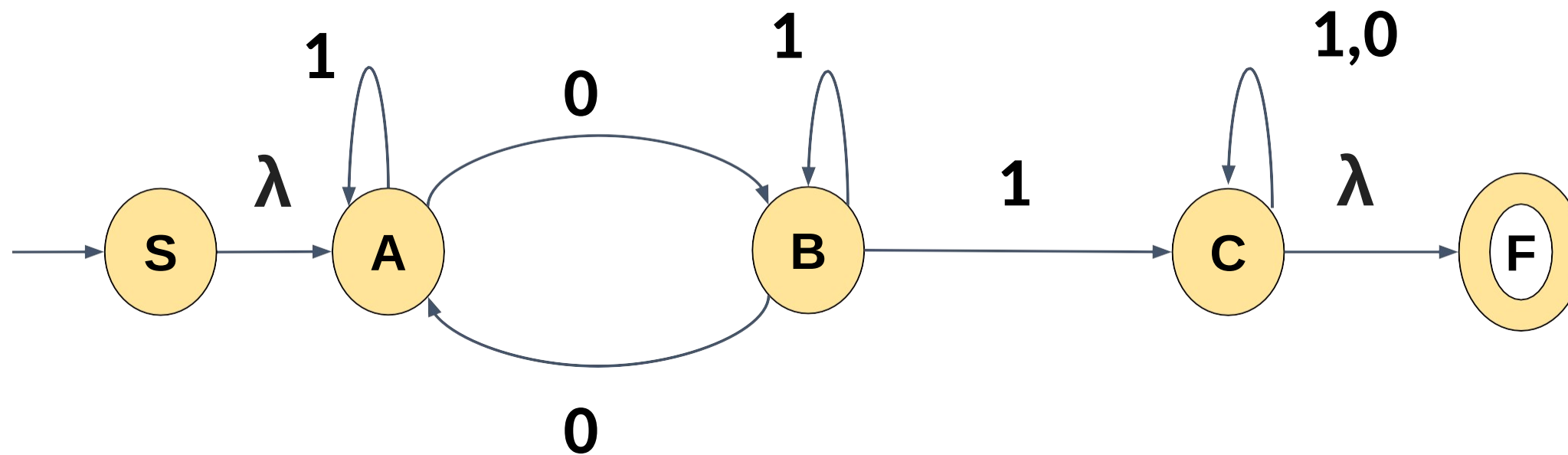
Example 7 :



A new start state (S) is introduced as there is an incoming edge to the existing start state

Unit 1 - Finite Automata to Regular Expression

Example 7 :

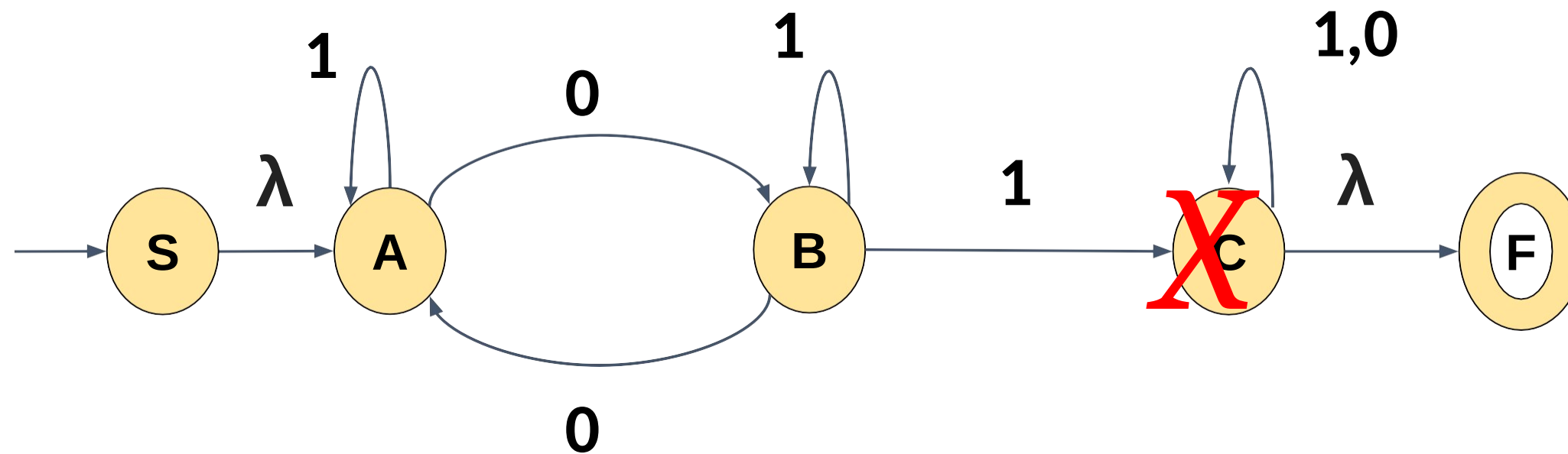


A new final state (F) is introduced as there is an outgoing edge from the existing final state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 7 :

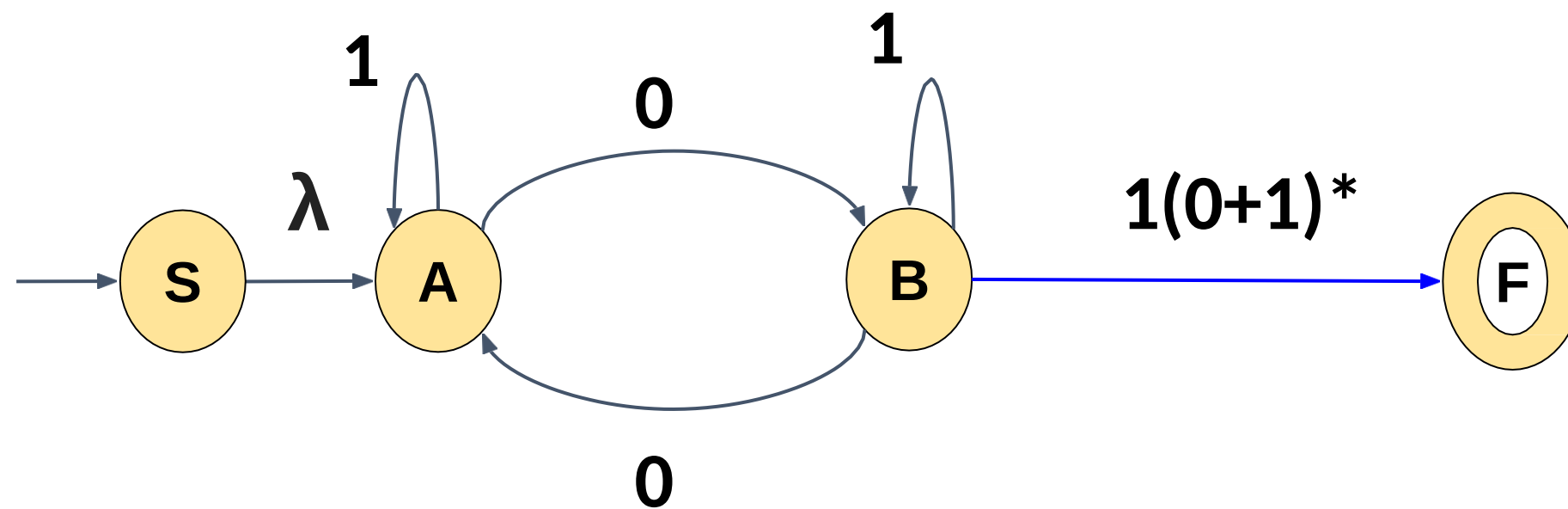


1. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 7 :

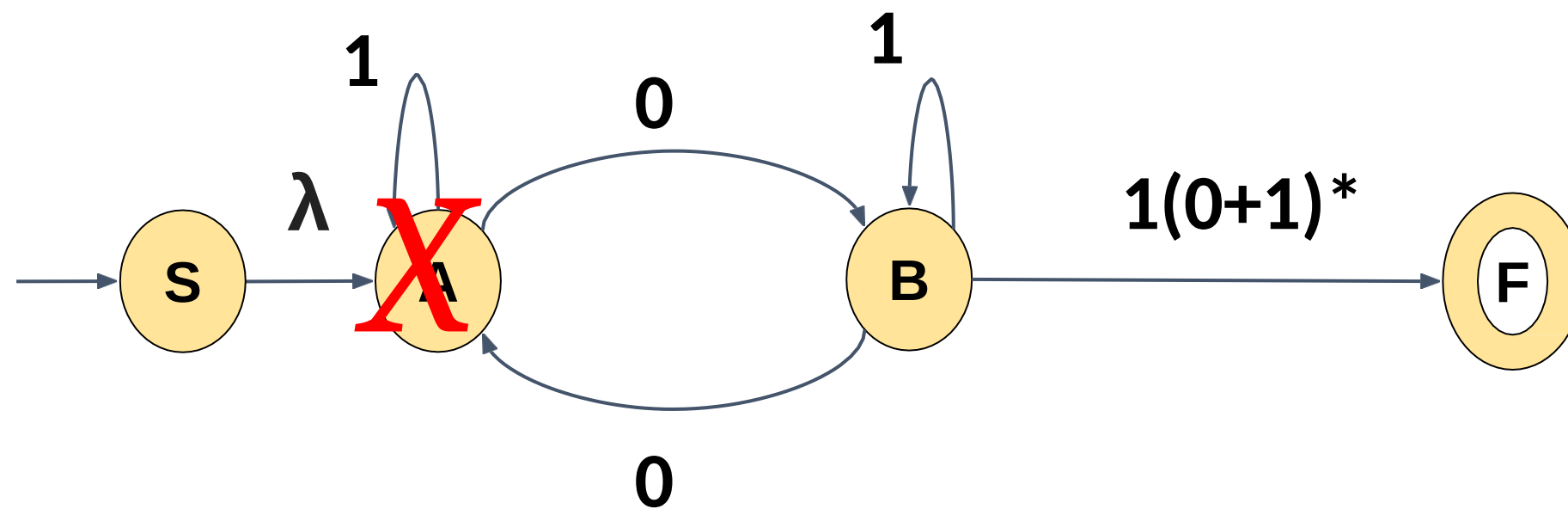


1. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 7 :

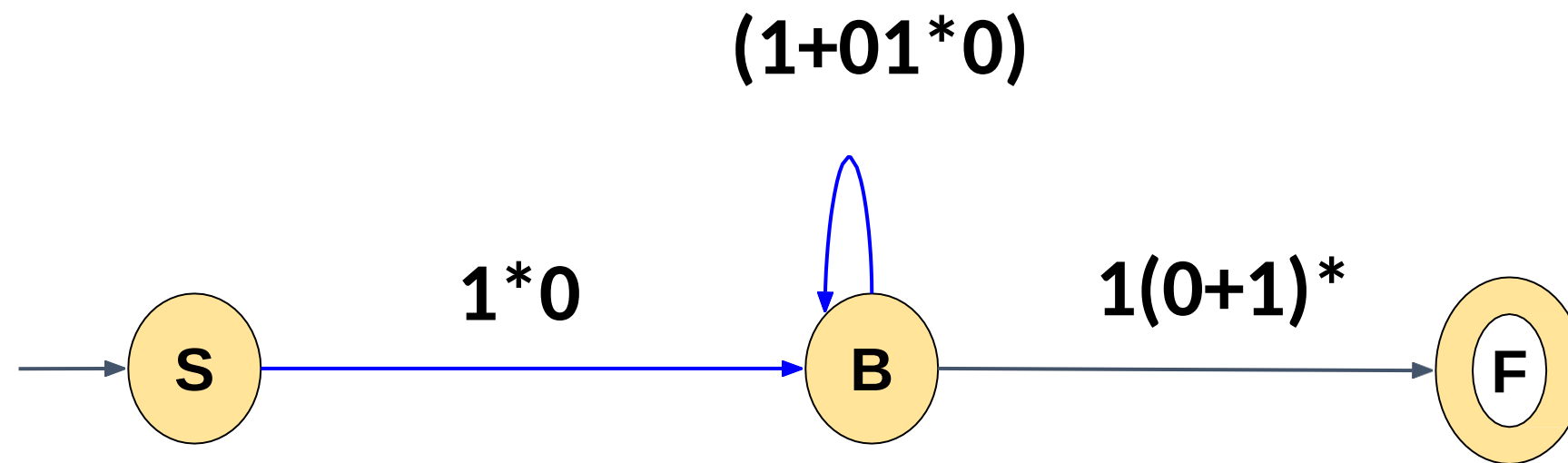


1. Eliminate C
2. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 7 :

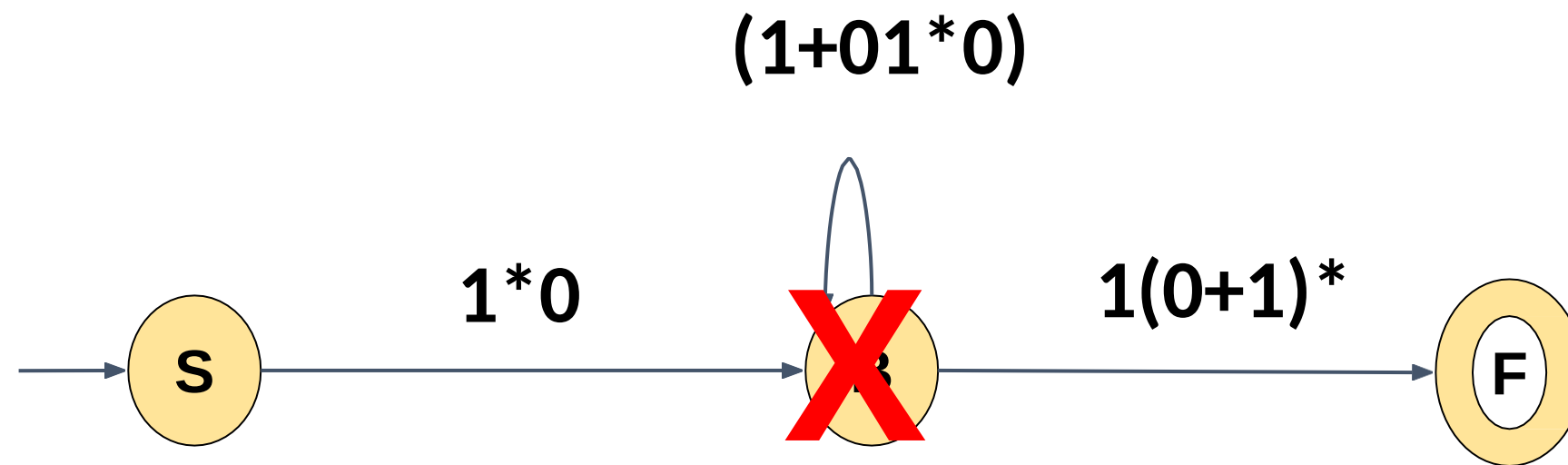


1. Eliminate C
2. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 7 :

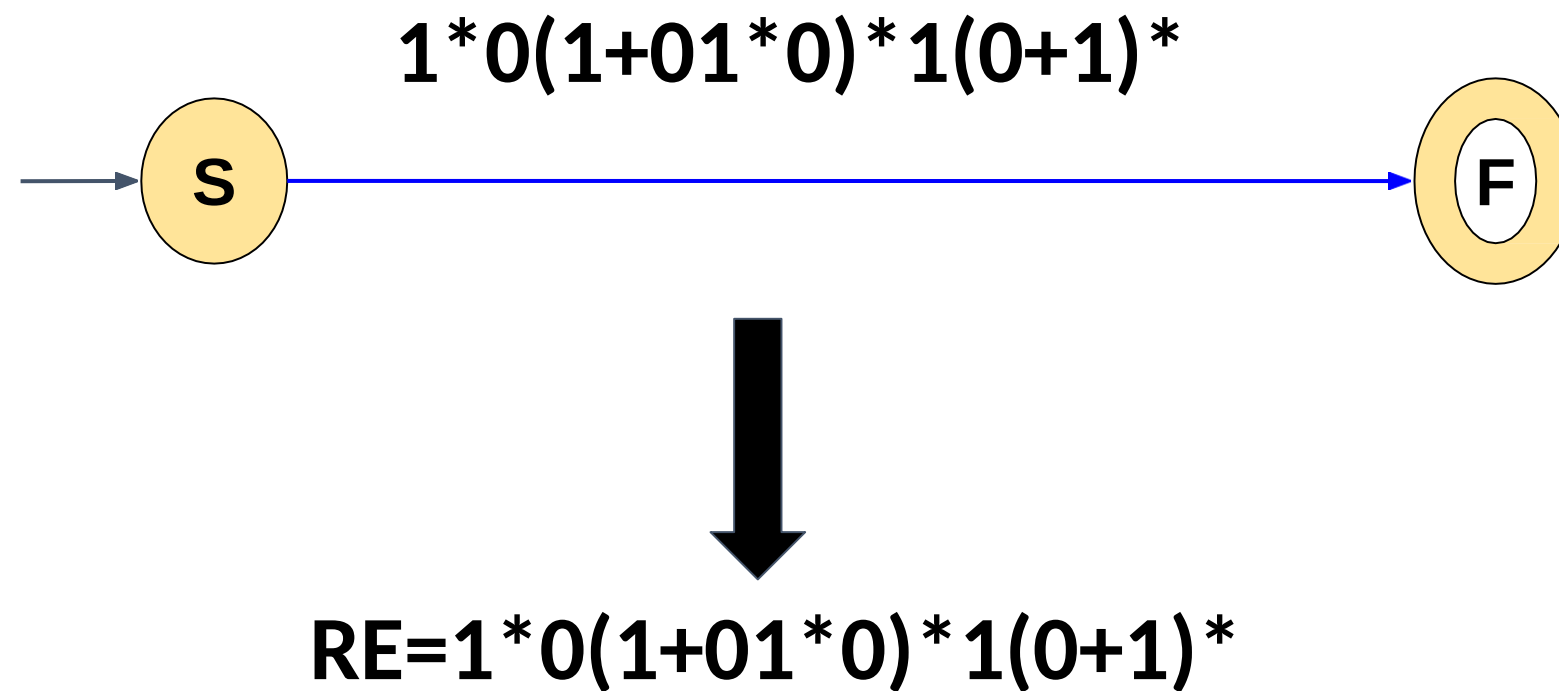


1. Eliminate C
2. Eliminate A
3. Eliminate B

Automata Formal Languages and Logic

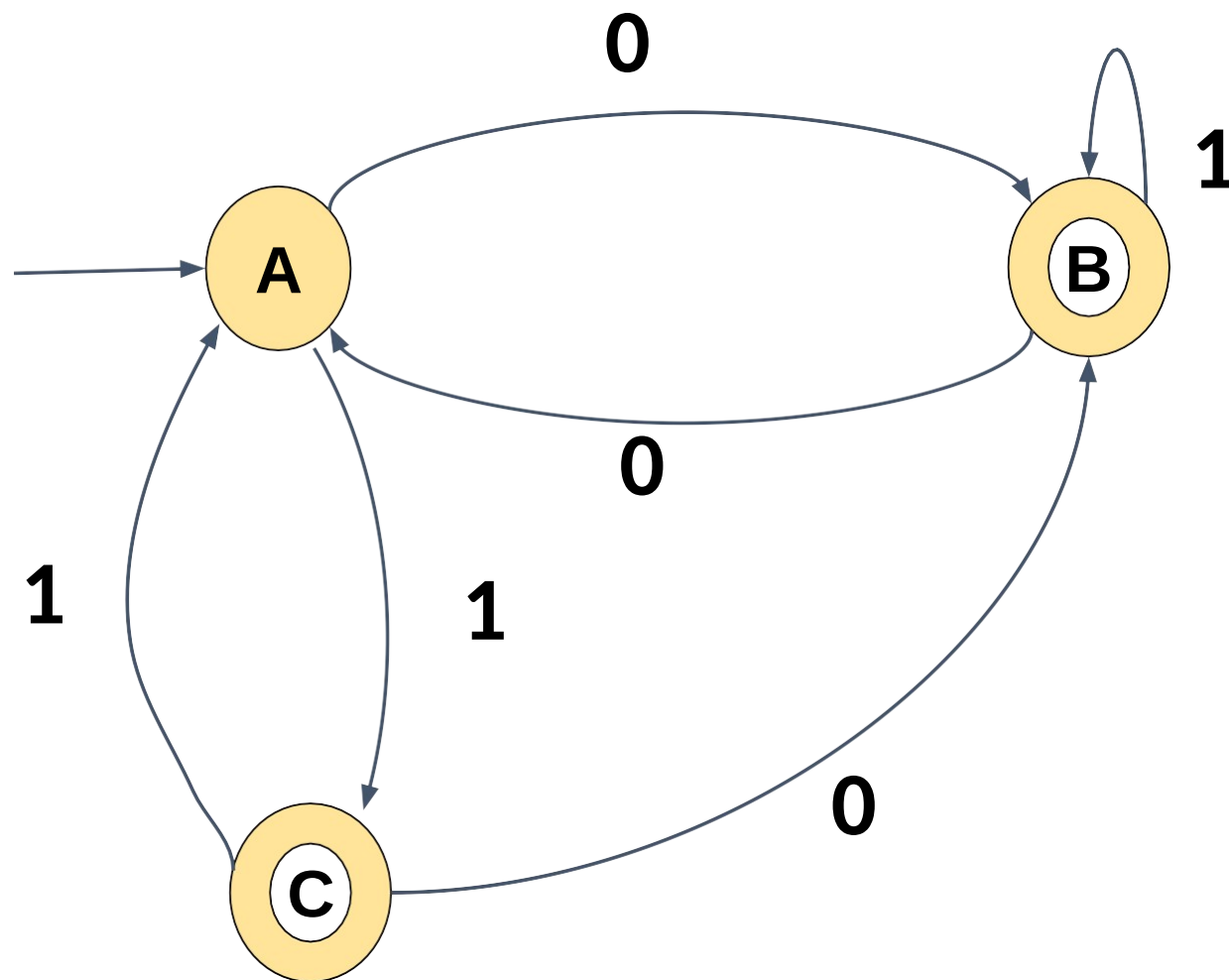
Unit 1 - Finite Automata to Regular Expression

Example 7 :



Unit 1 - Finite Automata to Regular Expression

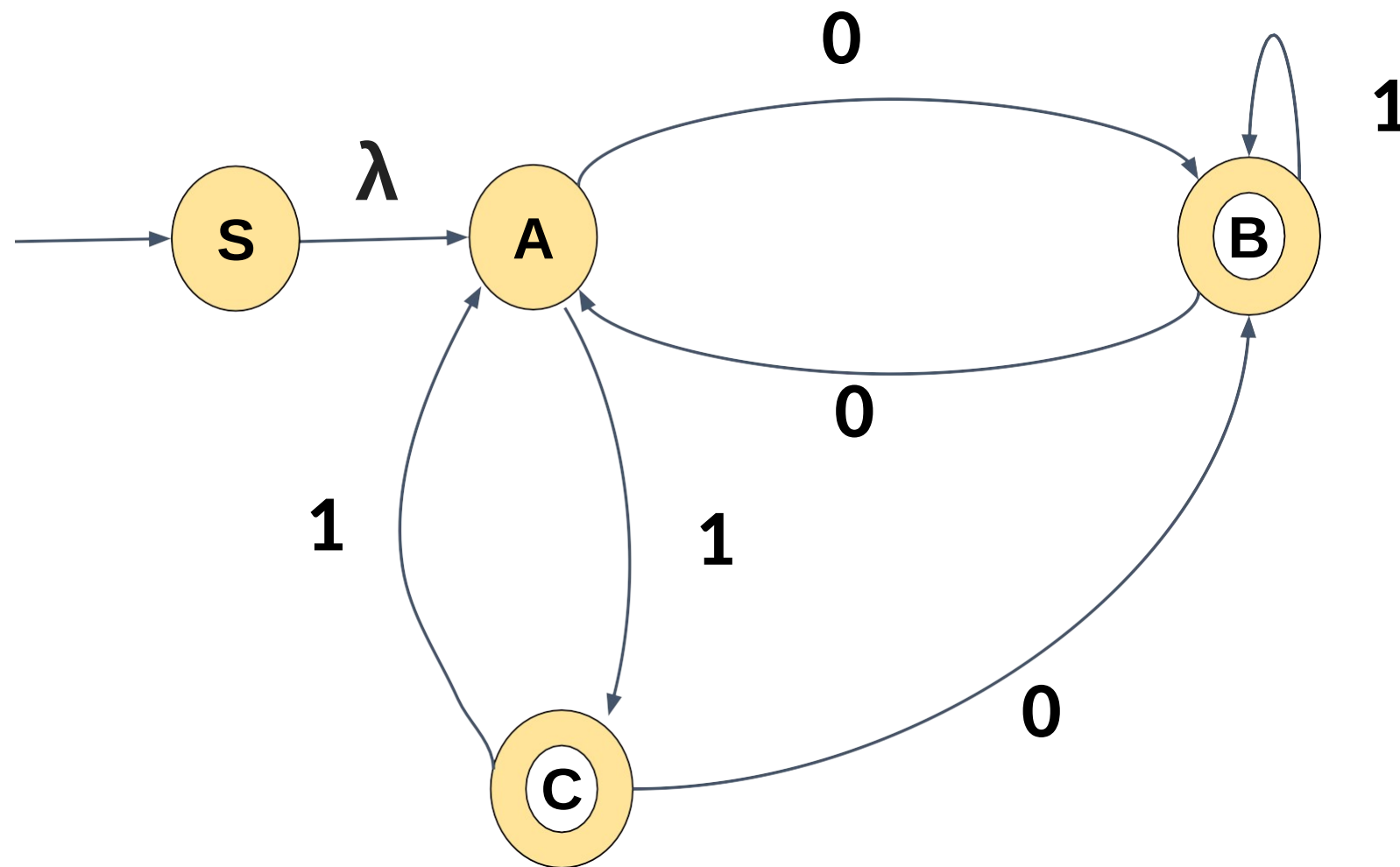
Example 8 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

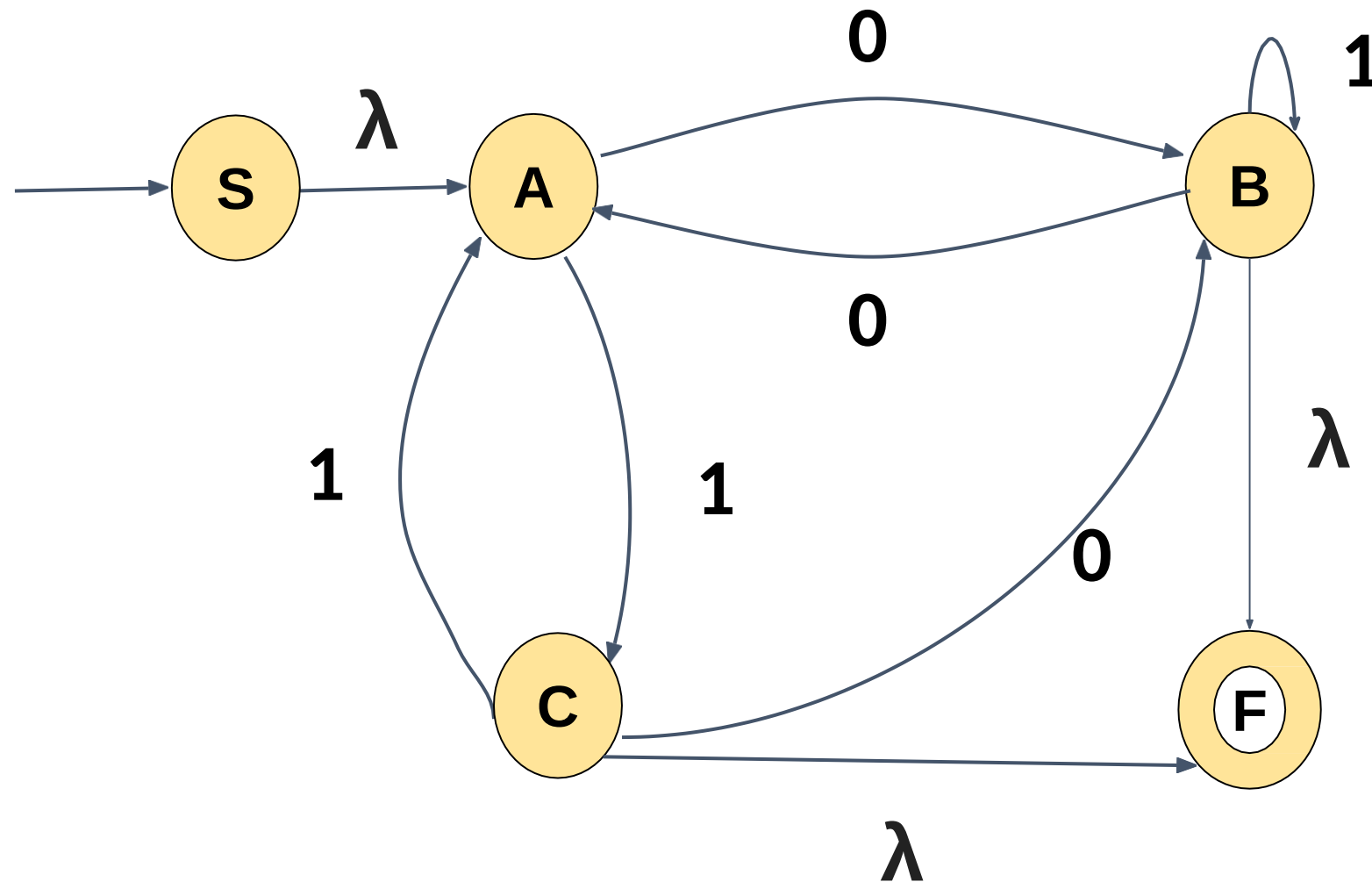


A new start state (S) is introduced as there is an incoming edge to the existing start state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

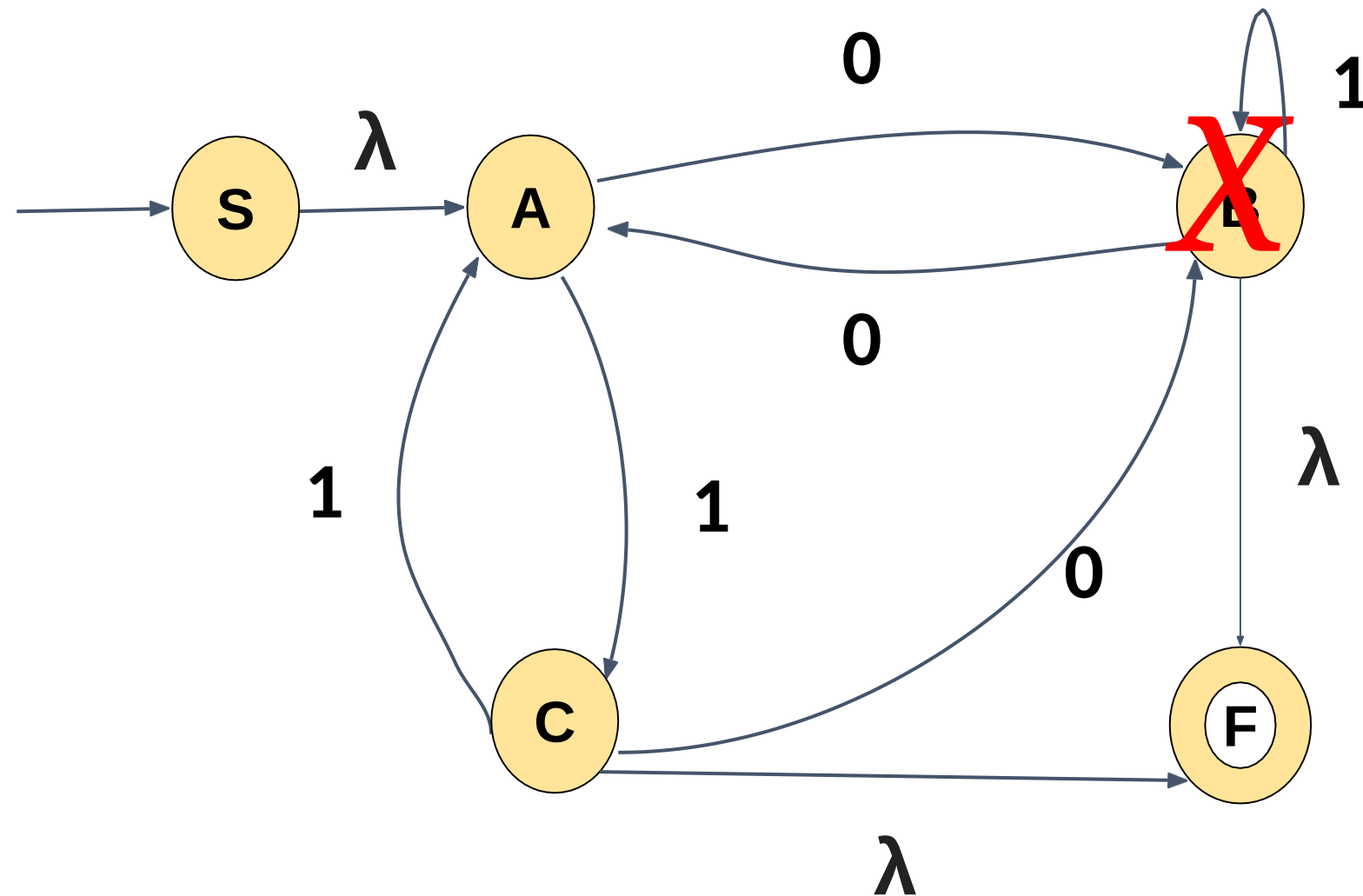


A new final state (F) is introduced as there is an outgoing edge from the existing final state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

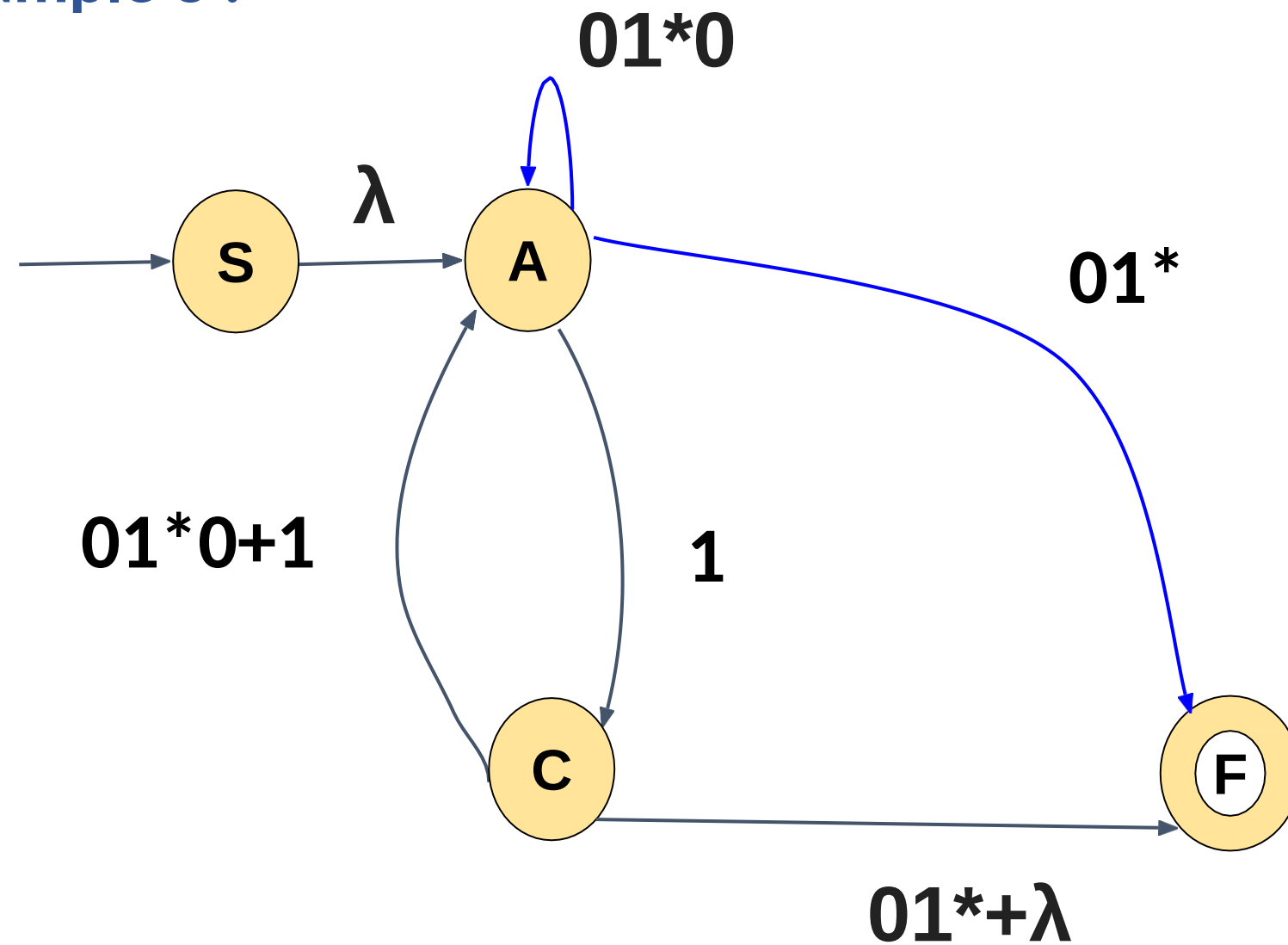


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

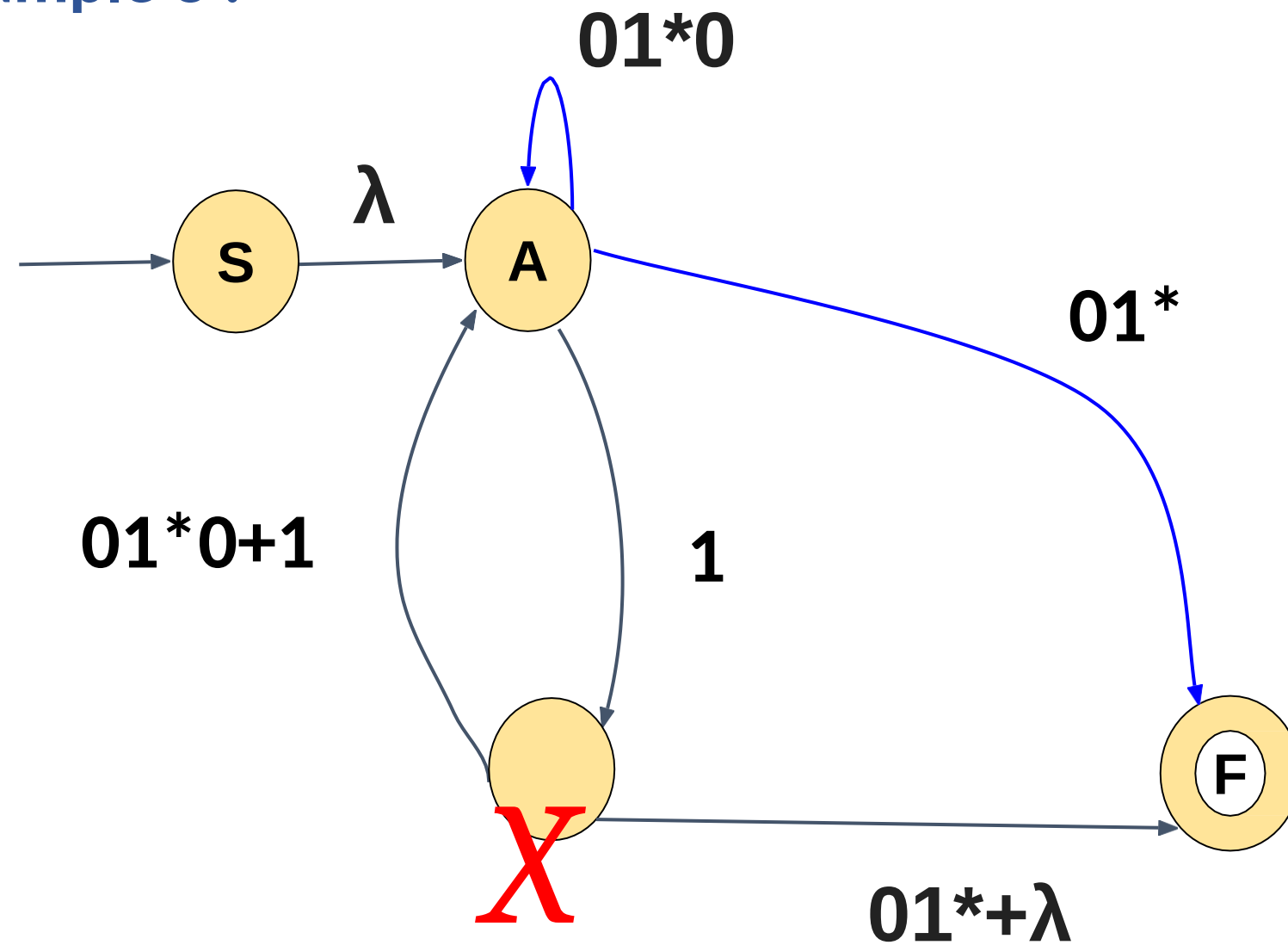


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

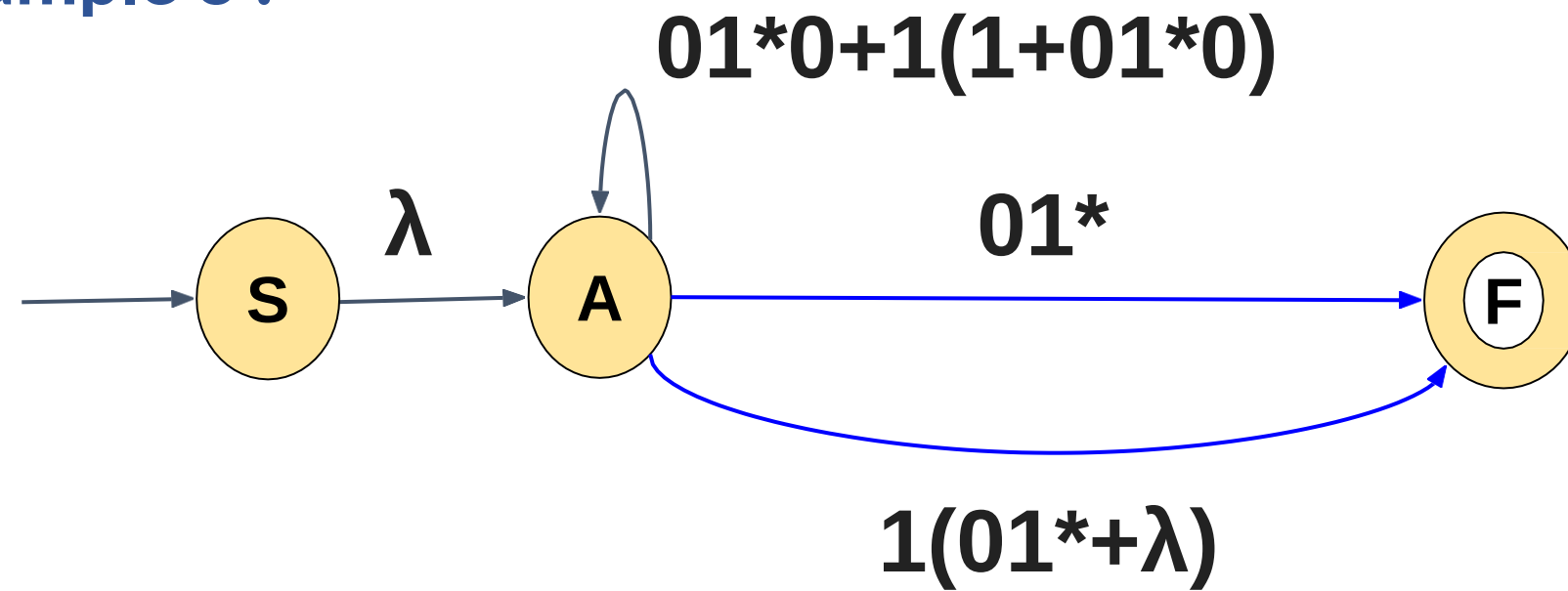


1. Eliminate B
2. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

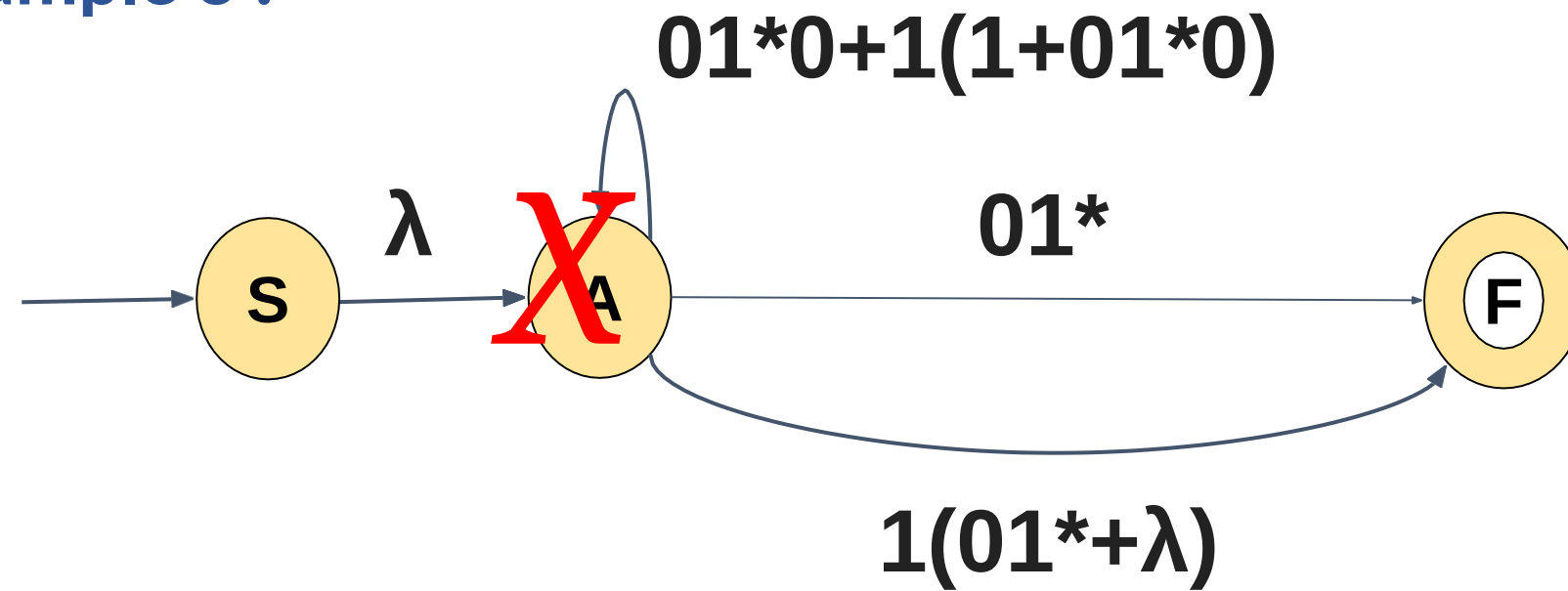


1. Eliminate B
2. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

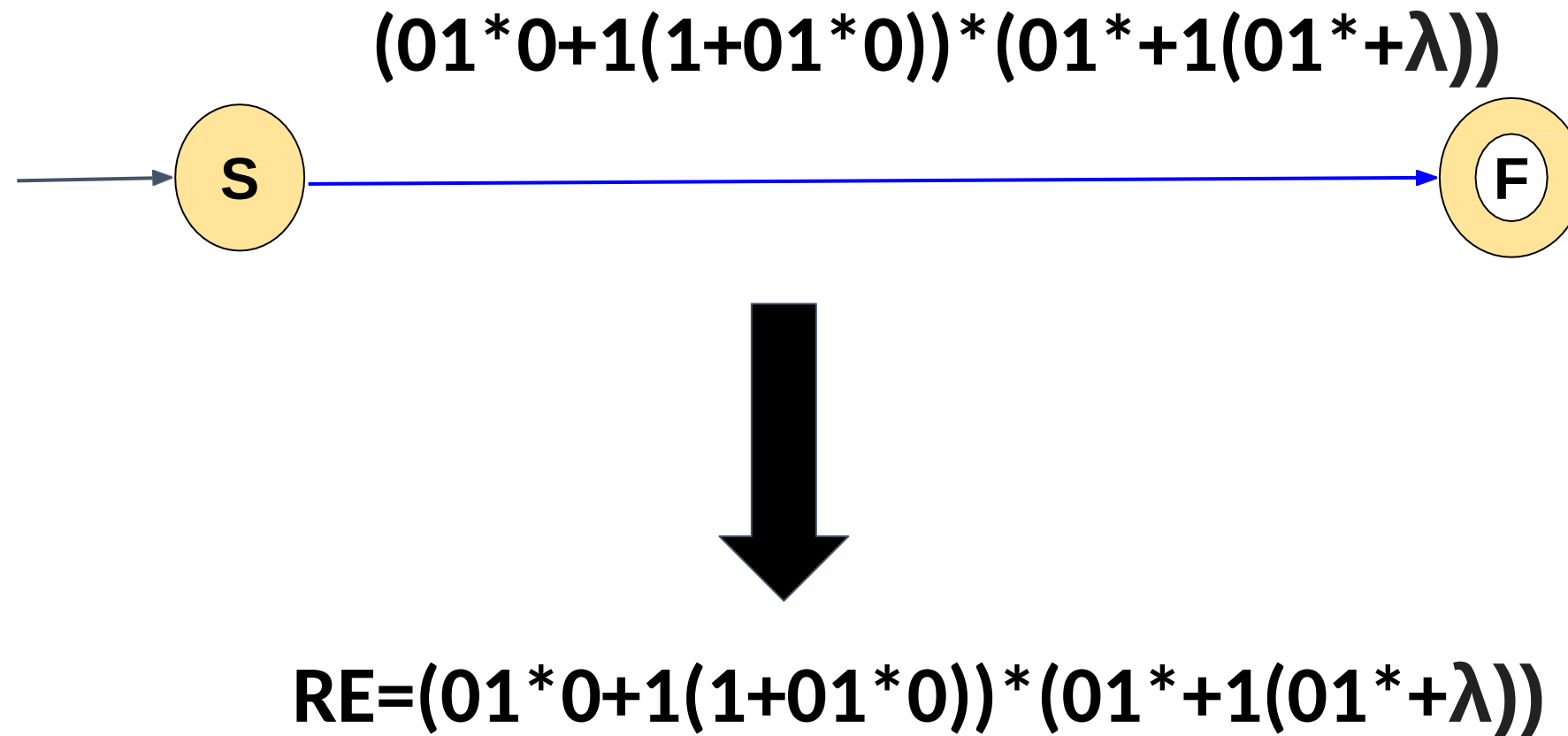


1. Eliminate B
2. Eliminate C
3. Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

Unit 1 - Finite Automata to Regular Expression

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda))^* (11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

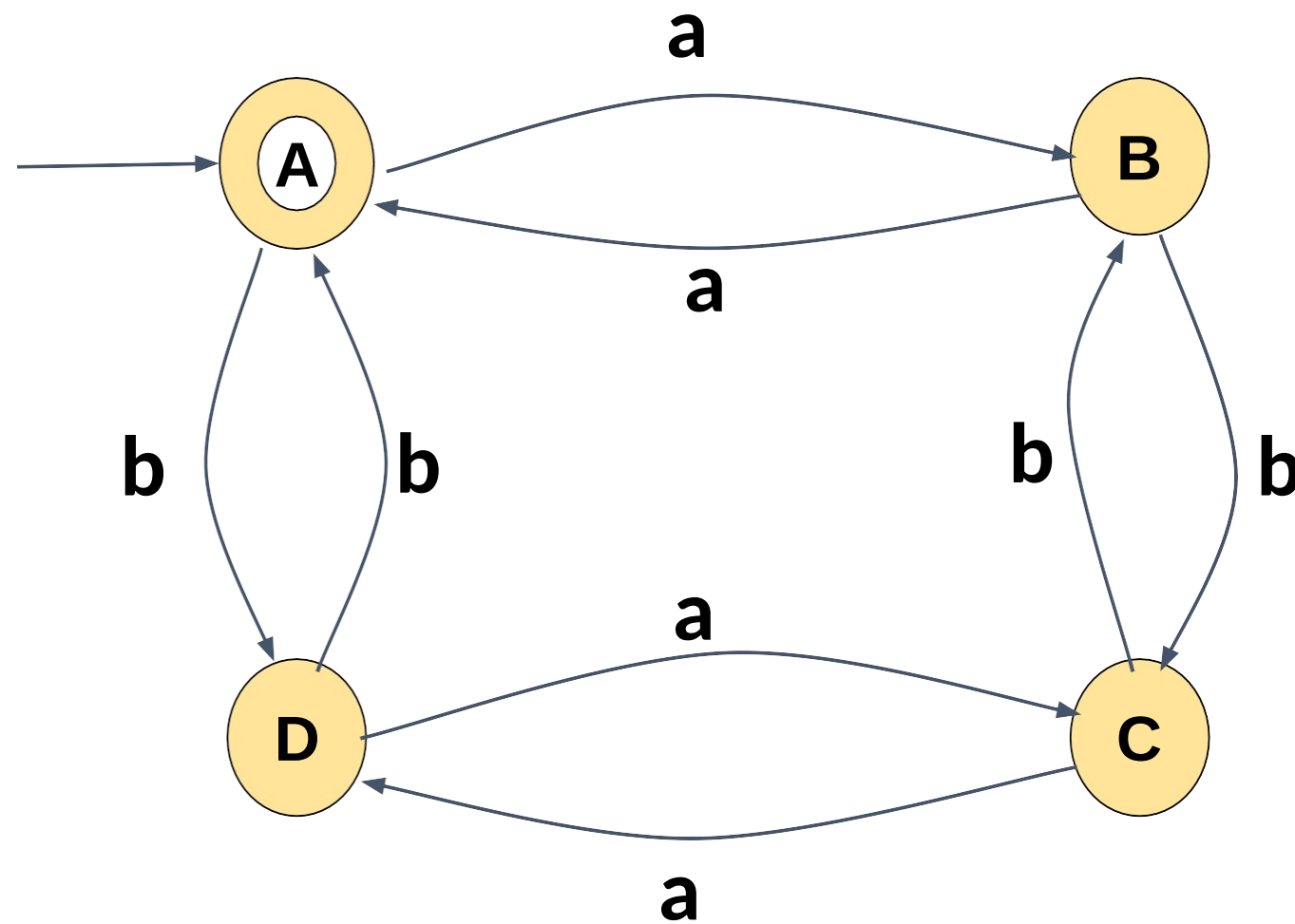
$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

$$RE = (01^*0(1 + \lambda))^* (11)^* (01^*(\lambda + 1) + 1)$$

Unit 1 - Finite Automata to Regular Expression

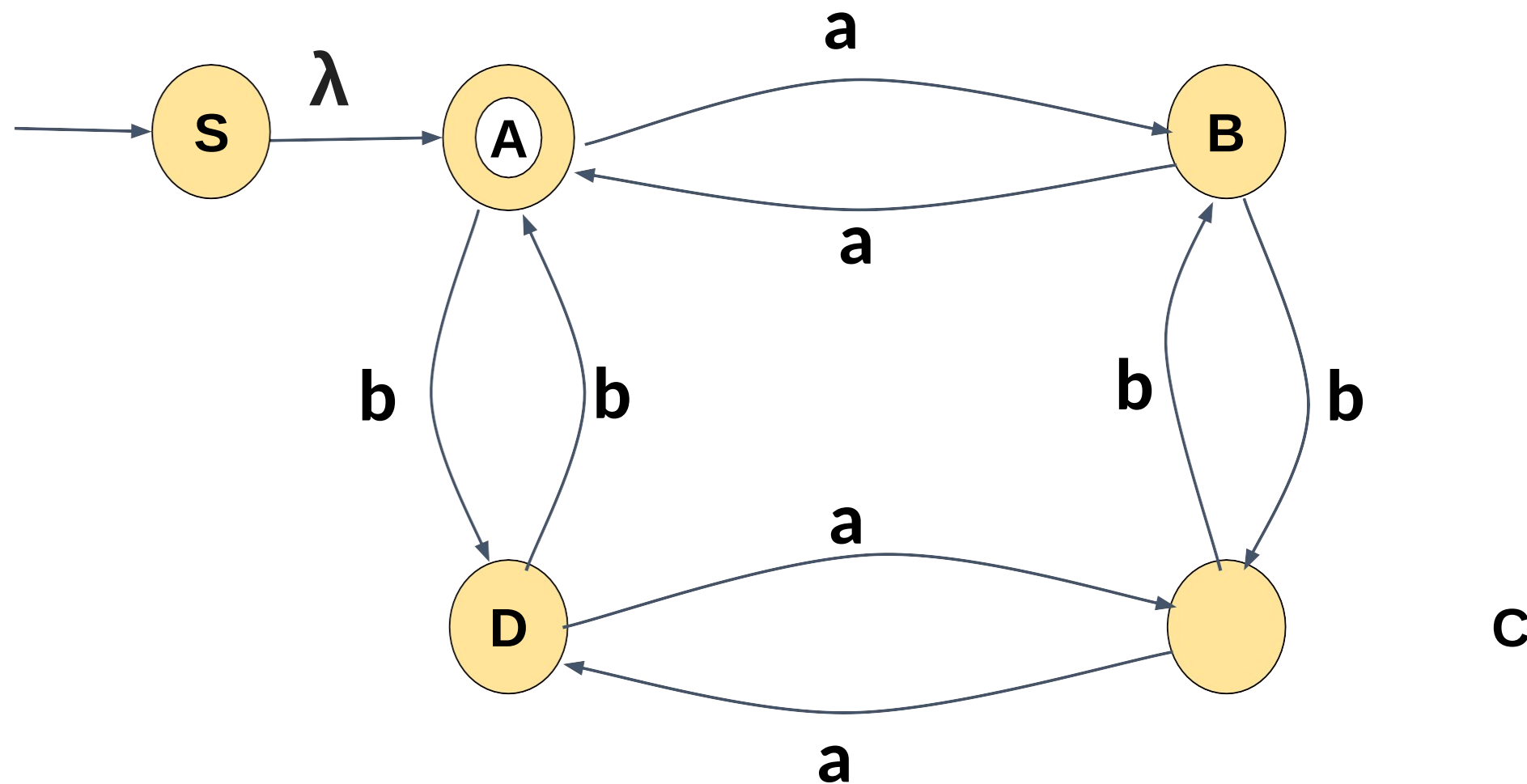
Example 9 :order of elimination(B,D,C,A)



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

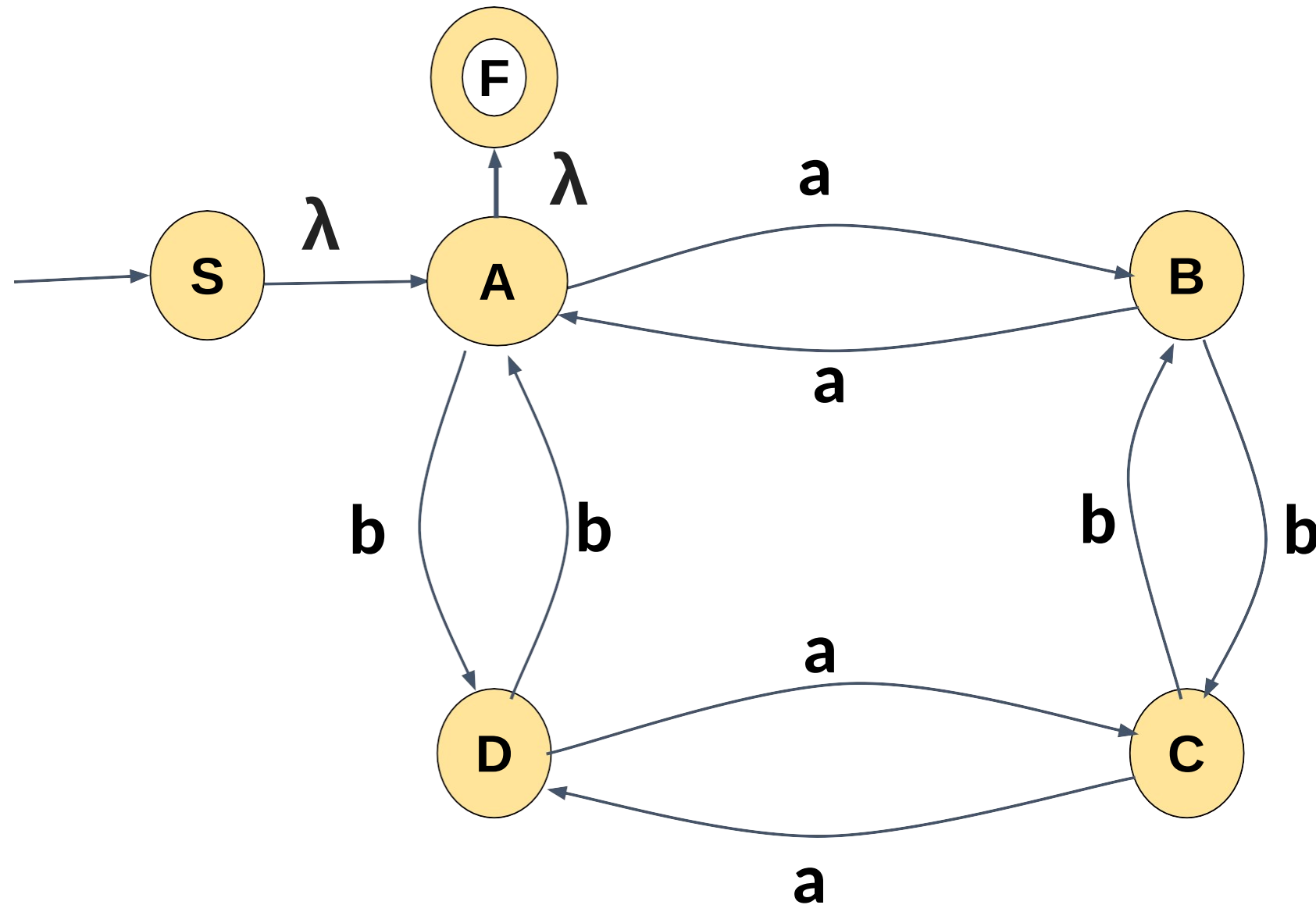


A new start state (S) is introduced as there is an incoming edge to the existing start state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

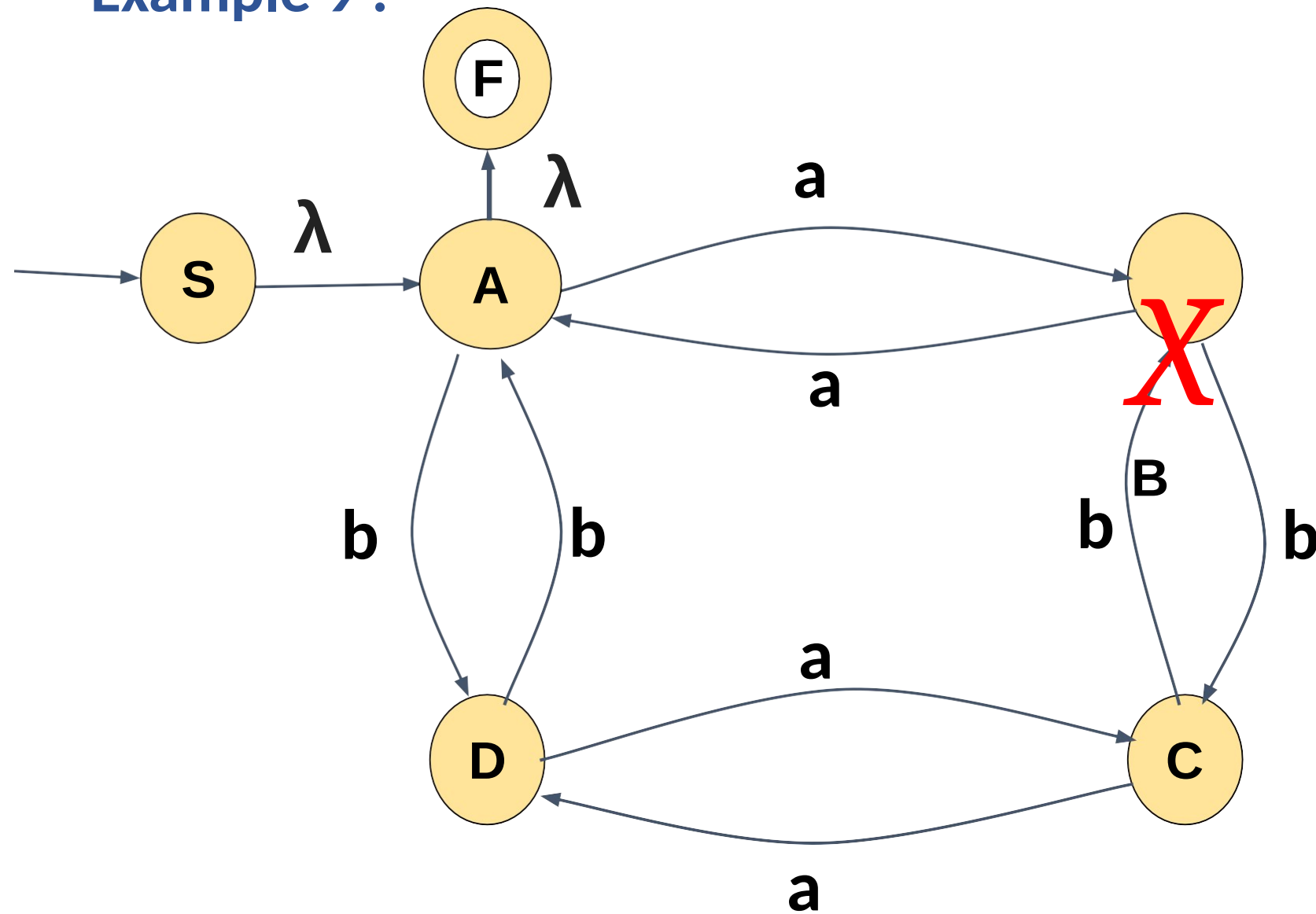


A new final state (F) is introduced as there is an outgoing edge from the existing final state

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

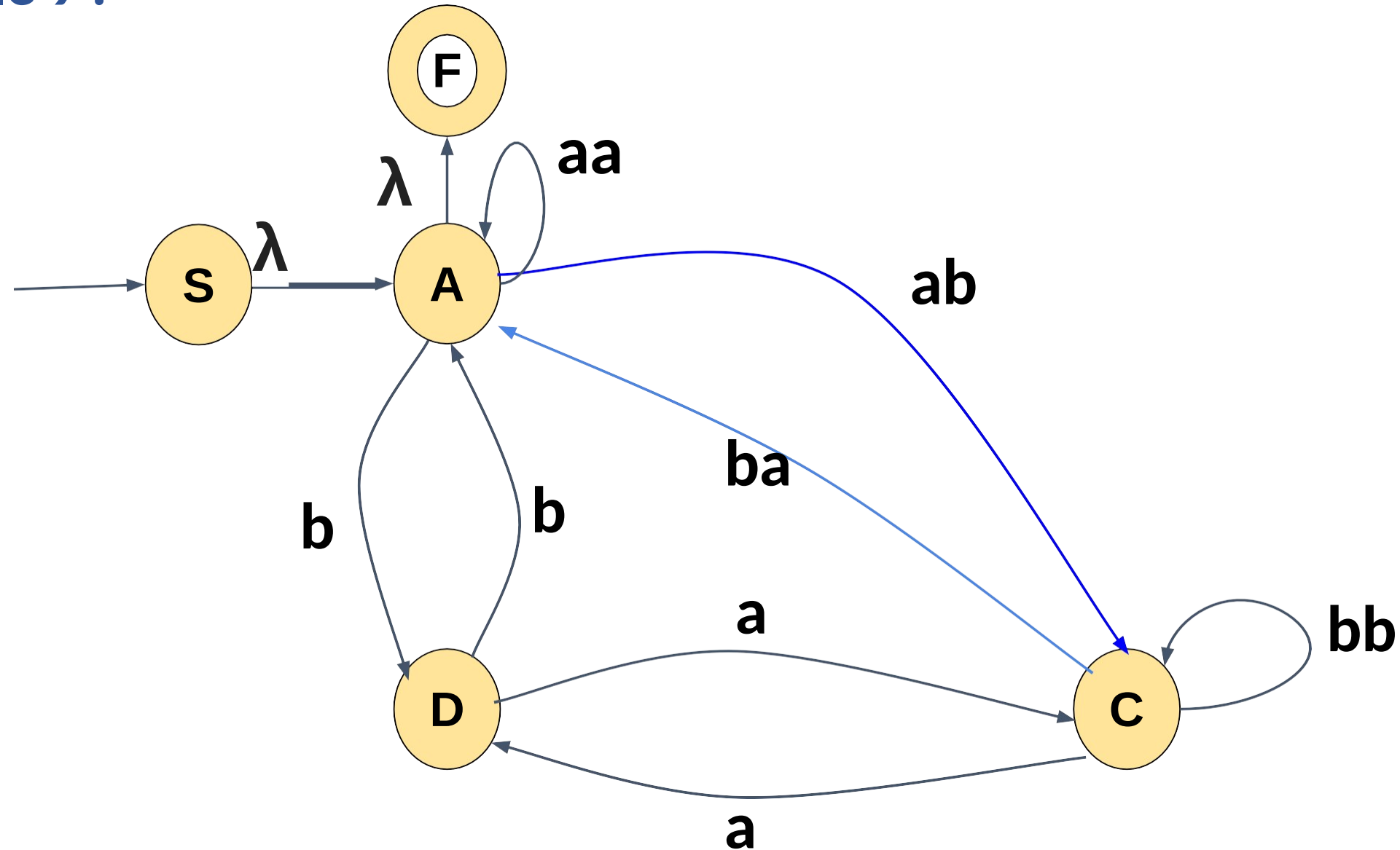
Example 9 :



1. Eliminate B

Unit 1 - Finite Automata to Regular Expression

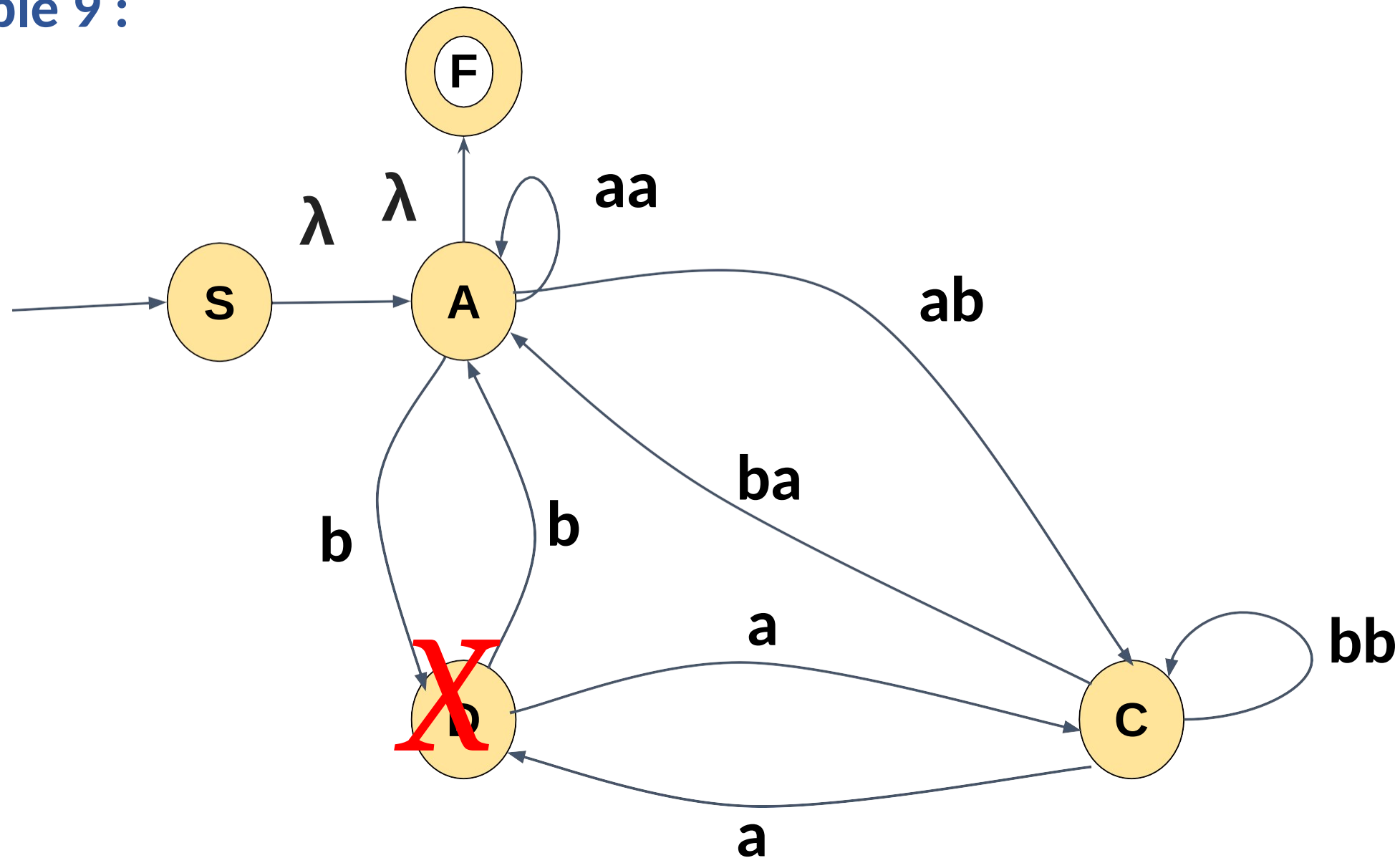
Example 9 :



1. Eliminate B

Unit 1 - Finite Automata to Regular Expression

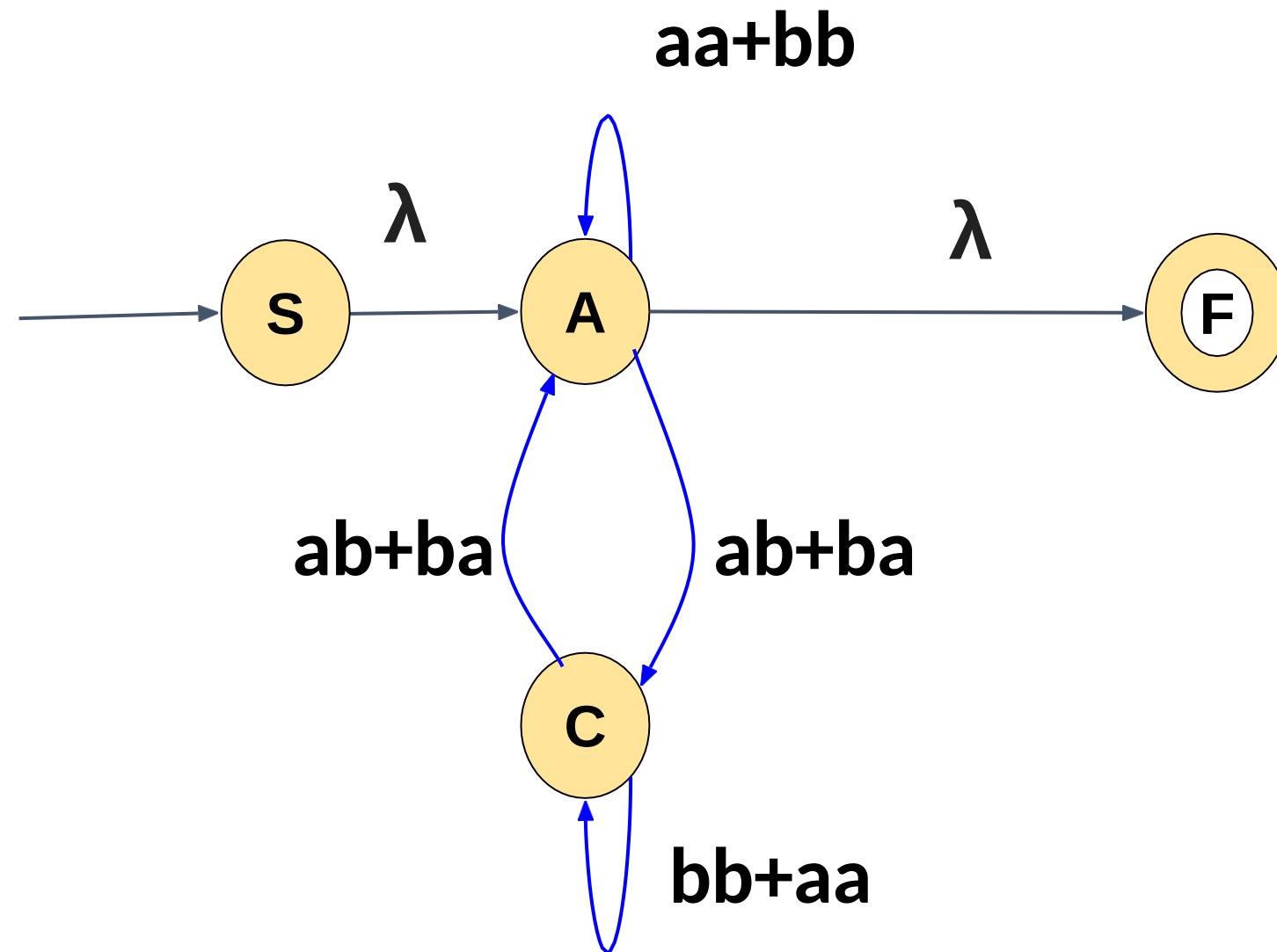
Example 9 :



1. Eliminate B
2. Eliminate D

Unit 1 - Finite Automata to Regular Expression

Example 9 :

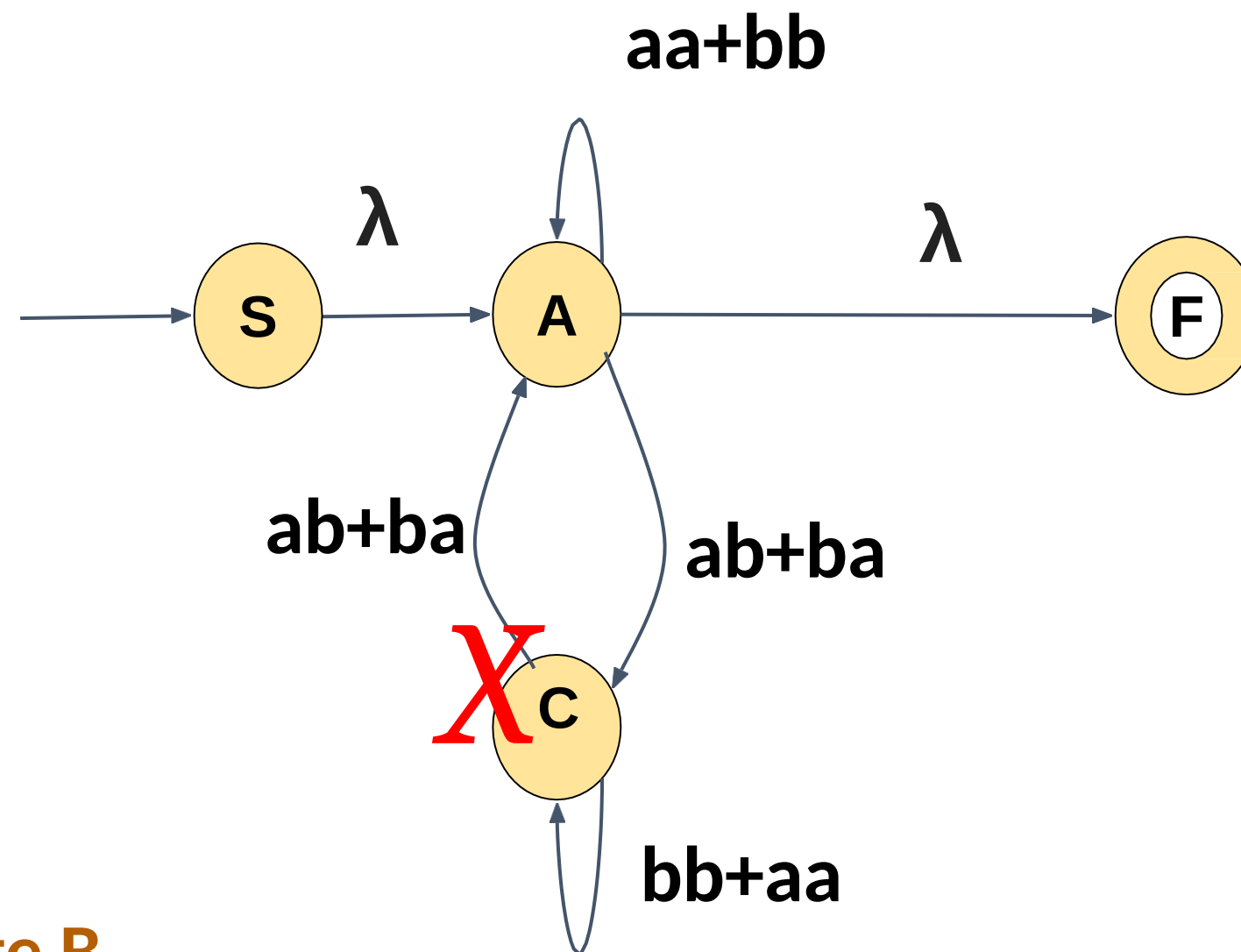


1. Eliminate B
2. Eliminate D

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

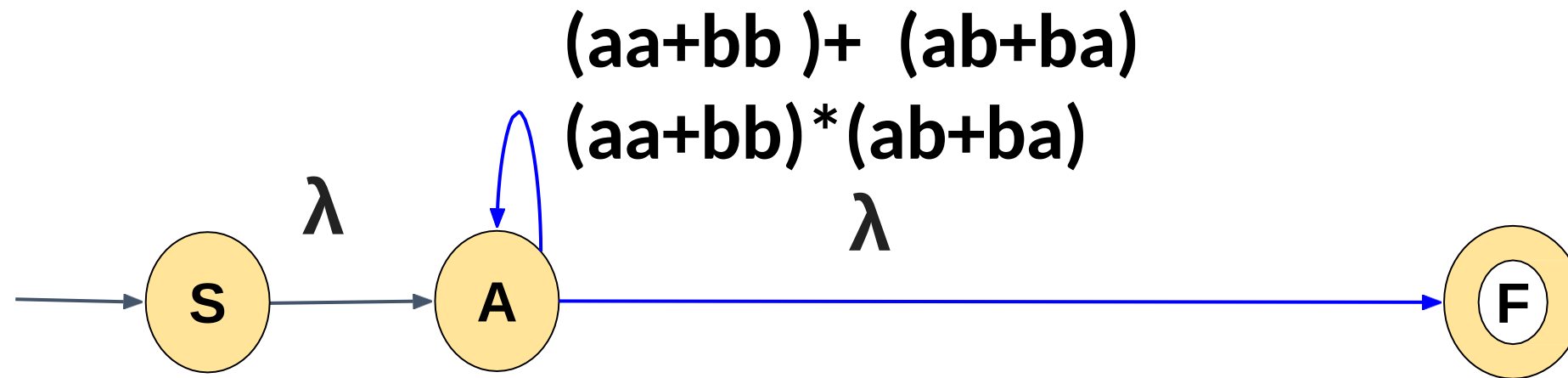


1. Eliminate B
2. Eliminate D
3. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

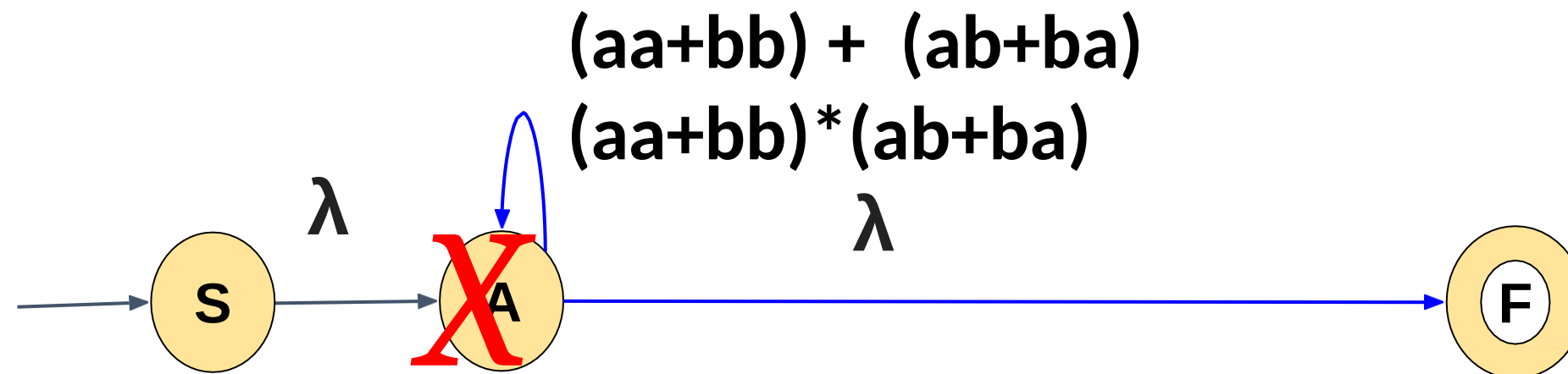


1. Eliminate B
2. Eliminate D
3. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

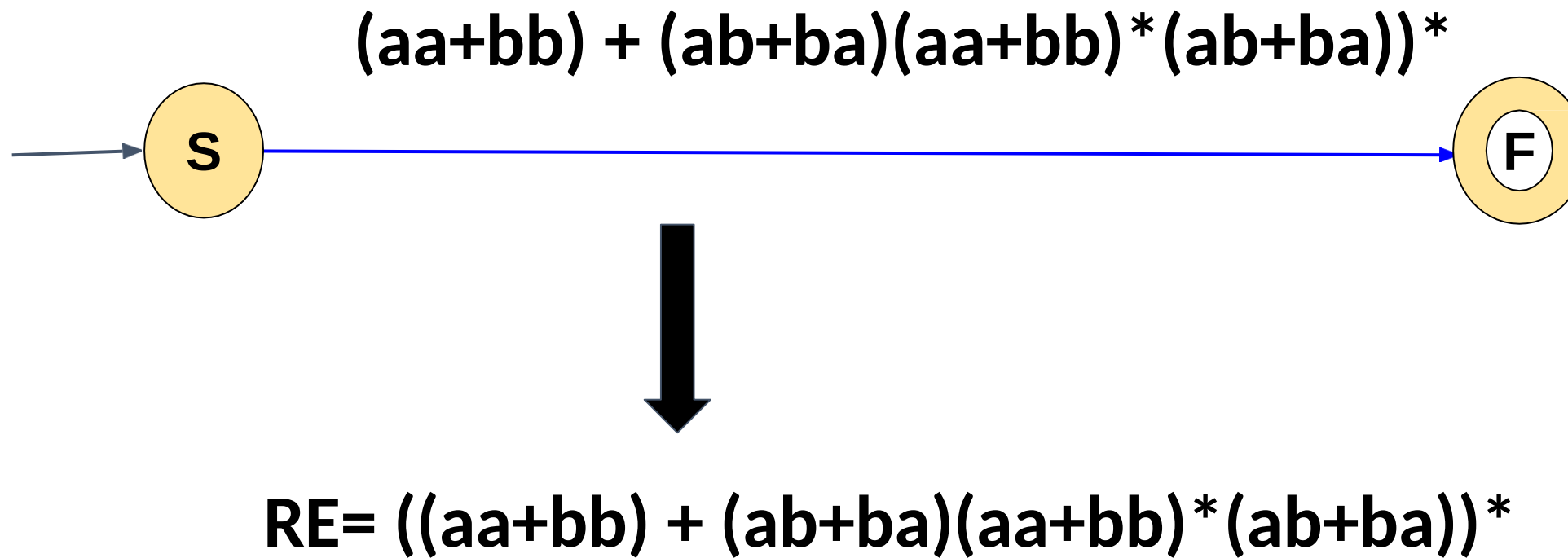
Example 9 :



1. Eliminate B
2. Eliminate D
3. Eliminate C
4. Eliminate A

Unit 1 - Finite Automata to Regular Expression

Example 9 :

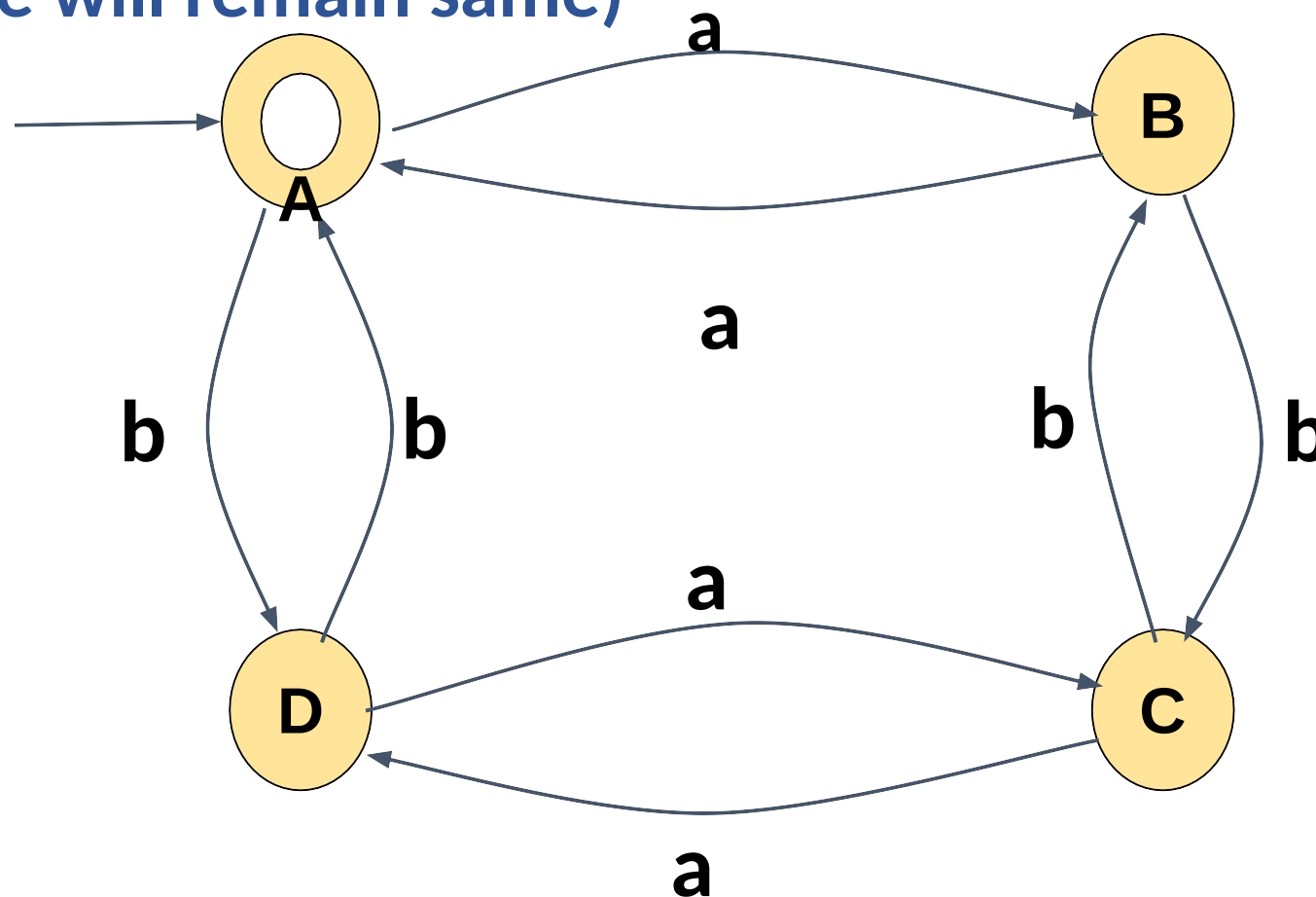


Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

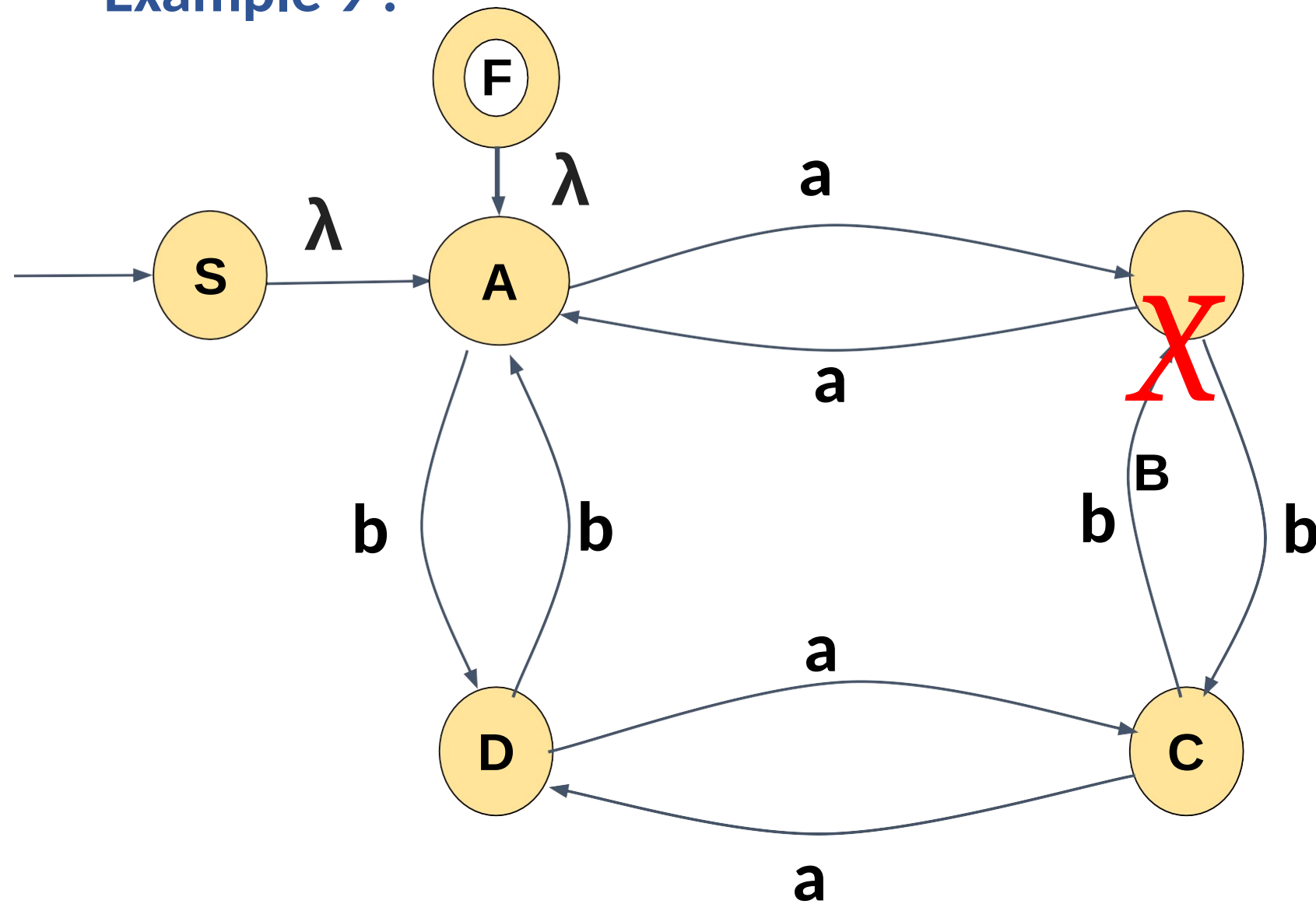
Consider the same example: Order of elimination: B,C, D ,A (Adding new start state and new final state will remain same)



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

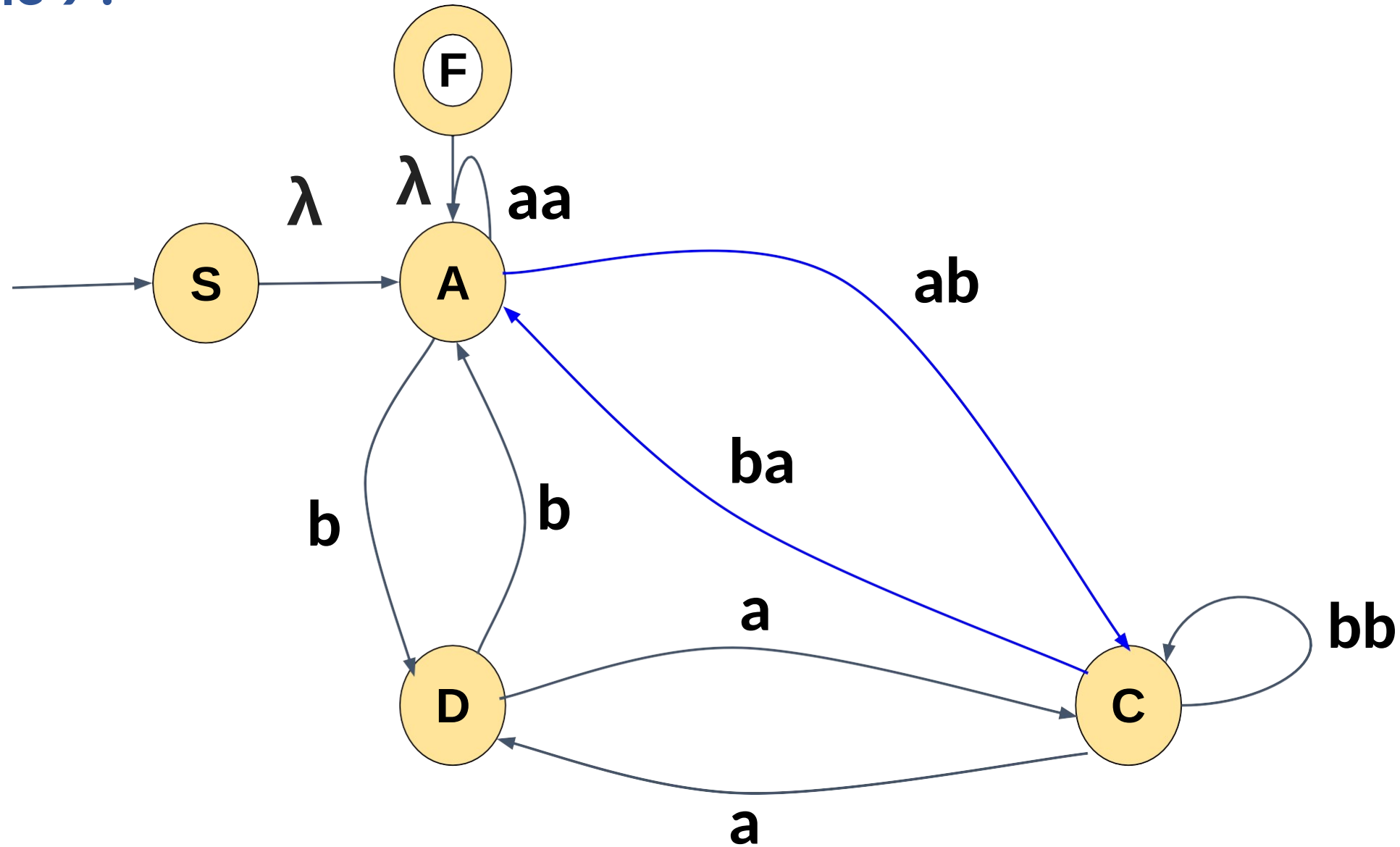
Example 9 :



1. Eliminate B

Unit 1 - Finite Automata to Regular Expression

Example 9 :

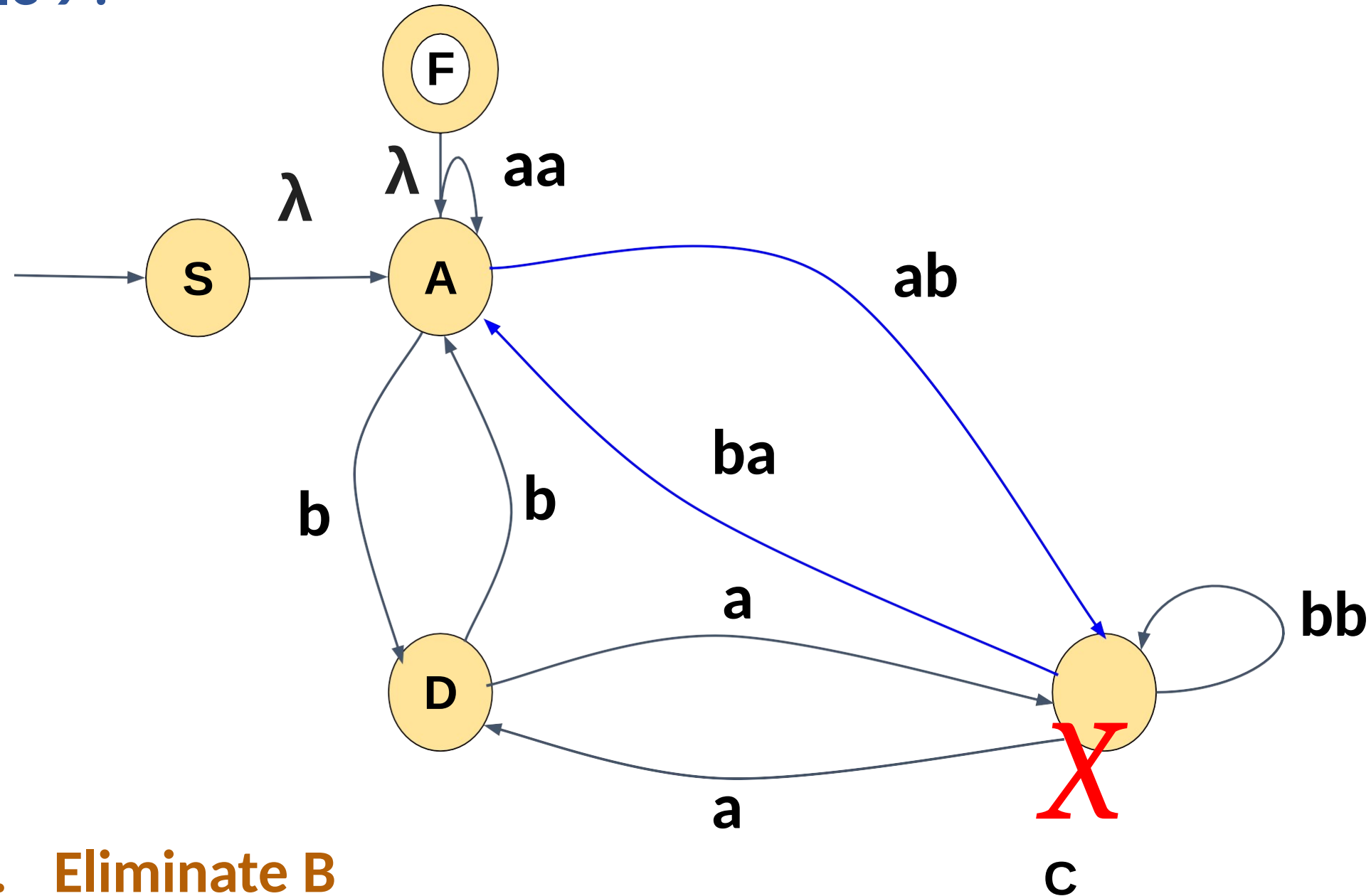


1. Eliminate B

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

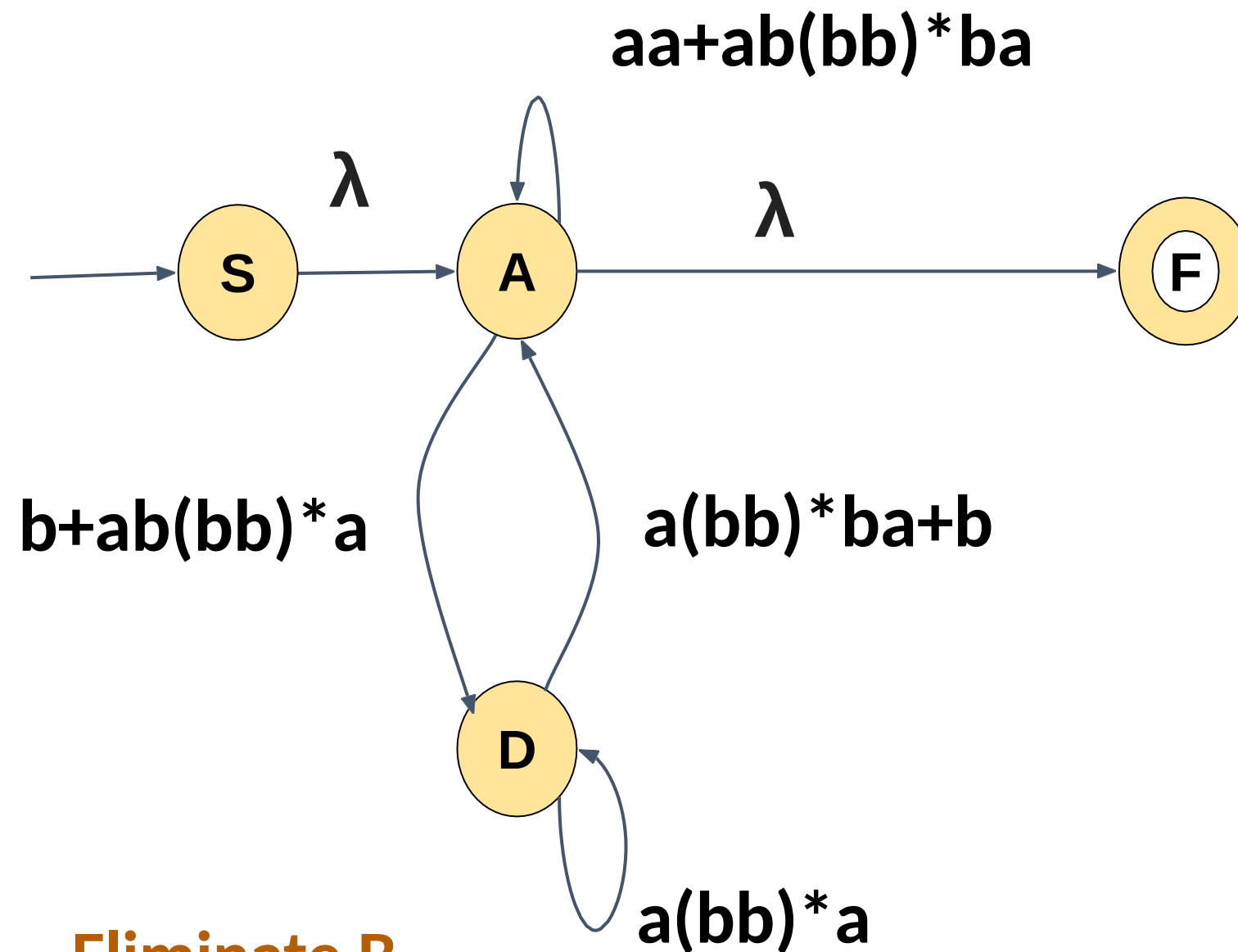


1. Eliminate B
2. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

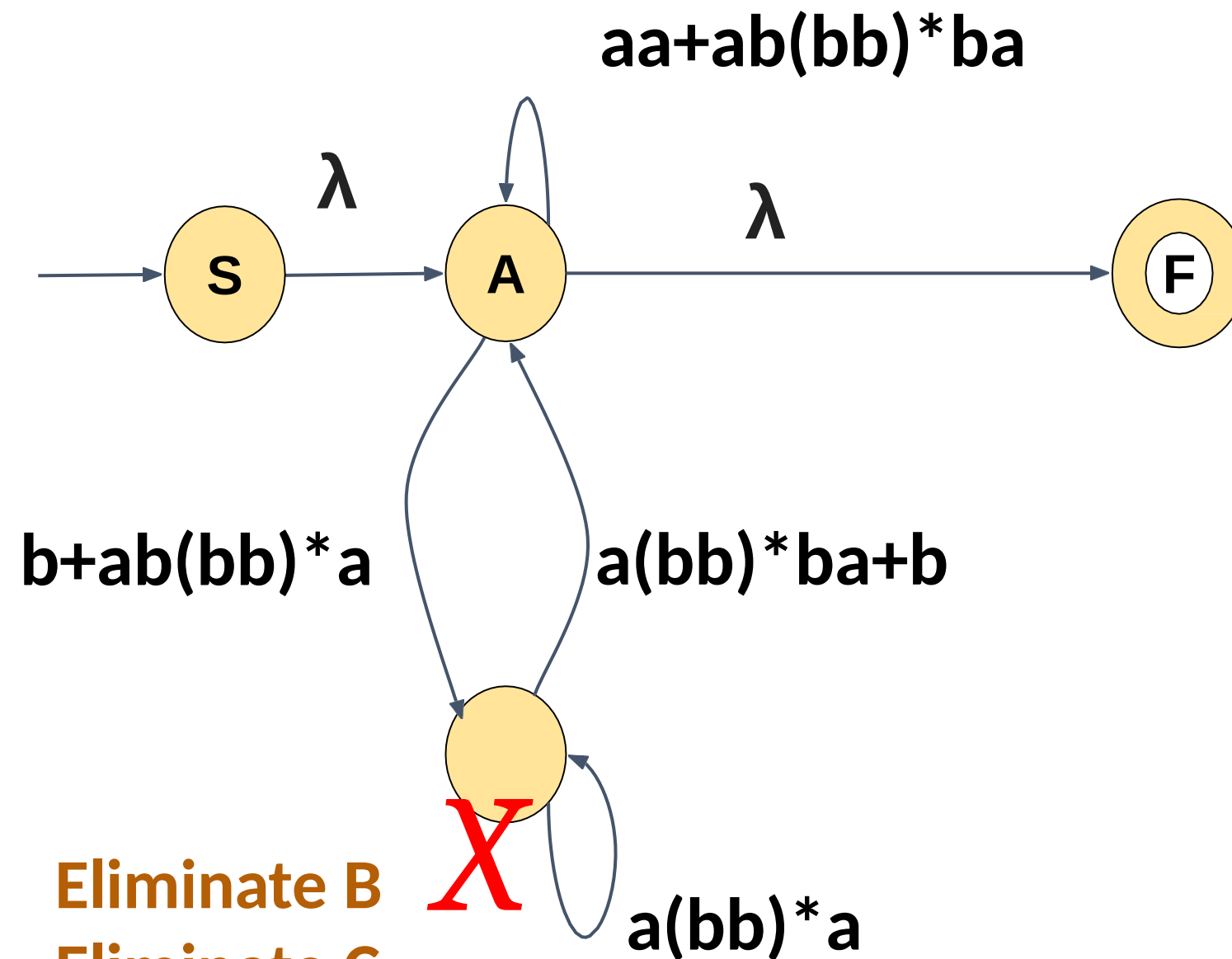


1. Eliminate B
2. Eliminate C

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :



1. Eliminate B
2. Eliminate C
3. Eliminate D

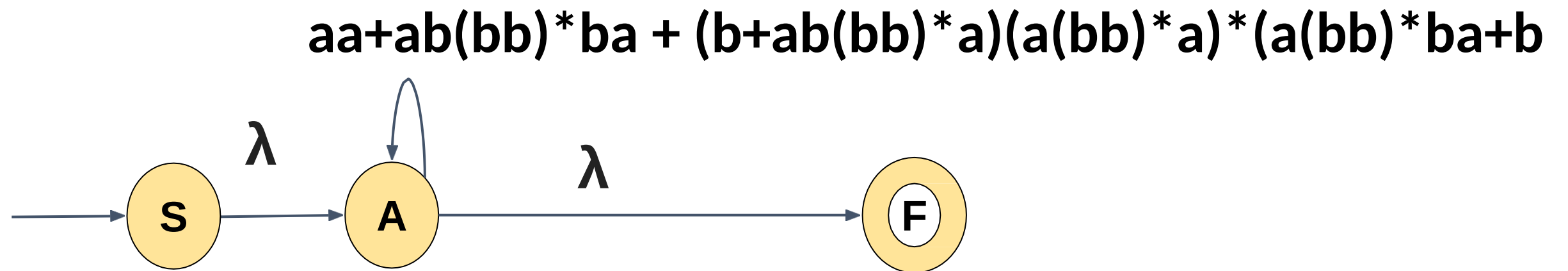
X

D

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :



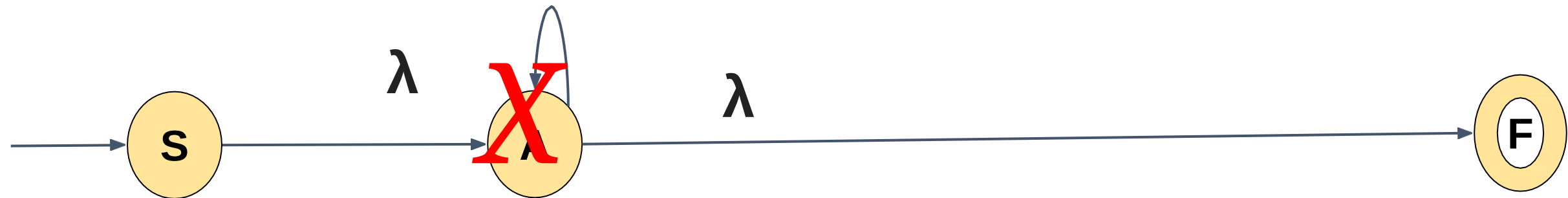
1. Eliminate B
2. Eliminate C
3. Eliminate D

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

$(aa+ab(bb)^*ba + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b)$

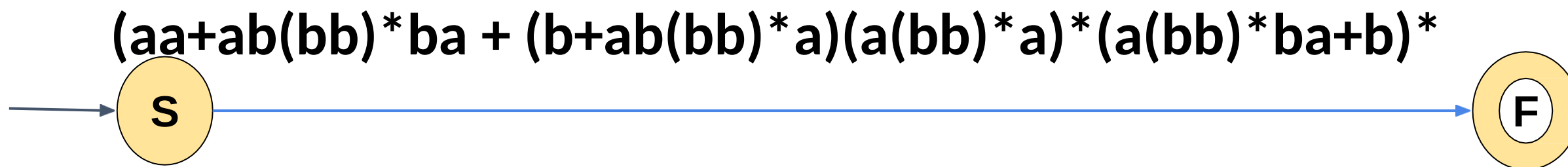


1. Eliminate B
2. Eliminate C
3. Eliminate
4. D
- Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

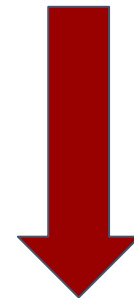
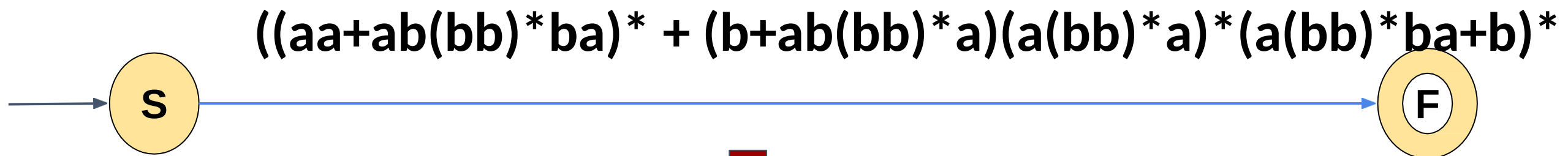


1. Eliminate B
 2. Eliminate C
 3. Eliminate
 4. D
- Eliminate A

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :

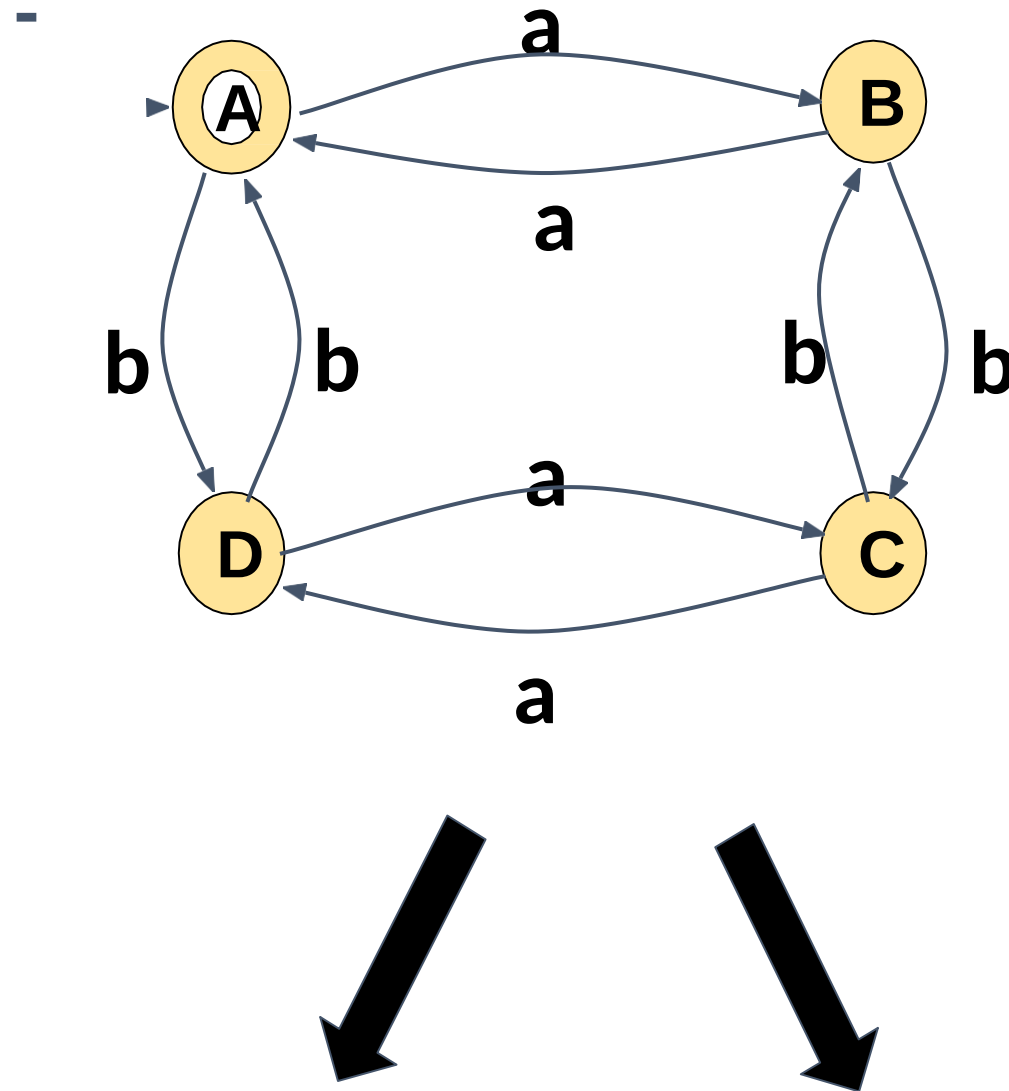


$$RE = ((aa+ab(bb)^*ba) + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b))^*$$

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Expression

Example 9 :



$(aa+bb + (ab+ba+(aa+bb)^*(ab+ba))^*$

Eliminate : B,D,C,A

$((aa+ab(bb)^*ba)^* + (b+ab(bb)^*a)$

$(a(bb)^*a)^*(a(bb)^*ba+b))^*$ Eliminate

B,C,D,A



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

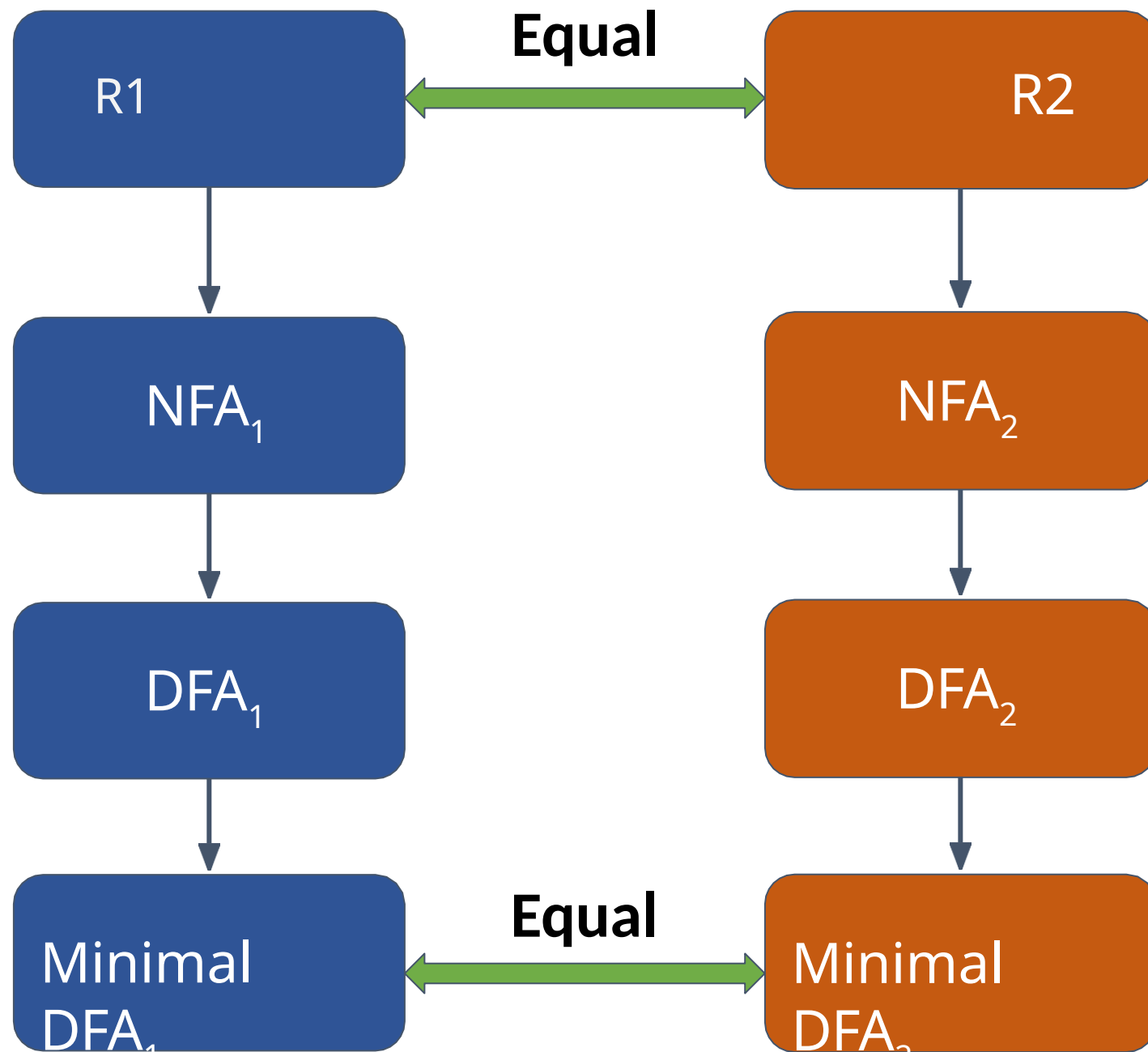
Two regular expressions(R_1 and R_2) are equivalent($R_1 = R_2$) iff:

$$L(R_1) = L(R_2)$$

We can determine the equivalence using:

- 1) A Formal method
- 2) An Informal method:

Formal method to prove Equivalence of two Regex



Informal Method -

We try proving $R_1 \neq R_2$

- Find a string that can be matched with only one of the regex hence proving that the two regex are not equivalent.
- Faster
- But, based on hit and trail.
- Can only be used to prove inequality!!

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Let us look at examples and find out whether two regex are equivalent or not??

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 1:

$$(0+1)^*(0+\lambda)$$
$$(1+\lambda)(1+0)^*$$

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 1:

$(0+1)^*(0+\lambda)$

**Formal
Method**

$(1+\lambda)(1+0)^*$

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

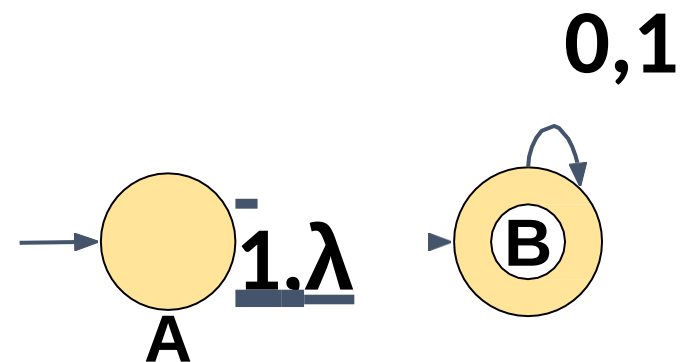
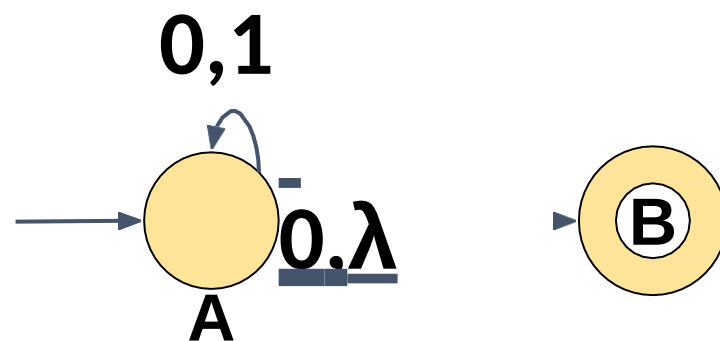
Example 1:

Formal Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

NFA



Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

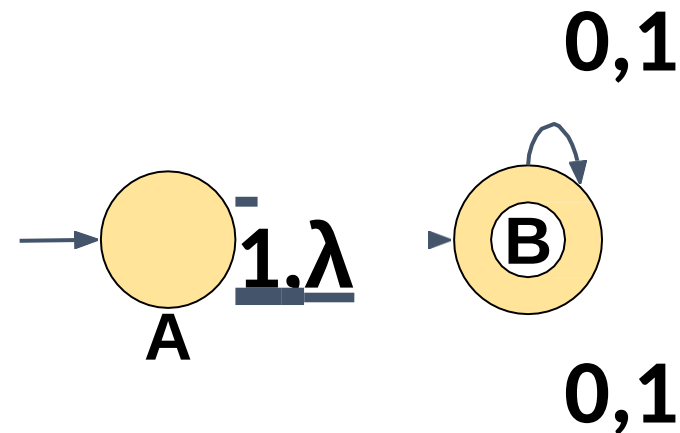
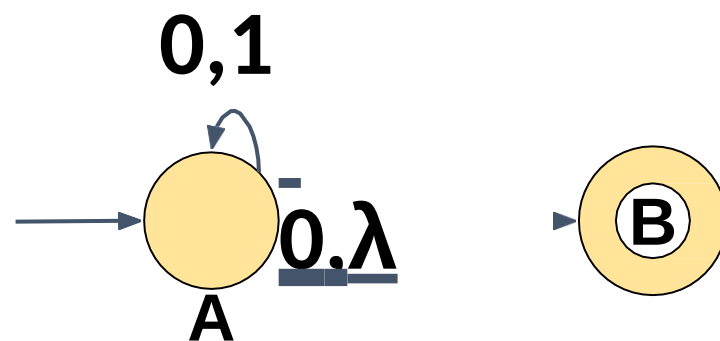
Example 1:

Formal Method

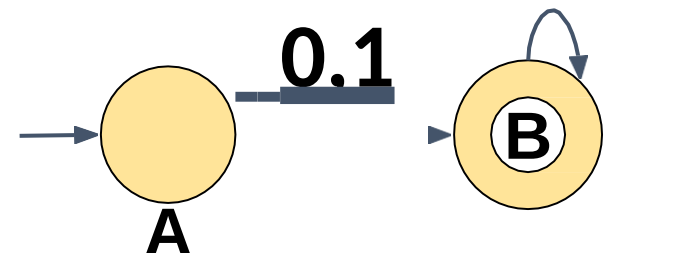
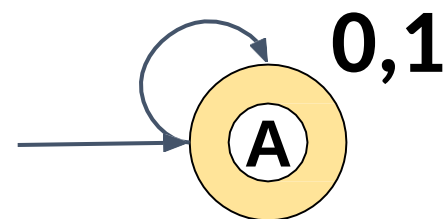
$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

NFA



DFA



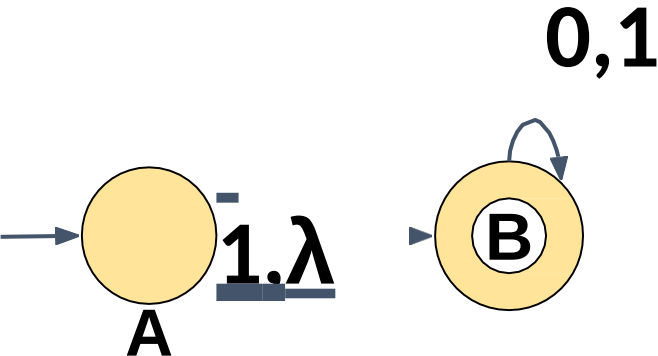
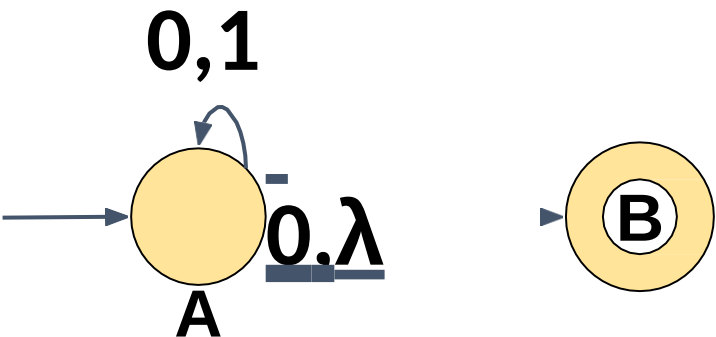
Example 1:

Formal Method

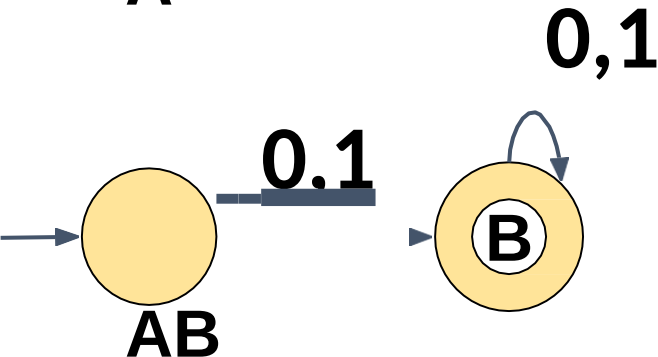
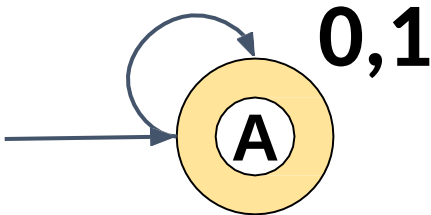
$$(0+1)^*(0+\lambda)$$

$$(1+\lambda)(1+0)^*$$

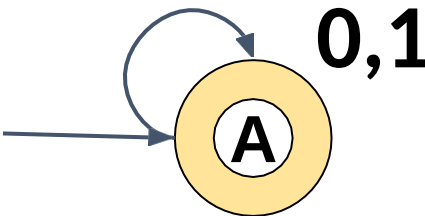
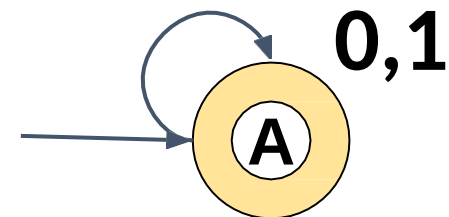
NFA



DFA



Minimal
DFA



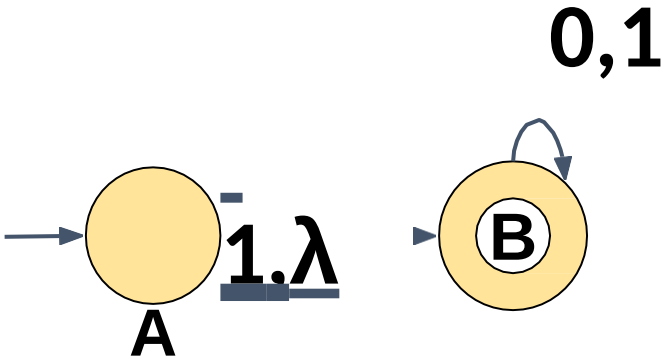
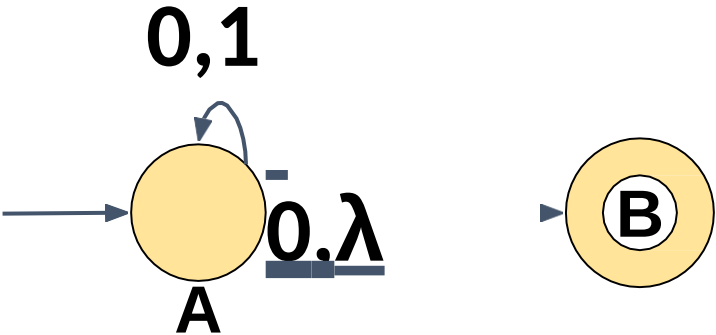
Example 1:

Formal Method

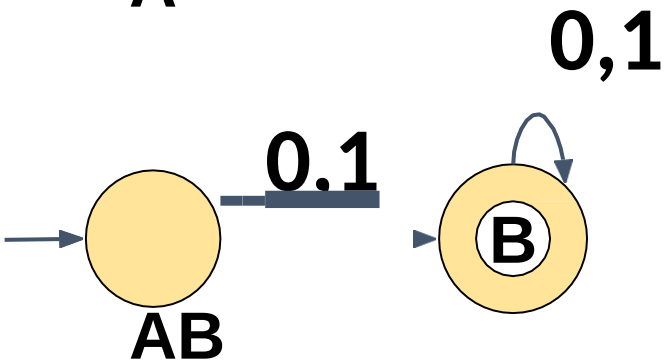
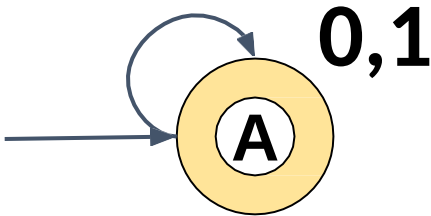
$$(0+1)^*(0+\lambda)$$

$$(1+\lambda)(1+0)^*$$

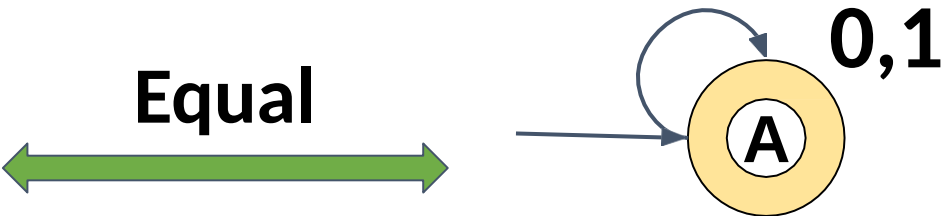
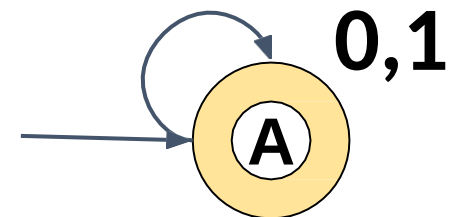
NFA



DFA



Minimal
DFA



Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

$$(0+\lambda)(11^*0)^*(1+\lambda)$$
$$(1+\lambda)(011^*)(0+\lambda)$$

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

$(0+\lambda)(11^*0)^*(1+\lambda)$

Informal

Method

$(1+\lambda)(011^*)(0+\lambda)$

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

Informal

$(0+\lambda)(11^*0)^*(1+\lambda)$

Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

Informal

$(0+\lambda)(11^*0)^*(1+\lambda)$

Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(1+\lambda)(011^*)(0+\lambda)$

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

Informal

$(0+\lambda)(11^*0)^*(1+\lambda)$

Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(1+\lambda)(011^*)(0+\lambda)$

01
1

Example 2:

Informal

$(0+\lambda)(11^*0)^*(1+\lambda)$

Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

01
1

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Example 2:

Informal

$(0+\lambda)(11^*0)^*(1+\lambda)$

Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

011 cannot be generated

01
1

Unit 1 - Equivalence of two regular expression

$(0+\lambda)(11^*0)^*(1+\lambda)$

$\neq$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

011 cannot be generated

$(1+\lambda)(011^*)(0+\lambda)$

011

Automata Formal Languages and Logic

Unit 1 - Equivalence of two regular expression

Can you answer whether the two regex are equivalent or not??

1) $(1+\lambda) (00^*1)^* 0^*$ and $(0+\lambda) (11^*0)1^*$

2) $0^*(10^*)^*$ and $(1^*0)^*1^*$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Regular Grammar

Automata Formal Languages and Logic

Unit 1 - Regular Grammar (outline)

- Formal Definition of a Grammar
- Terminology : Sentence , Sentential Form
- Examples on how to construct a Regular Grammar for a given L

Unit 1 - Regular Grammar Definition

Grammar

V -
Variables

expr, var, S, A, B..., w,x,y

T -
Terminals

a, b, c ..., 0, 1, ... id, num

P - Production
Rules

α or $\beta = (V \cup T)^*$
 $\alpha \rightarrow \beta$

S - Start
Symbol

$L(G) =$

$\{w \mid S \Rightarrow w, \text{ where } w \in \Sigma^*\}$

Automata Formal Languages and Logic

Unit 1 - Regular Grammar

Sentence (or a String)

A sentence is made up only of terminals.

Example:

For the Grammar $S \rightarrow aS \mid a \mid \lambda$

Sentence (or string) is { a or λ or aa or aaaa)

Automata Formal Languages and Logic

Unit 1 - Regular Grammar

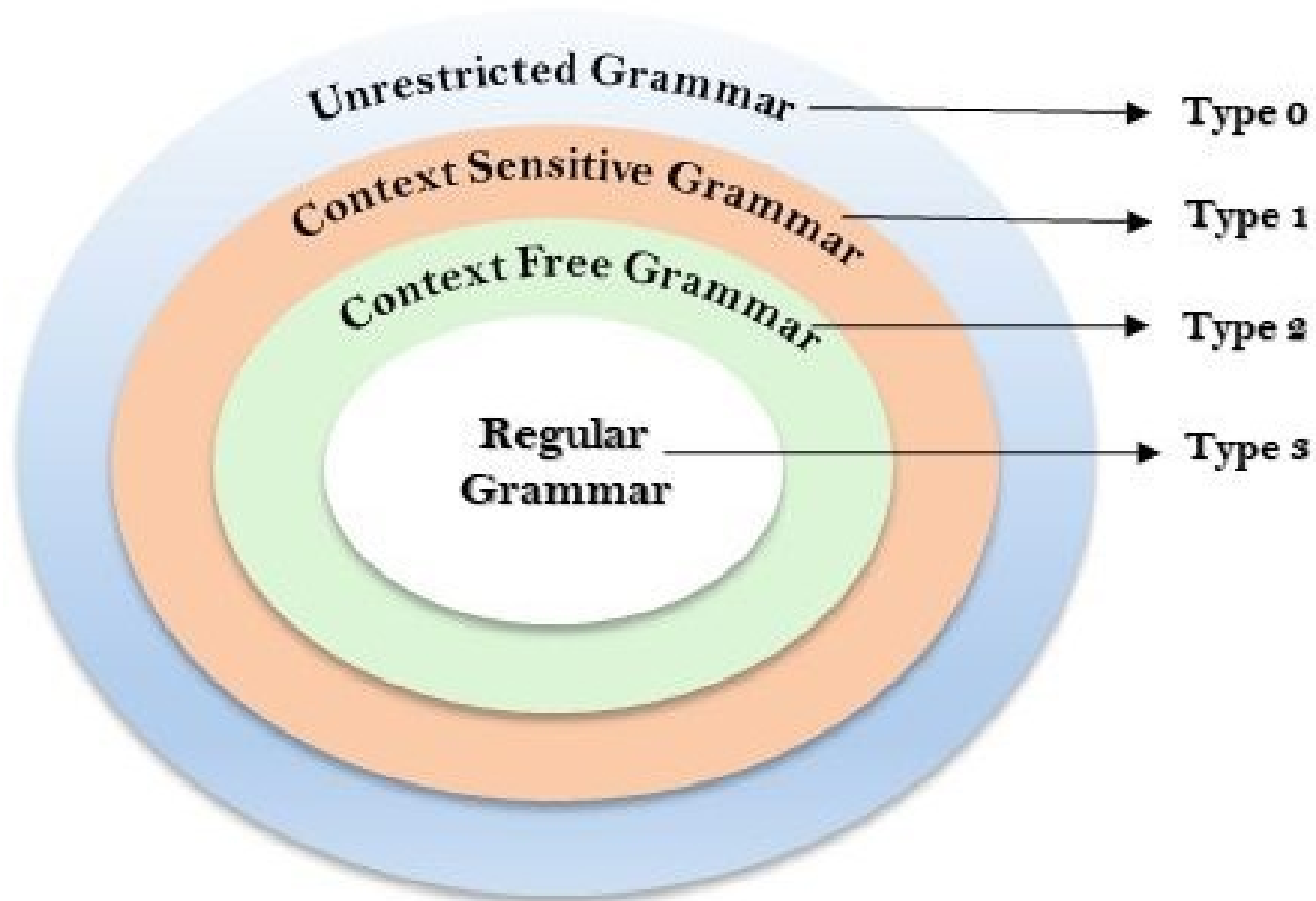
Sentential Form

A sentential form is made up over set of terminals and non-terminals $(VUT)^*$

Example:

For the Grammar $S \rightarrow aS \mid a \mid \lambda$

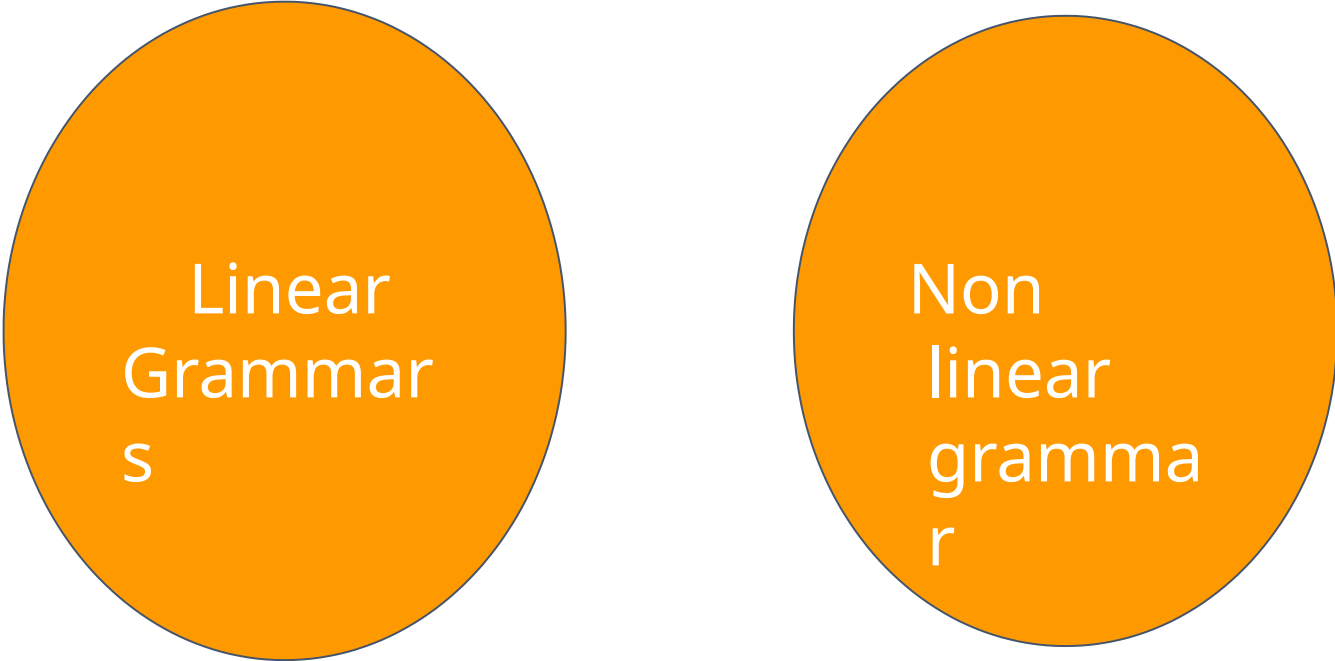
Sentential form is { a or λ or aS or aa or or aaS aaaa)



Noam Chomsky

Chomsky Hierarchy

Unit 1 - Regular Grammar



Linear
Grammars

Non
linear
grammar

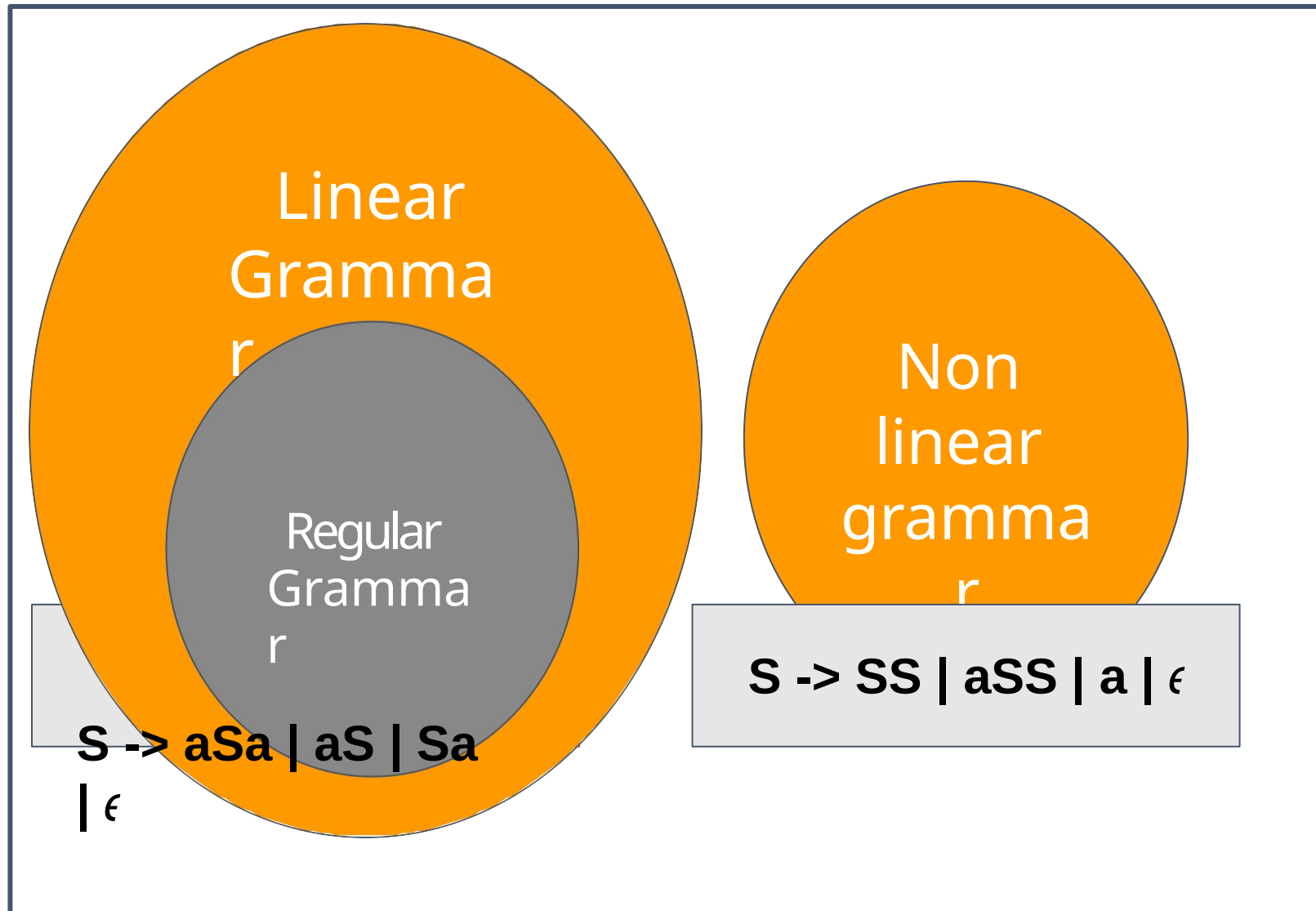
Unit 1 - Regular Grammar

Linear
Grammar
r s

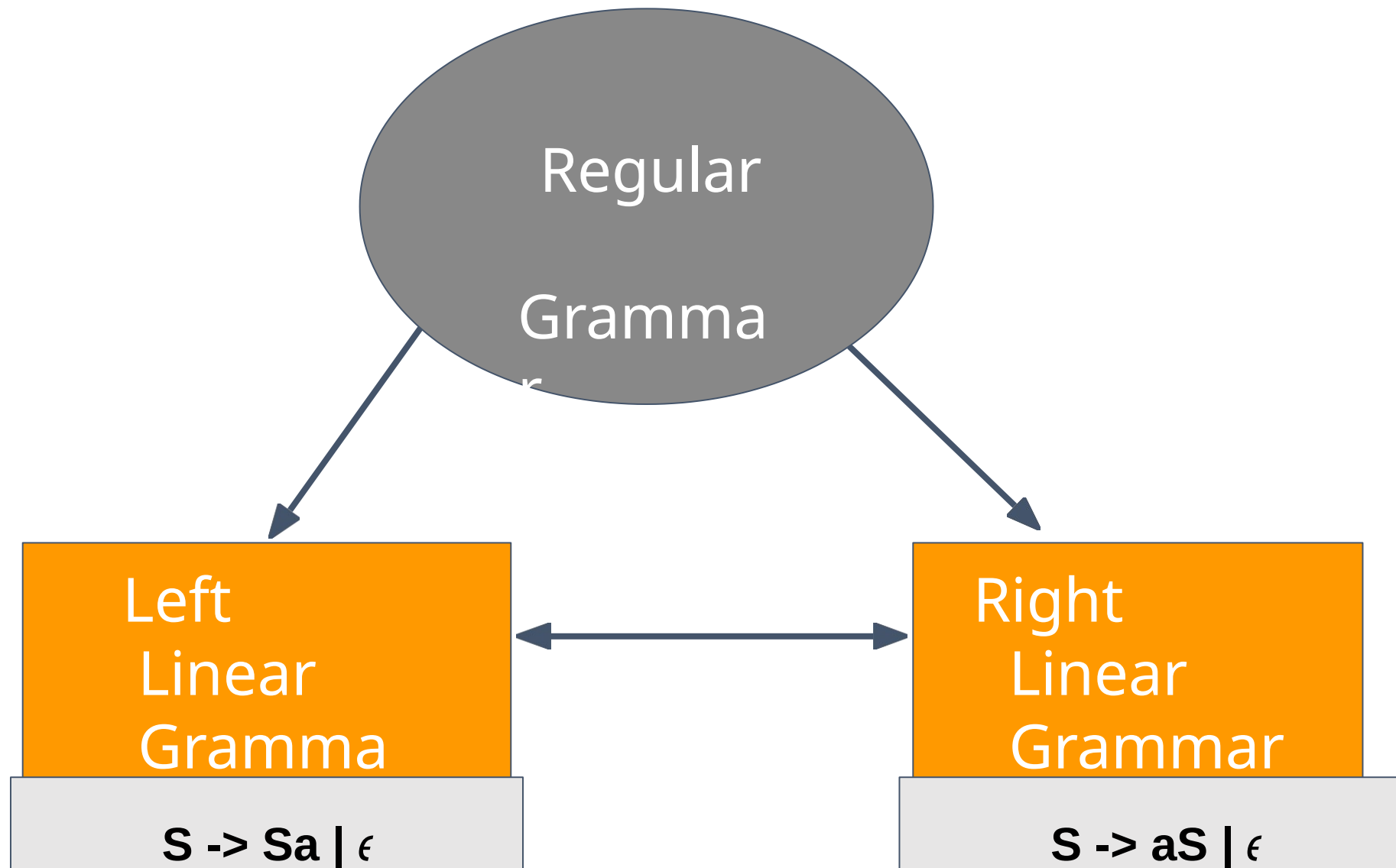
$S \rightarrow aSa \mid aS \mid Sa \mid \epsilon$

Non
linear
grammar
r

Unit 1 - Regular Grammar



Unit 1 - Regular Grammar



Construct a regular grammar for a given language description

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language

Language	RE	RG
$L = \{a\}$	a	$S \rightarrow a$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language

Language	RE	RG
$L = \{a, b\}$	$a+b$	$S \rightarrow a \mid b$ or $S \rightarrow a$ $S \rightarrow b$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language

Language	RE	RG
$L = \{ab\}$	ab	$S \rightarrow ab$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the language $L = \{a^n | n \geq 0\}$

Language	RE	RG
$L = \{\lambda, a, aa, aaa, \dots\}$	a^*	$S \rightarrow aS \lambda$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the language $L = \{a^n | n \geq 1\}$

Language	RE	RG
$L = \{a, aa, aaa, \dots\}$	a^+	$S \rightarrow aS \mid a$

Construct a regular grammar for the regular expression
 $(a+b)^*$

Language	RE	RG
$L =$ $\{\lambda, a, b, abb, baaaab, ..$ $.....\}$ $L = \{\text{any number of}$ $\text{'as and b's'}\}$	$(a+b)^*$	$S \rightarrow aS \mid bS \mid \lambda$

Construct a regular grammar for the regular expression $(a+b)^+$

Language	RE	RG
$L =$ $\{a,b,abb,baaaab,...$ $.....\}$ $L=\{\text{any number of 'as and b's'}\}$	$(a+b)^+$	$S \rightarrow aS \mid bS \mid a \mid b$

Construct a regular grammar for the regular expression $(ab)^*$

Language	RE	RG
$L =$ $\{\lambda, ab, ababab, \dots$ $\dots\}$ $L = \{\text{any number of}$ $ab's\}$	$(ab)^*$	$S \rightarrow abS \mid \lambda$

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the language $L=\{a^m b^n \mid n,m \geq 0\}$

Language	RE	RG
$L=\{\lambda,a,b,bb,aa,.....\}$ $L=\{\text{any number of a's or any number of b's}\}$	$a^* + b^*$	$S \rightarrow A \mid B$ A:any number of a's B:any number of b's $A \rightarrow aA \mid \lambda$ $B \rightarrow bB \mid \lambda$

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language
with even number of a's.

$$L=\{a^{2n} \mid n \geq 0\}$$

Language	RE	RG
$L=\{\lambda,$ aa, aaaa, aaaa aa	$(aa)^*$	$S \rightarrow aaS \mid \lambda$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language with

odd number of a's. $L = \{a^{2n+1} \mid n \geq 0\}$

Language	RE	RG
$L = \{a, aaa, aaaaa, aaaaaa, aa \dots\}$	$(aa)^*a$	$S \rightarrow aaS \mid a$

Automata Formal Languages and Logic

Unit 1 - Construct a regular grammar for the given language

Construct a regular grammar for the given language
with number of a's as multiples of 4. $L = \{a^{4n} \mid n \geq 0\}$

Language	RE	RG
$L = \{\lambda, aaaa, aaaaaaaaaa, aaaaaaaaaa, aaaa, \dots\}$	$(aaaa)^*$	$S \rightarrow aaaaS \mid \lambda$

Unit 1 - Construct a regular grammar for the given language

Convert finite automata to regular grammar for the language to accept at least one 'a' over the alphabet $\Sigma=\{a,b\}$

Language	RE	RG
$L=\{a, bb... a, ba \text{ any \# of a's \& b's}\}$	$b^* a (a+b)^*$	$\begin{aligned} S &\rightarrow bS \mid aA \\ A &\rightarrow aA \mid bA \\ &\mid \lambda \end{aligned}$

Automata Formal Languages and Logic

Unit 1 - Equivalence of regular grammar and Finite automata



Automata Formal Languages and Logic

Unit 1 - Equivalence of regular grammar and Finite automata

Let's look at examples to convert :

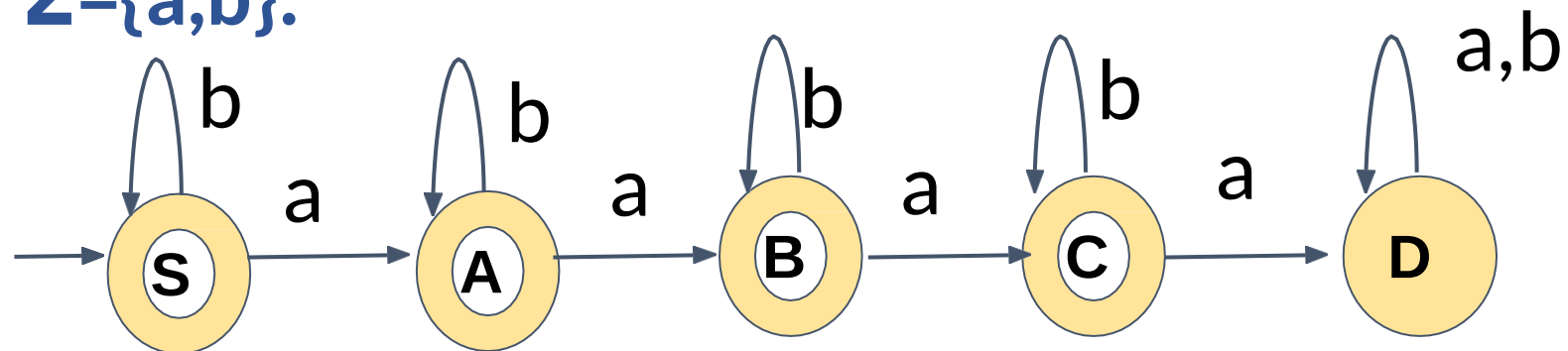
- 1. Finite Automata to Regular Grammar**
- 2. Regular Grammar to Finite Automata**

Automata Formal Languages and Logic

Unit 1 - Equivalence of regular grammar and Finite automata

1. Convert finite automata to regular grammar for the language to accept at most 3 'a's over the alphabet

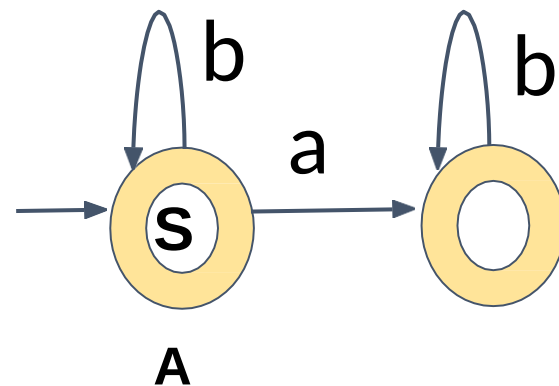
$\Sigma = \{a, b\}$.



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

Let's look at each transition and perform the conversion



$$S \rightarrow bS \mid aA \mid \lambda$$

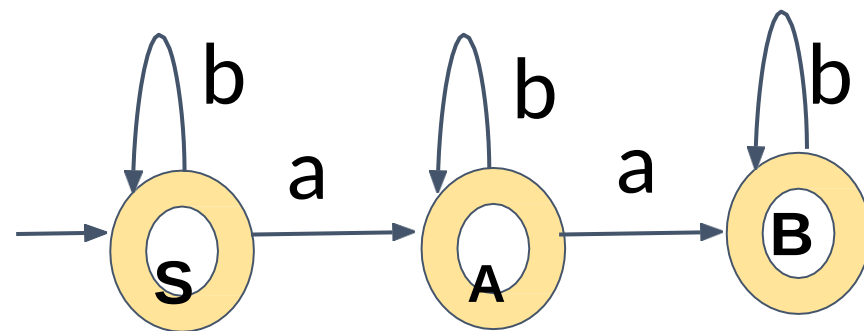
$$A \rightarrow bA \mid \lambda$$

λ on RHS indicates S
and A are final
states

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

Let's look at each transition and perform the conversion



$$S \rightarrow bS \mid aA \mid \lambda$$

$$A \rightarrow bA \mid aB \mid \lambda$$

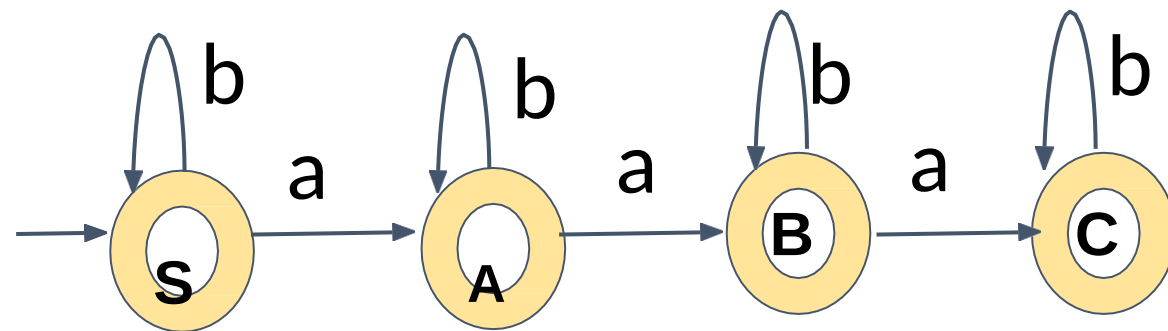
$$B \rightarrow bB \mid aC \mid \lambda$$

λ on RHS indicates
S, A and B are final
states

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

Let's look at each transition and perform the conversion



$$S \rightarrow bS \mid aA \mid \lambda$$

$$C \rightarrow bC \mid aD \mid \lambda$$

$$A \rightarrow bA \mid aB \mid \lambda$$

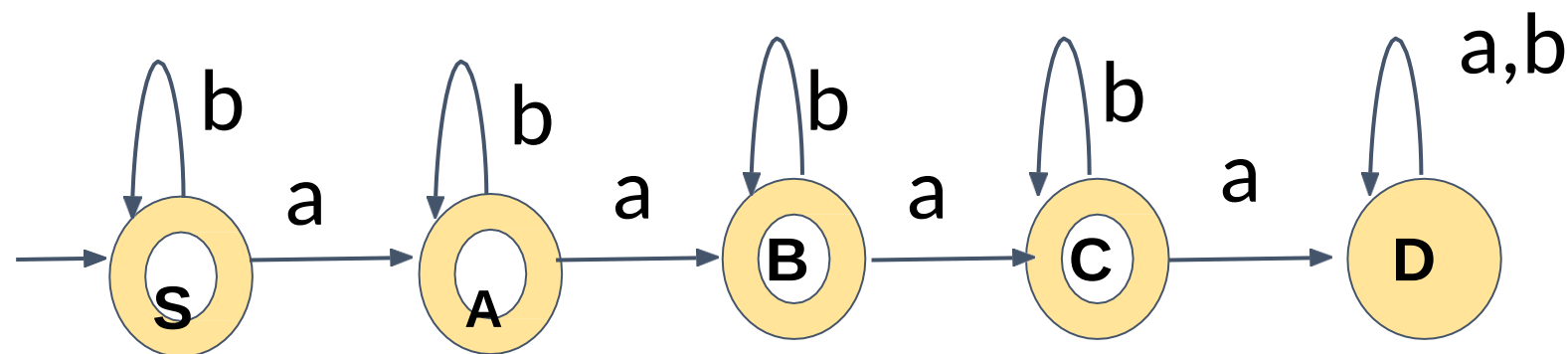
$$B \rightarrow bB \mid aC \mid \lambda$$

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

Let's look at each transition and perform the conversion

Since D is a dead state, we will not encode $C \rightarrow aD$ and $D \rightarrow aD \mid bD$ as D is a useless Non-terminal (that means it never terminates)



$$S \rightarrow bS \mid aA \mid \lambda$$

$$C \rightarrow bC \mid \lambda$$

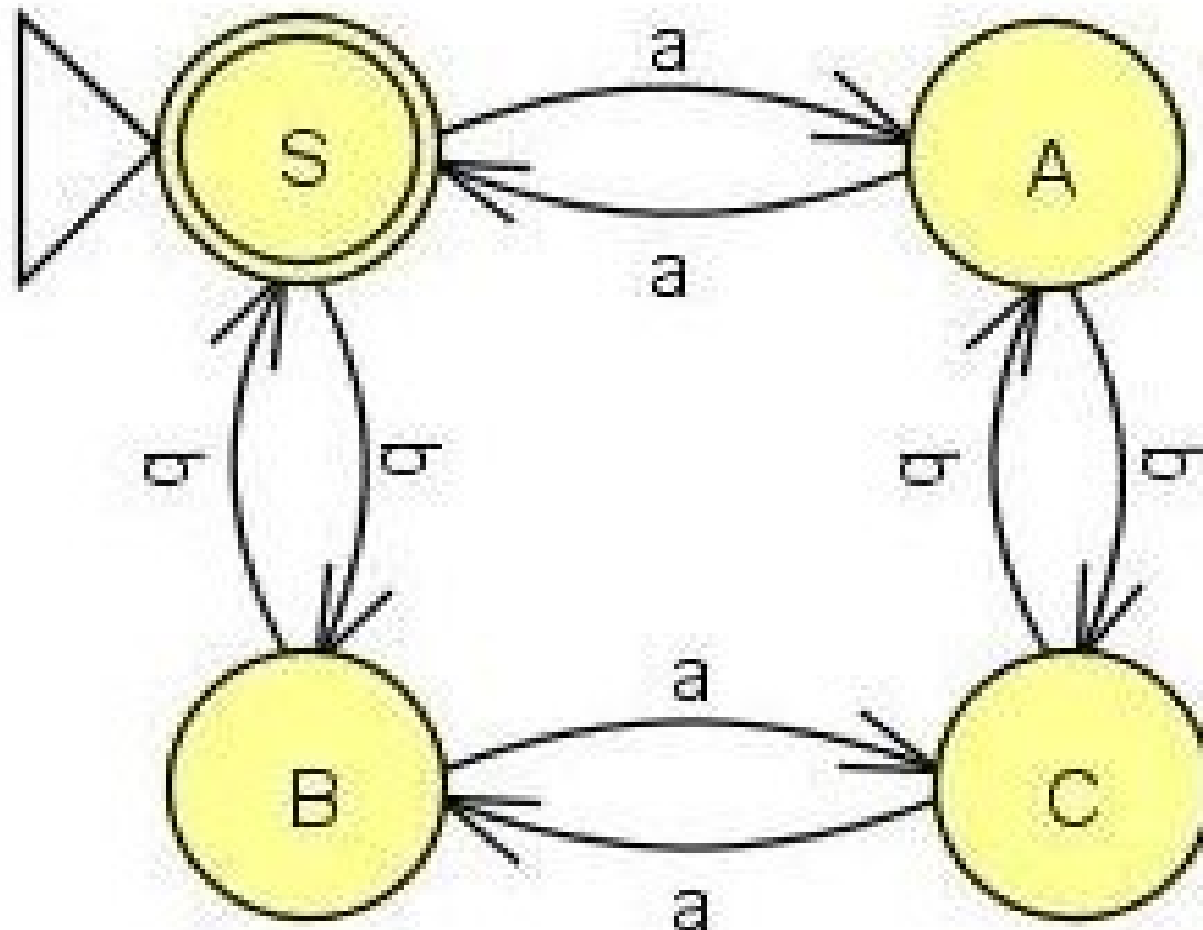
$$A \rightarrow bA \mid aB \mid \lambda$$

$$B \rightarrow bB \mid aC \mid \lambda$$

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

2. Convert given finite automata accepting
 $a^2=0$ $n_b(w) \bmod 2=0$ } to regular
 $L=\{n_a(w) \bmod 2=0 \text{ and } n_b(w) \bmod 2=0\}$
 gram



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

2. Convert a given finite automata accepting $L = \{n_a(w) \bmod 2 = 0 \text{ and } n_b(w) \bmod 2 = 0\}$ to regular grammar.

Start state of automata will be the start symbol of the grammar.

We start with S, S on seeing terminal a it moves to state A ($S \rightarrow aA$) and on seeing terminal b it moves to state B ($S \rightarrow bB$).

Since S is also the final state, we introduce the production $S \rightarrow$

λ . Grammar is

$S \rightarrow aA \mid bB \mid \lambda$

$A \rightarrow aS \mid bC$

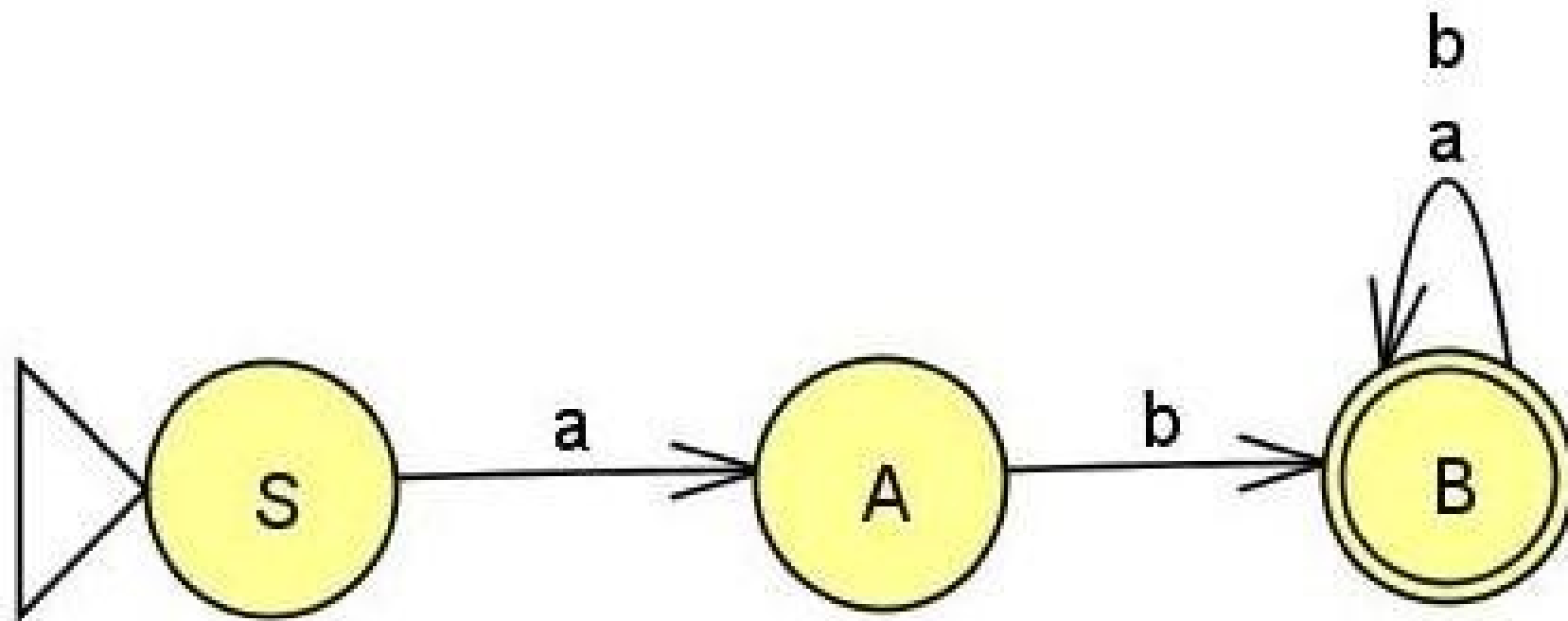
$B \rightarrow aC \mid bS$

$C \rightarrow aB \mid bA$

Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

3. Converting a given finite automata accepting $L=\{abw, w \in \{a,b\}^*\}$ to regular grammar.



Automata Formal Languages and Logic

Unit 1 - Finite Automata to Regular Grammar

3. Converting a given finite automata accepting $L=\{abw, w \in \{a,b\}^*\}$ to regular grammar.

Grammar is,

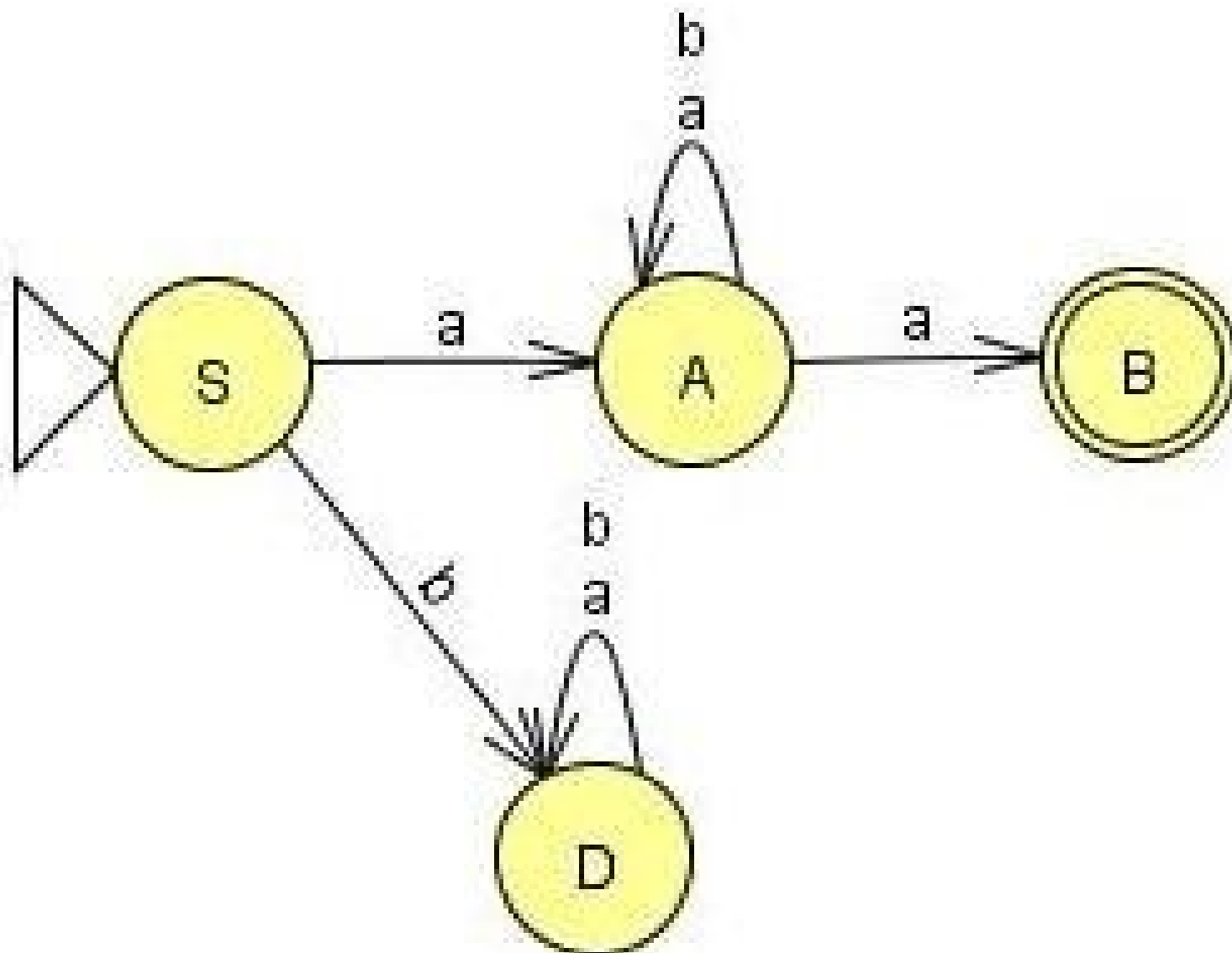
$S \rightarrow aA$

$A \rightarrow bB$

$B \rightarrow aB \mid$

$bB \mid \lambda$

4. Converting given finite automata accepting $L=\{awa, w \in \{a,b\}^*\}$ to regular grammar.



Automata Formal Languages and Logic

Unit 1 - Equivalence of regular grammar and Finite automata

4. Converting given finite automata accepting $L=\{awa, w \in \{a,b\}^*\}$ to regular grammar.

So the grammar is,

$S \rightarrow aA$

$A \rightarrow aA \mid bA \mid$

$aB \quad B \rightarrow \lambda$

Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

Converting Regular grammar to finite automata

Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

Variables-

>state

terminal symbols -> symbols of finite automata

Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

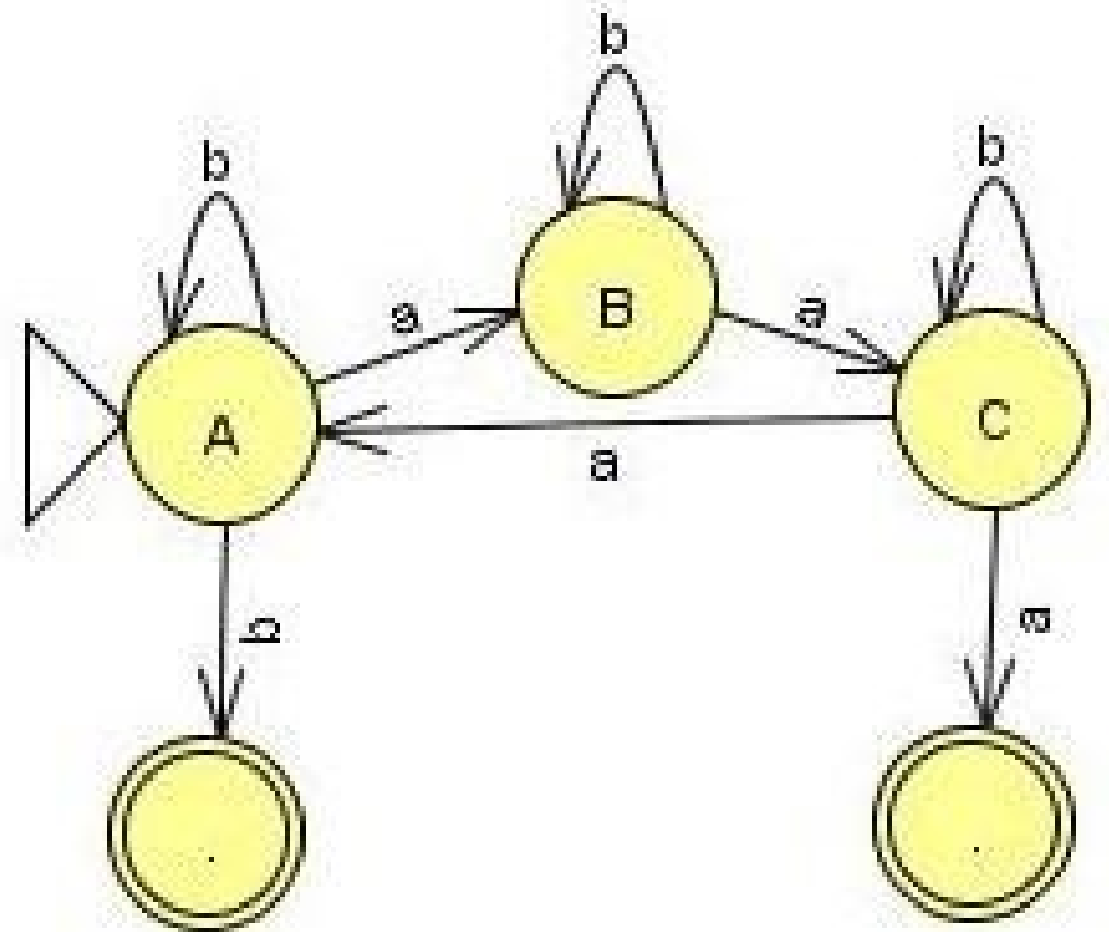
Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

- Transition $C \rightarrow aA$ to $C \rightarrow a$ when $A \rightarrow \lambda$ indicates A is a final state.



Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$

Automata Formal Languages and Logic

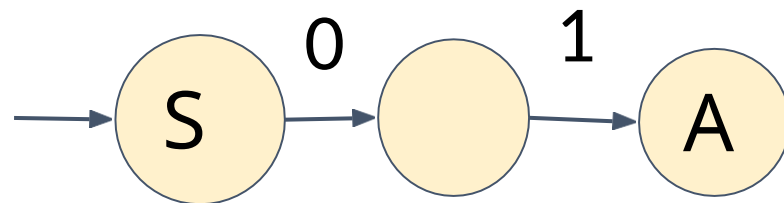
Unit 1 - Regular Grammar to Finite Automata

Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$



Automata Formal Languages and Logic

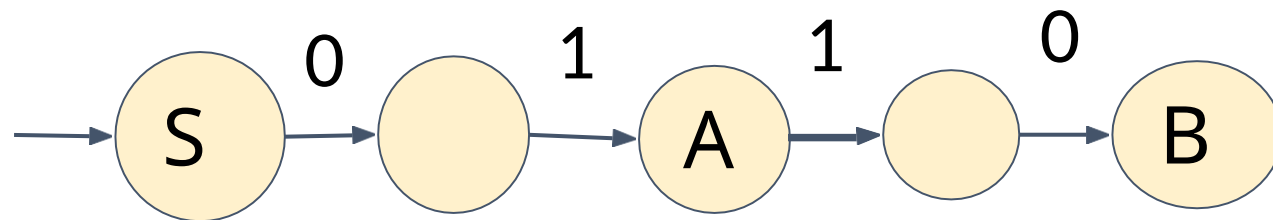
Unit 1 - Regular Grammar to Finite Automata

Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$



Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

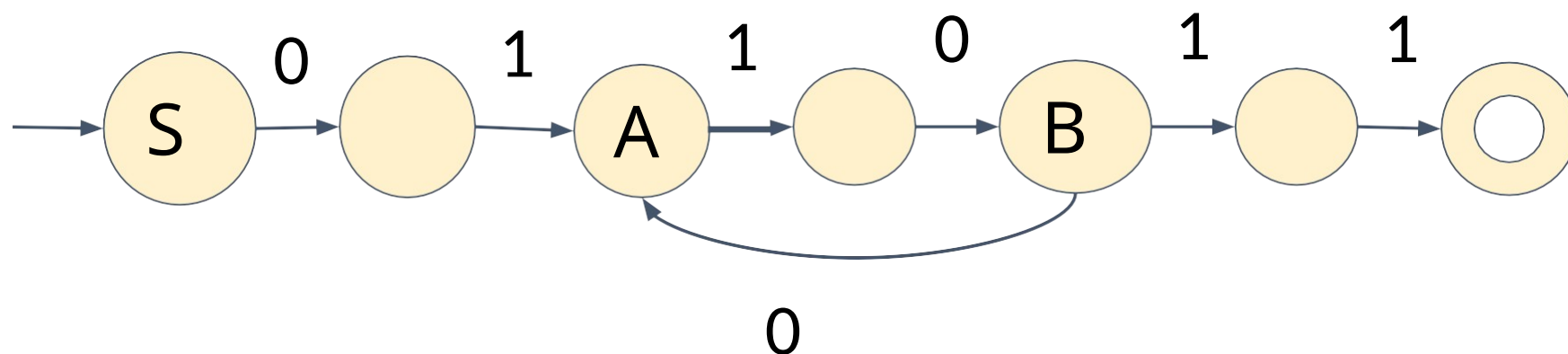
Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$

$B \rightarrow 11$ indicates on state B on consuming the input 11 we reach final state



Automata Formal Languages and Logic

Unit 1 - Regular Grammar to Finite Automata

Example 3 :

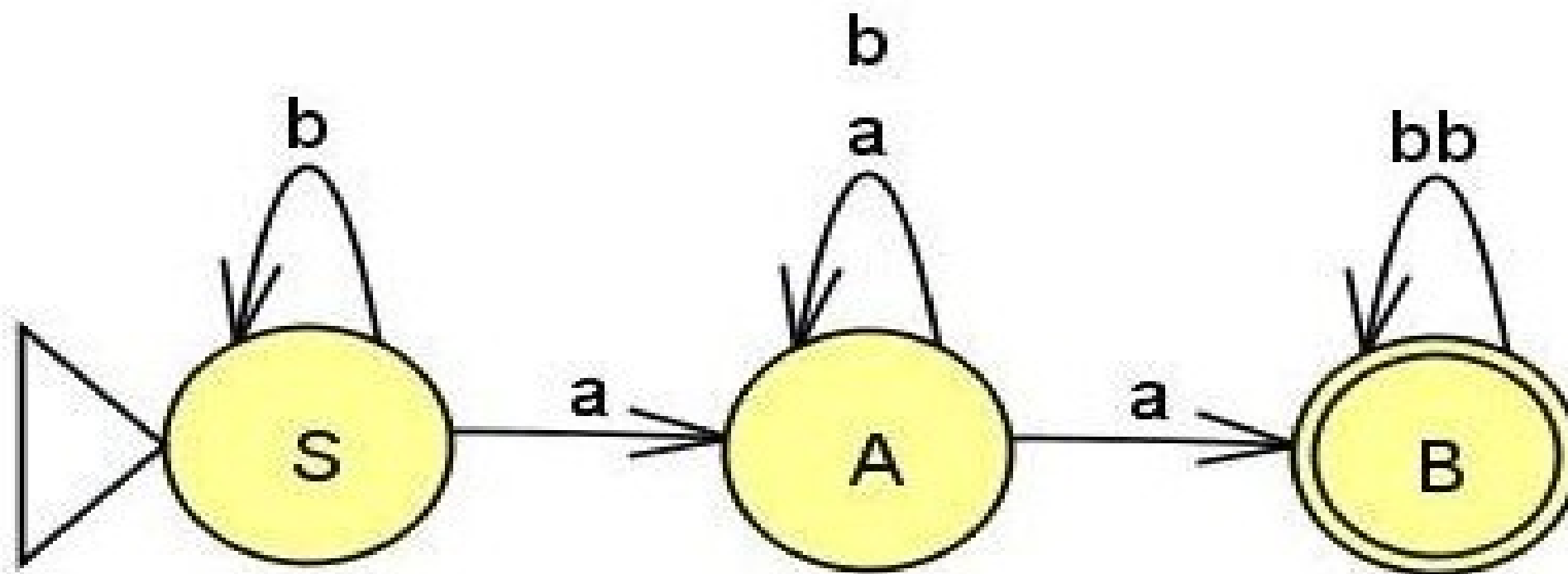
$S \rightarrow bS \mid aA$

$A \rightarrow aA \mid bA$

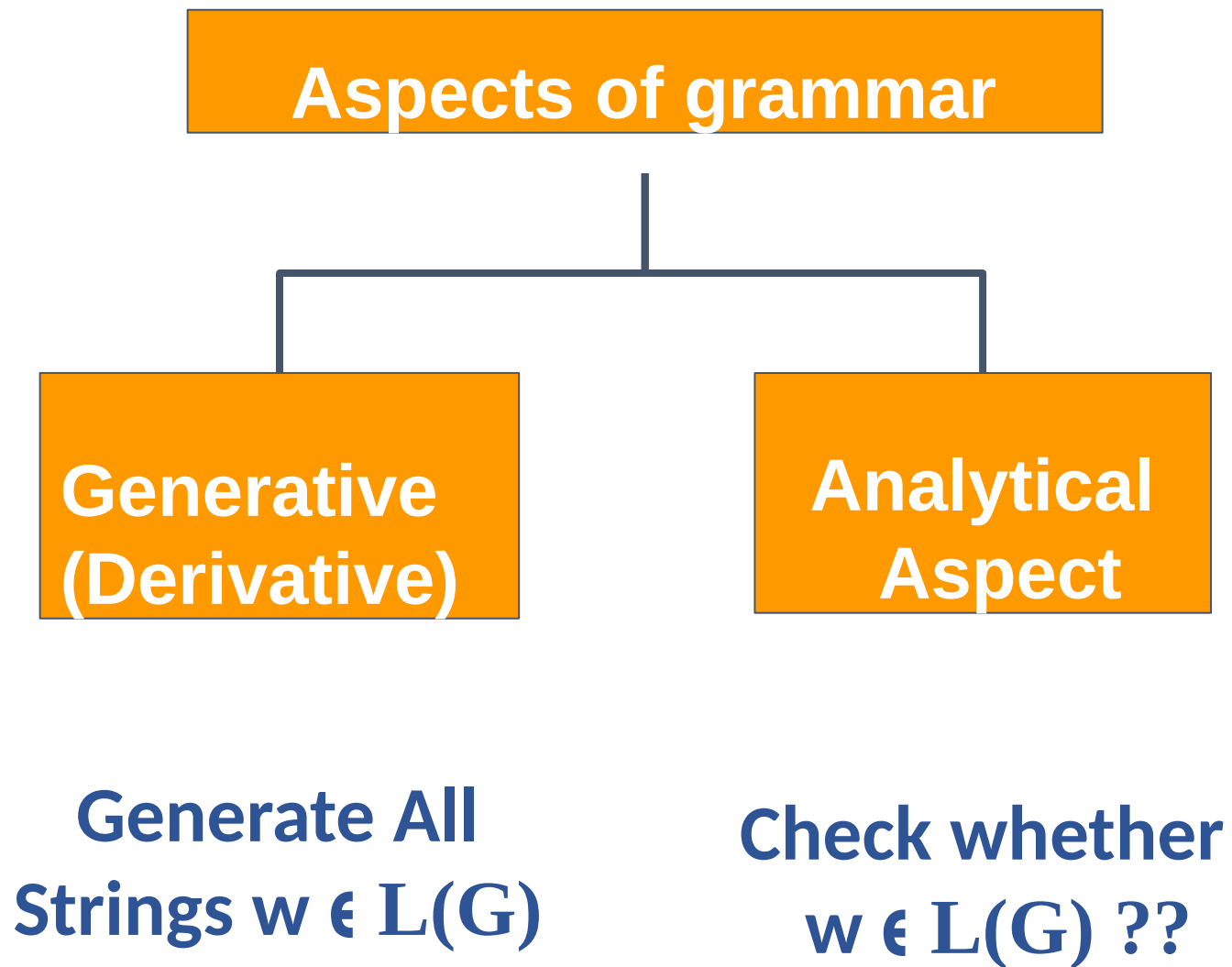
$A \rightarrow aB$

$B \rightarrow bbB$

$B \rightarrow \lambda$



Unit 1 - Aspects of Grammar



Parse Tree

|

Unit 1 - Parse Tree

What is a Tree?

Leaves

Branches

Root



Automata Formal Languages and Logic

Unit 1 - Parse Tree

Technically we represent a Tree upside down

Root-

Branches

Leaves



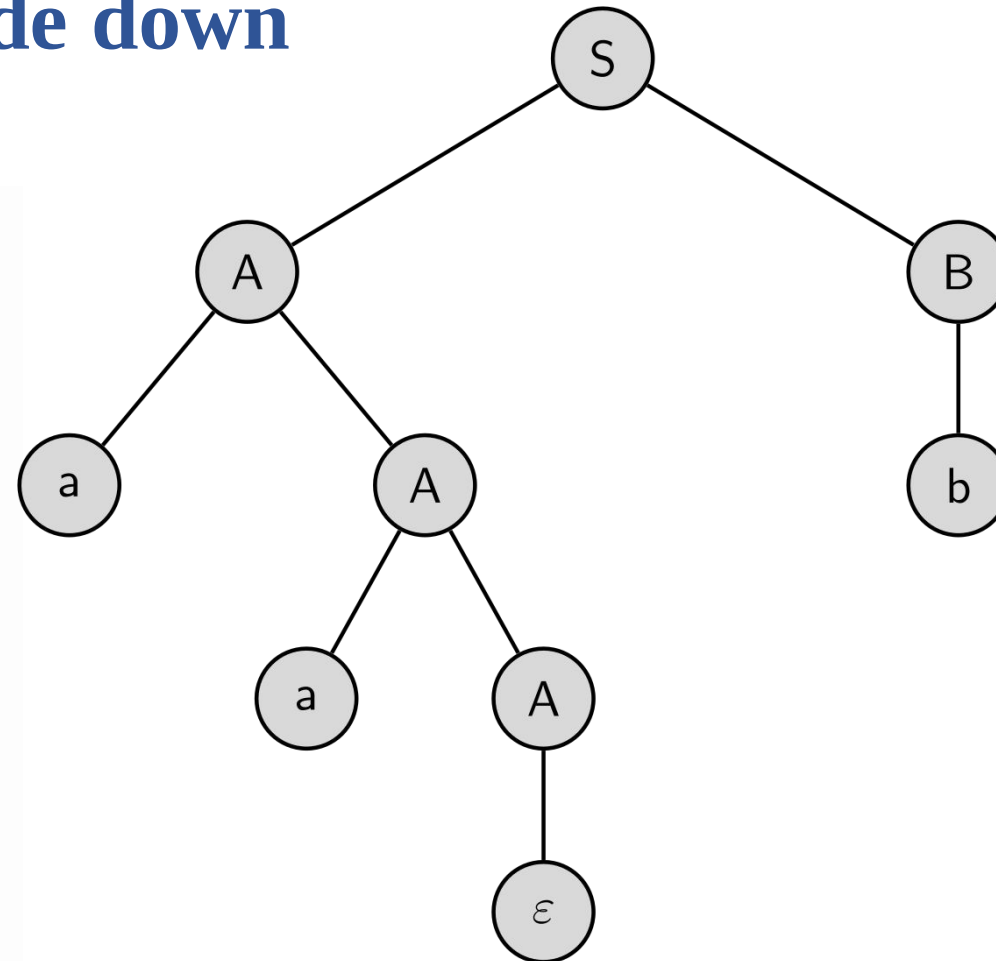
Unit 1 - Parse Tree

Technically we represent a Tree upside down

Root

Branches

Leaves



Unit 1 - Parse Tree

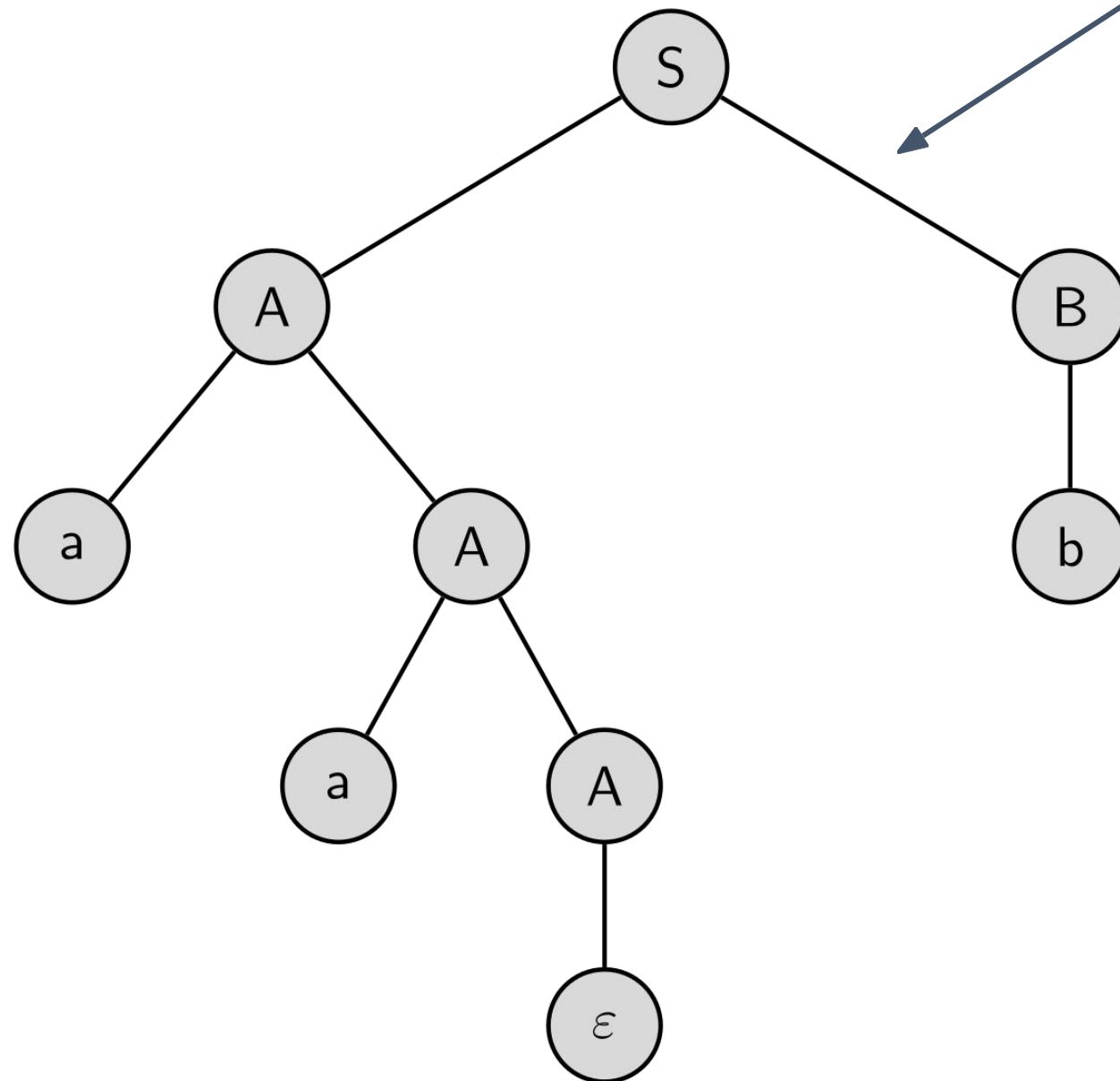
Tree

s

Edge

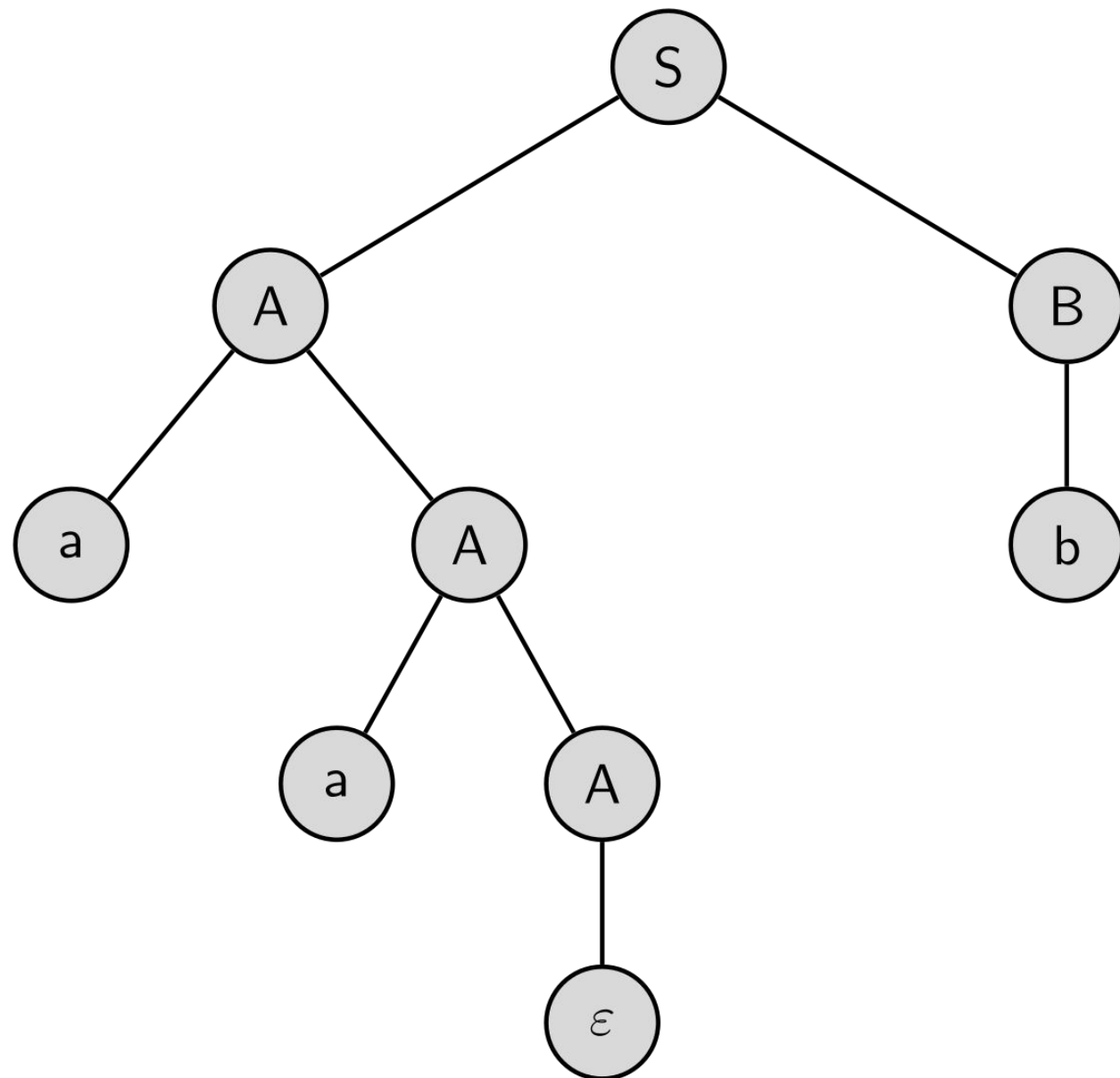
Node

Root Node



Unit 1 - Parse Tree

Parse Tree / Derivation Tree



S Root Node (Start Symbol)

V Non-Terminals - Interior Nodes

T Terminal Symbols - Leaf nodes

Yield of Parse Tree - aab

Unit 1 - Parsing

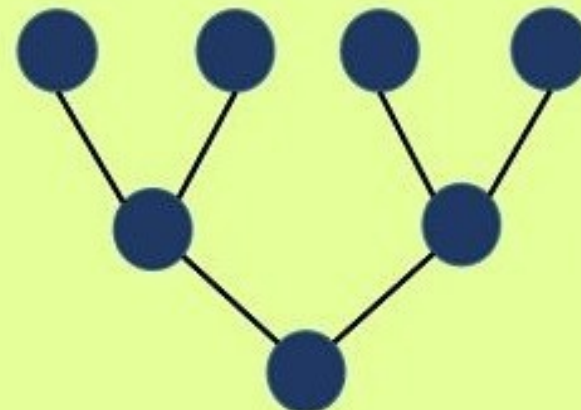
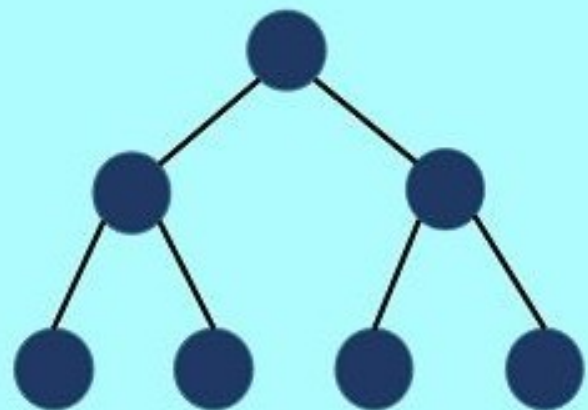
Parsing is the process of determining whether a String
 $w \in L(G)??$

Unit 1 - Types of Parsing

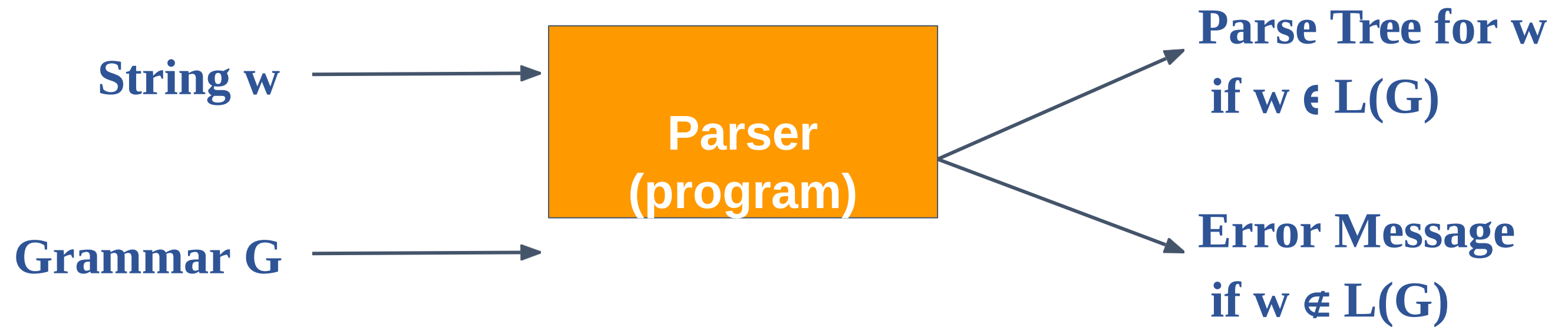
Parsing

Top-Down
Parsing

Bottom-up
Parsing



Unit 1 - Parser



Unit 1 - Construct a Parse Tree

Let us look at few examples and generate parse tree for a given String w and Grammar G .

Unit 1 - Parse Tree

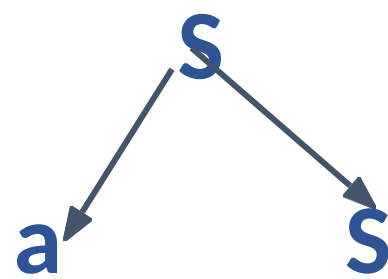
Parse Tree

S

Root node

Unit 1 - Parse Tree

Parse Tree

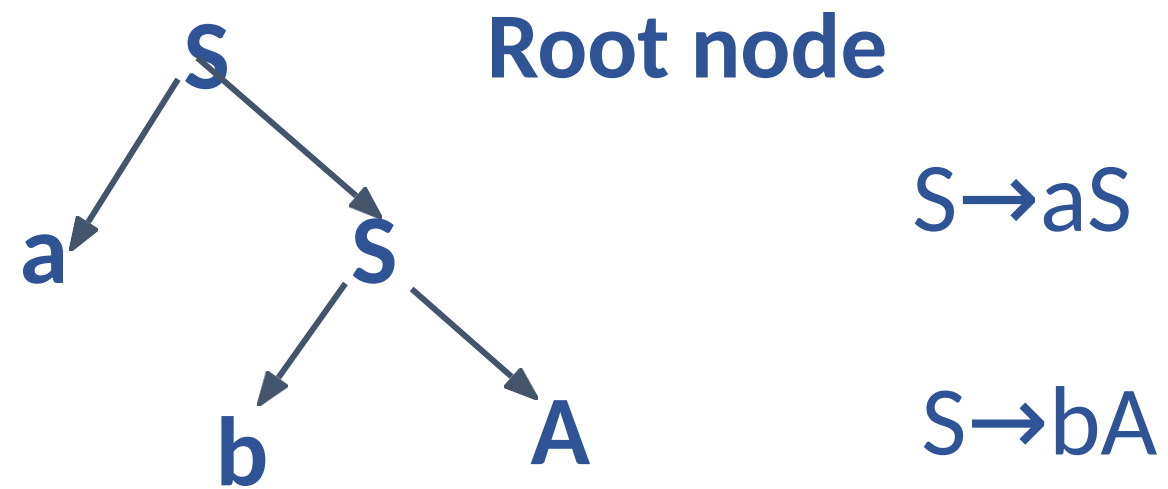


Root node

$S \rightarrow aS$

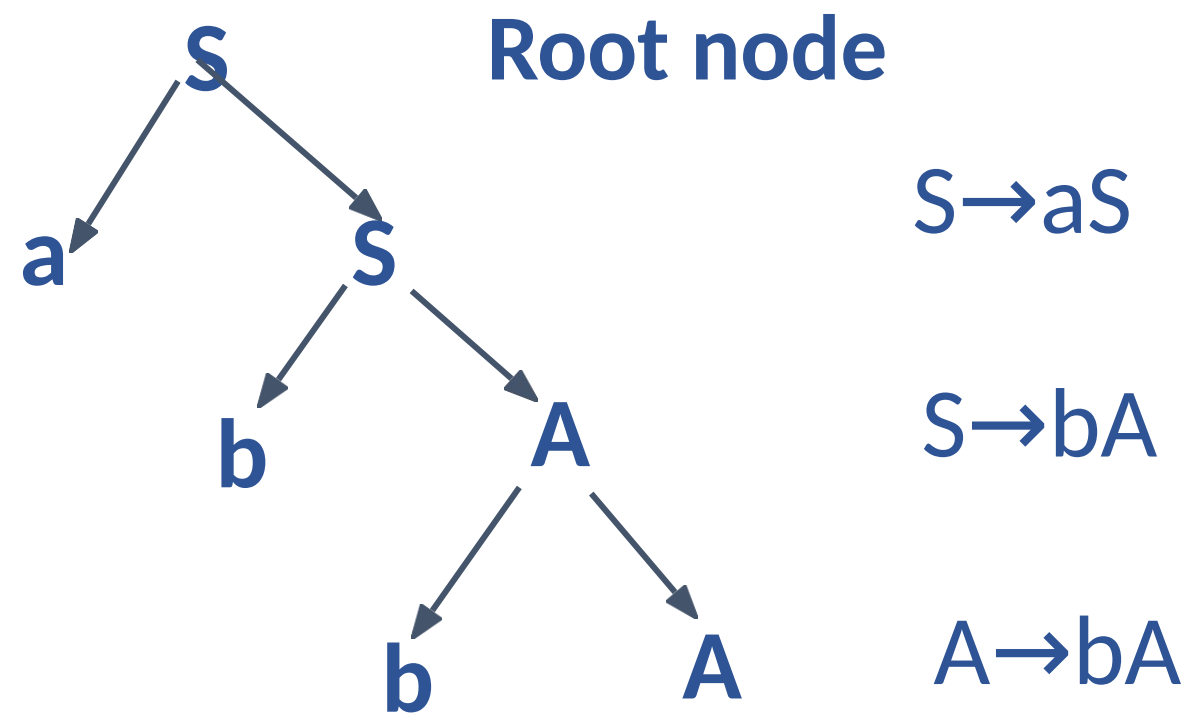
Unit 1 - Parse Tree

Parse Tree



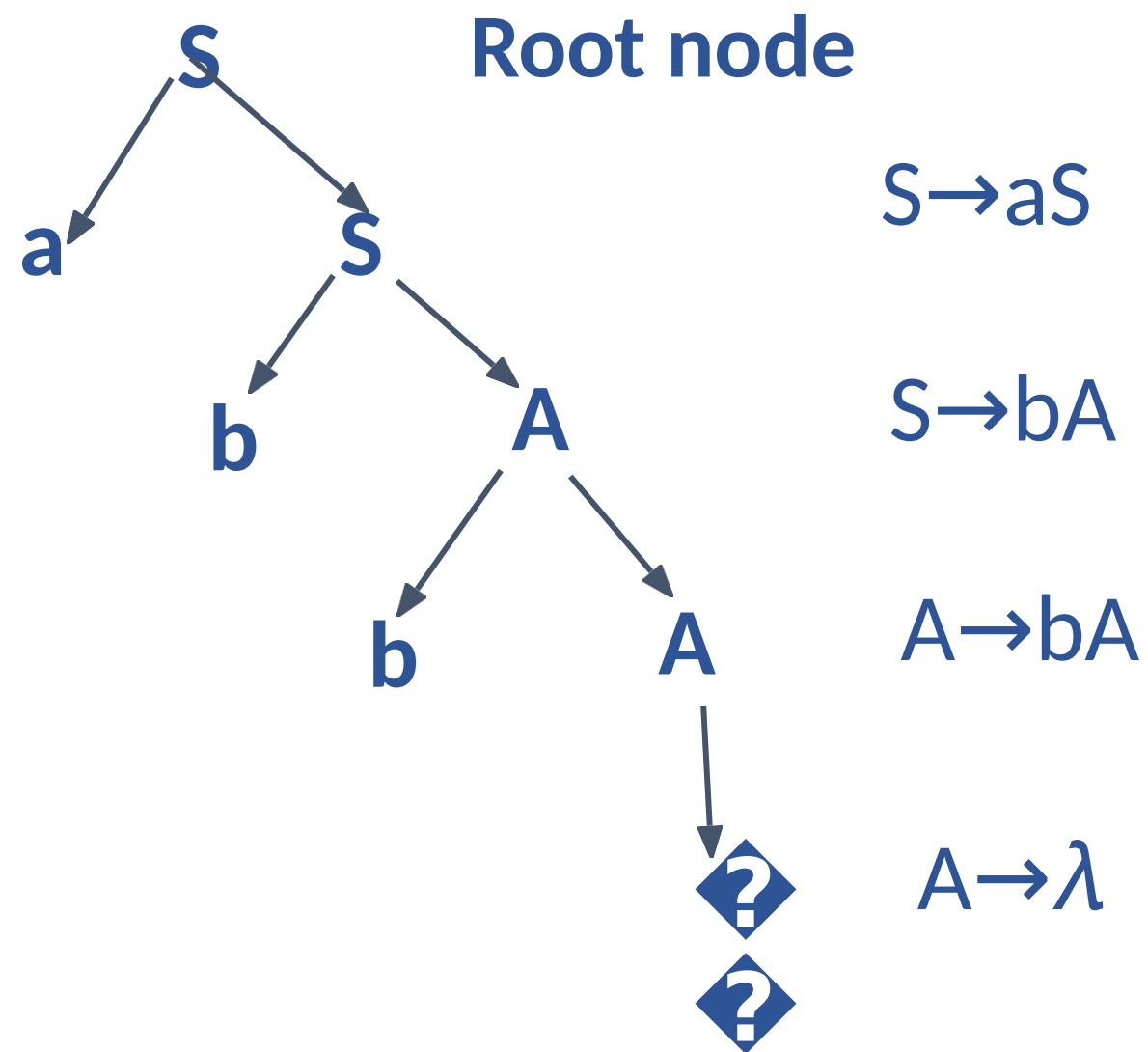
Unit 1 - Parse Tree

Parse Tree



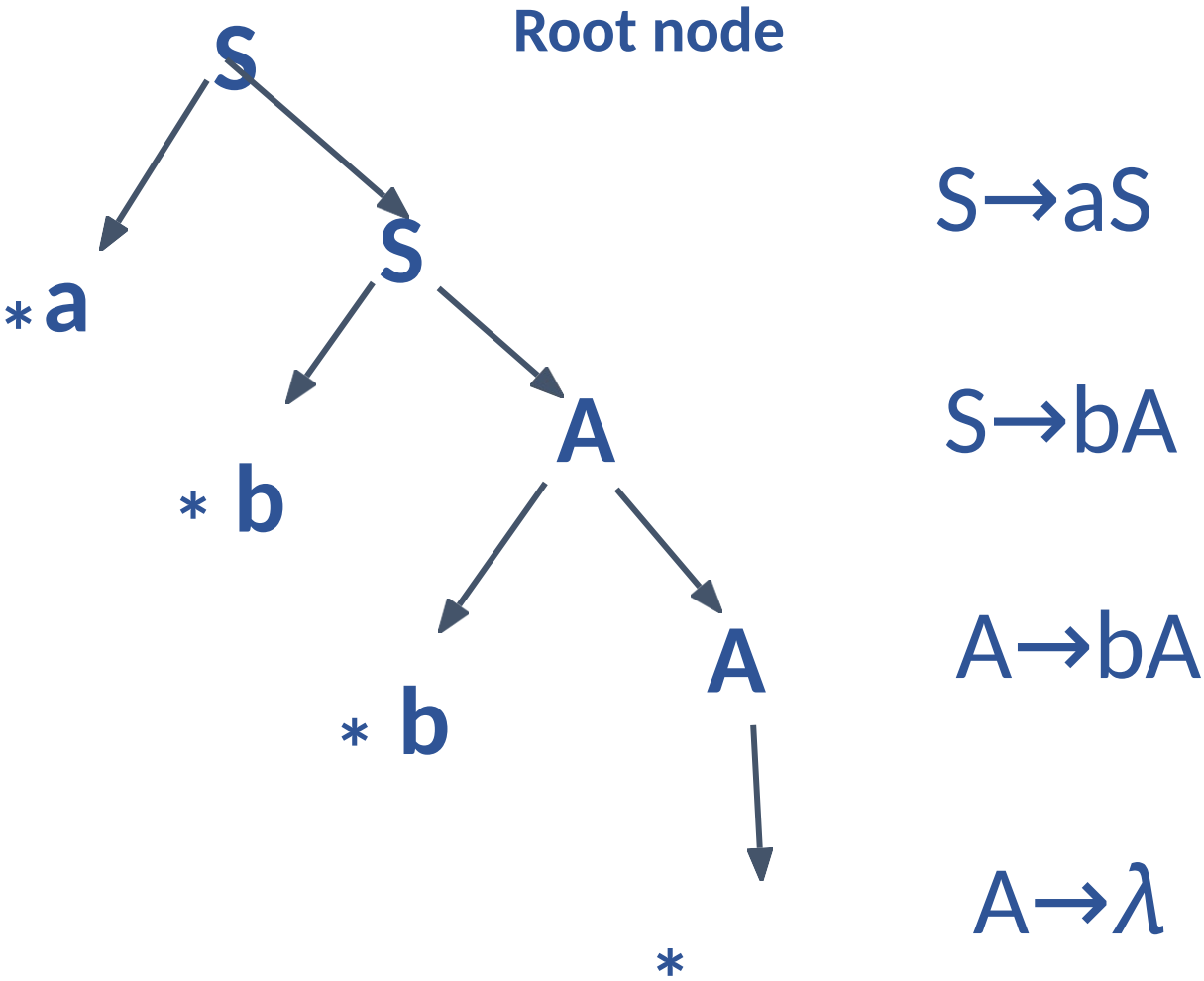
Unit 1 - Parse Tree

Parse Tree



Unit 1 - Parse Tree

Parse Tree

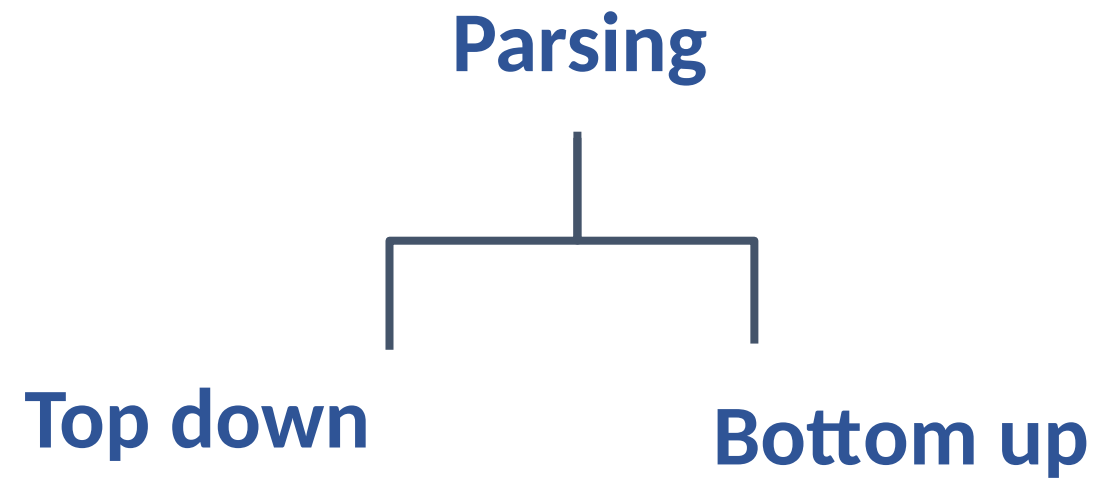


Leaf Node: *

Yield of the tree: $a b b$

Unit 1 - Analytical aspect

2. Analytical aspect



Unit 1 - Regular Grammar

2. Analytical aspect

Parsing



$S \rightarrow aS$

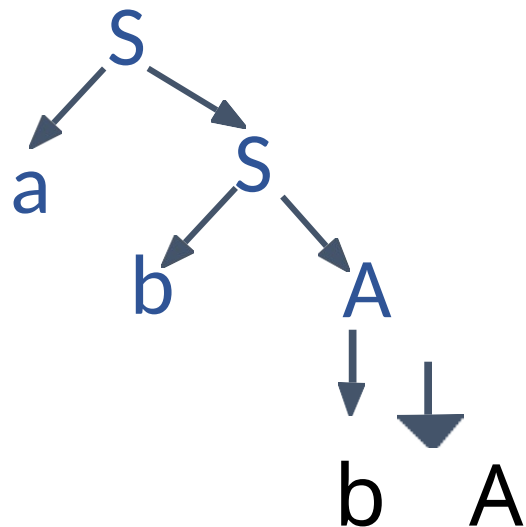
$S \rightarrow bA$

$A \rightarrow bA$

$A \rightarrow \lambda$

Top down

Bottom up



Unit 1 - Regular Grammar

2. Analytical aspect

Parsing



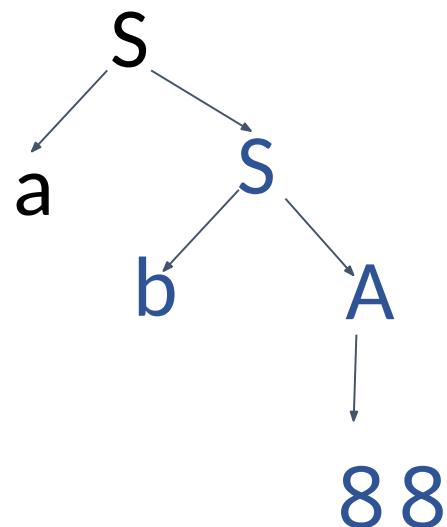
$S \rightarrow aS$

$S \rightarrow bA$

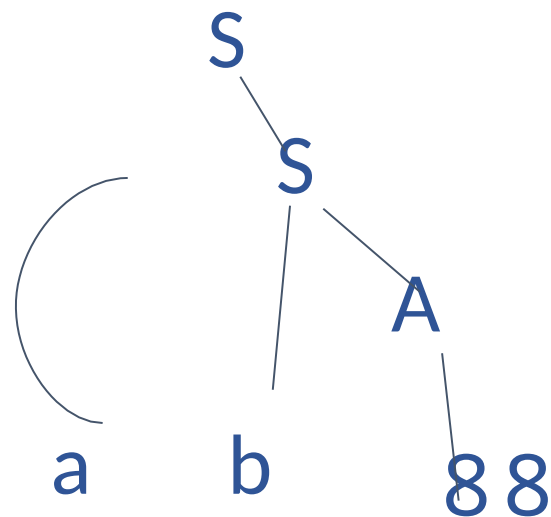
$A \rightarrow bA$

$A \rightarrow \lambda$

Top down



Bottom up



Automata Formal Languages and Logic

Unit 1 - Converting Right Linear Grammar to Left Linear Grammar

All about Left Linear Grammar

Automata Formal Languages and Logic

Unit 1 - Constructing a Left Linear Grammar

Right Linear

$A \rightarrow aB$

$B \rightarrow aB \mid$

$bB \mid \lambda$

$L = \{\text{starting with 'a'}\}$

Left Linear

??

Automata Formal Languages and Logic

Unit 1 - Constructing a Left Linear Grammar

Right Linear

$A \rightarrow aB$

$B \rightarrow aB \mid$

$bB \mid \lambda$

$L = \{\text{starting with 'a'}\}$

Reverse the
Symbols on RHS

Left Linear

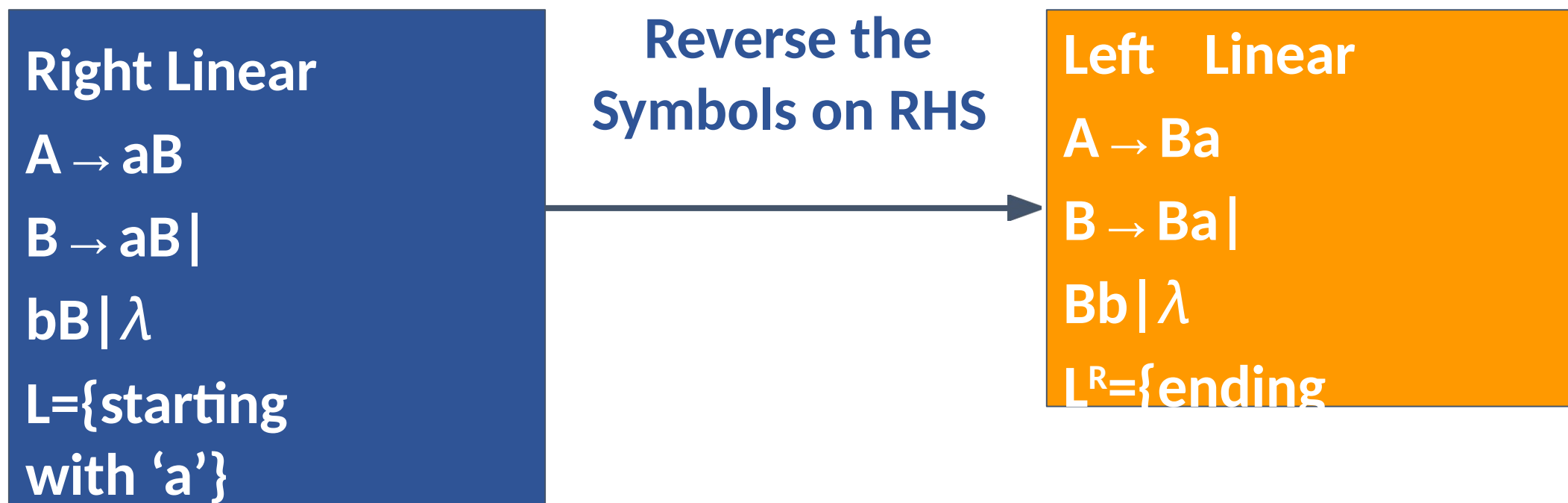
$A \rightarrow Ba$

$B \rightarrow Ba \mid Bb \mid \lambda$

$L = ?$

Automata Formal Languages and Logic

Unit 1 - Constructing a Left Linear Grammar



When we reverse the RHS in every production in the right linear grammar for “L” we get a left linear grammar which will represent L^R .

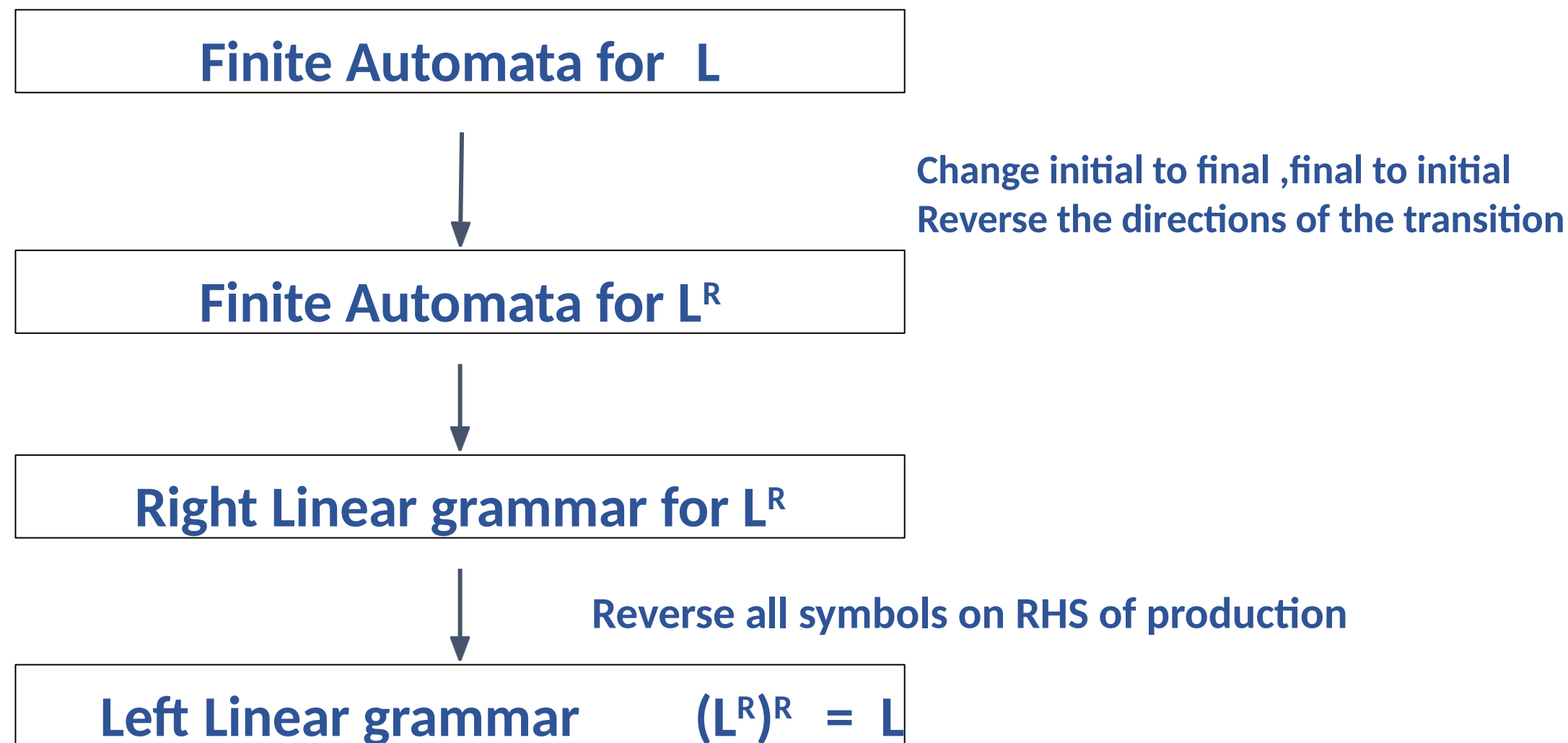
Automata Formal Languages and Logic

Unit 1 - Converting Finite automata to Left Linear Grammar

Converting Finite Automata to Left linear grammar

Automata Formal Languages and Logic

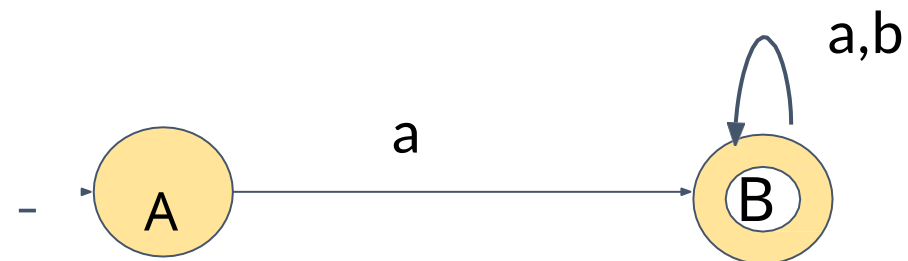
Unit 1 - Constructing a Left Linear Grammar



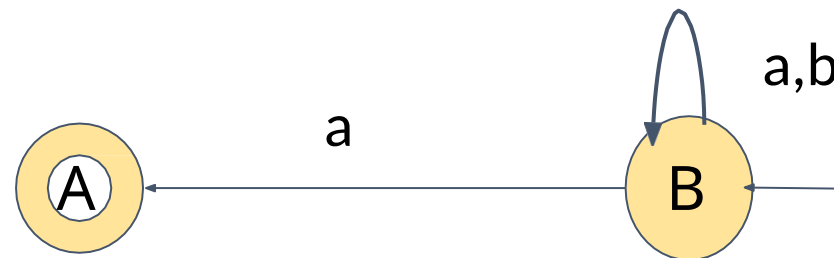
Automata Formal Languages and Logic

Unit 1 - Constructing a Left Linear Grammar

FA for $L = \{aw, w \in \{a,b\}^*\}$



FA for $L^R = \{wa, w \in \{a,b\}^*\}$



RLG for L^R

Right Linear Grammar

$B \rightarrow aB \mid bB \mid aA$

$A \rightarrow \lambda$

LLG for $(L^R)^R = L$

Left Linear Grammar

$B \rightarrow Ba \mid Bb \mid Aa$

$A \rightarrow \lambda$

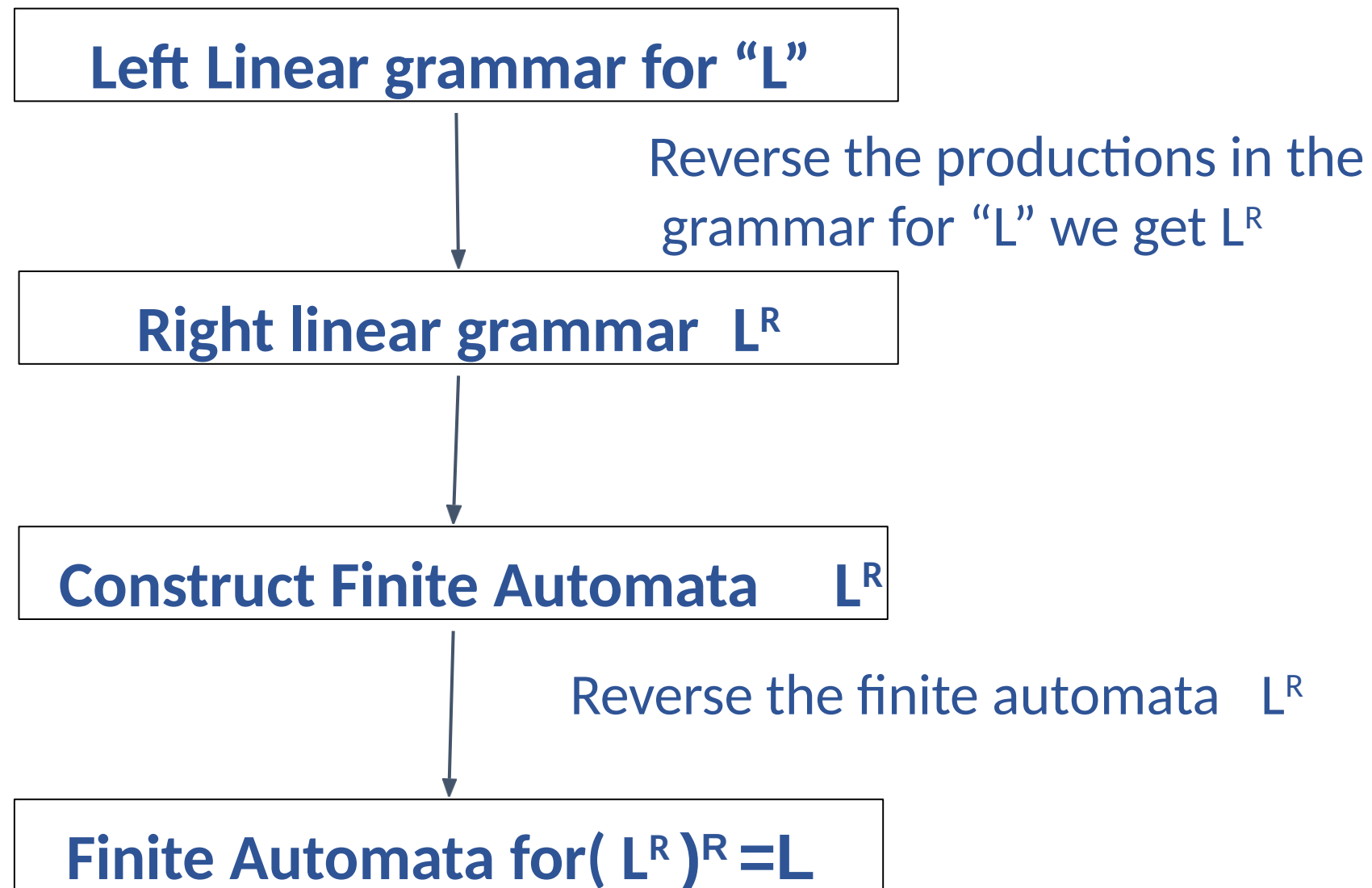
Automata Formal Languages and Logic

Unit 1 - Converting Left Linear Grammar to Finite automata

Converting Left linear grammar to Finite automata

Automata Formal Languages and Logic

Unit 1 - Converting left Linear Grammar to finite automata



Automata Formal Languages and Logic

Unit 1 - Constructing a Left Linear Grammar

LLG for L

Left Linear Grammar

$B \rightarrow Ba \mid Bb \mid Aa$

$A \rightarrow \lambda$

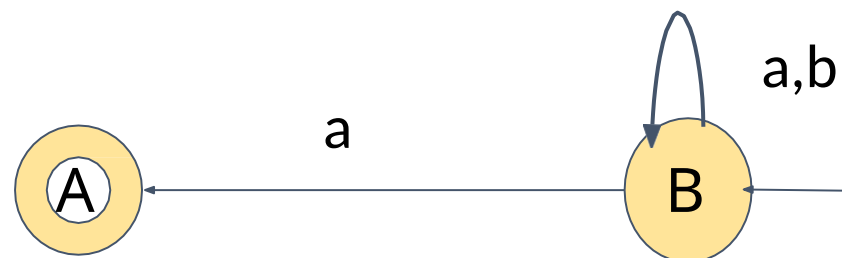
RLG for L^R

Right Linear Grammar

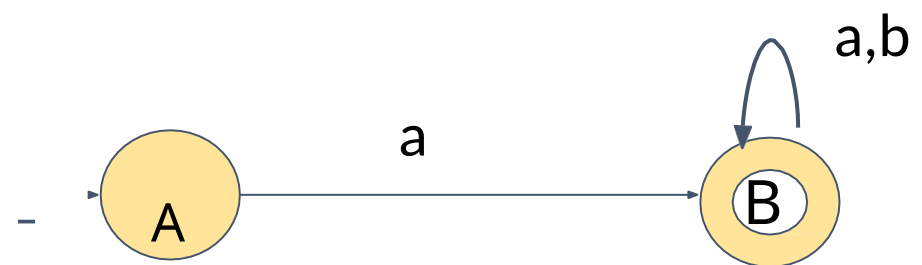
$B \rightarrow aB \mid bB \mid aA$

$A \rightarrow \lambda$

FA for $L^R = \{w\mathbf{a}, w \in \{a,b\}^*\}$



FA for $(L^R)^R = L = \{\mathbf{a}w, w \in \{a,b\}^*\}$



Automata Formal Languages and Logic

Unit 1 - LLG or RLG?

Which one is easier LLG or RLG??

Automata Formal Languages and Logic

Unit 1 - Which one is easier ,Left or Right Linear Grammar?

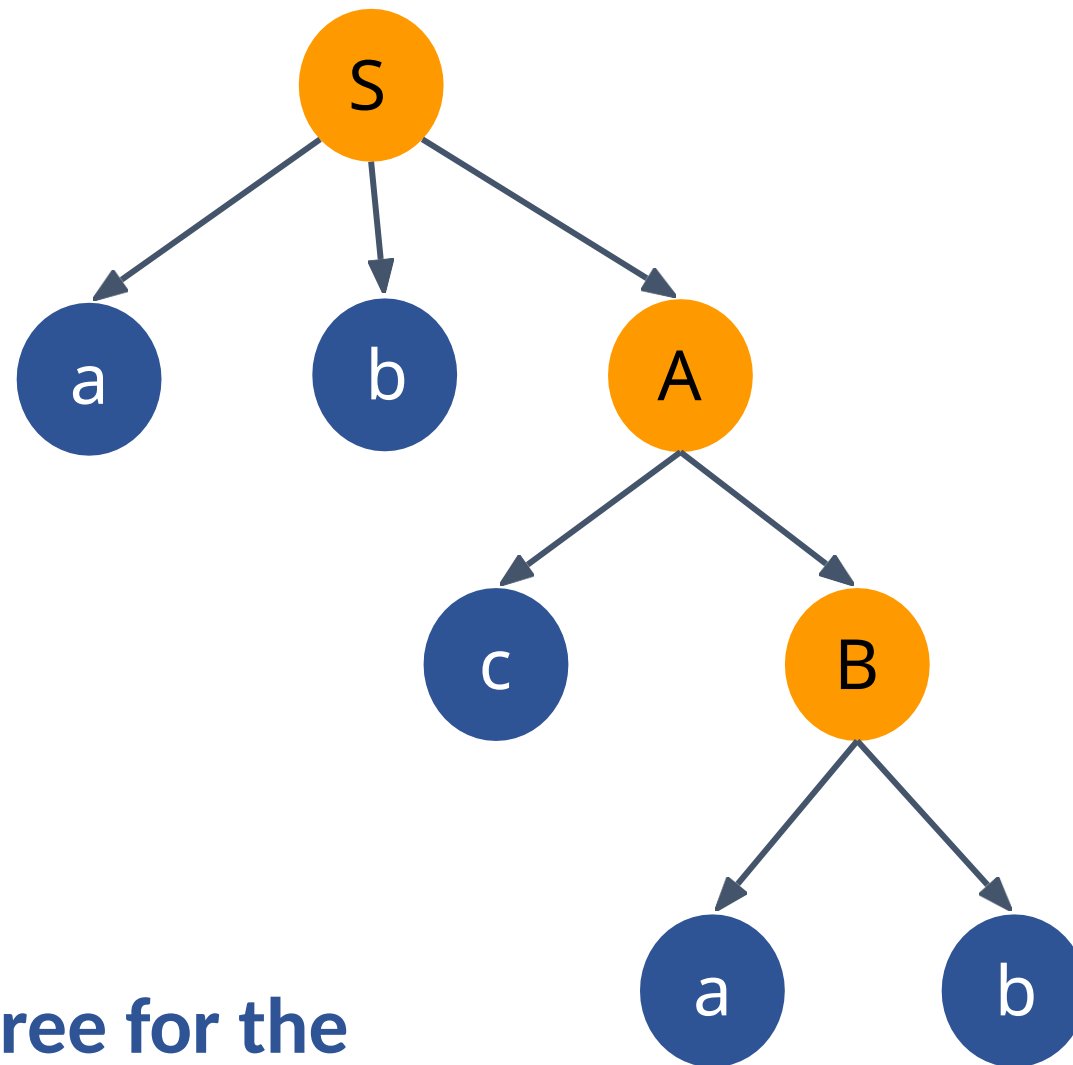
Right Linear

$S \rightarrow abA$

$A \rightarrow cB|aC$

$B \rightarrow ab$

$C \rightarrow b$



Construct Parse tree for the String "abcab"

Automata Formal Languages and Logic

Unit 1 - Which one is easier, Left or Right Linear Grammar?

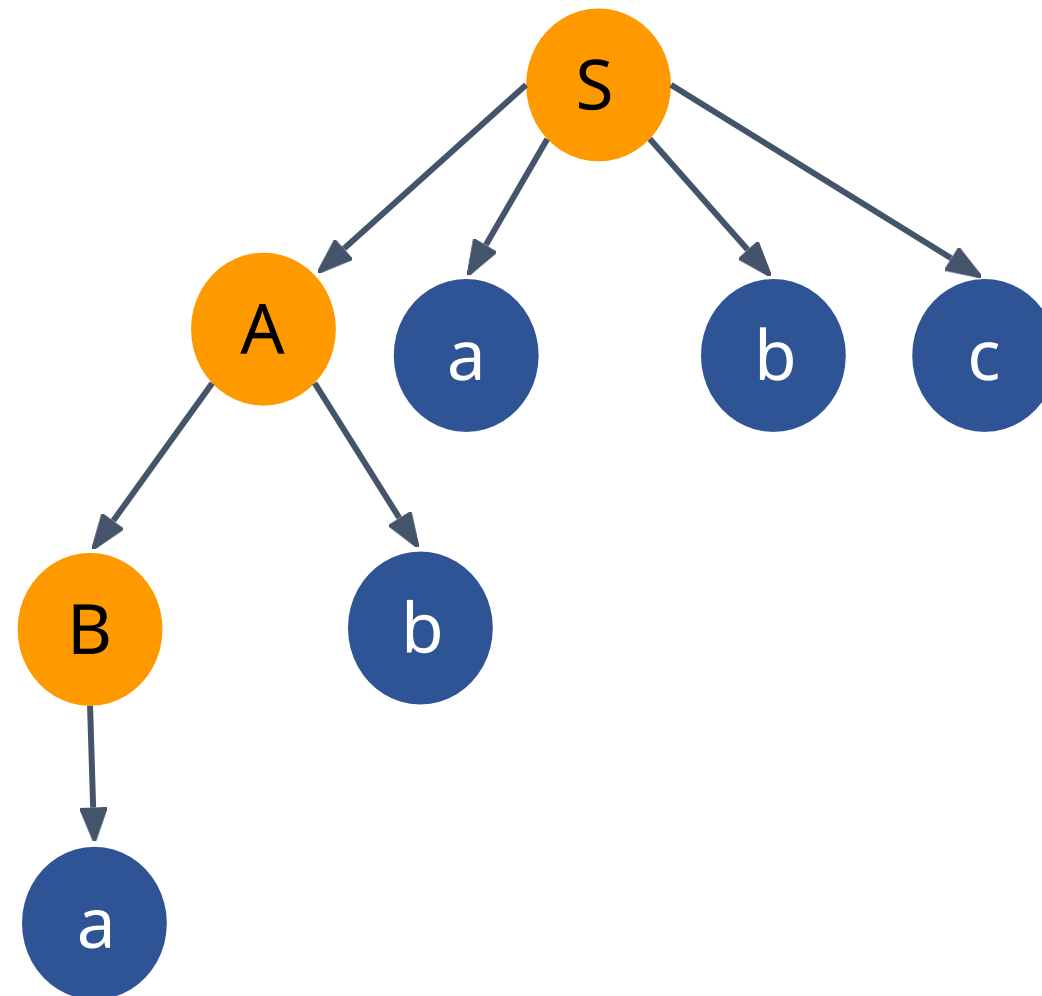
Left Linear

$S \rightarrow Aabc$

$A \rightarrow Bb|C$

$B \rightarrow a$

$C \rightarrow b$



Construct Parse tree for the String “ababc”

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Multiple Choice Questions

Q1. Consider the regular grammar: $S \rightarrow aS \mid bA \mid \lambda$, $A \rightarrow aA \mid bA \mid \lambda$. Which of the following is a valid sentence generated by this grammar?

- A. abba
- B. aba
- C. aaaabbb
- D. aabbbbcc

Q2. What is the distinguishing feature of a regular grammar when compared to context-free grammar?

- A. It has multiple start symbols
- B. All productions are of the form $A \rightarrow BA$
- C. Productions have only terminals on the right-hand side
- D. All productions are of the form $A \rightarrow aB$ or $A \rightarrow a$ or $A \rightarrow \lambda$

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Multiple Choice Questions

Q3. Which regular grammar generates the language $L = \{ab\}$?

- A. $S \rightarrow aS \mid bS \mid \lambda$
- B. $S \rightarrow ab$
- C. $S \rightarrow aA, A \rightarrow b$
- D. $S \rightarrow aS \mid b \mid \lambda$

Q4. Given the language $L = \{a^{2n+1} \mid n \geq 0\}$, which regular grammar correctly generates it?

- A. $S \rightarrow aaS \mid \lambda$
- B. $S \rightarrow aaS \mid a$
- C. $S \rightarrow aS \mid aa$
- D. $S \rightarrow aS \mid \lambda$

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Multiple Choice Questions

Q5.

Consider the following right-linear grammar with productions:

$S \rightarrow aS \mid bA \mid \lambda$

$A \rightarrow aB \mid bA \mid a$

$B \rightarrow bS \mid a$

Which of the following strings is **NOT** generated by this grammar?

A. abaaa

B. abba

C. aaa

D. abaab

Q6.

Given the left-linear grammar:

$S \rightarrow Sa \mid Sb \mid Aa$

$A \rightarrow Bb \mid \lambda$

$B \rightarrow Ab \mid a$

What is the language generated by this grammar?

A. Strings ending with 'a'

B. Strings where the number of a's equals the number of b's

C. Strings over $\{a, b\}$ ending with "ab"

D. All strings over $\{a, b\}$ except the empty string

Automata Formal Languages and Logic

Unit 1 - Regular Expression in Practice

Answers:

1. C
2. D
3. C
4. B
5. C
6. A



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724